FIRST PROGRAMMING LESSONS IN PYTHON

Laurent COUSOT **Patrick COUSOT**
Radhia COUSOT **Thibault COUSOT**

© 2026

For primary-school,
middle-school
and high-school pupils,
and beginners in computing.

Contents

Introduction **1**

1 Specification **5**

2 Interaction **13**
2.1 The robot . . . 13
2.2 Driving the robot . . . 15
2.2.1 Initial position of the robot . . . 15
2.2.2 Handling the robot's pen . . . 16
2.2.3 Turning the robot a quarter turn on the spot . . . 17
2.2.4 Turning the robot an eighth turn on the spot . . . 17
2.2.5 Moving the robot forward . . . 18
2.2.6 Making the robot turn along an arc . . . 19
2.2.7 No way out of the frame . . . 20
2.2.8 Correcting driving errors . . . 20
2.2.9 Finishing and reproducing the drawing . . . 20
2.2.10 Drawing a point, a cross, the grid of the robot's stopping points . . . 21
2.2.11 Drawing in colour . . . 21
2.2.12 Moving the robot quickly with the mouse . . . 21

3 Programming **27**

4 Writing, interpreting and running programs **33**
4.1 Structure of programs . . . 33
4.2 Comments . . . 38
4.3 Interpretation of programs . . . 39
4.4 Syntax errors . . . 40

4.5 Execution of programs . . . 41
4.6 Logical errors . . . 42
4.7 How to locate logical errors in the program . . . 45

5 Procedures 53
5.1 Copying text . . . 53
5.2 Examples of procedures . . . 56
5.3 Declaring procedures . . . 58
5.4 Identifiers . . . 60
5.5 Calling procedures . . . 60
5.6 Robot commands defined by procedures . . . 61

6 Robot jumps to the edge and to the centre of the frame 67

7 "for" loops 71
7.1 Repetitions . . . 71
7.2 The loop counter . . . 72
7.3 The "for" loop . . . 73
7.4 Using a "for" loop in a procedure . . . 75
7.5 Nested "for" loops (independent) . . . 76
7.6 Nested "for" loops (dependent) . . . 78
7.7 Invariants of "for" loops . . . 79

8 Integer expressions 93

9 Calling procedures with parameters 107
9.1 Making the robot move forward and backward . . . 107
9.2 Pen thickness . . . 109
9.3 Speed of the robot . . . 110

10 The robot's grid 115
10.1 Square grid . . . 115
10.2 Rectangular grid . . . 116

11 Constants 131

12 Painting **137**
12.1 Black and white . . . 137
12.2 Colour . . . 138

13 Declaring procedures with parameters **149**

14 Cartesian coordinates **157**

15 The "if" test **167**

16 Boolean expressions **177**
16.1 Parity . . . 177
16.2 Comparison of integers . . . 178
16.3 Negation, conjunction and disjunction . . . 180
16.4 Boolean expressions . . . 182

17 "while" loops **191**
17.1 Is the robot at the edge of the frame? . . . 191
17.2 "while" loop . . . 192

18 Coding **201**
18.1 Morse code . . . 201
18.2 Number systems . . . 202
18.3 Binary coding . . . 203
18.3.1 Natural numbers . . . 203
18.3.2 Characters (ASCII code) . . . 205
18.3.3 Text . . . 206
18.3.4 Character strings . . . 206
18.4 Music . . . 208
18.4.1 Note values . . . 208
18.4.2 Tempo . . . 210
18.4.3 Notes . . . 211
18.4.4 Rests . . . 212
18.4.5 Dynamics . . . 212

19 Rational expressions **223**
19.1 Rational numbers . . . 223
19.2 Parameters of type "float" . . . 224
19.3 Rational expressions . . . 225

19.4 Rounding errors . . . 226

20 Rotations and translations **231**
20.1 Size of the robot's movement grid in polar coordinates . . . 231
20.2 Rotations . . . 232
20.3 Translations . . . 234

21 Recursion **249**
21.1 Simple recursion . . . 249
21.2 Invariants . . . 253
21.3 Non-termination . . . 256
21.4 Mutual recursion . . . 256

22 Variables **275**
22.1 Variables and assignment . . . 275
22.2 Saving the state of the robot . . . 276
22.3 Counting . . . 278
22.4 Entering a value from the keyboard . . . 279
22.5 Displaying the value of an expression . . . 280

Bibliography **289**

Summary of the robot commands **293**

Index **306**

The programs contained in this book and in the accompanying folder are used at the users' own risk as regards their quality or their operation. The authors, the publisher and the distributors disclaim all liability concerning the use of the programs contained in this book and in the accompanying folder. They give no warranty, express or implied, that these programs are free of errors, that they meet any academic or commercial standard, or that they conform to any specification required for any application whatsoever, in particular those that might directly or indirectly result in a violation of the laws of any country. Under no circumstances shall they be held liable for any direct or indirect damage, of whatever nature, resulting from an error in the programs or in the book, even if they have been advised of the presence of such errors that might cause such damage.

Python is a free programming language whose development is supported by the Python Software Foundation, which owns the trademark "Python". Macintosh© is a registered trademark of Apple Inc., Windows© is a registered trademark of Microsoft Corporation. The library of commands of the Robot is protected by copyright.

Introduction

The use of computers is spreading to every field of economic activity. Most users employ application software to carry out one or more specific tasks: text editing, accounting, games, etc. Out of a desire to make the most of the power of the machine, for the fun of it, or simply out of a wish to understand how computers work, many budding computer scientists want to learn programming, sometimes on their own. The task often turns out to be long, difficult, even off-putting. We would like to make it a creative activity, one that trains the mind and is fascinating.

Many introductions to programming rely on prior knowledge of the architecture of computers or use mathematical notions that are beyond the reach of children or of adults without a scientific background [10][1]. (The small raised numbers refer to a footnote explaining the preceding word. For example, the note at the bottom of this page explains the use of numbers between square brackets such as [10].) We propose to proceed differently.

We use drawing, which is within everyone's reach. When the book was first published in 1991, this approach was relatively new in the teaching of computer science, since the striking progress of technology that brought microcomputers with very powerful graphic screens within reach of the general public dated only from the 1980s. It is amusing to note that the origins of our graphic activity go back to prehistory (more precisely to the period of cave art[2] between 22000 and 13000 BC). Computer graphics, which was born

[1] Scientists are used to 'citing their sources' by giving references to the articles in scientific journals and to the books of the authors whose ideas they use. The modern way to cite sources is to use a number placed between square brackets (as for example [10]) which refers to a list of references, placed at the end of the book, called the *bibliography* (see page 289).

[2] Painting on the walls of a cave.

for the general public in the 1980s, will therefore allow us, 20000 years later, to draw with the same pleasure as our distant ancestors.

This book is for everyone. Nursery-school children are able to make stylized drawings by hand and, with a little imagination, to reproduce them on the computer screen using an interactive drawing program (chapters 1 and 2). Primary-school pupils can write programs to produce figurative drawings (chapters 3 to 5). Very quickly they will discover how easily they can program the computer to make simple geometric drawings based on repetitive patterns that they would have neither the patience nor the precision to carry out by hand (chapters 6 and 7). Middle-school pupils will put their rudimentary knowledge of algebra and analytic geometry to work to write drawing programs using elementary calculations and Cartesian coordinate systems (chapters 8 to 17). They will find an intuitive basis for putting into practice concepts that may seem abstract. High-school pupils are able to take in more computer-oriented notions such as coding, or even less elementary notions such as recursion, which become obvious through drawing (chapters 17 to 22). University students will enjoy understanding simply the programming that sometimes seems obscure when it is presented too quickly in the first years of university.

Since programming languages do not offer the desirable graphic possibilities, we have added a library to the Python language[3] which allows us to obtain in a simple way effects that are simple (such as arcs of circles) or spectacular (such as painting) without being tied to the technology of current graphic screens and without resorting to advanced notions of programming such as data structures. This library is in the directory `LeRobot_Python_EN`, and all the examples and the solution programs of the exercises proposed in the book are provided in the folder `LeRobot_Python_EN/book/`[4]. It offers

[3] Python is a programming language designed by Guido van Rossum in the Netherlands in the early 1990s. It is used both for teaching programming and for writing large software. Our programs use Python (version 3) and its graphics library `tkinter`; we have tried them on Macintosh, where the window of the *Robot* is an ordinary macOS window.

[4] The directory `LeRobot_Manual_Python` contains the installation manual `LeRobot_Install.pdf` and a user manual `LeRobot_Manual.pdf` of the *Robot*. The programs of the book are in the folder `book`, sorted by chapter. To run a program, for example `Square.py`, just type the command `./robot.sh book/03_programming/Square.py` in a terminal window opened in the folder `LeRobot_Python_EN`. Another solution is to drag (with the mouse, (left) button pressed (do not release it), or with two fingers on the touchpad (trackpad)) the icon of the file containing the program onto the icon of the

many possibilities that are not explored in the book, such as, for specialists, creating interactive learning programs like those that are provided for addition, Cartesian coordinates and translation.

This book should be read in successive stages, taking the time and the pleasure to do many personal experiments on the computer, solving exercises, modifying the programs proposed as solutions and looking for new drawing themes.

Addenda

At the end of each chapter we give, as addenda, additional notes intended for readers already familiar with computer science.

In this book we introduce the basic activities of programming such as typing in the text, interpretation, execution and debugging of programs. The programming notions are presented in Python: structuring programs into procedures (functions), bounded iterations, integer expressions, constant declarations, parameters (passed by value), tests, boolean expressions, unbounded iterations, coding of information, rational (floating-point) expressions, recursion, variables and assignments. Our approach based on drawing gives a graphic intuition of these notions and avoids the more complex problems of specification and correctness of programs (which are visually obvious). Data structures (lists, tuples, dictionaries and objects in Python) are deliberately left for later, more advanced learning, but are illustrated graphically.

Learning programming at a very elementary level such as the one proposed in this book develops a sense of the passage from the concrete to the abstract, an intuition of formal structures, inductive reasoning and the ability to formalize, which are at the basis of programming but also the starting point of any scientific approach.

Acknowledgements: We thank our colleagues who used the manuscript of the first version of this book in the 1990s to introduce their children to programming, and most particularly Guy Cousineau, Michèle Soria, Denis and Marion; Colette and Jean-Marc Steyaert, Florent and Benoît; Jean Vuillemin and Thomas.

robot `LeRobotPythonEN.app`.

1

Specification

We are going to learn to program a computer by making it do drawings. To start simply, we use squared paper with small squares. The **drawing rule** to be respected is that *our drawings may only contain straight lines and quarter circles that must join two ends of the same horizontal side, of the same vertical side or of the same diagonal of a square of the grid.* This drawing rule therefore gives us the possibility of drawing:

- the horizontal segments _ corresponding to the horizontal sides of a square of the grid;
- the vertical segments | corresponding to the vertical sides of a square of the grid;
- the oblique segments / and \ corresponding to the diagonals of a square of the grid;
- the quarter circles ⌣ and ⌢ joining the ends of the same horizontal side of a square of the grid;
- the quarter circles (and) joining the ends of the same vertical side of a square of the grid;
- the quarter circles ◜ , ◞ , ◝ and ◟ joining the ends of the same diagonal of a square of the grid.

The small drawings of figure 1.1 (page 6) are simple examples drawn while respecting our drawing rule on squared paper. Although this rule does not leave complete freedom to our imagination, we can make figurative drawings such as those of the stegosaurus or of the fortified castle on page 6.

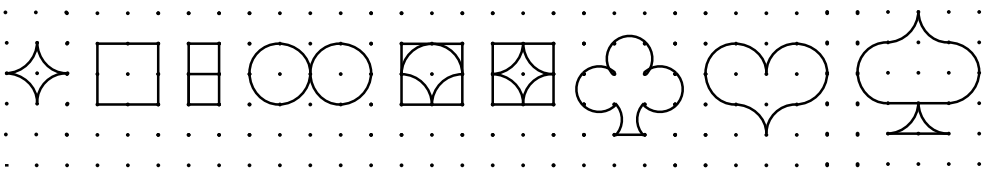


FIG. 1.1 – *Some simple geometric drawings*

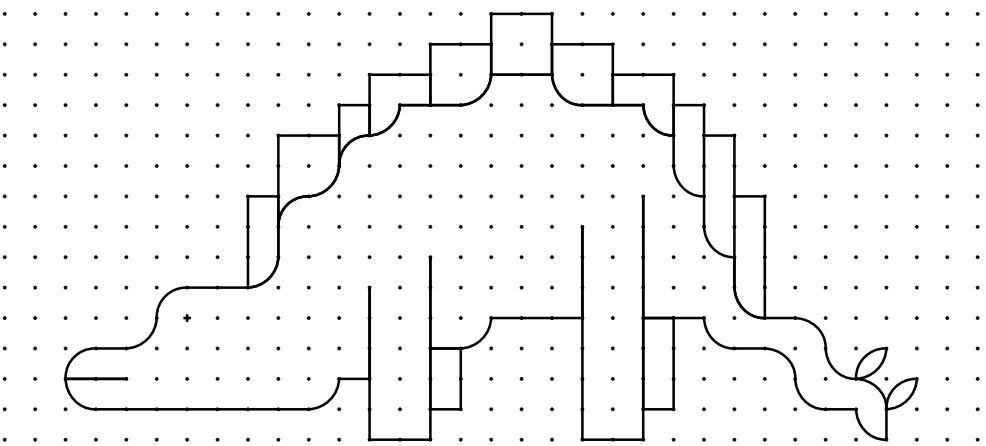


FIG. 1.2 – *Stegosaurus*

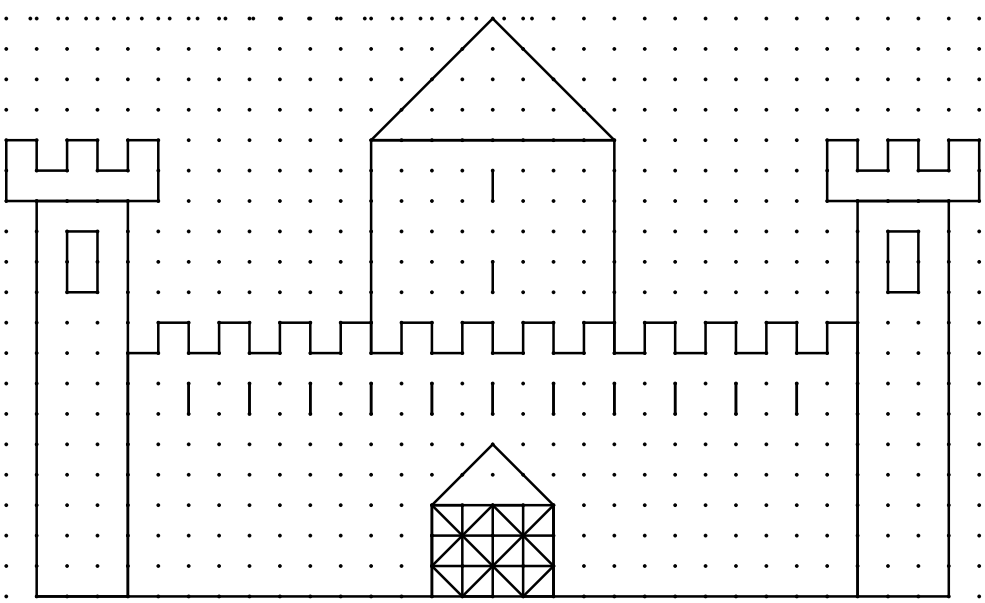


FIG. 1.3 – *Fortified castle*

Exercise 1

– *Complete the drawings of figure 1.4 below, using only horizontal* _ *, vertical* | *or oblique* ／ *and* ＼ *lines, which must join two vertices of the grid with successive numbers, the vertices being shown on this figure by small dots.*

– *Make geometric drawings on squared paper with small squares, respecting the drawing rule. Here are a few ideas (these drawings will later be made on the computer): a hopscotch, a trapezoid, a parallelogram, a rectangle, a square, a house, a stylized flower, an origami paper hen, a little man, the capital letter H in relief, the digits of a digital watch, the alphabet (in capital letters), a helicopter, a truck, a train, a barge, etc.* □[1]

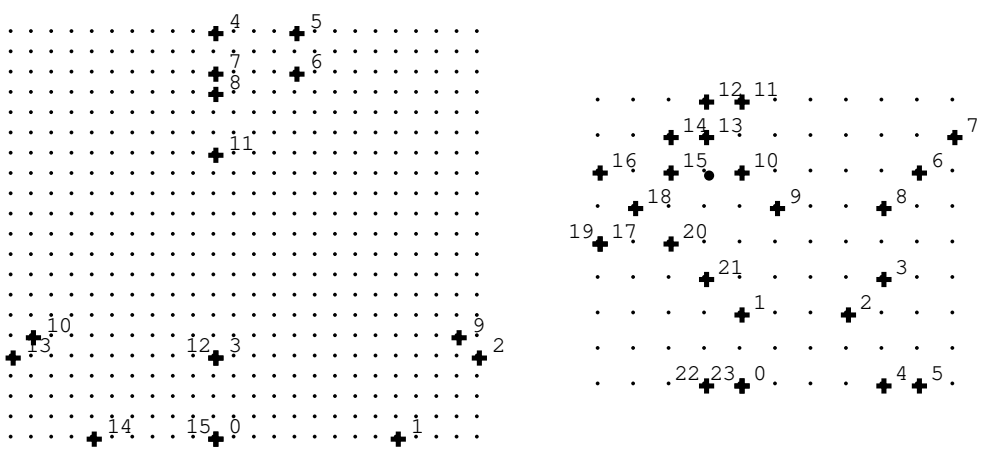


FIG. 1.4 – *Drawings to complete*

[1]The box □ marks the end of the text of an example, of an exercise, of a solution, etc. It allows the reader who does not want to read this text to find quickly where to resume reading.

Solution 1

_ *Once completed, the drawings of figure 1.4 (page 7) represent a boat and a dog.*

_ *Examples of drawings are given on pages 8, 9 and 10.* □[2]

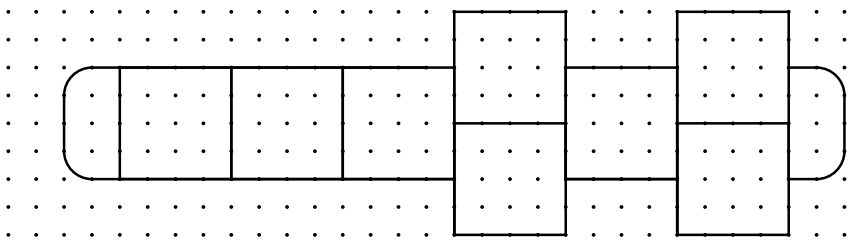


FIG. 1.5 – *Hopscotch*

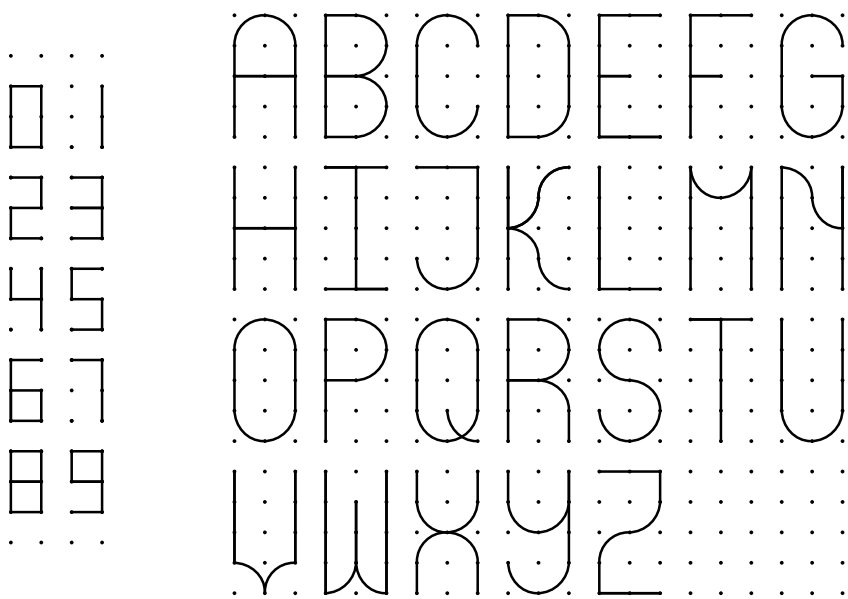


FIG. 1.6 – *Digital digits and alphabet (in capital letters)*

[2] We give the solutions of the exercises a few pages after their statements. We strongly advise the reader to read this book in small steps, taking care to do a few exercises after each chapter.

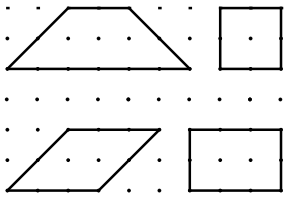


FIG. 1.7 – *Quadrilaterals*

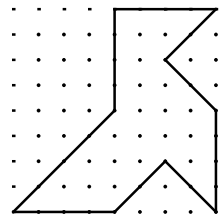


FIG. 1.10 – *Origami paper hen*

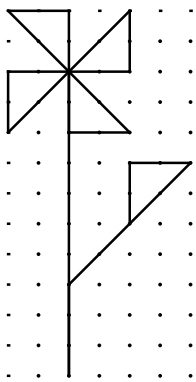


FIG. 1.8 – *Stylized flower*

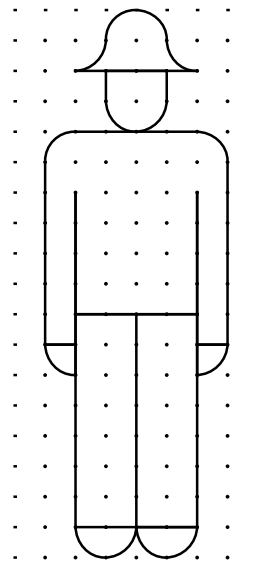


FIG. 1.11 – *Little man*

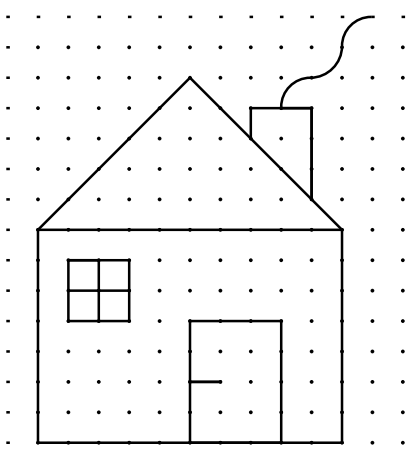


FIG. 1.9 – *House*

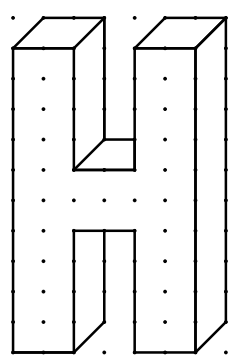


FIG. 1.12 – *Capital letter H in relief*

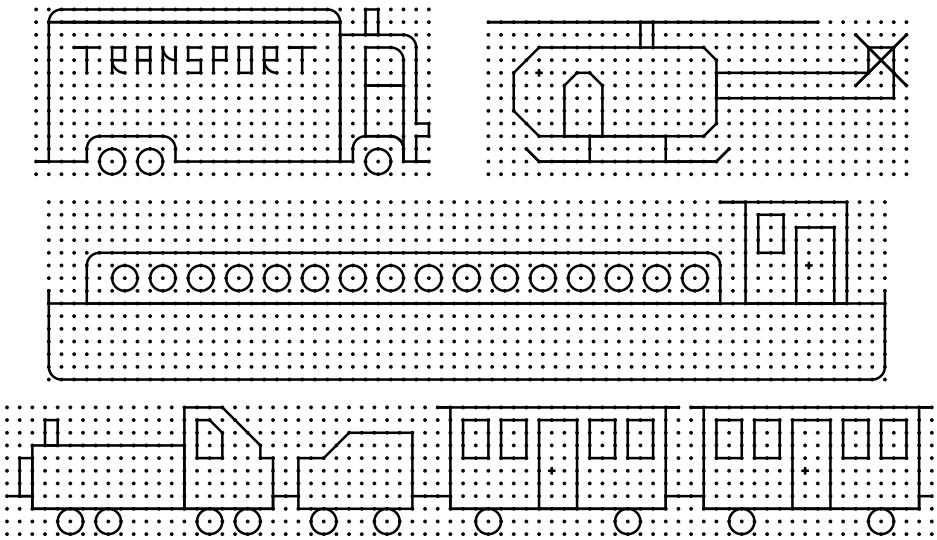


FIG. 1.13 – *Means of transport*

Addenda

Before writing a computer program to solve a problem, one must begin by specifying precisely the task that the computer will have to carry out. In our graphic approach to explaining the programming of computers, this *specification* is a drawing, which the computer must produce.

Drawing by hand while respecting the constraint of using only lines of the form ⎯, |, ⁄, \, ◜, ◞, ◠, ◡, ⌒, (or) is difficult for very young children. To ensure a progression from the simple to the complex, one begins by having them copy drawings made only of horizontal and vertical lines on paper with large squares, then one introduces the oblique segments, the quarter circles on the diagonals and finally the quarter circles on the sides of a square of the grid. These geometric drawings are an excellent preparation for learning to write. Then the children enjoy inventing their own drawings. They must then learn to model their ideas within the rather strict frame set for them by the drawing rules. This notion of *model*, in the sense of a simplified representation of a complex process or system, is at the basis of the *scientific approach*.

These geometric drawings, built using only a few primitive shapes, are also a first step towards programming, which consists of building a sometimes complex edifice from the elementary bricks that are the instructions of the programming language.

In the next chapter, we no longer make drawings by hand but draw them on the computer screen by driving a robot with the computer's keyboard. This lets us discover how the computer works in *interactive mode* and enter directly into the world of *computer graphics*, that is, the creation of images on a computer. By construction, the robot draws while respecting the drawing rules that have been set. It lets us correct our mistakes easily, save our drawing on disk, make reductions or enlargements of it and print as many copies of it as we want.

2

Interaction

2.1 The robot

To make our geometric drawings on the computer we are going to drive a little robot. Its name is $\mathcal{R}obot$[1]. To draw with the $\mathcal{R}obot$, double-click on its icon



LeRobot.app

Our $\mathcal{R}obot$ carries a pen which it can raise or lower while it moves about on the computer screen. When the $\mathcal{R}obot$ moves with its pen down, it leaves a trail on the screen, just as a snail would on a sheet of paper. When it moves with its pen up, it leaves no trace of where it has been. To draw, the $\mathcal{R}obot$ must follow the lines of the drawing exactly with its pen down. It can also go from one point of the drawing to another point which is not joined to it by a line by raising its pen. To guide the $\mathcal{R}obot$ we use the computer keyboard, which serves as a remote control. So we can drive the $\mathcal{R}obot$ as if it were a little radio-controlled car.

On the computer screen our $\mathcal{R}obot$ is represented by a small arrow ▲ which shows in which direction of the compass rose it is facing (see figure 2.1, page 14). It can face north ▲, north-east ◥, east ▶, south-east ◢, south ▼, south-west ◣, west ◀ or north-west ◤. The arrow is solid when the $\mathcal{R}obot$ has lowered its pen and hollow (△, ◹, ▷, ◺, ▽, ◿, ◁ or ◸) when the pen is up.

[1] The way its name is written leaves no doubt about its humanoid and friendly nature.

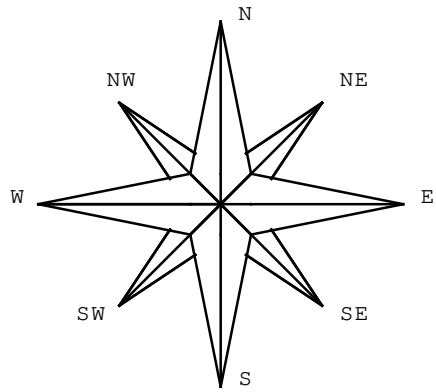


FIG. 2.1 – *Eight-point compass rose*

Like trains on their rails, our *Robot* cannot go just anywhere on the screen. It must always move along the lines or arcs joining two successive points of a grid whose size is fixed by the size of the computer screen. A small part of this grid is shown below in figure 2.2.

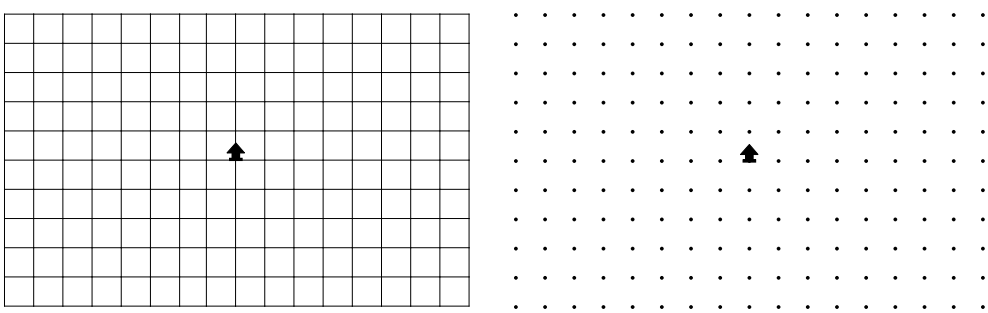


FIG. 2.2 – *Grid and stopping points of the Robot*

This imaginary grid corresponds to the squared paper we used to make our drawings by hand, except that the squares are not visible on the computer screen. After a move, the *Robot* always stops at the intersection of a horizontal line and a vertical line of the imaginary grid. Figure 2.3 (page 15) shows all the possible paths of the *Robot* between any two stopping points on the grid. Figure 2.4 (page 15) shows the exact position of the *Robot* when it stops on the grid. It places the tip of its pen, located in the middle at the back of the arrow, exactly on a stopping point of the grid.

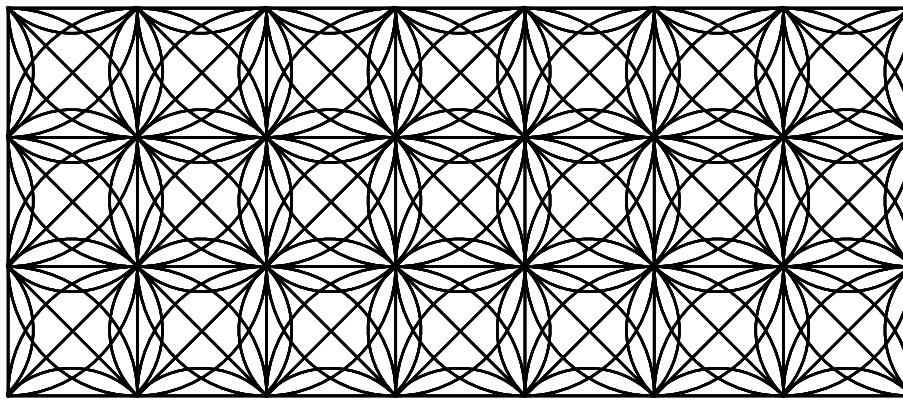


FIG. 2.3 – *Paths of the Robot (magnification* × 4.5*)*

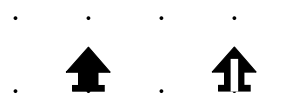


FIG. 2.4 – *The Robot at rest (magnification* × 2.5*)*

2.2 Driving the robot

To drive the *Robot* on the computer, you must "*run the program* `Draw`", which is part of the `LeRobot_Python_EN` library that comes with this book[2].

2.2.1 Initial position of the robot

When the program `Draw` starts running, the computer screen looks as shown, reduced, in figure 2.5 (page 16). So at the start the *Robot* is at rest in the middle of the imaginary grid, with its pen down, facing north ⬆. To drive the *Robot*, we use the keys of the computer keyboard, which serves as a remote control. We shall now see which keys to press to raise or lower the pen, move forward, turn on the spot, make a turn, etc.

[2] Just double-click on the icon of the *Robot* `LeRobotPythonEN.app`. You can also, in a terminal window opened in the folder `LeRobot_Python_EN`, type `python3 Draw.py`. On the Macintosh, the window of the *Robot* is an ordinary window, which comes to the front; if the keys have no effect, click in this window.

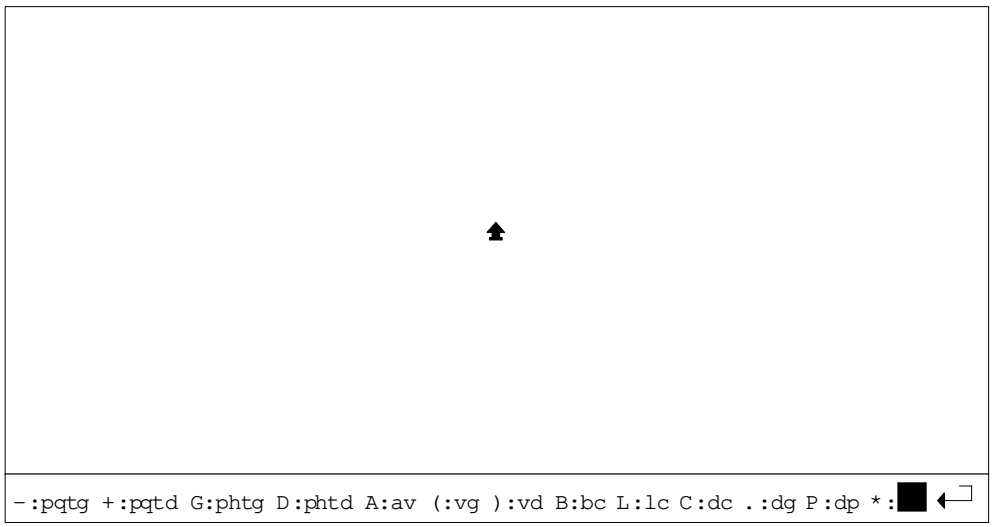


FIG. 2.5 – *The computer screen at the start* of the execution of the program `Draw`

2.2.2 Handling the robot's pen

By pressing the key marked [U] on the keyboard, the *Robot* raises its pen. By pressing the key [D], it lowers it. As there is not much room on the computer screen to recall the instructions for the *Robot* in detail, we use mnemonic abbreviations[3] placed at the bottom of the screen, as can be seen in figure 2.5 above. These abbreviations are made of the first letters of English words. For example:

- `u`:`pu` means "the key [U] raises the pen": pen up;
- `d`:`pd` means "the key [D] lowers the pen": pen down.

Letters are typed in lower case, without pressing the shift key (a capital letter is not a command of the *Robot*).

The effect of the keys [U] and [D] that handle the pen of the *Robot* is summarised in table 2.1 (page 17).

[3] Mnemonic abbreviations help the memory by association of ideas.

U (pu)	Before	↑	↗	→	↘	↓	↙	←	↖
	After	↑	↗	→	↘	↓	↙	←	↖
D (pd)	Before	↑	↗	→	↘	↓	↙	←	↖
	After	↑	↗	→	↘	↓	↙	←	↖

TAB. 2.1 – *Handling the pen of the Robot*

2.2.3 Turning the robot a quarter turn on the spot

The key – turns the *Robot* on the spot a quarter turn to the left (`qtl` for short, from quarter turn left). For example, if the *Robot* is facing north ↑, the key – turns it on the spot to face west ←. If it is facing east →, it turns to face north ↑. The key + turns the *Robot* on the spot a quarter turn to the right (`qtr` for short, from quarter turn right).

When we speak of the left side of the *Robot*, you must imagine yourself in the place of the *Robot*, looking in the direction of the arrow. For example, the left of → is the top of this page, whereas the left of ↓ is on the right of this sheet!

The effect of the commands – (`qtl`) and + (`qtr`) is summarised in the table below for the case where the pen is down. The effect is the same when the pen is up.

– (qtl)	Before	↑	↗	→	↘	↓	↙	←	↖
	After	←	↖	↑	↗	→	↘	↓	↙
+ (qtr)	Before	↑	↗	→	↘	↓	↙	←	↖
	After	→	↘	↓	↙	←	↖	↑	↗

TAB. 2.2 – *Quarter turns of the Robot on the spot*

2.2.4 Turning the robot an eighth turn on the spot

The key L turns the *Robot* on the spot an eighth turn to the left (`etl` for short, from eighth turn left; the key `L` stands for "left"). For example, if it is facing north ↑, the *Robot* turns to face north-west ↖. If you keep pressing the key L, the *Robot* turns an eighth turn each time, going through the

successive positions ◀ then ◤, ▼, ◣, ▶, ◥, ... The key [R] turns the *Robot* on the spot an eighth turn to the right (`etr` for short, from eighth turn right; `R` stands for "right"). The effect of the keys [L] (`etl`) and [R] (`etr`) is summarised below:

[L] (`etl`)	Before	▲	◥	◀	◤	▼	◣	▶	◥
	After	◥	◀	◤	▼	◣	▶	◥	▲
[R] (`etr`)	Before	▲	◥	▶	◣	▼	◤	◀	◥
	After	◥	▶	◣	▼	◤	◀	◥	▲

TAB. 2.3 – *Eighth turns of the Robot on the spot*

2.2.5 Moving the robot forward

The key [F] makes the *Robot* move straight forward by the length of one side of a square of the grid. The mnemonic abbreviation of this command of the *Robot* is `fd`, from forward (and `F` is the first letter of "forward"). If the pen is down, this move draws a short line. If the pen is up, this move leaves no trace on the drawing. In both cases the *Robot* moves straight ahead in the direction it is facing without changing direction:

[F] (`fd`)	Before	▲	◥	▶	◣	▼	◤	◀	◥
	After	▲	◥	▶	◣	▼	◤	◀	◥

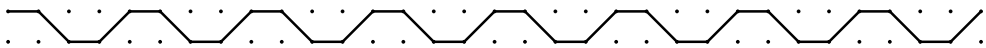
TAB. 2.4 – *Forward moves of the Robot in the eight directions*

Example 1 (Notched frieze) By pressing the keys [F] (`fd`), [+] (`qtr`), [F] (`fd`), [+] (`qtr`), [F] (`fd`), [−] (`qtl`), [F] (`fd`), [−] (`qtl`), [F] (`fd`), [+] (`qtr`), [F] (`fd`), [+] (`qtr`), [F] (`fd`), [−] (`qtl`), [F] (`fd`), [−] (`qtl`), ... one after the other, we get the following frieze:



□

Example 2 (Battlement frieze) By pressing the keys [R] (`etr`), [R] (`etr`), [F] (`fd`), [R] (`etr`), [F] (`fd`), [L] (`etl`), [F] (`fd`), [L] (`etl`), [F] (`fd`), [R] (`etr`), [F] (`fd`), [R] (`etr`), [F] (`fd`), ... one after the other, we get the following frieze:



□

2.2.6 Making the robot turn along an arc

The key [(] makes the $\mathcal{R}^{o}_{b}{}^{o}t$ turn to the left along an arc (`cl` for short, from curve left) whereas the key [)] makes it turn to the right along an arc (`cr`, curve right). These keys were chosen because parentheses suggest the idea of a bend and the left parenthesis [(], which is on the left of the keyboard, corresponds to the left turn, whereas the right parenthesis [)], which corresponds to the right turn, is on the right of the keyboard. If the pen is down, this move draws a small arc of a circle. If the pen is up, this move leaves no trace on the drawing. In both cases the $\mathcal{R}^{o}_{b}{}^{o}t$ moves on the grid and changes direction as shown in the table below:

[(] (`cl`)	Before								
	After								
[)] (`cr`)	Before								
	After								

TAB. 2.5 – *Turns of the $\mathcal{R}^{o}_{b}{}^{o}t$ along an arc*

Example 3 (Frieze of braces) By pressing the keys [)] (`cr`), [(] (`cl`), [+] (`qtr`), [+] (`qtr`), [(] (`cl`), [)] (`cr`), [+] (`qtr`), [+] (`qtr`), [)] (`cr`), [(] (`cl`), [+] (`qtr`), [+] (`qtr`), ... one after the other, we get the following frieze:



□

2.2.7 No way out of the frame

By pressing the key F several times in a row, the $\mathcal{R}\breve{o}\breve{b}ot$ ⏶ moves straight ahead drawing a line until it reaches the edge of the frame that bounds its field of action. If we keep pressing the key F to try to make it leave the frame, this is an *error* in the sense that the $\mathcal{R}\breve{o}\breve{b}\breve{o}t$ refuses to do it and gives a "beep-beep" of protest[4] (it also writes "`** EXIT FORBIDDEN **`" in the terminal window)!

2.2.8 Correcting driving errors

We often make mistakes when driving the $\mathcal{R}\breve{o}\breve{b}\breve{o}t$. For example, instead of making it turn to the right with a right parenthesis) (`cr`), we type a left parenthesis ((`cl`) and the $\mathcal{R}\breve{o}\breve{b}\breve{o}t$ turns to the left! The *delete* key, marked with an arrow pointing to the left ← or delete, lets us correct errors by going backwards and erasing the last lines in the reverse order in which they were drawn by the $\mathcal{R}\breve{o}\breve{b}ot$.

2.2.9 Finishing and reproducing the drawing

While the program `Draw` is running, it is possible, at any moment, to finish the drawing by pressing the return key ↩ or return, also marked `Enter` or `Ret` on some computer keyboards (`Entrée` on French ones). This is recalled at the bottom right of the computer screen by a blinking symbol ↩.

The drawing that has just been finished is not lost: it is possible to reproduce it and to print it[5].

---

[4] The way its name is written leaves no doubt about its displeasure.

[5] After the return key, the $\mathcal{R}\breve{o}\breve{b}ot$ asks in the bottom line of the screen `Do you want to save in my_drawing.py? (y/n)`. By pressing the key Y, the drawing is saved as a Python program in the file `my_drawing.py`, which begins with **from** `LeRobot` **import** `*`; it is reproduced by typing `make my_drawing` (or `python3 my_drawing.py`) in the folder `LeRobot_Python_EN`. The next drawing saved will replace this file: to keep the file for good, you must rename it and then run it by dragging it onto the application `LeRobotPythonEN.app`.

2.2.10 Drawing a point, a cross, the grid of the robot's stopping points

The key P (`dp`, draw a point) draws a point, without moving the *Robot*. We use the key C (`dc`, draw a cross) to draw a small cross `+`. The key . (`dg`, draw the grid) draws the grid of the stopping points of the *Robot* as can be seen for example in figure 1.13 (page 10).

2.2.11 Drawing in colour

The *Robot* can draw with pens of different colours. While the program `Draw` is running, the colour of the pen used by the *Robot* is shown in a small square at the bottom right of the screen (see figure 2.5 page 16). To change the colour of the pen, just press the key * (`pc`, pen colour) once or several times to go through all the possible colours one after the other.

To paint an area enclosed by a line, place the *Robot* so that its back is inside the area, then press the key W (`paint`): the area takes the colour of the pen (see chapter 12).

2.2.12 Moving the robot quickly with the mouse

To move the *Robot* without drawing anything, we can raise the pen U (`pu`), then move forward once or several times F (`fd`), possibly changing direction (R (`etr`) or L (`etl`)), and finally lower the pen D (`pd`). To go faster, we can use the mouse to move the *Robot* to any point of the grid of stopping points without drawing anything: just click on this point (or drag the mouse to it with the button held down), or press the arrow keys.

Summary of the interactive commands for driving the robot

The commands for driving the *Robot* to draw interactively with the program `Draw` are summarised in table 2.6 (page 22).

Key:	*Command:*	*Effect:*
Grid of the stopping points of the Robot:		
.	`dg`	draws the grid of stopping points (draw the grid).
Turns of the Robot on the spot:		
R	`etr`	eighth turn to the right (eighth turn right).
+	`qtr`	quarter turn to the right (quarter turn right).
L	`etl`	eighth turn to the left (eighth turn left).
–	`qtl`	quarter turn to the left (quarter turn left).
Moves of the Robot:		
F	`fd`	moves straight forward to the next stopping point (forward).
)	`cr`	quarter circle to the right (curve right).
(	`cl`	quarter circle to the left (curve left).
← → ↑ ↓		moves by one stopping point, without drawing.
Handling the pen of the Robot:		
U	`pu`	pen up: moves without drawing (pen up).
D	`pd`	pen down: draws while moving (pen down).
P	`dp`	draws a point, without moving (draw a point).
C	`dc`	draws a cross, without moving (draw a cross).
Colour and painting:		
*	`pc`	next pen colour (pen colour).
W	`paint`	paints the closed area behind the *Robot* with the pen colour.
Driving errors of the Robot:		
← or `delete`		erases the last lines drawn, in reverse order.
Stopping the Robot:		
↩ or `return`		finishes the drawing; Y then saves it in `my_drawing.py` (to reproduce and print it).

TAB. 2.6 – *Summary of the interactive commands of the Robot*

Exercise 2 *Suppose that the* Robot *is initially facing north with its pen down* ⬆. *Among the digits of a digital watch shown in figure 1.6 (page 8), which one do you get by giving the robot the* sequence of commands[6] *below:*

```
qtr(); fd(); qtl(); fd(); fd(); qtl()
fd(); qtl(); fd(); qtl(); fd()
```

that is, by pressing one after the other:

[+] [F] [−] [F] [F] [−] [F] [−] [F] [−] [F] *?* □

Exercise 3 *Which capital letter of the alphabet of figure 1.6 (page 8) do you get by giving the* Robot ⬆ *the following sequence of commands:*

```
fd(); fd(); qtr(); fd(); qtr(); qtr(); fd()
qtr(); fd(); fd(); qtr(); fd(); fd()
```

that is, by pressing:

[F] [F] [+] [F] [+] [+] [F] [+] [F] [F] [+] [F] [F] *?* □

Exercise 4 *What series of commands must be given to the* Robot ⬆ *for it to draw the capital letter* `T` *shown in figure 1.6 (page 8)?* □

Exercise 5 *What series of commands must be given to the* Robot ⬆ *for it to draw the digit 4 shown in figure 1.6 (page 8)?* □

Exercise 6 *What sequence of commands must be given to the* Robot *to draw the octagon of figure 2.6 below?* □

Exercise 7 *What sequence of commands must be given to the* Robot *to draw the three tangent circles of figure 2.6 below?* □

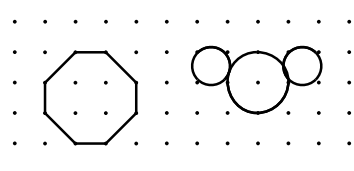


FIG. 2.6 – *Octagon and tangent circles of different sizes*

[6] A sequence of commands is a succession or series of commands given in a certain order.

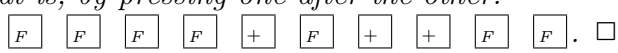
Solution 2 *By trying it on the computer we find the digit nine.* □

Solution 3 *The letter F.* □

Solution 4 *The capital letter T of figure 1.6 (page 8) is obtained by giving the Robot the following commands one after the other:*

```
fd(); fd(); fd(); fd(); qtr(); fd()
qtr(); qtr(); fd(); fd()
```

that is, by pressing one after the other:

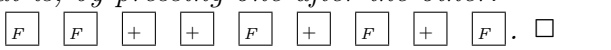


□

Solution 5 *The digit 4 of a digital watch is obtained by giving the Robot the following sequence of commands:*

```
fd(); fd(); qtr(); qtr(); fd()
qtr(); fd(); qtr(); fd()
```

that is, by pressing one after the other:

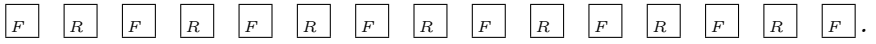


□

Solution 6 *The octagon of figure 2.6 (page 23) is obtained by giving the Robot the sequence of commands below:*

```
fd(); etr(); fd(); etr(); fd(); etr(); fd(); etr()
fd(); etr(); fd(); etr(); fd(); etr(); fd()
```

that is, by pressing one after the other:

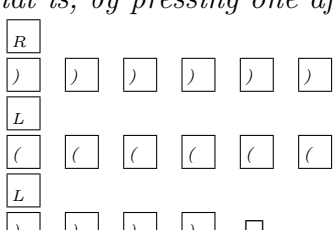


□

Solution 7 *The tangent circles of figure 2.6 (page 23) are obtained by giving the Robot the sequence of commands below:*

```
etr()
cr(); cr(); cr(); cr(); cr(); cr()
etl()
cl(); cl(); cl(); cl(); cl(); cl()
etl()
cr(); cr(); cr(); cr()
```

that is, by pressing one after the other:



□

Addenda

In this chapter we have learnt to drive the $\mathcal{R}^{o}_{b}{}^{o}t$ interactively. *Interactive mode* is the way many computer programs work, programs called *interpreters*. These programs hold a dialogue with the user by means of a communication language called the *command language*. The user directs the work of the interpreter by giving it commands by means of the computer keyboard, the mouse, etc. There are very many examples. Let us mention text editors, spreadsheets, the Macintosh 'Finder' or the command interpreter of operating systems[a] (such as Microsoft `Windows` on PCs or Unix on scientific workstations), and transaction systems for querying databases (such as those that used to be queried by Minitel, the French videotex terminal). The obvious advantage of this interaction with the computer is that the user can correct mistakes as soon as they happen. The main drawback is that repetitive tasks very quickly become tedious.

For our $\mathcal{R}^{o}_{b}{}^{o}t$, the command language comes down to a few very simple commands given on the keyboard (C , D , F , L , P , R , U , + , − , (,) , . , * , W , ⟵ or return), not forgetting the mouse, which is a very quick way of pointing to a precise point on the screen. The $\mathcal{R}^{o}_{b}{}^{o}t$ only communicates with the user by "beep-beeps" telling the user not to make it leave the frame. Drawing errors can be corrected with the delete key ⟵ or delete , which cancels the effect of the last commands given to the $\mathcal{R}^{o}_{b}{}^{o}t$.

During interactive drawing, the $\mathcal{R}^{o}_{b}{}^{o}t$ generates a Python program (the file `my_drawing.py`) which can then be run to reproduce and print the drawing. This can be compared to the painting robots of the car industry. A skilled worker paints a car with a spray gun. All the worker's movements are recorded on the computer and then used to generate a program. By running this program, the computer can then control a robot with a manipulator arm holding a spray gun, which faithfully reproduces all these movements and paints car after car, endlessly. This is a rudimentary example of *learning*, an active research topic in *artificial intelligence*, where the computer generates its own program to carry out certain tasks from examples, more or less numerous, that are given to it.

[a] Operating systems are programs that manage the internal organisation and the allocation of the resources of computers.

Interactive drawing on the computer with the $\mathcal{R}^{o}_{b}{}^{o}t$ is no more difficult than any video game, and it is certainly more educational. It teaches children to use a simple *formalism*. It requires an effort of imagination to find one's way in the plane, since the commands are not relative to the observer but to the $\mathcal{R}^{o}_{b}{}^{o}t$. However, the task soon becomes tiresome for repetitive geometric drawings. Only programming, which we tackle in the next chapter, makes it possible to obtain such drawings effortlessly.

3

Programming

So far we have used the computer to drive the $\mathcal{R}^{o}_{\smile}{}^{o}t$ in *interactive* or *conversational* mode. In this mode of operation we press a key on the computer keyboard, which passes a command to the $\mathcal{R}^{o}_{\smile}{}^{o}t$, which carries it out immediately, unless by mistake we try to make it leave the frame, in which case it answers with a "beep-beep". We go on like this, correcting driving mistakes as the drawing takes shape, until we tell the computer that the drawing is finished.

Whereas the interactive control of the $\mathcal{R}^{o}_{\smile}{}^{o}t$ corresponds to the immediate execution of a task, the programmed control of the $\mathcal{R}^{o}_{\smile}{}^{o}t$ consists in separating the description of the task from its later executions.

More generally, the idea of *programming* is the following: to get something done, the *programmer* writes a *program* (in a certain *language*) which describes precisely how to do this thing. Then he asks a *computer* to run the program, following exactly the instructions given by the program. In other words, instead of doing the thing himself right away, the programmer explains to a computer how to do it in his place later! In this way we avoid having to carry out long and repetitive tasks, provided we know how to describe these tasks properly.

If he has made a mistake, the programmer must correct his errors by modifying the program. This is called *debugging* the program. The *debugging* of the program consists in finding and then removing the *bugs*, that is, the errors which show up during execution as malfunctions (such as, for example, `qtl()` instead of `qtr()`, or `st()` forgotten at the end of the program)[1]. The

[1] In the French edition we use the vocabulary recommended by the French ministerial

principle of computers controlled by a program has analogies with many human activities, as the table below shows:

Programmer	*Program*	*Computer*	*Drawing*
Head chef	Recipe	Cooks	Meal
Composer	Musical work	Musicians	Concert
Author	Text of a play	Actors	Theatre performance
Choreographer	Choreography	Dancers	Ballet
Director	Film	Film projector	Film showing
Weaver	Punched cards	Jacquard loom	Jacquard fabric
Course designer	Course plan	Rider and horse	Show-jumping round

TAB. 3.1 – *Examples of programmed activities*

We are now going to learn to program computers in Python. Driving the $\mathcal{R}^{o}_{o}t$ to make drawings will be our main application. To make our drawings in this programmed mode of operation, we proceed as follows:

1. We begin by writing a Python program in which we state the sequence of commands that the $\mathcal{R}^{o}_{o}t$ must carry out to make the drawing we have in mind, or that we have drawn beforehand on a sheet of squared paper. This program is written with the help of the computer (with a text editor), which keeps it on disk in a file whose name ends with `.py`, for example `Square.py`.
2. Next we ask a program of the computer, the Python *interpreter*, to read this program, to check that it is correctly written and to translate it into the *machine language* of the computer (or into a form very close to it). We say that the interpreter *interprets* the program. The translated program and the original Python program are equivalent in the sense that they describe exactly the same sequence of commands for the $\mathcal{R}^{o}_{o}t$. They are simply written in different languages.

terminology commission [7], which translates *bug* by *bogue*, a French word that sounds the same. The American word *bug* means small harmful insects and, more familiarly, germs and other viruses, as well as the little things that go wrong, such as errors in programs. Since the French *bogue* means the prickly husk of the chestnut, the idea of nuisance is kept. The husk also calls to mind a typographical slip. The French translation does not keep the American word's idea of a little grain of sand with dramatic consequences.

3. Finally the interpreter has the translated program *executed* by the computer. In doing so, the computer drives the *Robot*, asking it to carry out the commands in the order in which we wrote them in the original Python program.

The last two steps are done with a single command, typed in a terminal window (see the next chapter): `./robot.sh Square.py` has the program `Square.py` interpreted, that is, checked, translated and then executed. You can also drag the icon of the file `Square.py` onto that of the application `LeRobotPythonEN.app`.

Example 4 (Square) Suppose we want to draw the square opposite. The Python program will be the following:



```
from LibRobot import *

# Draw a square
fd(); fd(); qtr(); fd(); fd(); qtr()
fd(); fd(); qtr(); fd(); fd(); qtr()
st()
```

When it runs this program, the computer passes to the *Robot* the order to carry out the sequence of commands `fd()` then `fd()` then `qtr()` then `fd()` ... and finally `qtr()`, exactly as if we had typed on the keyboard F then, one after the other, F, +, F, F, +, F, F, +, F, F and finally +. The last command `st()` (from *stop*) tells the computer that it must wait until we press the escape key Esc to end the execution of the program (another key makes it draw the drawing again). This leaves us time to admire the *Robot*'s drawing at our leisure before it disappears. The drawing is not lost since we just have to run the program again to reproduce it. □

We can see from this first example that the Python language, although designed in the Netherlands by Guido van Rossum at the Cwi (the Centre for Mathematics and Computer Science in Amsterdam), uses English words such as '**from**' and '**import**': the first line of the program imports from the library `LibRobot` all its commands, which is what the star '`*`' stands for. Computer science keeps the traces of its English-speaking origins!

For the moment it is enough for us to know that the computer that runs this program asks the *Robot* to carry out the commands that come after the line '**from** `LibRobot` **import** `*`', one after the other, line after line, and from left to right on each line. Each command is followed by `()`, which means that it needs no further detail to be carried out. The semicolons that separate

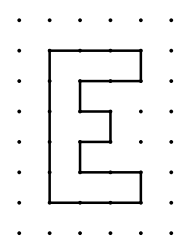
the commands on the same line can be understood as meaning 'then'. There is no semicolon at the end of a line: going on to the next line is enough to say 'then'. The line that begins with the sign '#' is a comment, which the computer ignores (we shall come back to this in the next chapter).

Exercise 8 *Run the program* `Draw` *for the interactive control of the* Robot *and give it the orders described by the following Python program (the correspondence between the keys of the keyboard and the commands of the* Robot *is given in table 2.6 page 22):*

```
from LibRobot import *
etr(); fd(); fd(); fd(); fd()
etl(); fd(); fd(); fd(); fd()
qtr(); fd(); fd(); fd(); fd()
qtr(); etr(); fd(); fd(); qtl(); fd(); fd()
etr(); fd(); fd(); fd(); fd()
qtr(); etr(); fd(); fd(); qtl(); fd(); fd()
etr(); fd(); fd(); fd(); fd()
st()
```

What drawing did you ask the Robot *to make?* □

Exercise 9 *We would like the program below to draw the capital letter* `E` *opposite:*



```
from LibRobot import *
# Draw a thick capital letter E
fd(); fd(); fd(); fd(); fd(); qtr()
fd(); fd(); fd(); qtr(); fd(); qtr()
fd(); fd(); qtr()
fd(); qtl()
fd(); qtr(); fd(); qtr(); fd(); qtl()
fd(); qtl()
fd(); fd(); qtr(); fd(); qtr()
fd(); fd()
st()
```

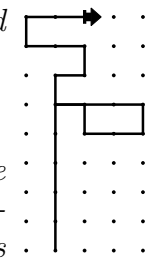
This faulty program contains two bugs. Can you find them? □

Exercise 10 *Run the Python program* `Stegosaurus` *found in the folder* `book` *that comes with this book.* □

Solution 8 *A paper bird (an origami "cocotte").* □

Solution 9 *The program* `LetterEWithBugs` *on page 30 contains two bugs:*

1. *the command* `qtr()` *on the third line should be a command* `qtl()`*,*
2. *a command* `fd()` *is missing on the last line but one.*



The result of running this program is shown opposite (with the R◎bot *in its final position). The debugged program, which correctly draws the thick letter* `E`*, is the following (the corrections are underlined):*

```
from LibRobot import *

# Draw a thick capital letter E
fd(); fd(); fd(); fd(); fd(); qtr()
fd(); fd(); fd(); qtr(); fd(); qtr()
fd(); fd(); qtl()
fd(); qtl()
fd(); qtr(); fd(); qtr(); fd(); qtl()
fd(); qtl()
fd(); fd(); qtr(); fd(); qtr()
fd(); fd(); fd()
st()
```

□

Solution 10 *The Python program* `Stegosaurus` *is stored in the file* `Stegosaurus.py` *of the folder* `book/04_writing_running`*. In a terminal window, opened in the folder* `LeRobot_Python_EN`*, just type the command*

```
./robot.sh book/04_writing_running/Stegosaurus.py
```

which has the Python program `Stegosaurus.py` *interpreted: the window of the* R◎bot *opens and the stegosaurus is drawn. You get the same result by dragging the icon of the file* `Stegosaurus.py` *onto that of the application* `LeRobotPythonEN.app`*.*

□

Addenda

We owe to Seymour Papert, the author of LOGO [31], the spreading of the idea of using drawing to learn the basic concepts of programming. However, the introduction of a grid spares us the use of angles and of lengths counted in pixels[a]. The grid makes it very easy to draw arcs of circles, which is not straightforward in LOGO.

The idea of using a grid to make geometric drawings is not new. It has long been known at school (see [41] for example). It is found, as the basis of graphics programming exercises, in many high-level computing works, as in TEX[b] for example (see [22] pages 389–390). The grid removes the problems of the discrete representation of continuous mathematical concepts on a finite screen. As a result our segments look like segments as we draw them on paper (and not like lines put side by side in a staircase), and our circles look like circles (and not like squares, as happens on graphics screens for circles that are too small). Admittedly, the use of large squares limits the possibilities of drawing. Programming is all the easier if we take advantage of geometric shapes to guide the structure of programs. Once the basic notions have been learnt, we shall be able to vary the size of the grid so that, going to the limit set by the pixels of the screen, we recover all the possibilities offered by today's high-resolution graphics screens.

In the next chapter we begin to write our first Python programs. The algorithms[c] remain very simple since they consist in codifying the movement of the hand that draws.

---

[a] The *pixel* is the smallest element of the graphics screen that can be lit (white), unlit (black) or coloured, with more or fewer brightnesses and colours depending on the quality of the screen.

[b] TEX, or rather its variant LATEX [23], is the programming language we have used to typeset this book.

[c] An *algorithm* is the description of a sequence of operations on objects to accomplish a task. This word comes from the Latin version of the name of the Persian mathematician and astronomer al-Khwarizmi of the 9th century AD. His treatise *Al-Jabr wa l-muqabala* teaches how to solve algorithmically equations of the first and second degree with numerical coefficients. The term *algebra* derives from the very title of this treatise.

4

Writing, interpreting and running programs

4.1 Structure of programs

Just as writing correct English requires following precise rules of spelling and grammar, writing correct programs is subject to a number of *syntax rules*, fortunately simpler than those of English. The difference is that English sentences containing spelling or grammar mistakes may still be understood by English speakers, whereas a program containing *syntax errors* cannot be translated into the language of the computer, and therefore cannot be run. This translation is carried out by a program (called an *interpreter*) designed to point out our errors but not to correct them. To correct our errors in a satisfactory way, the interpreter would have to understand our intentions, a very difficult problem.

The program `PaperHen` below is an example of a correct Python program.

Example 5 (Syntactically correct program)

```
from LibRobot import *
# Draw an origami paper hen facing west
# Back
etr(); fd(); fd(); fd(); fd()
etl(); fd(); fd(); fd(); fd()
# Top
qtr(); fd(); fd(); fd(); fd(); qtr(); etr()
# Forehead
fd(); fd(); qtl(); fd(); fd()
```

```
etr(); fd(); fd(); fd(); fd()
# Bottom
qtr(); etr(); fd(); fd(); qtl(); fd(); fd()
etr(); fd(); fd(); fd(); fd()
# Stopping the robot
st()
```

□

All our Python programs for driving the $\mathcal{R}\!\!\overset{\smile}{o}\!\!bot$ have the same form:

Python program:

from ␣ `LibRobot` ␣ **import** ␣ `*`
`#` Comment explaining what the program does
... Instructions of the program (one per line, or separated by ';') ...

A Python program is stored in a file whose name ends with `.py`. The name of the program is the name of this file, without the `.py`: the program `PaperHen` is stored in the file `PaperHen.py`.

This name is chosen by the programmer as he or she likes, so as to suggest what the program is for. The only rule to follow is that the name of the program must contain only unaccented lowercase or uppercase letters[1], digits or underscores '_'[2] and must begin with a letter. For example `S`, `TheStegosaurus` or `The_Stegosaurus` are correct program names whereas `1S`, `s!`, `The Stegosaurus` or `The-Stégosaurus` do not follow the rule for writing program names.

A Python program begins with the line '**from** `LibRobot` **import** `*`'. This line imports from the *library* `LibRobot` all (that is the meaning of the star '`*`') the commands of the $\mathcal{R}\!\!\overset{\smile}{o}\!\!bot$, without which the computer would not know them. It also opens the window in which the $\mathcal{R}\!\!\overset{\smile}{o}\!\!bot$ draws.

There must be a blank between the word **from** and the name `LibRobot`, as well as between `LibRobot` and the word **import**. In a book there are blanks everywhere and even, if you look closely, more blank than anything else. Putting a blank in a book to say that you must type a blank is therefore silly, because most readers are not interested in the blanks in books but simply in what is written there. We shall therefore use the symbol '␣' to

[1] Accents do not exist in English (with very rare exceptions, as in the words 'naïve', 'protégé' or 'rôle', borrowed from French).

[2] The underscore '_' must not be confused with the hyphen '-'.

write our blanks in black and white! In other words we write ␣ to say that you must press the long bar at the bottom of the computer keyboard, called the *space bar*, to obtain a blank ' ' in the program. We do not do so in the programs given as examples, which are shown as they appear on the screen. Likewise, at the end of a line you must press the return key ⏎ or return to go to the beginning of the next line. We do not write it in books because all readers know that they must go to the beginning of the next line when they reach the end of a line.

After the comment that explains what the program does come the instructions of the program. They are written in the order in which they must be executed by the computer, starting at the beginning of the line (with no blank in front: we shall see in the next chapter that blanks at the beginning of a line have a precise meaning in Python). You can write one instruction per line or else several instructions on the same line, separating them by semicolons, with no semicolon after the last one on the line. The instructions of the program can be commands of the *Robot*: `fd`, `cl`, `cr`, `qtr`, `qtl`, `etr`, `etl`, `pu`, `pd`, `dc`, `dp`, `dg` or `st`, each followed by `()`. In general, programs for driving the *Robot* end with the command `st()`, which waits until we press a key before the drawing disappears at the end of the execution of the program.

Texts that begin with the sign `#` and continue to the end of the line are *comments*, ignored by the computer.

The program `WrongPaperHen` below contains several errors that are very common among beginners.

Example 6 (Syntactically incorrect program)

```
"""Draw an origami paper hen facing west

# Back
etr(); fd(); fd(); fd(); fd()
rtl(); fd(); fd(); fd(); fd()
# Top
qtr(); fd(); fd(); fd(); fd(); qtr(); etr()
# Forehead
fd(); fd(); qtl(); fd(); fd()
etr(); fd(); fd(); fd(); fd()
# Bottom
qtr(); etr(); fd(); fd(); qtl(); fd(); fd()
etr(); fd(); fd(); fd(); fd()
# Stopping the robot
st(
```

The errors in this program are the following:

1. *the line '*`from LibRobot import *`*' has been forgotten;*
2. *the text that explains what the program does has been written between triple quotes* `"""` *(it is a* documentation string*, which can be used as a comment spanning several lines) but the closing triple quotes* `"""` *have been forgotten;*
3. *the instruction* `rtl()` *is not a command of the* $\mathcal{R}\breve{o}\breve{b}\breve{o}t$*;*
4. *the opening parenthesis of the last command* `st(` *is not closed.*

□

A complicated example of a syntactically correct Python program is given below:

Example 7 (The stegosaurus) The Python program for driving the $\mathcal{R}o\breve{b}ot$ to draw the stegosaurus shown in figure 1.2 (page 6) is the following:

```
from LibRobot import *
# Go to the tip of the tail
# Down
pu(); qtr(); qtr(); fd(); fd(); fd(); fd(); fd()
fd(); fd()
# On the right
qtl(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); qtl(); pd()
# Top of the tail
fd(); qtr(); cl(); qtl(); cl(); qtr(); qtr()
# 1st tail spike
cl(); qtr(); qtr(); cl(); qtl(); cl(); qtr()
# 2nd tail spike
cr(); cl(); fd()
# Draw the 11 bony back plates from back to front
# 1
cr(); fd(); fd(); fd(); qtr(); fd(); qtr(); fd()
fd(); fd(); fd(); qtr(); cr(); fd()
# 2
qtl(); cr(); fd(); fd(); fd(); qtr(); fd()
qtr(); fd(); fd(); qtr(); pu(); fd(); pd()
# 3
cr(); fd(); fd(); qtr(); fd(); qtr(); fd()
qtr(); pu(); fd(); pd()
# 4
cr(); qtl(); fd(); qtr(); fd(); qtr(); fd()
fd(); qtr(); fd(); fd(); qtr(); cr(); qtl()
fd()
# 5
```

```
fd(); cr(); fd(); qtr(); fd(); fd(); qtr(); fd()
fd(); qtr(); fd(); cr(); qtl()
# 6
fd(); fd(); qtr(); fd(); fd(); qtr(); fd(); fd()
qtr(); fd(); fd(); qtr(); fd(); fd(); qtl()
# 7
cr(); fd(); qtr(); fd(); fd(); qtr(); fd(); fd()
qtr(); fd(); cr(); fd()
# 8
fd(); qtl(); cr(); qtr(); fd(); fd(); qtr()
fd(); fd(); qtr(); fd(); qtr(); fd(); qtl()
cr()
# 9
cl(); qtr(); qtr(); fd(); fd(); qtr(); fd()
qtr(); fd(); qtr(); cl()
# 10
cr(); cl(); qtr(); qtr(); fd(); fd(); fd()
qtr(); fd(); fd(); qtr(); fd(); cr(); cl()
# 11
fd(); cr(); qtr(); fd(); fd(); fd(); qtr(); fd()
qtr(); fd(); fd(); cr()
# Draw the head
# Skull
fd(); fd()
# Eye
pu(); qtl(); fd(); dc(); qtr(); qtr(); fd()
qtl(); pd()
# Forehead
cl(); cr()
# Top of the snout
fd(); cl()
# Mouth
qtl(); fd(); fd(); qtr(); qtr(); fd(); fd()
qtl()
# Underside of the head
cl(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
cl(); qtr(); fd()
# Draw the front legs
# Left leg
qtl(); fd(); fd(); fd(); qtr(); qtr(); fd()
fd(); fd(); fd(); fd(); qtl(); fd(); fd(); qtl()
fd(); fd(); fd(); fd(); fd(); fd(); qtr(); qtr()
fd(); fd(); fd()
# Right leg
fd(); fd(); qtl(); fd(); qtl(); fd(); fd()
```

```
qtl(); fd(); qtr(); qtr(); fd()
# Draw the belly
cl(); qtr(); fd(); fd(); fd()
# Draw the hind legs
# Left leg
qtl(); fd(); fd(); fd(); qtr(); qtr(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); qtl(); fd()
fd(); qtl(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); qtr(); qtr(); fd(); fd(); fd(); fd()
fd(); fd()
# Right leg
fd(); qtl(); fd(); qtl(); fd(); fd(); fd()
qtl(); fd(); qtr(); qtr(); fd()
# Complete the underside of the tail
fd(); qtr(); cl(); fd(); cr(); cl(); fd(); qtr()
cl(); qtr(); st()
```

□

4.2 Comments

A Python program may contain *comments*, which are any texts written after the sign `# ...`, up to the end of the line. A comment may take up a whole line or follow the instructions of a line, as in `cl()  # ERROR: cr`. A text of several lines may also be placed between triple quotes `""" ... """`: it is a *documentation string*, which the computer reads but which gives no command to the Robot. The computer takes no account of these comments, which are only meant to explain the use and the working principle of the program to the readers of this program. If all the comments of a program are removed, it becomes unreadable for a human but nothing is changed for the computer.

Example 8 (The unreadable stegosaurus) By removing all the comments of the program `Stegosaurus` on page 36, as well as the useless blanks and line breaks, we obtain a Python program that is totally unreadable but which draws perfectly well the stegosaurus shown in figure 1.2 (page 6):

```
from LibRobot import *
pu(); qtr(); qtr(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
qtl(); fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); qtl(); pd(); fd(); qtr(); cl(); qtl()
cl(); qtr(); qtr(); cl(); qtr(); qtr(); cl(); qtl(); cl()
qtr(); cr(); cl(); fd(); cr(); fd(); fd(); fd(); qtr(); fd()
qtr(); fd(); fd(); fd(); fd(); qtr(); cr(); fd(); qtl(); cr()
```

```
fd(); fd(); fd(); qtr(); fd(); qtr(); fd(); fd(); qtr(); pu()
fd(); pd(); cr(); fd(); fd(); qtr(); fd(); qtr(); fd(); qtr()
pu(); fd(); pd(); cr(); qtl(); fd(); qtr(); fd(); qtr(); fd()
fd(); qtr(); fd(); fd(); qtr(); cr(); qtl(); fd(); fd(); cr()
fd(); qtr(); fd(); fd(); qtr(); fd(); fd(); qtr(); fd(); cr()
qtl(); fd(); fd(); qtr(); fd(); fd(); qtr(); fd(); fd()
qtr(); fd(); fd(); qtr(); fd(); fd(); qtl(); cr(); fd()
qtr(); fd(); fd(); qtr(); fd(); fd(); qtr(); fd(); cr(); fd()
fd(); qtl(); cr(); qtr(); fd(); fd(); qtr(); fd(); fd()
qtr(); fd(); qtr(); fd(); qtl(); cr(); cl(); qtr(); qtr()
fd(); fd(); qtr(); fd(); qtr(); fd(); qtr(); cl(); cr(); cl()
qtr(); qtr(); fd(); fd(); fd(); qtr(); fd(); fd(); qtr()
fd(); cr(); cl(); fd(); cr(); qtr(); fd(); fd(); fd(); qtr()
fd(); qtr(); fd(); fd(); cr(); fd(); fd(); pu(); qtl(); fd()
dc(); qtr(); qtr(); fd(); qtl(); pd(); cl(); cr(); fd(); cl()
qtl(); fd(); fd(); qtr(); qtr(); fd(); fd(); qtl(); cl()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); cl(); qtr(); fd()
qtl(); fd(); fd(); fd(); qtr(); qtr(); fd(); fd(); fd(); fd()
fd(); qtl(); fd(); fd(); qtl(); fd(); fd(); fd(); fd(); fd()
fd(); qtr(); qtr(); fd(); fd(); fd(); fd(); fd(); qtl(); fd()
qtl(); fd(); fd(); qtl(); fd(); qtr(); qtr(); fd(); cl()
qtr(); fd(); fd(); fd(); qtl(); fd(); fd(); fd(); qtr()
qtr(); fd(); fd(); fd(); fd(); fd(); fd(); fd(); qtl(); fd()
fd(); qtl(); fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
qtr(); qtr(); fd(); fd(); fd(); fd(); fd(); fd(); fd(); qtl()
fd(); qtl(); fd(); fd(); fd(); qtl(); fd(); qtr(); qtr()
fd(); fd(); qtr(); cl(); fd(); cr(); cl(); fd(); qtr(); cl()
qtr(); st()
```

This example also shows that the layout of the instructions of the program is important. Spacing between instructions and proper alignment make programs much easier to read. □

4.3 Interpretation of programs

The *interpretation* of a Python program consists in translating it into the language of the computer, called *machine language*, and having it executed by the computer. It is done by a program called the Python *interpreter*[3]. The interpreter works in two stages:

[3] The command `./robot.sh PaperHen.py`, typed in a terminal window opened in the folder `LeRobot_Python_EN`, calls the interpreter `python3` to run the program `PaperHen.py`. Dragging the icon of `PaperHen.py` onto that of the application `LeRobotPythonEN.app` does the same thing.

- it first reads the *whole* program to check that it follows the syntax rules of Python and to translate it; if it finds a syntax error, it reports it and stops without running anything;
- then, if the program is syntactically correct, it has the computer execute its instructions, one after the other.

Some errors can be discovered only during this second stage, when the computer reaches the faulty instruction, for example the call of a command that does not exist. Execution then stops, but the $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{o}t$ has already made all the drawing requested by the preceding instructions. Obviously the Python program and its translation drive the $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{o}t$ in the same way. This way of proceeding saves us from having to learn machine language. It is very difficult to understand this machine language because all programs are written only with 0s and 1s!

4.4 Syntax errors

To be able to run a Python program, it must not contain any syntax errors violating the rules for writing programs. In case of an error the interpreter explains briefly, in English, the cause of the error and points out the place in the program where the error was found (the line number, for example `line 15`, then the line itself and, underneath, one or more carets `^` under the faulty text). The message begins with the name of the error, for example `SyntaxError`. You must then correct the error and run the program again.

An error discovered during execution is reported in the same way, but the message begins with `Traceback (most recent call last):` and ends with the name of the error, for example `NameError`.

Example 9 (Syntactically incorrect program) The program `WrongPaperHen` on page 35 contains several of the most frequent errors for such a simple program. When we run this program `WrongPaperHen`, the interpreter starts by reporting '`SyntaxError: unterminated triple-quoted string literal (detected at line 15)`', pointing at the `"""` of the first line: the closing triple quotes were forgotten at the end of the text that explains what the program does, so that the whole rest of the program, up to the last line 15, is part of this text! Having added the forgotten `"""` at the end of the first line, a new attempt reports '`SyntaxError: '(' was never closed`' pointing at the parenthesis of `st(` on line 15. We close it by writing

`st()`. The program is now syntactically correct and its execution can begin. It stops at once, on the first instruction, with the message '`NameError: name 'etr' is not defined. Did you mean: 'str'?`' (the suggestion `str`, a name known to Python, is of no help here). Since the line '**from** `LibRobot` **import** `*`' was forgotten, the interpreter does not know the meaning of the commands of the $\mathcal{R}\breve{o}\breve{b}\breve{o}t$, and the window of the $\mathcal{R}o\underset{\smile}{b}ot$ has not even opened. After adding this line at the beginning of the program, we can run it again. The window opens, the $\mathcal{R}o\underset{\smile}{b}ot$ draws the first line of the back of the paper hen, then everything stops with the message '`NameError: name 'rtl' is not defined. Did you mean: 'etl'?`', which points at the line where `rtl()` is. It is a typing mistake, which we correct by replacing `rtl` by `etl`. We thus laboriously obtain the program `PaperHen` on page 33, which is correct (apart from the comment, written with `#` in `PaperHen`). The interpreter stops at the first error it finds: you must therefore run the program again after each correction. Note that the two syntax errors were reported before any execution, whereas the two unknown names were discovered only during execution, when the computer reached the faulty instruction. □

4.5 Execution of programs

The *execution* of a program consists in having the computer carry out the actions defined by the instructions of the program. Since in our case the instructions are commands of the $\mathcal{R}o\underset{\smile}{b}ot$, executing the program makes it possible to drive the $\mathcal{R}o\underset{\smile}{b}ot$.

When the $\mathcal{R}o\underset{\smile}{b}ot$ has finished its drawing, the command `st()` displays at the bottom of the window the keys that let you continue:

- decrease (by pressing the minus key [-] once or several times) or increase (by pressing the plus key [+] once or several times) the speed of the $\mathcal{R}o\underset{\smile}{b}ot$ (which can vary from 0 to 10);
- have the $\mathcal{R}o\underset{\smile}{b}ot$ redraw the whole drawing, at the chosen speed, by pressing another key, for example the return key [↩] or [return]; when the $\mathcal{R}o\underset{\smile}{b}ot$ draws too fast for you to watch the drawing unfold, just decrease its speed and then have it redraw the drawing;
- stop the execution of the program for good by pressing the escape key [Esc] (or one of the keys [§] or [#]), which gives the screen of figure 4.2 (page 43).

During the execution of a program driving the *Robot*, we can also stop the drawing of the *Robot* at any moment by pressing the escape key Esc (or § or #). The drawing then stays displayed as it was at that instant, until we press a key. For example if we interrupt the execution of the program `Stegosaurus` on page 36 when the *Robot* finishes drawing the left front leg, we obtain a screen similar to the one in figure 4.1 on page 43.

4.6 Logical errors

Having eliminated all the syntax errors, that is, the programming errors that come from violating the grammar rules of the Python language, our troubles are not yet over before we can admire the drawing! There remain the *logical errors*, which we shall discover when running the program.

The first possible error is that the sequence of commands that our program asks the computer to give to the *Robot* leads it outside the frame. This is the case of the following program:

```
from LibRobot import *
# Program leading to a run-time error because it makes
# the robot leave the frame.
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd()
```

This program (`LeaveTheFrame`) gives the sad result below, accompanied in the terminal window by the message '`Error line 8 of LeaveTheFrame.py: the robot would leave the frame (fd)`':

The second possible error is that we made a mistake in the program and that the drawing obtained does not look at all like what we had imagined. For example, the program `WrongStegosaurus` below is a logically wrong but

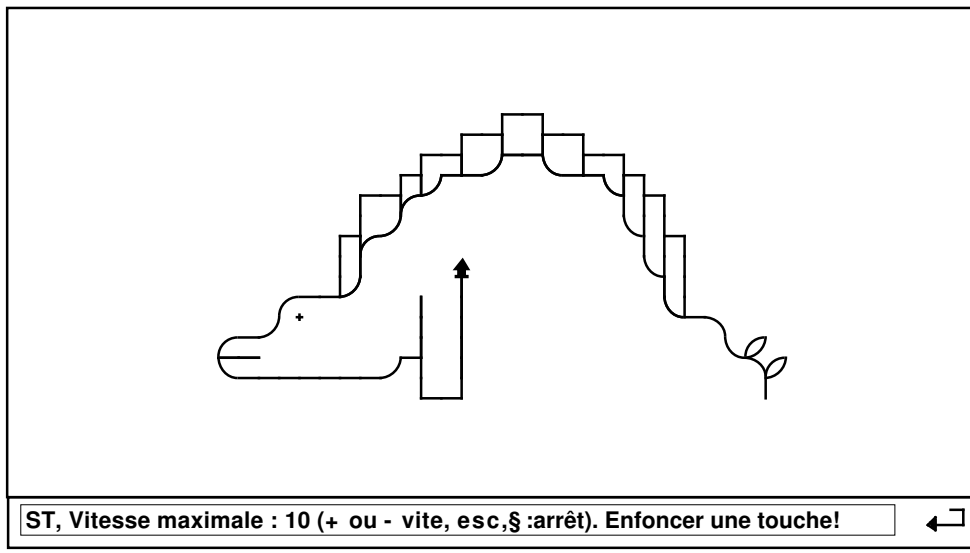


FIG. 4.1 – *Computer screen after a stop*

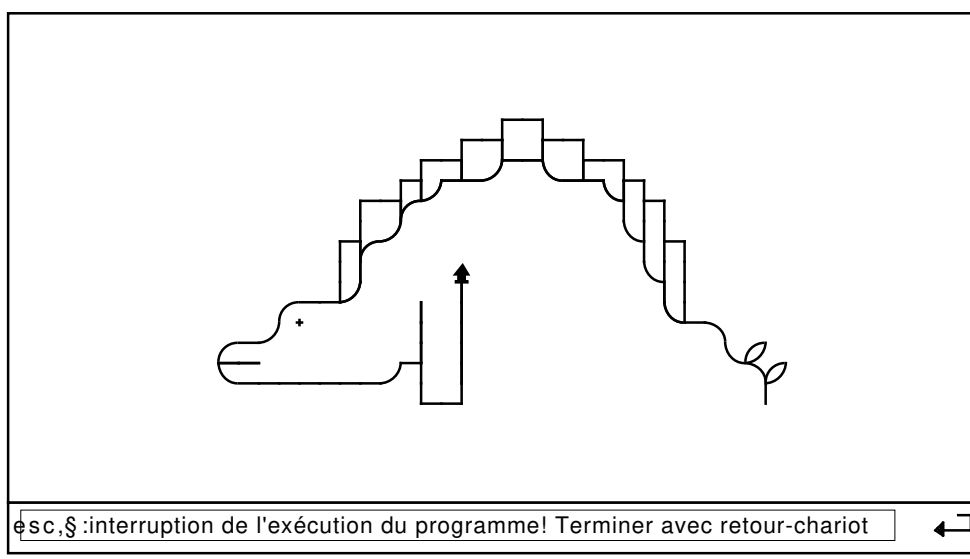


FIG. 4.2 – *Computer screen before the final stop*

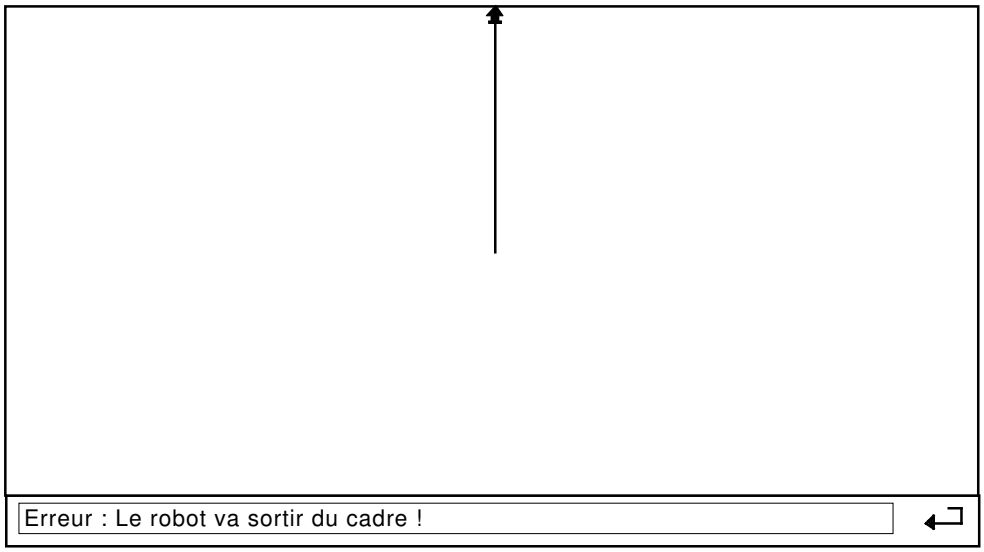


FIG. 4.3 – *Computer screen when the robot leaves the frame*

syntactically correct version of the program `Stegosaurus` on page 36.

```
from LibRobot import *

...
# 10
cl()  # ERROR: cr
cl(); qtr(); qtr(); fd(); fd(); fd()
qtr(); fd(); fd(); qtr(); fd(); cr(); cl()
...
# Draw the belly
cl(); qtl()  # ERROR: qtr
fd(); fd(); fd()
...
st()
```

The author of this program `WrongStegosaurus` twice mixed up his left and his right. The result of running this program, as shown below (figure 4.4), would surprise more than one palaeontologist!

A third error, typical of Python beginners, is to forget the parentheses `()` after a command of the $\mathcal{R}^{o}_{b}{}^{o}t$, for example '`fd; qtr()`' instead of '`fd(); qtr()`'. The program is syntactically correct but the $\mathcal{R}^{o}_{b}{}^{o}t$ does not move forward, because `fd` without `()` names the command without asking for it to be executed. The Python interpreter reports nothing at all: no error,

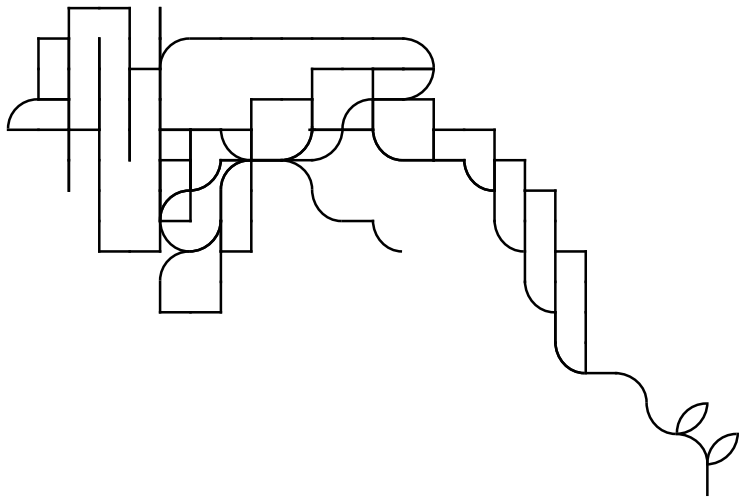


FIG. 4.4 – *A rather botched stegosaurus*

not even a *warning*. Only the drawing, which is missing a line, reveals the error. So you must always check that every command is indeed followed by its parentheses.

4.7 How to locate logical errors in the program

To find the logical errors in the program you have to imagine that you are executing the program in place of the $\mathcal{R}\breve{o}\breve{b}\breve{o}t$. When the program is very long this is not easy. The best way is therefore to follow the execution of the program in *step-by-step* mode, which lets you follow the commands transmitted by the computer to the $\mathcal{R}\breve{o}\breve{b}\breve{o}t$ so as to locate the error precisely. When the error is located, you must stop the execution of the program in order to correct it in the program. Once this is done, you can run the program again for a new attempt.

- To switch to step-by-step mode we can:
  - at the end of the drawing, reduce the speed to zero by pressing the key [–] several times, then have the drawing redrawn by pressing the return key [↩] or [return];

- or insert an instruction `ss()` in the Python program, a little before the place where the error is suspected. This instruction `ss()` (single step) switches the computer to step-by-step execution mode.

- In step-by-step mode, the computer waits for us to give it the order to execute each command of the *Robot* by pressing a key, for example the return key ↩ or return. We can thus follow the execution of the program instruction after instruction, which in general makes it fairly easy to find the error.
- When the error has been precisely located in the program, you can stop the execution of the program by pressing the escape key Esc (or § or #).
  It is also possible to program the return to a normal execution speed by inserting an instruction `ex()` in the program.

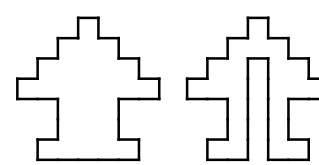
It almost always takes several attempts before the program is logically correct. When it is, one is often tempted to improve it. The hardest thing is then to know when to stop!

`st()`	stops the execution of the program at the end of the drawing. The speed can be decreased with − (minimum 0) or increased with + (maximum 10). Any other key has the drawing redrawn at the indicated speed. Execution stops for good when you press esc or § or #.
`ss()`	executes the rest of the program in step-by-step mode (single step, speed 0): each command of the *Robot* is executed when you press a key.
`ex()`	executes the rest of the program in normal execution mode (speed 10).

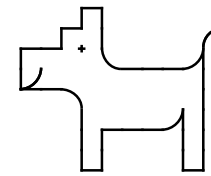
TAB. 4.1 – *Commands controlling the driving mode of the Robot*

Exercise 11

– *Write, debug and run Python programs drawing the Robot as it is stylized opposite, with its pen down and with its pen up.*



– *Write, debug and run a Python program that drives the Robot to make it draw the dog shown opposite.*



– *Program the drawings on page 9.*

– *Program the drawing of your initials in capital letters in the style of the letter* `H` *in relief shown in figure 1.12 (page 9).*

□

Solution 11 *We give one program (*`Dog`*), the others being in the folder* `book/04_writing_running`*.*

```
from LibRobot import *
# Rear outline of the front leg
fd(); fd(); qtr()
# Belly
fd(); fd(); fd(); cl()
# Hind leg
qtr(); qtr(); fd(); fd(); fd(); qtl(); fd()
qtl(); fd(); fd(); fd(); fd()
# Tail
fd(); fd(); cr(); qtr(); qtr(); cl(); cr()
# Back
fd(); fd(); fd(); cr()
# Head
# Ear
fd(); fd(); qtl(); fd(); qtl(); fd()
# Eye
pu(); fd(); dc(); qtr(); qtr(); fd(); pd()
qtl(); fd(); qtl(); fd()
# Muzzle
qtr(); fd(); fd(); qtl(); fd(); fd(); qtl()
# Mouth
cl(); qtr(); qtr(); cr(); qtr(); qtr(); fd()
fd(); cr()
# Front outline of the front leg
fd(); fd(); fd(); qtl(); fd(); st()
```

□

Addenda

The cycle of writing, syntactic debugging, translation, logical debugging and execution of programs has become considerably shorter in recent years thanks to the fantastic progress in computer architecture, which should be understood very schematically. A microcomputer comprises:

- an arithmetic and logic unit to do calculations and handle data,
- a memory to keep data and programs temporarily (its content is lost when the power is switched off),

- one or more magnetic or solid-state (SSD) hard disks to keep files containing data or programs permanently,
- (USB) ports for plugging in memory sticks, which make it possible to keep files containing data or programs permanently, or to exchange them with other computers,
- a keyboard to enter data or programs into memory,
- a graphics screen to display texts and images,
- a mouse to point at places on the graphics screen,
- a bus to allow fast exchanges of data and programs between these various components.

When the computer is switched on, it is wired to load into its memory and execute a program stored permanently on disk, called the *operating system*. The essential task of this program is to manage all the components of the computer and to dialogue with the user by means of a *command interpreter*, which waits for the user to give it orders through the keyboard and executes them immediately, one after the other, until the user gives it the order to switch off the computer. Among the commands of the interpreter are, for example, deleting a file stored on disk or launching the execution of a program written in machine language (generally called an *application program*) such as, for example, a *text editor* to create a text and store it in a file, or an *interpreter* to translate a Python program into machine language and have it executed.

The first interfaces between man and machine were purely textual. You typed on the keyboard and saw the result on a printer, soon replaced by a screen with only lines of text at most 80 characters long. Black-and-white screens were replaced by colour screens (as can still be seen in the `terminal` application). Then came graphics screens made up of very small coloured pixels. Windowing gives the illusion of several graphics screens on a single one. Then came the idea of icons to represent files and applications, on which you can double-click to launch an application. For example, a double click on the icon of `LeRobotPythonEN.app` launches the interactive drawing program, whereas dragging the icon of a program of the $\mathcal{R}^{o}\underline{b}^{o}t$ onto this icon of the application `LeRobotPythonEN.app` runs this program, making the drawing window and the terminal appear. Note that you must click on one or the other of the windows to use it. You can stop the program with Control+C or ctrl+C (which computer scientists write ^C) after selecting the terminal window, or by closing this window, or else with the escape key (`esc`) in the graphics window.

Computers are *binary* in the sense that all data, in particular programs, are represented by sequences of 0s and 1s. The computer works by transferring an instruction of the program from a cell of its memory into its arithmetic and logic unit (via the bus) and executing it (which in general modifies the content of the memory, of the disk, of the screen, etc.), then starting again in the same way with the instruction stored in the next cell of its memory. Some instructions, called *branch instructions*, make it possible to indicate that the next instruction is stored in the memory cell with a given number.

To execute a program written in machine language and stored on disk, the operating system asks the arithmetic and logic unit to load this program into memory, that is, to copy the program into a part of the memory that was unoccupied, and then to execute a branch instruction to the first instruction of this program. The program must end with a branch instruction back to the operating system. Modern computers also have the ability to interrupt the execution of the current program and then to resume it later as if nothing had happened. In the meantime, it is the operating system that runs, to react according to the cause of the interruption. For example if the user has moved the mouse, its new coordinates are stored in memory. If the clock ticks, which happens every sixtieth of a second for example, the time is updated, etc. In particular the user has buttons or keys (Esc, §, # for our $\mathcal{R}^{o}\underset{\smile}{b}^{o}t$) to interrupt the execution of the program if he or she notices an error.

Among the applications that the operating system can run, there is one that particularly interests us: the Python interpreter.

Modern programming environments (such as `IDLE`, supplied with Python) take care of editing the text of the program, that is, of creating and modifying it by interaction with the user. The Python program is displayed on the screen and stored in memory as the user types it on the keyboard. The user can modify it using various function keys and the mouse. The user is also given the possibility of saving the text on the hard disk in a file whose name must be given.

When text editing is finished, the environment can have the program translated into an equivalent program written in machine language. For so-called *compiled* languages, such as OCaml or C, this translation is done once and for all by a *compiler*: the compiled program is stored on disk, then the user goes back to the operating system level to load into memory and execute the compiled program as explained above.

During the debugging phase of the program, it is faster to store the translated program directly in memory, the translator itself taking care of its execution and then taking back control when this execution is finished or interrupted. This is how the Python interpreter proceeds, every time it is asked to run a program.

One can see the considerable progress that has been made since the first computers, for which the wiring had to be modified to change the program [25]:

- since the program is stored in memory (following the idea of Von Neumann[a]) and the memory is connected to fast disks, it is possible to change the program of the machine almost instantly;
- the use of high-level languages such as Python or OCaml and of efficient interpreters or compilers makes it possible to avoid having to program in a thankless machine language;
- the alphanumeric screen lets the user edit his or her program interactively at the keyboard, which saves a considerable amount of time compared with handling the cardboard punched cards still used in the 1970s;
- computing power, linked to the density of transistors on computer chips (which ranges from a few billion to several hundred billion transistors in 2016), makes it possible to manage graphics screens directly connected to the memory, which allows very fast changes and hence the use of windows, mice, etc.;
- the size of the memory (a few billion characters, GB, gigabytes) and of the disks (a few thousand billion characters, TB, terabytes) makes it possible to consider ever larger programs and data, and therefore ever more sophisticated applications.

[a] John von Neumann, 1903–1957, American mathematician.

The size of the largest programs that can be written today now runs to tens or hundreds of millions of lines, which is considerable and not without difficulties, since the limits of human understanding are being reached. A training of the mind, a new rigour, even more precise than the logic of mathematicians, becomes the prime quality which, together with imagination and a sense of aesthetics, makes good computer scientists.

A little method also helps a lot. The whole history of programming languages is full of inventions making it possible to formulate more clearly how to organize data in memory or the computations of the computer. Each time, human capacities, and therefore the maximum size of the largest programs, have been pushed back. A very old idea for structuring computations, due to Von Neumann and Turing[a], is that of the *procedure*, formerly called a *routine* or *subprogram*. We begin to introduce it in the next chapter.

[a] Alan Turing, 1912–1954, English mathematician.

5

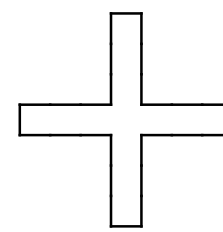
Procedures

5.1 Copying text

In the program `GreekCross_1` below we find the same sequence of commands four times, to draw the four identical arms of the cross and turn. Each arm breaks down into two equal sides of length 3 and an end of length 1 (the unit being the length of a side of a square of the grid).

```
from LibRobot import *

# Draw a Greek cross
fd(); fd(); fd(); qtl(); fd(); qtl()
fd(); fd(); fd(); qtr()
fd(); fd(); fd(); qtl(); fd(); qtl()
fd(); fd(); fd(); qtr()
fd(); fd(); fd(); qtl(); fd(); qtl()
fd(); fd(); fd(); qtr()
fd(); fd(); fd(); qtl(); fd(); qtl()
fd(); fd(); fd(); qtr()
st()
```



This situation, where the same sequence of commands of the *Robot* is found several times in a row in a program, is very frequent because geometric drawings are often made up of identical pieces. It is then very tedious to type the same text several times. This can be avoided by using the facilities offered by the computer for copying a text already recorded in its memory and pasting it elsewhere in the program. How to do this is explained for PCs (under Windows or Linux) in figure 5.1 (page 54) and for the Macintosh in figure 5.2 (page 55).

Selecting and copying a text on a PC (Windows or Linux)

1. Click with the mouse at the beginning of the text to be copied to place the blinking edit cursor there.
2. Hold the shift key ⇧ down and click at the end of the text to be copied, so that the text to be copied appears *highlighted* (you can also drag the mouse from the beginning to the end of the text while holding its button down).
3. Hold the control key Ctrl down while pressing the key C to *copy* the selected text.
4. Click with the mouse at the place where the text is to be copied.
5. Hold the control key Ctrl down while pressing the key V to *paste* the copied text.

The copying operation (Ctrl V) at any place in the program can be repeated as many times as necessary. In case of error, Ctrl Z undoes the last change.

FIG. 5.1 – *Copy and paste on a PC (Windows or Linux)*

Selecting and copying a text on the Macintosh

1. Place the blinking edit cursor at the beginning of the text to be copied by moving it with the arrow keys (↑, ↓, ←, →) or by clicking with the mouse at the desired place.
2. Hold the shift key ⇧ or shift down, without releasing it.
3. Place the blinking edit cursor at the end of the text to be copied by moving it with the arrow keys (↑, ↓, ←, →) or by clicking with the mouse at the desired place, so that the text to be copied is shown in *reverse video*, that is, in white on a black background (if by mistake you type text on the keyboard, it replaces the text selected in reverse video; the delete key ← removes the text selected in reverse video; in both cases, choose the `Undo` item of the `Edit` menu to restore the original).
4. Release the shift key ⇧ or shift.
5. Select the `Copy` item of the `Edit` menu (or, equivalently, hold the command key ⌘ down and press the key C).
6. Place the blinking edit cursor at the place where the text is to be copied by moving it with the arrow keys (↑, ↓, ←, →) or by clicking at the desired place with the mouse.
7. Select the `Paste` item of the `Edit` menu (or, equivalently, hold the command key ⌘ down and press the key V).

The copying operation (`Paste`) at any place in the program can be repeated as many times as necessary.

FIG. 5.2 – *Copy and paste on the Macintosh*

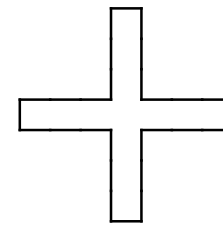
Making many copies by hand takes a long time. Moreover, one can easily imagine that in a program several pages long, the reader has trouble finding the identical parts of text. This phenomenon is illustrated in the program `Number888_1` below. The difficulty would obviously be even greater if the identical parts of text were on different pages.

```
from LibRobot import *
# Draw the number 888 in the style
# of digital watches.
# Draw the hundreds 8
fd(); qtr(); fd(); qtl(); fd(); qtl(); fd()
qtl(); fd(); qtl(); fd(); qtr(); fd(); qtr()
fd()
# Leave a space and turn the robot back to the north
pu(); qtr(); qtr(); fd(); fd(); qtl(); pd()
# Draw the tens 8
fd(); qtr(); fd(); qtl(); fd(); qtl(); fd()
qtl(); fd(); qtl(); fd(); qtr(); fd(); qtr()
fd()
# Leave a space and turn the robot back to the north
pu(); qtr(); qtr(); fd(); fd(); qtl(); pd()
# Draw the units 8
fd(); qtr(); fd(); qtl(); fd(); qtl(); fd()
qtl(); fd(); qtl(); fd(); qtr(); fd(); qtr()
fd()
st()
```



5.2 Examples of procedures

Rather than copying texts by hand ourselves, we can ask the Python interpreter to do it for us by using *procedures*. Let us begin by rewriting the program `GreekCross_1` (page 53) using procedures to bring out the decomposition of the Greek cross into four identical arms, each of them breaking down into two equal sides of length 3 and an end of length 1.



```
from LibRobot import *

# Draw a Greek cross
def side():
   fd(); fd(); fd()

def end():
   qtl(); fd(); qtl()

# Draw the left side, the end and then the right
# side of an arm, and turn the robot towards
# the next arm.
def arm():
   side(); end(); side(); qtr()

def cross():
   arm(); arm(); arm(); arm()

cross(); st()
```

Using a procedure involves:

- the *procedure declaration*, which defines a sequence of instructions (called the *body of the procedure*) and gives it a name (called the *name of the procedure*);
- the *procedure call*, which indicates that the sequence of instructions making up the body of the procedure must be copied in place of the name of the procedure.

For the reader of the program, using procedures is preferable to copying texts. Each time he or she sees the name of a procedure, the reader understands that it stands for a copy of a text that can be looked up in the procedure declaration. On meeting several calls of the same procedure, the reader need make no effort to understand immediately that it is the same text that is copied. The program is therefore clearer. Isn't that the case for the program `Number888_2` below, which is easier to understand than the program `Number888_1`?

```
from LibRobot import *

# Draw the number 888 as on
# digital watches.

# Draw the digit 8, starting at the bottom left of the
# 8 with the robot facing north and ending at the bottom
# left of the 8 with the robot facing west.
def eight():
   fd(); qtr(); fd(); qtl(); fd(); qtl(); fd()
```



```
    qtl(); fd(); qtl(); fd(); qtr(); fd(); qtr()
    fd()
# Starting with the robot facing west at the bottom
# left of the 8, back up to the bottom right of the 8 then
# one space and turn the robot north again.
def space():
    pu(); qtr(); qtr(); fd(); fd(); qtl(); pd()
eight(); space()  # the hundreds 8
eight(); space()  # the tens 8
eight()  # the units 8
st()
```

5.3 Declaring procedures

The declarations of the procedures of a program are placed between the line '**from** `LibRobot` **import** `*`' and the instructions of the program. The structure of a Python program is therefore the following:

Python program:

from ␣ `LibRobot` ␣ **import** ␣ `*`
`#` Comment explaining what the program does
... Declarations of the procedures of the program ...
... Instructions of the program[a] (not indented) ...

[a] Including the calls of the procedures of the program.

In Python, a procedure declaration has the following form (we shall use more elaborate forms later on):

Procedure declaration:

`#` Specification: what the procedure does
def ␣ `procedureName():`
    ... Body of the procedure[a] (**necessarily indented**) ...

[a] Including the calls of previously declared procedures.

The English word **def** is short for *define*. The `procedureName` is chosen by the programmer as he or she likes, so as to suggest what this procedure is for.

The parentheses `()` that follow it indicate that it is a procedure, which will have to be called by writing `procedureName()`, exactly like the commands of the *Robot* `fd()`, `qtr()`, etc. Indeed, the commands of the *Robot* are procedures declared in the library `LibRobot`. The **def** line must end with a colon '`:`', which announces the body of the procedure.

The instructions of the body of the procedure are written on the following lines, all shifted to the right by the same number of blanks, usually 4. They are said to be *indented*. As in the program, one or more instructions can be written per line, separated by semicolons. It is the indentation, and the indentation alone, that tells Python which instructions are part of the body of the procedure: the body ends at the first line that is no longer indented. Blanks at the beginning of a line are therefore not simply a matter of layout: they are part of the syntax of Python. If you forget to indent the first line of the body, Python reports for example '`IndentationError: expected an indented block after function definition on line 3`'. If on the contrary you indent an instruction of the program by mistake, it reports '`IndentationError: unexpected indent`'. And if a line of the body does not have the same number of blanks as the previous ones, it reports '`IndentationError: unindent does not match any outer indentation level`'. An empty line after the body of each procedure, as in the program `Number888_2` above, is not compulsory but makes the end of the procedure clearly visible.

Warning: the body of a procedure **must** be indented, and **all** its lines by the same number of blanks. A line of the body that you forget to indent is not always reported as an error: it is simply no longer part of the procedure. In the example below, `qtr()` is not indented: Python executes it only once, at the beginning of the program, and not at each call of `corner()`. Instead of a square, the *Robot* draws a straight line eight steps long, without any error message:

```
def corner():
   fd(); fd()
qtr()            # Error: not indented, outside the procedure
corner(); corner(); corner(); corner()
```

For the *Robot* to turn at each corner, `qtr()` must be indented like `fd(); fd()`, with the same four blanks.

Remark: in Python, procedures are special cases of what are called *functions*. We shall come back to them later.

5.4 Identifiers

The name of a procedure is called an *identifier* in Python. Here is a precise definition, to which you should refer every time we speak of identifiers:

Identifier:

An *identifier* is a sequence of unaccented lowercase or uppercase letters, digits or underscores '_', which does not begin with a digit and which is different from the English words reserved by Python such as **from**, **import**, **def**, **for**, **in**, **if**, **while**, **return**, **True**, etc.

For example `side`, `greekCross`, `greek_cross` or `side2` are identifiers whereas `2side`, `left-side` or `for` are not. If you write '**def** `2side():`', Python answers '`SyntaxError: invalid decimal literal`' because it thinks it is reading a number; for '**def** `left-side():`' or '**def** `for():`', it simply answers '`SyntaxError: invalid syntax`'. Uppercase and lowercase letters are distinguished: `Side` and `side` are two different identifiers. If you declare '**def** `Side():`' and then call `side()`, Python answers '`NameError: name 'side' is not defined. Did you mean: 'Side'?`'. Python allows an identifier to begin with an uppercase letter, but the custom is to begin the names of procedures with a lowercase letter. To make an identifier made up of several words readable, we put an uppercase letter at the beginning of each word except the first, as in `saintAndrewsCross` or `innerEdgeOfLens`[1].

5.5 Calling procedures

To call a procedure, you just write its name followed by `()`. The call `procedureName()` is then replaced by the body of the procedure, that is, by the list of indented instructions that follows the line '**def** `procedureName():`' in its declaration.

Note that if you declare a procedure without ever calling it, it is of absolutely no use.

[1] Many Python programmers prefer to separate the words with underscores, as in `saint_andrews_cross`.

A procedure can be called in the program or in another procedure. For example in the program `GreekCross_2`, we call the procedure `cross`, which calls the procedure `arm`, which itself calls the procedures `side` and `end`.

The rule that a procedure must be declared before being called must absolutely be followed. Suppose for example that we have declared a procedure under the name `procName`. Suppose also that, because of a typing mistake, we have called this procedure under the name `progName`. When execution reaches this call, the interpreter stops, reporting the error '`NameError: name 'progName' is not defined`'. Not knowing that we wrote `progName` by mistake instead of `procName`, the interpreter assumes that the procedure `procName` has been declared but has never been used. This is useless but allowed. It also notes that the procedure `progName` is called but has not been declared. This is forbidden. It notices, however, that the two names look alike and suggests '`Did you mean: 'procName'?`'.[2] We get the same '`NameError`' error if the program calls a procedure whose declaration is placed further on: Python executes the declarations in the order in which they are written, and at the time of the call it does not yet know the procedure. Like all errors discovered during execution, these errors appear only once the *Robot* has made the part of the drawing that precedes the faulty call.

5.6 Robot commands defined by procedures

Some commands of the *Robot* are equivalent to a sequence of elementary commands and can therefore be expressed with procedures. For example, to turn a quarter turn, it is enough to turn an eighth of a turn twice in a row.

[2] We often speak of programs in an anthropomorphic way, that is, by attributing to them characteristics proper to human beings. In fact we are thinking of the programmers who wrote them. For example, the computer does not make mistakes; it is its manufacturer who made a mistake, or the programmer who made a logical error in his or her program.

We therefore have:

```
# Turns the robot on the spot a quarter turn
#   to the right
def qtr():
   etr(); etr()
# Turns the robot on the spot a quarter turn
#   to the left
def qtl():
   etl(); etl()
```

Here are new commands of the *Robot* that can be expressed by means of procedures using commands we already know:

```
# Turns the robot on the spot 3 eighths of a
#   turn to the right
def tetr():
   qtr(); etr()
# Turns the robot on the spot 3 eighths of a
#   turn to the left
def tetl():
   qtl(); etl()
# Turns the robot on the spot a half turn
def ht():
   qtr(); qtr()
```

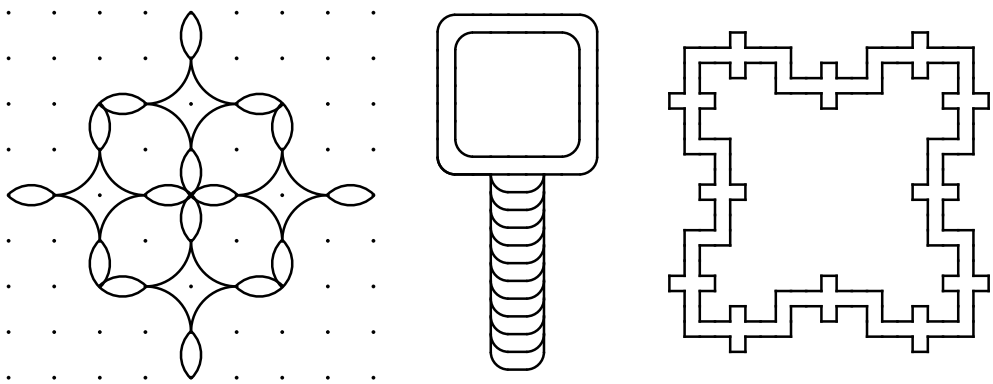


FIG. 5.4 – *Astroid star, magnifying glass and notched double square*

Exercise 12 *Write Python programs with procedures to draw the crosses, the astroid star, the magnifying glass or the notched double square below.* □

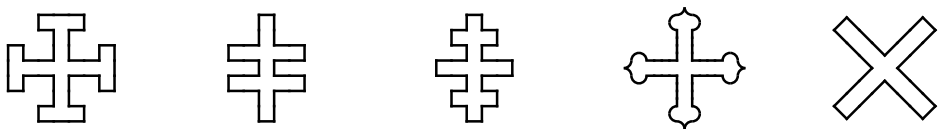


FIG. 5.3 – *Cross potent, double cross, papal cross, cross botonny and Saint Andrew's cross*

Solution 12 *We give one example of a program (*`AstroidStar`*), the others being solved in the folder* `book/05_procedures`*.*

```
from LibRobot import *

# Draw a star made of four looped
# astroids
def arc():
   etl(); cr(); etl(); cr(); etl(); cr()

def astroid():
   arc(); tetr(); arc(); tetr(); arc(); tetr()
   arc()

def star():
   astroid(); etr(); astroid(); etr()
   astroid(); etr(); astroid()

star(); st()
```

□

Addenda

The notion of procedure introduced by Von Neumann and Turing was used in 1949 under the name of *routine* on the EDSAC (*Electronic Delay Storage Automatic Computer*), generally considered the first stored-program computer to have worked. This machine was built from 1947 at the University of Cambridge in England by the team of Wilkes[a] and ran its first programs in May 1949 [25].

Procedures illustrate the concept of *abstraction*: the properties of an object are isolated so as to use them or study them separately. More precisely, a procedure gives a name to an algorithm for constructing part of a drawing. In the body of the procedure, we are interested in how to construct the drawing. In the calls we are interested only in the finished drawing, while wishing to ignore completely how the drawing is obtained. We thus separate the object from the way of constructing it, the what from the how.

[a] Maurice Wilkes, one of the inventors of radar.

We find it useful to describe the drawing made by a procedure in a comment placed at the beginning of the procedure, called the *specification* of the procedure. Note that for a procedure to be usable without ever looking at its body, the specification of the procedure must describe which drawing it makes but also what the positions and orientations of the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ are before and after the drawing is made. This brings out a notion of *invariant*, to which we shall return in the following chapters on iteration.

Procedures are the basis of the methods for structuring very large programs: to be understandable, programs of several thousand lines must be split into procedures no longer than a few pages. Procedures are therefore an elegant means of dividing a program up logically. However, the concept of procedure is not sufficient to structure programs of several million lines. One then uses the notion of *module* (also called *unit*) and of *library* to group together sets of procedures.

Procedures grouped together in a *library* (in Python also called a *module*) can be supplied to programmers to help them in their tasks. To use a procedure, the programmer must know its name and its specification, without needing to know how it works internally. The line '**from** `LibRobot` **import** $*$' at the beginning of the program is there to tell the interpreter which library is used in this program. The commands of the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ are procedures grouped together in libraries.

Analysing geometric drawings to bring out a structure that can be described algorithmically by means of procedures is very good training for the mind. In this sense learning to program can play for children a formative role similar to the one once played by Euclidean geometry.

6

Robot jumps to the edge and to the centre of the frame

Our $\mathcal{R}^{o}_{b}{}^{o}t$ can jump to the edge or to the centre of the frame (which delimits the drawing area on the computer screen), without changing its orientation and without drawing anything. It is exactly as if we could carry it by hand and put it down at the edge or at the centre of the frame, as one can do with a remote-controlled car. The computer does it for us thanks to the commands given in the table below and illustrated in figure 6.1 (page 69) (the underlined letters give the abbreviation of the names of the commands):

`gl()`	(go to the left) places the $\mathcal{R}^{o}_{b}{}^{o}t$ on the left of the screen, without changing its vertical position or its orientation, and without drawing anything.
`gr()`	(go to the right) places the $\mathcal{R}^{o}_{b}{}^{o}t$ on the right of the screen, without changing its vertical position or its orientation, and without drawing anything.
`gt()`	(go to the top) places the $\mathcal{R}^{o}_{b}{}^{o}t$ at the top of the screen, without changing its horizontal position or its orientation, and without drawing anything.
`gb()`	(go to the bottom) places the $\mathcal{R}^{o}_{b}{}^{o}t$ at the bottom of the screen, without changing its horizontal position or its orientation, and without drawing anything.
`gc()`	(go to the centre) places the $\mathcal{R}^{o}_{b}{}^{o}t$ at the centre of the screen, without changing its orientation, and without drawing anything.

TAB. 6.1 – *Jumps of the* $\mathcal{R}^{o}_{b}{}^{o}t$ *to the edge and to the centre of the frame*

Exercise 13 *Write a Python program that draws crosses at the edge and at the centre of the screen as shown in the figure below.* □

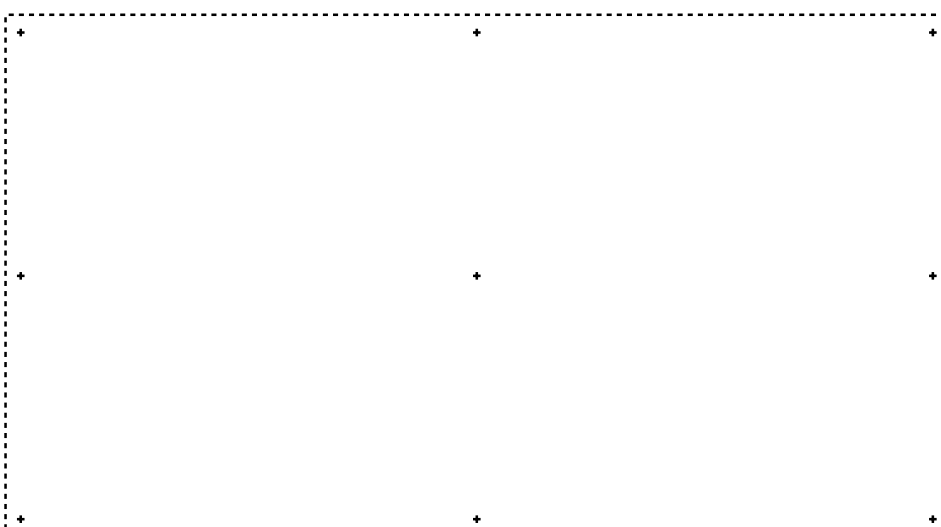


FIG. 6.2 – *Crosses at the edge and at the centre of the frame*

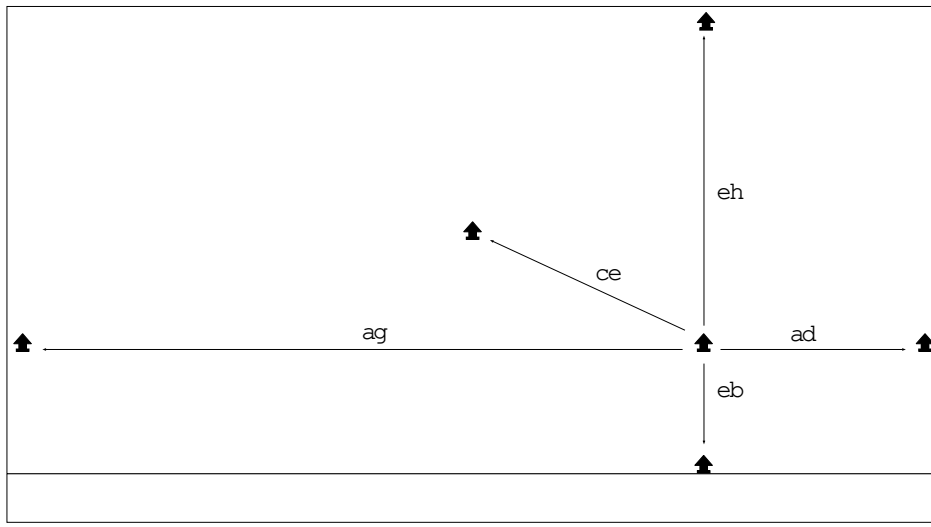


FIG. 6.1 – *Jumps of the Robot to the edge and to the centre of the frame*

Solution 13

```
from LibRobot import *

# Draw crosses on the edge and at the centre of the frame
gl(); dc()  # middle of the left edge
gr(); dc()  # middle of the right edge
gc(); gt(); dc()  # middle of the top edge
gb(); dc()  # middle of the bottom edge
gt(); gl(); dc()  # in the top left corner
gr(); dc()  # in the top right corner
gb(); gl(); dc()  # in the bottom left corner
gr(); dc()  # in the bottom right corner
gc(); dc()  # at the centre
st()
```

□

Addenda

The commands that move the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ to the edge of the frame will be defined in Python later, after we have explained the notions of test and iteration. We introduce them right now so that we can easily draw friezes. In the next chapter these friezes with repetitive patterns will allow us to introduce simply the notion of bounded iteration.

The command `gc()`, which brings the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ back to the centre, will be essential as soon as we use a Cartesian coordinate system to designate the points of the grid.

7

“for” loops

7.1 Repetitions

We have already seen a few examples of Python programs in which a sequence of *Robot* commands or of procedure calls is repeated many times in a row. This is the case, for example, in the program `FriezeOfN_1`, whose execution produces the frieze below:



```
from LibRobot import *

# Draw a frieze of N
gl()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
fd(); fd(); ht(); cl(); cr(); ht()
```

```
fd(); fd(); ht(); cl(); cr(); ht()
st()
```

To avoid repeating the same sequences of instructions over and over, we are going to study a new statement of the Python language called the *"for" loop* ("for" is an ordinary English word, so the statement reads almost like an English sentence[1]). Here is the program `FriezeOfN_1` drawing the frieze below, rewritten using a "for" loop:



```
from LibRobot import *
# Draw a frieze of N with a "for" loop
gl()
for j in range(16):
   fd(); fd(); ht(); cl(); cr(); ht()
st()
```

In plain English, the loop '**for** `j` **in** `range(16):` ...' can be read as 'for j among the first 16 numbers do ...'. The English word **in** has its usual meaning. The English word `range` means 'sequence' or 'span'.

The sequence of statements to be repeated is written on the lines that follow the colon '`:`', shifted to the right. It is called the *body of the loop*. The *number of iterations*, that is, the number of times the body of the loop must be repeated, is given between the parentheses of `range`. In our example, the body of the loop '`fd(); fd(); ht(); cl(); cr(); ht()`' must be repeated 16 times in a row.

7.2 The loop counter

Just after the word **for** we find the name `j` of a *counter*. The computer uses this counter to count the number of times it has to repeat the sequence of statements in the "for" loop, exactly as we might do by counting on our fingers! To do so, the computer uses a cell of its memory, which successively contains the integers 0, 1, 2, ..., 15. Indeed, `range(16)` denotes the sequence of 16 integers that starts at 0 and stops just before 16 (like computer scientists, Python counts from 0). We say that `j` is an integer *variable* since its value varies from one iteration to the next.

[1] We call it a "for" loop, using the very word that is written in the program.

We have to give a name to this counter. It is an identifier that we can choose as we please (here, for example, `j`), following the rule for identifiers: it begins with a letter (or an underscore '_'), contains only letters, digits and underscores, and must not be a reserved word of Python. If we write '**for** `1st` **in** ...', Python answers '`SyntaxError: invalid decimal literal`' (since `1st` begins with a digit). If we write '**for in in** ...', it answers '`SyntaxError: invalid syntax`'.

In Python, there is no need to announce in advance that we are going to use a counter[2]: the counter is created by the "for" loop itself. Python knows that it is an integer since it takes the values 0, 1, 2, ... After the loop, the counter keeps the last value it took (15 in our example), but it is better not to use it: if the loop did not perform any iteration, the counter does not exist and Python reports the error '`NameError: name 'j' is not defined`'.

For the counter to take the values 1, 2, 3, ..., 16, we would write '**for** `j` **in** `range(1, 17):`'. In general, `range(`a`, `b`)` denotes the sequence of integers from a to $b-1$: it starts at a and stops just *before* b. Thus `range(1, 5)` denotes 1, 2, 3, 4 and `range(`n`)`, which is an abbreviation of `range(0, `n`)`, denotes the n numbers 0, 1, ..., $n-1$. For the counter to go from 1 to n, we therefore write '**for** `i` **in** `range(1, `n` + 1):`'.

The loop counter of the program `FriezeOfN_2` is only used to count the iterations. It is then simpler to write `range(16)`. We will see in the next chapter that its value can also be used in the body of the loop.

7.3 The "for" loop

A *"for" loop* has the following form:

"For" loop:
for ␣ Identifier ␣ **in** ␣ `range(`Number of iterations`):` ... Statements of the body of the loop (indented) ...

The identifier is the name of the iteration counter. The sequence of statements that is repeated is called the *body of the loop*. Like the body of a procedure, it is written on the lines that follow the colon ':', all shifted to

[2] Other languages such as Pascal require the counter to be *declared* at the beginning of the program or of the procedure, stating that it denotes an integer.

the right by the same number of blanks (usually four), one statement per line or several statements on the same line separated by '`;`'. It is this shift, called *indentation*, that tells Python where the body of the loop begins and ends: the loop ends at the first line that is no longer shifted.

Do not forget the blanks ␣ around the word **in**. If we write `jin` instead of `j` **in**, or `inrange` instead of **in** `range`, Python no longer recognizes the word **in** and reports '`SyntaxError: invalid syntax`'. If we forget the colon '`:`' at the end of the **for** line, Python reports '`SyntaxError: expected ':'`'.

A "for" loop is itself a statement. The statements that follow it are written without any shift, at the same level as the **for**, like the `st()` of the program `FriezeOfN_2`. If we also shifted the line `st()`, it would be part of the body of the loop and would be repeated 16 times! If we forget to shift the body of the loop, Python reports '`IndentationError: expected an indented block after 'for' statement on line 5`'. If the lines of the body are not all shifted by the same number of blanks, Python reports '`IndentationError: unexpected indent`' or '`IndentationError: unindent does not match any outer indentation level`'.

If the 'Number of iterations' is smaller than 1 (0, for example), the statements of the body of the loop are not executed at all. Do not put a semicolon ';' after the colon: Python then reports '`SyntaxError: invalid syntax`' just after the colon.

When the body of the loop is short, the whole loop can be written on a single line, as in '**for** `i` **in** `range(10): fd()`'. Extra blanks inside a line only serve to make the program easier to read, but those at the beginning of a line matter.

Example 10 (Frieze of spikes) The following frieze:



is produced by the program below:

```
from LibRobot import *

# Draw a frieze of spikes
gl(); qtr()
for j in range(32):
   pu(); fd(); pd(); ht(); cr(); ht()
   fd(); fd(); ht(); cr()
st()
```

□

7.4 Using a "for" loop in a procedure

The body of a procedure can contain "for" loops, like any other statement:

Procedure declaration:

```
def␣procedureName():
```
... Statements of the body of the procedure[a] (indented) ...

[a] Including "for" loops using counters `i`, `j`, ...

Since the counter of a loop is only used in the body of that loop and since it starts again from its first value at each new loop, the same identifier can serve as a counter in several different loops, whether they are in different procedures or one after the other in the same procedure. This is illustrated in the program `Arrow` below, where different loop counters, used in different procedures (`fletching` and `shaft`), have the same name `i`.

Example 11 (Arrow)

```
from LibRobot import *
# Draw a vertical arrow
def feather():
   fd(); tetl(); fd(); pu(); tetl(); fd(); fd()
   tetl(); pd(); fd(); etr()
def fletching():
   for i in range(5):
      feather()
def shaft():
   for i in range(10):
      fd()
def head():
   qtl(); fd(); tetr(); fd(); qtr(); fd()
   tetr(); fd()
gb(); fletching(); shaft(); head(); st()
```



□

7.5 Nested "for" loops (independent)

The body of a "for" loop:

for ␣ Identifier1 ␣ **in** ␣ `range(`Number of iterations1`):`

...

can itself contain "for" loops:

for ␣ Identifier1 ␣ **in** ␣ `range(`Number of iterations1`):`

...

for ␣ Identifier2 ␣ **in** ␣ `range(`Number of iterations2`):`

...

...

We then say that the "for" loops are *nested*. The loop using the counter Identifier2 to perform Number of iterations2 iterations is called the *inner loop*. The loop using the counter Identifier1 to perform Number of iterations1 iterations is called the *outer loop*. The body of the inner loop is shifted twice: once because it is part of the body of the outer loop, a second time because it is the body of the inner loop. The inner loop ends at the first line that returns to the shift of its **for**; the outer loop at the first line that returns to the shift

of the first **for**. So it is the shift that indicates, without ambiguity, which **for** each statement belongs to. Consider, for example, the drawing of a square:

```
from LibRobot import *
# Draw a square with procedures
# Draw one side of the square and turn right
def side():
   for j in range(4):
      fd()
   qtr()
# Draw a square
def square():
   for i in range(4):
      side()
square(); st()
```



By replacing the procedure calls by their bodies, we obtain two nested loops:

```
from LibRobot import *
# Draw a square with nested "for" loops
# Draw a square
for i in range(4):
   # Draw one side of the square and turn right
   for j in range(4):
      fd()
   qtr()
st()
```

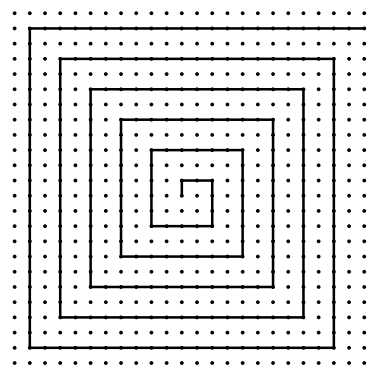
The inner loop is used to build one side while the outer loop draws the four sides.

7.6 Nested "for" loops (dependent)

In nested "for" loops, the number of iterations of the inner loop may be different each time the body of the outer loop is executed. Let us begin by considering the simple example of line segments of lengths 1, 2, 3, ... arranged in a spiral (the drawing is shown below at scale $\frac{1}{2}$ with the grid):

```
from LibRobot import *

# Draw a square spiral
for i in range(1, 23):
    for j in range(i):
        fd()
    qtr()
st()
```



The number of iterations in the inner "for" loop is equal to the value of the counter `i` of the outer loop, that is, successively 1, 2, 3, ..., 22 (since `range(1, 23)` stops just before 23). The program `SquareSpiral_1` is therefore equivalent to the program `SquareSpiral_2` below, in which the "for" loops have been eliminated to show exactly which sequence of commands is executed by the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$.

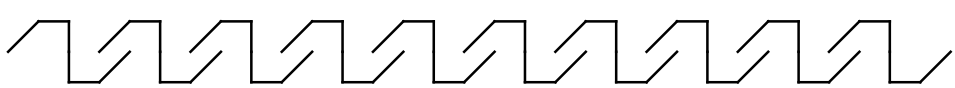
```
from LibRobot import *

# Draw a square spiral without "for" loops
fd(); qtr()
fd(); fd(); qtr()
fd(); fd(); fd(); qtr()
fd(); fd(); fd(); fd(); qtr()
fd(); fd(); fd(); fd(); fd(); qtr()
fd(); fd(); fd(); fd(); fd(); fd(); qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd()
qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
```

```
fd(); fd(); qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); fd()
qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); qtr()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); fd(); fd()
fd(); fd(); fd(); fd(); fd(); fd(); qtr()
st()
```

7.7 Invariants of "for" loops

When we draw a frieze like this one:



with a "for" loop, the $\mathcal{R}obot$ must always be facing the same direction and be in the same position with respect to the next pattern to be drawn by executing the body of the loop. In other words, at the end of the body of the loop we must put the $\mathcal{R}obot$ back in its starting position:

```
from LibRobot import *

# Draw a frieze of tildes

# Draw a tilde-shaped pattern ~
def tilde():
   etr(); fd(); etr(); fd()
   qtr(); fd(); fd(); qtl()
   fd(); etl(); fd()

gl()
for i in range(10):
   tilde()
   # Back up to the next tilde's start
   pu(); tetl(); fd(); pd()
   # Turn the robot to face north
   qtr()
st()
```

We say that the position of the $\mathcal{R}obot$ is an *invariant* of the loop, in the sense that it must be the same at each iteration.

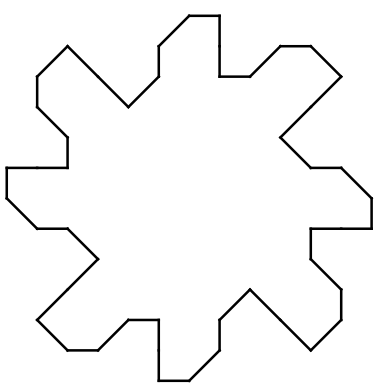
If, in our example, we draw one pattern and then the next one without putting the $\mathcal{R}obot$ back in its starting position in between, we get the following result, which, it must be admitted, does not look much like the original:

```
from LibRobot import *

# Draw a frieze of tildes (first mistake)

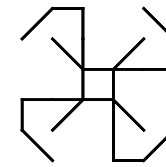
# Draw a tilde-shaped pattern ~
def tilde():
   etr(); fd(); etr(); fd()
   qtr(); fd(); fd(); qtl()
   fd(); etl(); fd()

for i in range(10):
   tilde()
st()
```



If we remember to move the $\mathcal{R}obot$ back but forget to turn it back in the right direction, the result is even stranger:

```
from LibRobot import *
# Draw a frieze of tildes (second mistake)
# Draw a tilde-shaped pattern ~
def tilde():
    etr(); fd(); etr(); fd()
    qtr(); fd(); fd(); qtl()
    fd(); etl(); fd()
for i in range(13):
    tilde()
    # Back up to the next tilde's start
    pu(); tetl(); fd(); pd()
st()
```



A good way to find the errors in a "for" loop is therefore to look for the invariant, that is, the position and orientation in which the $\mathcal{R}\text{o}\underset{\frown}{\text{b}}\text{o}t$ must be before drawing the pattern by executing the body of the loop. We must then check that the $\mathcal{R}\text{o}\underset{\frown}{\text{b}}\text{o}t$ is correctly placed when we enter the loop for the first time and that it is correctly placed again after executing the body of the loop.

The same notion of invariant applies to procedures. Before each call of a procedure, the robot must be in one or several possible states, which must be perfectly defined by whether the $\mathcal{R}\text{o}\underset{\frown}{\text{b}}\text{o}t$'s pen is up or down and by the position and orientation of the $\mathcal{R}\text{o}\underset{\frown}{\text{b}}\text{o}t$ with respect to the figure drawn by the procedure.

Exercise 14

_ *Write Python programs using "for" loops to draw some of the friezes of figure 7.3 page 83.*

_ *Write Python programs using "for" loops in procedures to draw some of the friezes of figure 7.4 page 84.*

_ *Write Python programs to draw some of the rosettes, friezes and tiling of figure 7.5 page 85, using independent nested "for" loops.*

_ *Write Python programs to draw some of the spirals, grids and other organ pipes of figure 7.1 below, using dependent nested "for" loops.*

_ *Write a Python program to draw the tiling of figure 7.2 below, stating precisely the invariants of the procedures and of the "for" loops.* □

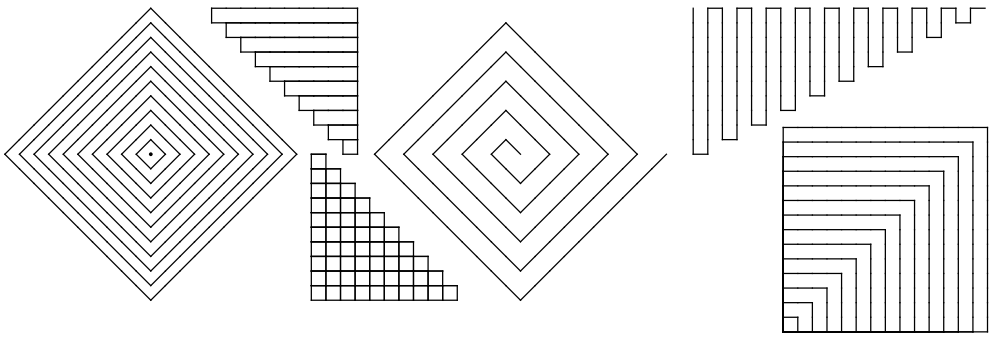


FIG. 7.1 – *Grids, squares, spiral and organ pipes*

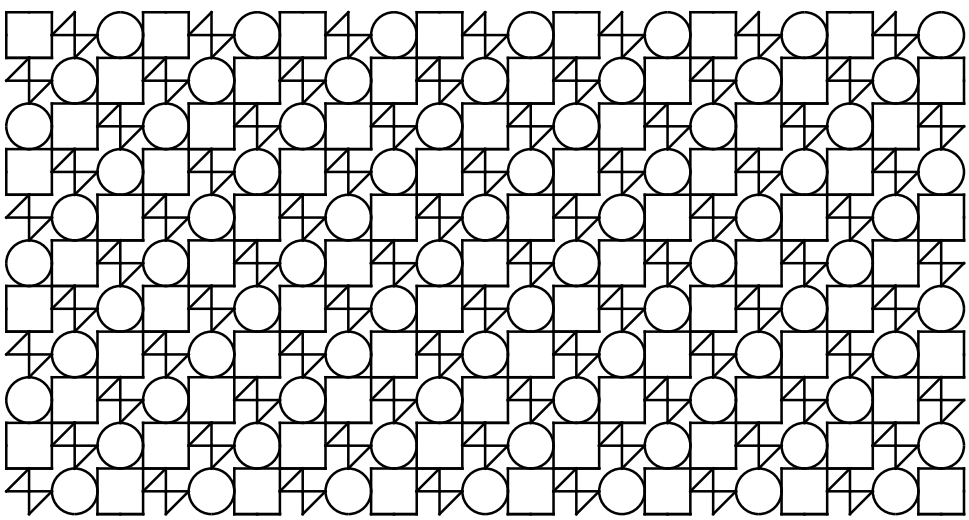


FIG. 7.2 – *Tiling of squares, triangles and circles*

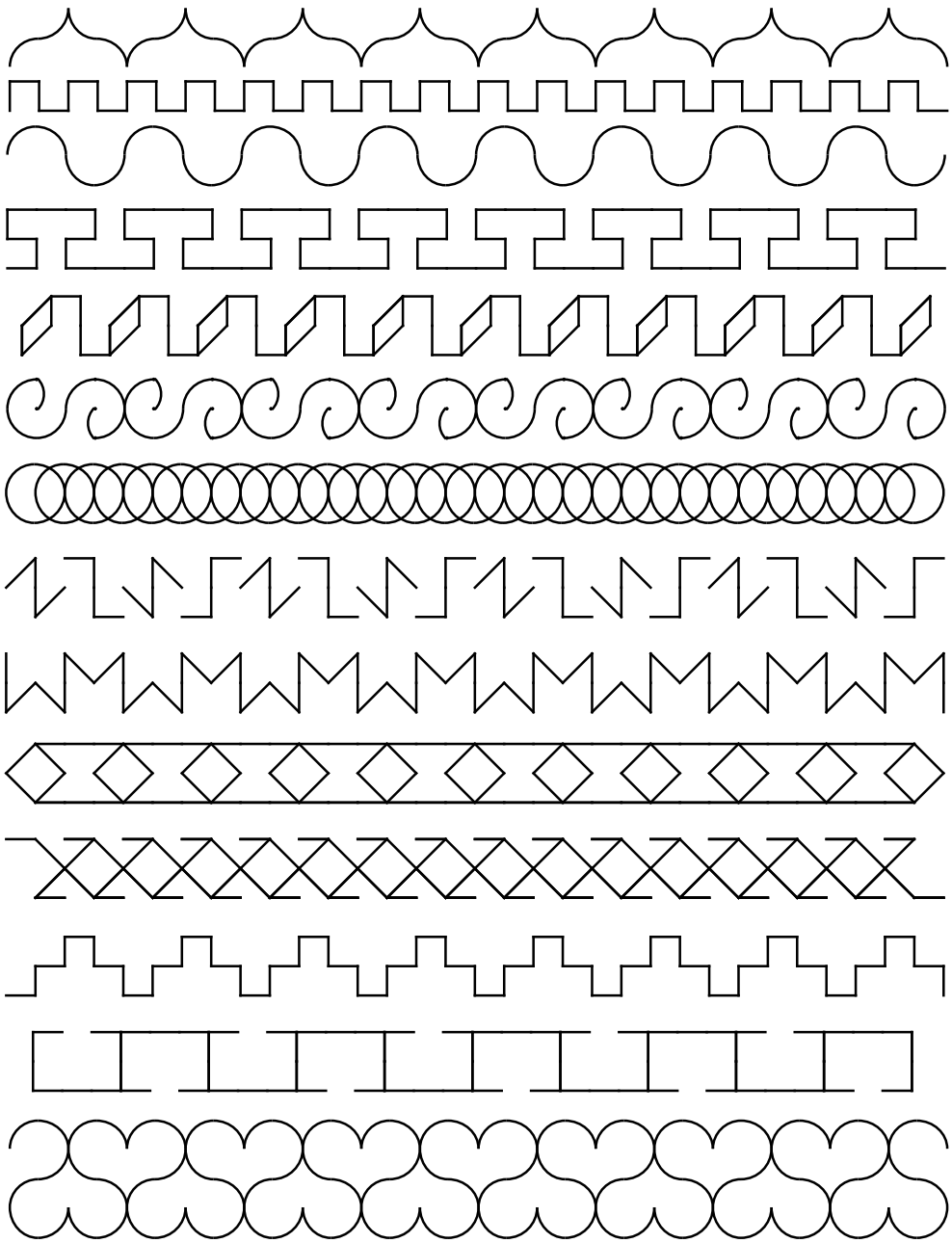


FIG. 7.3 – *Friezes (1)*

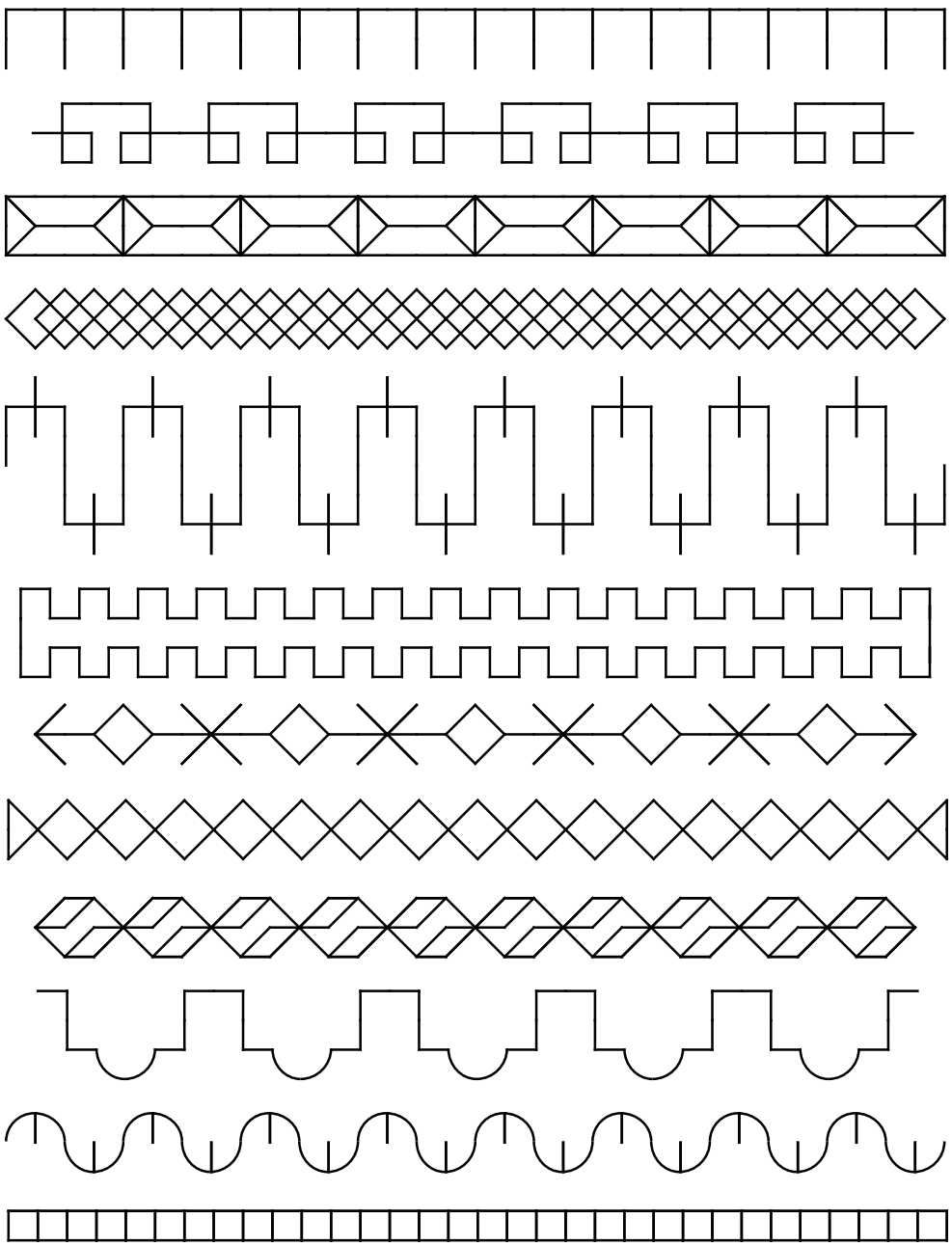


FIG. 7.4 – *Friezes (2)*

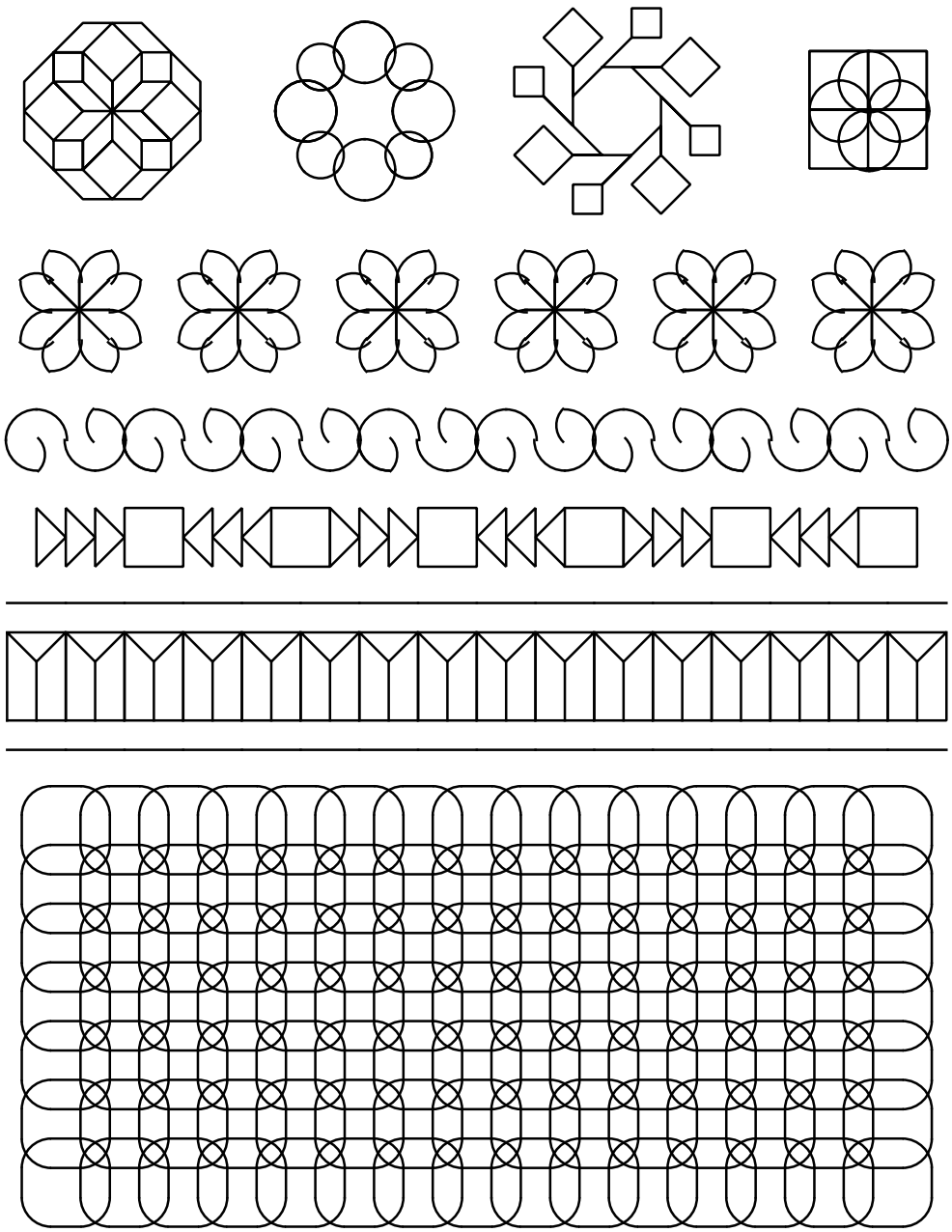


FIG. 7.5 – *Rosettes, friezes and tiling*

Solution 14 *We give a few programs; the others are in the folder* `book/07_for`*:*

- ```
  from LibRobot import *
  # Draw a notched frieze

  # Draw the pattern 'n_'
  def notch():
      fd(); qtr(); fd(); qtr(); fd(); qtl(); fd()
      qtl()

  gl()
  for i in range(20):
      notch()
  st()
  ```

- ```
  from LibRobot import *
  # Draw a triangular grid pattern
  # Draw 10 rows of squares side by side
  # horizontally
  qtr()
  for i in range(1, 11):
      # Draw a row of i squares side by side
      # horizontally, starting at the top left and
      # arriving at the top right, the robot facing
      # east in both cases.
      for j in range(i):
          # Draw a square, starting from the upper
          # left corner and arriving at the upper
          # right corner, the robot facing east in
          # both cases.
          for k in range(4):
              fd(); qtr()
          fd()
      # Go back, without drawing, to the start of the
      # next row, the robot facing east at
      # departure and on arrival.
      pu(); ht()
      for j in range(i):
          fd()
      qtl(); fd(); qtl(); pd()
  st()
  ```

- ```
  from LibRobot import *
  # Draw a tiling of squares, triangles and circles

  # Draw a square with sides of size 2, the robot
  ```
  ```

```
# starting from the lower left corner facing east,
# and arriving pen down at the lower right corner
# also facing east.
def square():
   # Draw the square.
   fd(); fd(); qtl(); fd(); fd(); qtl()
   fd(); fd(); qtl(); fd(); fd(); qtl()
   # Advance to the lower right corner.
   fd(); fd()

# Draw two triangles joined at a vertex
# inscribed in an invisible square of the grid of
# side 2, starting from the lower left corner of the square
# facing east, and arriving at the lower right
# corner, pen down and facing east.
def triangles():
   pu(); fd(); pd()
   qtl(); fd(); fd(); tetl(); fd(); tetl()
   fd(); fd(); tetr(); fd(); tetl()
   pu(); fd(); pd()

# Draw a circle of radius 1 inscribed in an invisible
# square of the grid of side 2, starting from the
# lower left corner of the square facing east, and
# arriving at the lower right corner of the square, pen
# down and facing east.
def circle():
   pu(); fd(); pd()
   cl(); cl(); cl(); cl()
   pu(); fd(); pd()

# Draw a row made of a pattern repeated 7
# times, the pattern being made of a square, triangles
# and a circle side by side, the robot starting from the
# lower left corner of the row facing east, and
# arriving at the lower right corner also facing
# east.
def rowSquare():
   for i in range(7):
      # The robot is pen down, facing east, at the bottom
      # left of the pattern
      square(); triangles(); circle()

# Like rowSquare, with the pattern made of triangles,
# a circle and a square.
def rowTriangles():
   for i in range(7):
      triangles(); circle(); square()
```

```
# Like rowSquare, with the pattern made of a circle,
# a square and triangles.
def rowCircle():
   for i in range(7):
      circle(); square(); triangles()

# Bring the robot from the lower right corner of a row
# to the lower left corner of the next row, without
# drawing, facing east and pen down.
def newRow():
   pu(); gl(); qtr(); fd(); fd(); qtl(); pd()

# Tile the screen with 11 rows of patterns starting with
# a square, then with triangles, then with a
# circle, from the upper right corner to the lower
# right corner of the screen, the robot facing east.
def tiling():
   # Draw rows 1 to 9 from top to bottom.
   for j in range(3):
      newRow(); rowSquare()
      newRow(); rowTriangles()
      newRow(); rowCircle()
   # Draw rows 10 then 11.
   newRow(); rowSquare()
   newRow(); rowTriangles()

# Go to the upper right corner, facing east.
gt(); gr(); qtr()
# Tile the screen.
tiling(); st()
```

□

Addenda

The "for" loop is one of the first linguistic notions to have appeared in computer science, since it was invented at ETH Zürich in 1952 by Heinz Rutishauser [35].

Comparing the drawings that can be obtained using the "for" loop with those that can be made in interactive mode gives a measure of the power of computers when it comes to performing repetitive tasks quickly. In this respect the computer is very clearly different from non-programmable calculators. We also understand that this computing power has nothing to do with any form of thought or intelligence, except that of the programmer of the computer.

In Python, we use a loop **for** `i` **in** `range(`n_1`,` n_2`):` ... to repeat the body of the loop (the indented statements that follow the colon) $n_2 - n_1$ times, unless $n_2 \leq n_1$, in which case the body of the loop is not executed. At each iteration, the loop counter `i` takes the successive values n_1, $n_1 + 1$, $n_1 + 2$, ..., $n_2 - 1$. In the form **for** `i` **in** `range(`n_1`,` n_2`, -1):` ..., `i` takes the successive values n_1, $n_1 - 1$, $n_1 - 2$, ..., $n_2 + 1$, unless $n_2 \geq n_1$, in which case the loop is not executed; for example, **for** `i` **in** `range(`n`, 0, -1):` makes `i` take the values n, $n-1$, ..., 1 (the third number, here -1, is the step that is added to the counter at each iteration). If the body of the loop modifies the value of the counter, this has no effect on the following iterations: at each iteration, the counter takes the next value of the `range` sequence. When the counter is of no use in the body of the loop, Python programmers often name it '`_`', as in **for** `_` **in** `range(16):` ...

Using "for" loops trains the ability to perceive when a drawing can be decomposed into a number of identical sub-drawings (in the case of a simple loop) or similar ones (in the case of nested loops). This is a daily exercise for the computer scientist, who must know how to decompose a complex task into simple subtasks, which sometimes requires an enormous amount of imagination. It is a first step towards acquiring the inductive reasoning of scientists who, from specific observations, create a general hypothesis accounting for these observations, a hypothesis that remains to be validated in all possible cases, here in a completely experimental way, by looking at the drawing on the screen. Mathematicians call such a hypothesis, induced from experience and not yet proved, a *conjecture*.

This inductive reasoning was long confused with reasoning by mathematical induction, which is essential in mathematics. It took more than a millennium for mathematicians (Fermat[a], Pascal[b] in particular) to fully understand reasoning by induction ([21] page 17).

For the time being we are only at the beginnings, the absence of a correct induction still being accepted in reasoning by induction: if the drawing is right for the basic pattern and for the first repetition of the pattern, then it is deemed correct, the only remaining problem, fitting the drawing into the window, being solved by simple experimentation.

The notion of *invariant* (that is, for our $\mathcal{R}\text{o}\underbrace{b}\text{o}t$, the description of its possible positions and orientations at the beginning of the body of the loops) is due to Turing[c] in 1949; it was forgotten, then rediscovered by Naur[d] in 1966 and Floyd[e] in 1967. It relies on reasoning by induction: if the invariant is true when we enter the loop for the first time (that is, the position and orientation of the $\mathcal{R}\text{o}\underbrace{b}\text{o}t$ conform to the description given in the invariant) and if, assuming that the invariant is true at the beginning of the execution of the body of the loop, it is still true after the execution of this loop body, then the invariant is always true at the beginning of each iteration of the loop and is still true when we exit the loop. The notion applies equally well to procedures: we can use an *entry invariant* describing the positions and orientations of the $\mathcal{R}\text{o}\underbrace{b}\text{o}t$ before the call and an *exit invariant* describing these positions and orientations after the call, so that the body of the procedure can be understood without having to look at all the places where it is called to find out under which conditions the procedure is called.

---

[a] Pierre de Fermat, 1601–1665, French mathematician.
[b] Blaise Pascal, 1623–1662, French mathematician.
[c] Alan Turing, English computer scientist.
[d] Peter Naur, Danish computer scientist.
[e] Robert Floyd, American computer scientist.

The computer can be used to manipulate programs, in particular to discover automatically invariants of simple form at each point of a program. This is the purpose of the *abstract interpretation* of programs [9]. For the $\mathcal{R}^{o}_{b}{}^{o}t$, for example, a simple form of invariant is the set of its possible orientations, ignoring the position of its pen, its coordinates on the screen, etc. The possible orientations of the concrete $\mathcal{R}^{o}_{b}{}^{o}t$ can be computed by executing the program with an abstract $\mathcal{R}^{o}_{b}{}^{o}t$ that executes the commands of the concrete $\mathcal{R}^{o}_{b}{}^{o}t$ without moving but simply changing its orientation. For this abstract $\mathcal{R}^{o}_{b}{}^{o}t$, the command `fd` does nothing since it does not change the orientation of the concrete $\mathcal{R}^{o}_{b}{}^{o}t$; the command `cr` has the same effect as `qtr`; etc. During the abstract execution, the possible orientations of the abstract $\mathcal{R}^{o}_{b}{}^{o}t$ are written as comments at each point of the program. The abstract execution of the program stops once the results have stabilized. For example, for the program `FriezeOfN_2` (page 72), we find:

```
from LibRobot import *
# Draw a frieze of N with a "for" loop
# North
gl()
# North
for j in range(16):
   fd(); fd()  # North  # North
   ht(); cl()  # North  # South
   cr(); ht()  # East   # South
# North
# North
st()
# North
```

The rule of signs is an example of abstract interpretation used in mathematics. The sign of an expression can be computed using the sign of the variables, which replaces their values, thanks to abstract operations such as $\oplus + \oplus = \oplus$, $\oplus - \ominus = \oplus$, $\oplus \times \ominus = \ominus$, etc. The price to pay for this simplification is of course the imprecision of the results, as for $\oplus + \ominus$.

8

Integer expressions

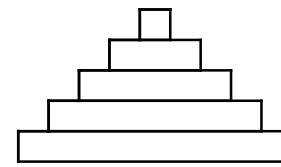
A computer can perform *additions* $(15+3) = 18$, *subtractions* $(15-3) = 12$, *multiplications* $(15 * 3) = 45$ and *divisions* of integers $(15 \;//\; 3) = 5$. If the division does not come out exactly, the result is the integer quotient. The remainder is given by the *modulo* operation, written `%`. For example $(17 \;//\; 3) = 5$ and $(17 \;\%\; 3) = 2$ because dividing the dividend 17 by the divisor 3 gives the quotient 5 and the remainder 2. In mathematics a cross '$\times$' is used to denote multiplication, whereas in Python the asterisk $*$ is used. This avoids confusing the multiplication sign '$\times$' with the lowercase letter `x`. Likewise, two slashes '`//`' are used for integer division rather than the sign '$\div$', which is not found on computer keyboards. Be careful not to write a single slash '`/`': in Python, it does not give the integer quotient ($17 \;/\; 3$ is not 5). We will see later that it is used to compute with decimal numbers.

To do a calculation involving several operations, we use parentheses. For example $((2 * 3) - 1)$ is equal to 5. The calculation begins by evaluating the innermost expression in parentheses. For example $((((5+1) * 3) - 4) \;//\; 2) = (((6 * 3) - 4) \;//\; 2) = ((18 - 4) \;//\; 2) = (14 \;//\; 2) = 7$.

In these *integer expressions* we can also use the value of the counters of "for" loops. For example, during the successive iterations in the body of the loop "**for** `i` **in** `range(1, 6)`: ...", the counter `i` is 1, then 2, then 3, then 4 and finally 5. In this loop body the expression `(2*i) - 1` therefore successively has the value $((2 * 1) - 1) = 1$, $((2 * 2) - 1) = 3$, $((2 * 3) - 1) = 5$, $((2 * 4) - 1) = 7$ and finally $((2 * 5) - 1) = 9$.

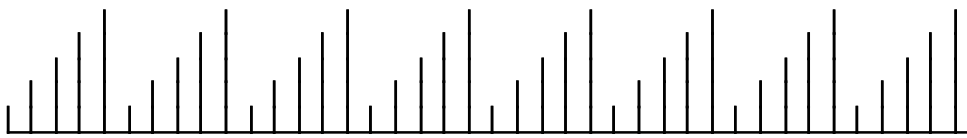
Example 12 (Pyramid) *To draw the pyramid below, we can start from the top and draw, from top to bottom, successive rectangles of lengths 1, 3,*

5, 7, 9. The length of the side of the $\texttt{i}^{\text{th}}$ *rectangle counting from the top is therefore* $((2 * \texttt{i}) - 1)$. *In the program, this expression is written '*`2 * i - 1`*' between the parentheses of* `range` *to give the number of iterations of the loop that draws one side of the rectangle.*



```
from LibRobot import *
# Draw a pyramid with 5 levels.
# Turn the robot to the west.
qtl()
for i in range(1, 6):
   # Starting from the bottom left corner of a level
   # with the robot facing west, draw the bottom left
   # corner of the level below and turn to the east.
   fd(); qtl(); fd(); qtl()
   # Draw the i-th rectangular level.
   # Bottom side
   for j in range(2 * i - 1):
      fd()
   # Right end
   qtl(); fd(); qtl()
   # Top side
   for j in range(2 * i - 1):
      fd()
   # Go back down to the bottom left corner and turn
   # the robot to the west.
   qtl(); fd(); qtr()
st()
```

Exercise 15 *Complete the program* `SawtoothStakes`*, replacing the ellipsis '. . . ' by an integer expression, so as to obtain the row of stakes shown below:*

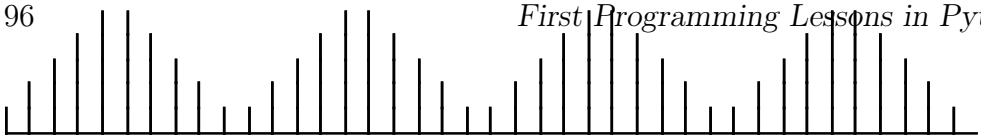


```
from LibRobot import *
# Draw a row of stakes in a sawtooth pattern
gl()
for i in range(1, 41):
   for j in range(2):
      for k in range(...):
         fd()
      ht()
   qtr(); fd(); qtl()
st()
```

□

Exercise 16 *Write Python programs commanding the* Robot *to produce the geometric drawings of figure 8.1 (page 96).* □

Exercise 17 (Difficult) *Complete the program* `WavesOfStakes` *to draw the row of stakes shown below:*



```
from LibRobot import *

# Draw a row of stakes in waves
gl()
for i in range(1, 9):
    for j in range(1, 6):
        for k in range(2):
            for l in range(...):
                fd()
            ht()
        qtr(); fd(); qtl()
st()
```

□

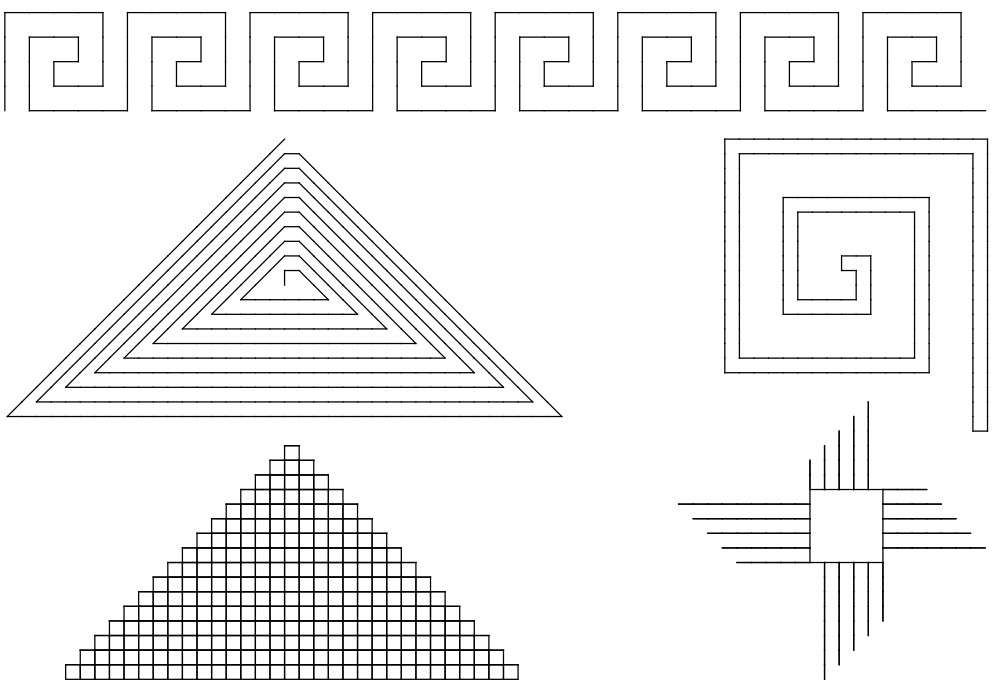


FIG. 8.1 – *Greek key, spirals, triangular tiling and fringed square*

Exercise 18 (Difficult)

1. *Write a Python program to draw the tiling of Nicomachus*[1] *shown in figure 8.2 page 98 using dependent nested "for" loops. Describe the loop invariants by comments in the program. The tiling of Nicomachus is made of 10 rings consisting of squares whose side lengths are respectively 1, 2, 3, 4, . . . , 10.*
2. *Compute the area of the square in two different ways suggested by figure 8.3 page 101:*
   (a) *by computing the length of the half-side of the square, obtained by adding those of the squares of the tiling,*
   (b) *by computing the area of the quarter of the large square delimited by the diagonals of this large square (see this part in green in figure 8.3, page 101), the area of this quarter of the large square being obtained by adding the areas of the small squares that make it up.*
3. *Deduce the equality:*

$$1^3 + 2^3 + 3^3 + 4^3 + \ldots + 10^3 = (1 + 2 + 3 + 4 + \ldots + 10)^2$$

□

Solution 15 *The missing integer expression in the program* `SawtoothStakes` *is:*

$$(1 + ((\texttt{i}-1)\ \%\ 5))$$

which successively takes the values 1, 2, 3, 4, 5, 1, 2, 3, . . . *when* `i` *takes the values* 1, 2, 3, 4, 5, 6, 7, 8, . . . □

Solution 16 *We give one program; the others are in the folder* `book/08_integer_expressions`*:*

```
from LibRobot import *
# Draw a trapezoidal spiral
fd(); qtr()
for i in range(1, 10):
   # Draw the top side
   fd()
   # Draw a right side
   etr()
```

[1] Nicomachus of Gerasa, Greek mathematician, c. 1st century AD.

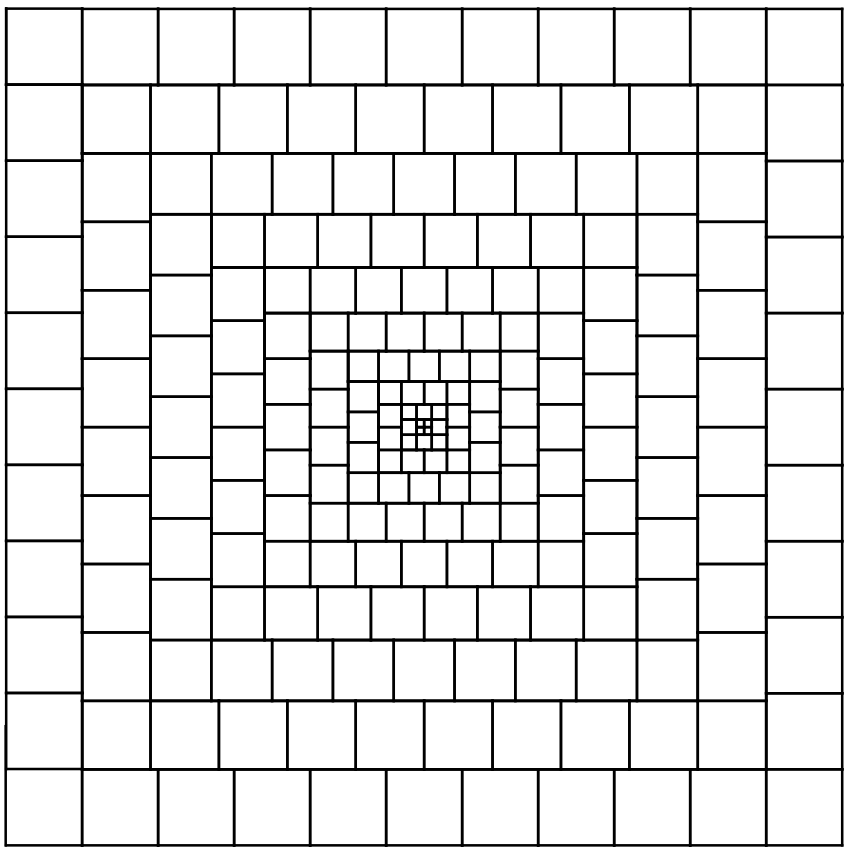


FIG. 8.2 – *Tiling of Nicomachus (Scale* $\frac{1}{4}$*)*

```
    for j in range(2 * i):
       fd()
    # Draw the bottom side
    tetr()
    for j in range(4 * i + 2):
       fd()
    # Draw a left side
    tetr()
    for j in range(2 * i + 1):
       fd()
    etr()
st()
```

□

Solution 17 *The missing integer expression in the program* `WavesOfStakes` *is the following:*

$$(((6-\texttt{j}) * ((\texttt{i}+1) \,\%\, 2)) + (\texttt{j} * (\texttt{i} \,\%\, 2)))$$

When `i` *is odd,* $((\texttt{i}+1) \,\%\, 2) = 0$ *and* $(\texttt{i} \,\%\, 2) = 1$*, so the expression is equal to* `j`*. When* `i` *is even,* $((\texttt{i}+1) \,\%\, 2) = 1$ *and* $(\texttt{i} \,\%\, 2) = 0$*, so the expression is equal to* $(6-\texttt{j})$*.* □

Solution 18

```
from LibRobot import *

# Tile a square with successive rings made of
# squares of sizes 1, 2, 3, ..., starting from the
# centre.
for i in range(1, 5):
   # Draw the 4 sides of the i-th ring, tiling them
   # with squares of size i.
   for j in range(4):
      # Draw one side of the i-th ring, tiled with
      # i squares of side length i, the robot
      # starting from the first square and arriving
      # at the last square in the same corner and in
      # the same direction as at the start.
      # Draw i - 1 squares of the i-th ring
      for k in range(i - 1):
         # Draw a square of length i, the robot
         # starting and arriving at the same corner
         # and in the same direction.
         for l in range(4):
            for m in range(i):
```

```
                fd()
             qtr()
          # Move forward to the next square of the
          # i-th ring.
          qtr()
          for l in range(i):
             fd()
          qtl()
       # Draw the i-th square of length i of the
       # i-th ring.
       for k in range(4):
          for l in range(i):
             fd()
          qtr()
       # Turn a quarter turn to place the robot at the
       # beginning of the next side of the i-th ring.
       qtr()
    # Go to the starting point of the (i + 1)-th
    # ring.
    qtl()
    for j in range(i):
       fd()
    qtr()
    for j in range(i):
       fd()
st()
```

If we divide one side of the large square in two, the length of this half-side is the sum $(1+2+\ldots+10)$ *of the lengths of the squares of sizes 1, 2, 3, . . . , 10 of the tiling. This can be seen clearly in figure 8.3 page 101 thanks to the ruler that we have placed below and to the right of the large square. The area of the large square is therefore the square of the length of its side, that is:*

$$(2 \times (1 + 2 + \ldots + 10))^2 = 4 \times (1 + 2 + \ldots + 10)^2$$

If we divide the large square along its diagonals, we obtain a triangle, as shown in green in figure 8.3 (page 101). This triangle contains 1 square of side length 1, 2 squares of side length 2, . . . , 10 squares of side length 10. Its area is therefore $1 \times 1^2 + 2 \times 2^2 + \ldots + 10 \times 10^2$*, that is,* $1^3 + 2^3 + \ldots + 10^3$*. By turning this triangle 4 times, we cover the large square completely. The area of the large square is therefore:*

$$4 \times (1^3 + 2^3 + \ldots + 10^3)$$

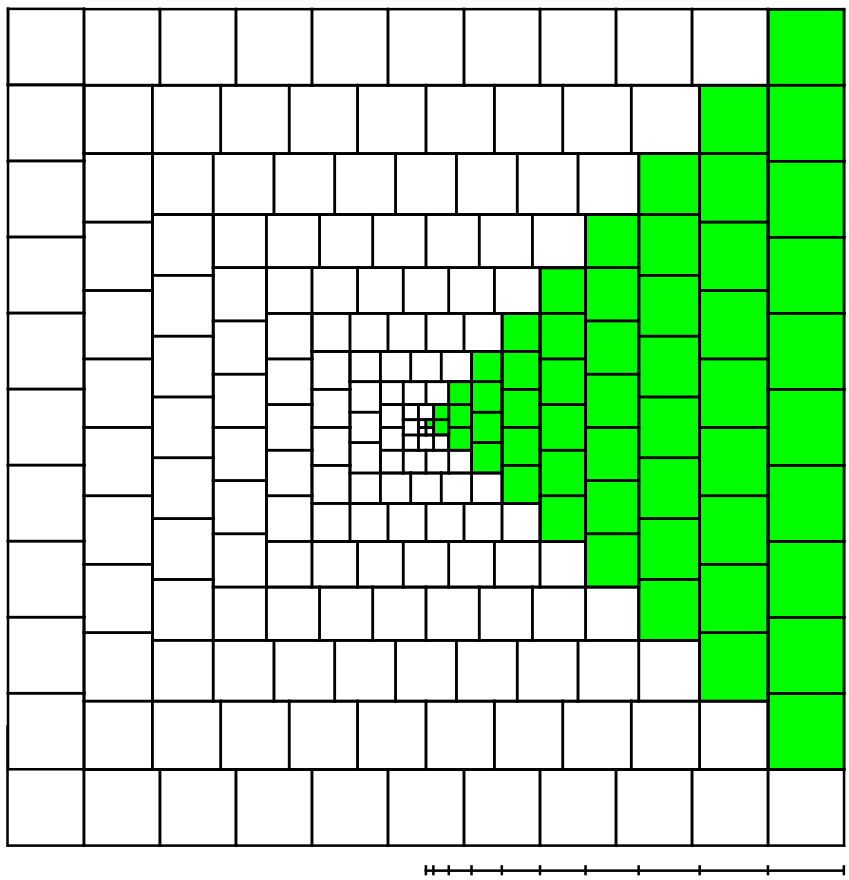


FIG. 8.3 – *Side and area of the square of Nicomachus (Scale* $\frac{1}{4}$*)*

The result is the same whichever way we compute the area of the square of Nicomachus. Consequently:

$$4 \times (1^3 + 2^3 + \ldots + 10^3) = 4 \times (1 + 2 + \ldots + 10)^2$$

that is:

$$1^3 + 2^3 + \ldots + 10^3 = (1 + 2 + \ldots + 10)^2$$

Another proof of this identity was given by Al-Karaji[2] *and relies on tiling the square with L-shaped pieces (gnomons), as illustrated below:*

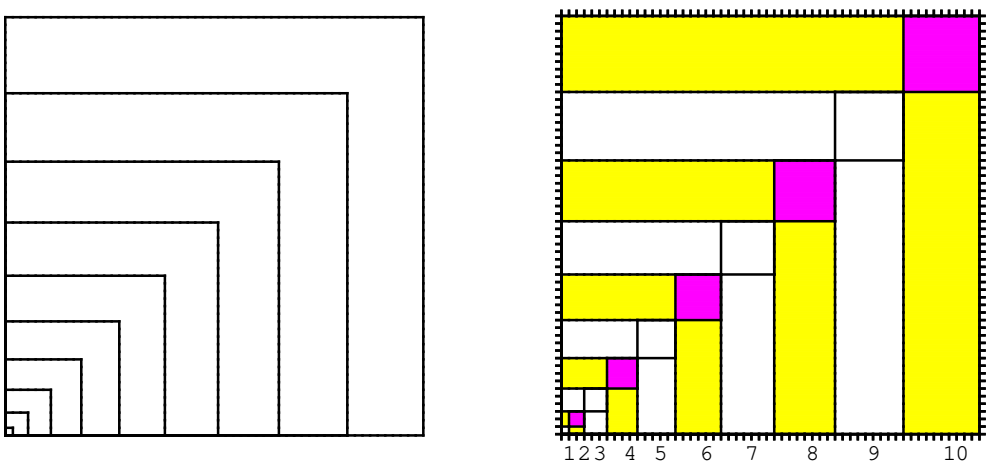


FIG. 8.4 – *Tiling of Al-Karaji*

The area of the outer L-shape with short side 10 is:

$$\begin{aligned}(2 \times 10 \times (1 + 2 + \ldots + 10)) - 10^2 &= 2 \times 10 \times \tfrac{10 \times (10+1)}{2} - 10^2 \\ &= -10^3 + 10^2 - 10^2 = 10^3\end{aligned}$$

The area of the L-shape with short side 9 is 9^3*, that of the L-shape with short side 8 is* 8^3*, ..., the area of the small square is* $1 = 1^3$*. Consequently, the area of the large square, which is the sum of the areas of the small square and of the L-shapes, is* $1^3 + 2^3 + \ldots + 10^3$*. On the other hand, the length of the side of the large square is* $(1 + 2 + \ldots + 10)$*, so its area is* $(1 + 2 + \ldots + 10)^2$*. Since the area of the large square is the same whichever way we compute it, we obtain:*

$$1^3 + 2^3 + \ldots + 10^3 = (1 + 2 + \ldots + 10)^2$$

[2] Al-Karaji, late 10th – early 11th century, Persian mathematician.

To justify that $(1+2+\ldots+10)=\frac{10\times(10+1)}{2}$*, we use a rectangle of width 10 and length (10 +1), half of which is tiled as follows:*

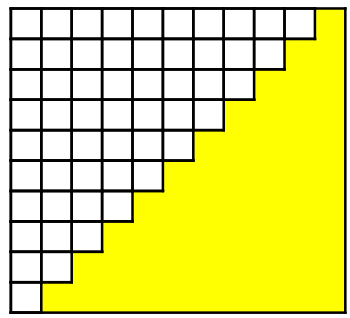


FIG. 8.5 – *Computing* $1+2+3+\ldots+10$

Another geometric justification of the equality $1^3+2^3+\ldots+10^3=(1+2+\ldots+10)^2$*, proposed by S. Golomb [14], is illustrated below:*

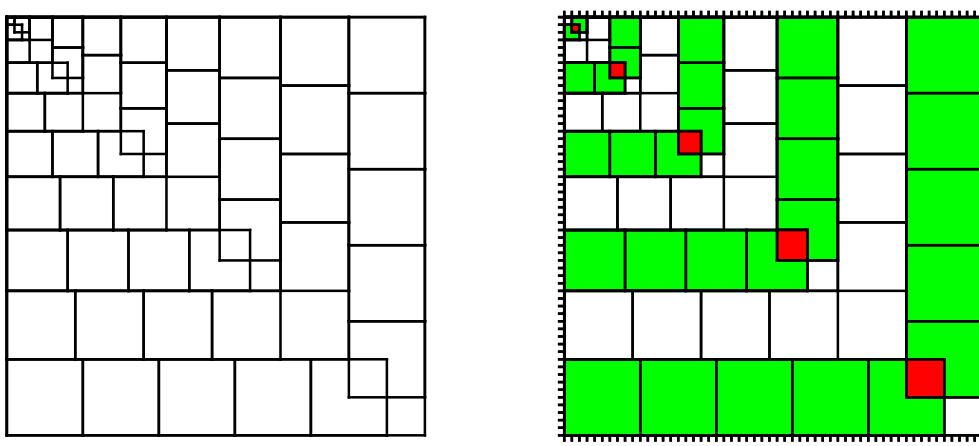


FIG. 8.6 – *Tiling of Golomb*

The length of the side of the large square is indeed $(1+2+\ldots+10)$*, so its area is* $(1+2+\ldots+10)^2$*. This area is equal to the sum of the areas of the squares making up the tiling, that is, 1 square of side 1, 2 squares of side 2, . . . , 10 squares of side 10. Note that for the squares with an even side length, shown in yellow in the figure, there is an overlap of two small squares, shown in red. But each of these small squares is adjacent to an uncovered region of the same size, shown in white. We can therefore remove one of the two*

overlapping small squares to fill the hole and obtain a perfect tiling. The area is therefore equal to $1^3 + 2^3 + \ldots + 10^3$. □

Addenda

To make learning the syntax of Python easier, we write integer expressions with parentheses. *Unary* integer expressions (with a single *operand* E) have the form $(-E)$ where $-$ is the minus sign *operator*. *Binary* integer expressions (with a left *operand* E_1 and a right operand E_2) have the form $(E_1\ \mathbf{op}\ E_2)$ where **op** is one of the *operators* of addition $+$, subtraction $-$, multiplication $*$, integer division `//` or remainder of integer division `%` (modulo). The operands E, E_1 and E_2 are an integer constant (positive such as 1, 17345 (without blanks), ... or negative such as -10, -5247, ...), a variable (for now a "for" loop counter such as `i`) or an integer *subexpression* of the form we have just described.

Computing the value of an expression is called *evaluating* the expression. The rules for evaluating fully parenthesized expressions are very simple. The value of a constant is the integer denoted by this constant. The value of a variable `i` used as a counter in the body C of a loop **for** `i` **in** `range(1,` n `+ 1):` C (where $n \geq 1$ is an integer constant) is successively 1, then 2, then ..., then n during the successive iterations of the loop. The value of an expression $(-E)$ is obtained by computing the value v of the expression E and then applying the minus sign to it. The value of an expression $(E_1\ \mathbf{op}\ E_2)$ is obtained by first computing the value v_1 of the expression E_1, then computing the value v_2 of the expression E_2, then computing the value $v_1\ \mathbf{op}\ v_2$ of the result. The same evaluation rules apply to the subexpressions E, E_1 and E_2.

To avoid cumbersome notation, in Python we can follow the common practice in mathematics of using *priority rules* (also called *precedence rules*) that allow us to remove some redundant parentheses. The evaluation rules for integer expressions are defined and illustrated below:

- In Python, as in mathematics, the integer expression $10 - 10 - 10$ is equal to $((10 - 10) - 10)$, that is, -10. Likewise $2 * 7 \,//\, 3 \,\%\, 2$ is equal to $(((2 * 7) \,//\, 3) \,\%\, 2)$, that is, 0. In both cases the rule is that operators of the same *priority* (namely $+$ and $-$ on the one hand; $*$, `//` and `%` on the other) are evaluated from left to right.
- The expression $8 + -2 * 3$ is equal to $(8 + ((-2) * 3))$, that is, 2. Likewise $1 - 5 \,//\, 2$ is equal to $(1 - (5 \,//\, 2))$, that is, -1. The rule is that when there are operators of different priorities, the operators of highest priority are evaluated first.

In Python the priorities are as follows:

Priority of integer operators		
$-$ (unary)	unary operators	high priority
$*$ // %	multiplicative operators	medium priority
$+$, $-$ (binary)	additive operators	low priority

- The expression $(8+-2)*3$ is equal to $((8+(-2))*3)$, that is, 18. Likewise $(1-5)\,//\,2$ is equal to $((1-5)\,//\,2)$, that is, -2. In both cases we begin by evaluating the subexpressions in parentheses.
- When the division of a negative number does not come out exactly, Python rounds the quotient down, that is, towards the smaller integer: $-7\,//\,2$ is equal to -4 (and not -3 as in OCaml or in PASCAL) and the remainder -7 % 2 is equal to 1 (and not -1 as in OCaml), so that we always have $((-7\,//\,2)*2)+(-7\,\%\,2)=-7$. With a positive divisor, the remainder % is therefore always positive or zero.

Equivalently, the order of evaluation of expressions can be defined by stating to which operators the operands of an expression are bound:

- An operand appearing between two operators of different priorities is bound to the operator of higher priority (so $2+3*5=(2+(3*5))$).
- An operand appearing between two operators of the same priority is bound to the operator on the left (so $2-3-5=((2-3)-5)$).
- Expressions in parentheses are evaluated before being used as operands.

The rules for evaluating expressions change from one programming language to another. For example, in APL [18], there are no operator priorities (because there are too many operators in the language to hope to remember them) and expressions are right-associative, so that $10-10-10=(10-(10-10))=(10-0)=10$. This rule, contrary to mathematical tradition, has the advantage that the grammatical analysis of expressions is greatly simplified for the computer.

Programming languages more classical than APL follow the mathematical convention of left-to-right evaluation with priorities.

However, some languages have a great many operators. For example, the C language [20] has 43 different operators divided into 15 levels of priority. The priorities of the operators are therefore not easy to remember.

Moreover, the rules change from one language to another, which makes life difficult for computer scientists who use several languages every day. To avoid mistakes they use redundant parentheses, which is what we advise beginners to do.

Finally, note that in Python integers are not limited: we can compute with integers of several hundred digits, the only limit being the size of the computer's memory. This is not the case in all languages. In PASCAL on the first personal computers, integers had to lie between -32768 and 32767. In OCaml, they lie between `min_int` $= -4611686018427387904$ and `max_int` $= 4611686018427387903$ and beyond that, the calculation gives a wrong result without warning: (`max_int`$+1$) is equal to `min_int`!

9

Calling procedures with parameters

9.1 Making the robot move forward and backward

We saw in chapter 5 that a procedure declaration gives a name to a sequence of commands that the $\mathcal{R}^{o}\underline{b}^{o}t$ can then execute once or several times when the procedure is called by its name. A procedure can have one or several *parameters*. In that case the procedure can perform actions that are similar but different when it is called with different parameters.

For example the procedure `fdn` (forward n times) tells the $\mathcal{R}^{o}\underline{b}^{o}t$ to move straight ahead one or several times by the length of the side of a square of the grid paper. It has one parameter which gives the number of times the robot must move forward. In Python, to call a procedure with a parameter, we write the name of the procedure followed by the value of the parameter between parentheses. '`fdn(1)`' is equivalent to '`fd()`' whereas '`fdn(4)`' is equivalent to '`fd(); fd(); fd(); fd()`'. When the parameter is zero, the command '`fdn(0)`' does not change the position of the $\mathcal{R}^{o}\underline{b}^{o}t$. Finally, when the parameter is a negative number, that is to say a number preceded by the minus sign −, the $\mathcal{R}^{o}\underline{b}^{o}t$ moves backward the given number of times without changing its direction. For example '`fdn(−4)`' is equivalent to '`ht(); fd(); fd(); fd(); fd(); ht()`'.

The parentheses must not be forgotten. If we write '`fdn 4`', Python does not understand and reports '`SyntaxError: invalid syntax`'. If we write

'`fdn −4`', Python understands that we want to subtract 4 from `fdn`, which makes no sense, and reports during execution '`TypeError: unsupported operand type(s) for -: 'function' and 'int'`' (an integer (`int`) cannot be subtracted from a procedure (`function`)). The parameter can also be an integer expression, as in '`fdn(2 * j)`': Python first computes the value of the expression `2 * j` and then calls `fdn` with this value. If we forget the parameter, writing '`fdn()`', Python reports '`TypeError: fdn() missing 1 required positional argument: 'n'`' (`fdn()` is missing a required parameter) and if we give too many, as in '`fdn(1, 2)`', it reports '`TypeError: fdn() takes 1 positional argument but 2 were given`' (`fdn()` takes one parameter but it was given two).

We can now understand the empty parentheses `()` that follow the commands of the *Robot* we have used so far, such as `fd()` or `qtr()`: in Python the parentheses order the call of the procedure, and they are empty for procedures that need no information to run. Beware: if we forget these empty parentheses, writing `fd` instead of `fd()`, Python reports no error but the procedure is not called and the *Robot* does not move forward!

Example 13 The program `OctagonalStripes` below draws the triangular stripes spread along the sides of an invisible octagon shown in figure 9.1 (page 109):

```
from LibRobot import *

# Draw stripes on the sides of an octagon
pu(); fdn(5); pd(); gs(4)
for i in range(8):
    # Draw 10 stripes of lengths 2, 4, ..., 20.
    for j in range(1, 11):
        # Draw a stripe of length (2 * j) centred
        # on a side of the octagon.
        fdn(j); fdn(-2 * j); fdn(j)
        # Move on to the next stripe (without drawing
        # the side of the octagon).
        pu(); qtr(); fd(); qtl(); pd()
    # Turn the robot perpendicular to the next
    # side.
    etr()
st()
```

□

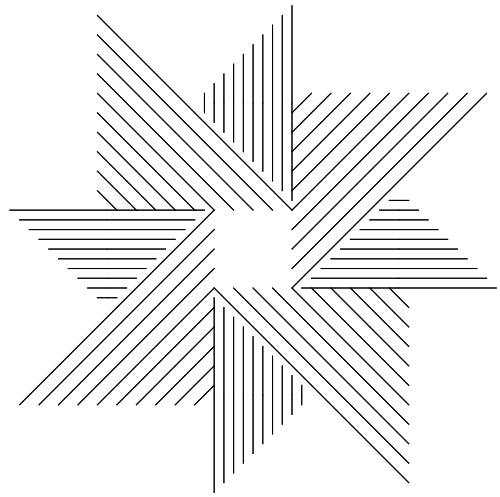
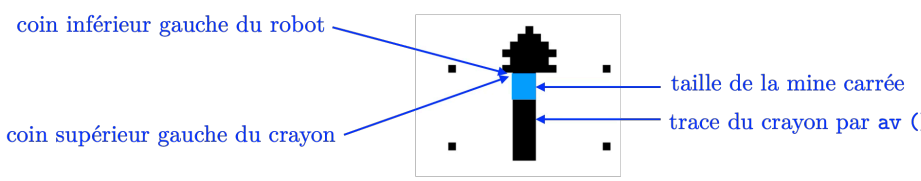


FIG. 9.1 – *Stripes on an octagon (scale $\frac{1}{3}$)*

9.2 Pen thickness

The command `pw` (pen width) sets the thickness of the pen of the *Robot*. Its integer parameter gives the thickness of the pen that is chosen. So far we have used a pen of thickness 1, which is the one the *Robot* uses by default, that is to say when it is not told otherwise. The command `pw(2)` makes it draw with a line twice as thick. The unit of length chosen to measure the thickness of the pen is the *pixel*, that is, let us recall, the size of the smallest dot that can be drawn on the computer screen.

When using a very thick pen, one must know that the pen of the *Robot* has a square tip and that the *Robot* holds this tip by its upper left corner.



Example 14 (Thick-tipped pen) The program below, `PenWidth`, lets us observe the effect of executing an `fd()` command with a standard grid of 12 pixels when the *Robot* uses successive pen thicknesses of 12, 24, 36 and 48

pixels:

□

```
from LibRobot import *
# Effect of fd () with thicker and thicker
# pens
pu(); fdn(5); qtr(); gsX(6); fdn(-1); gsX(12)
for i in range(1, 5):
   pu(); qtr(); fdn(i); qtl(); pd()
   pw(12 * i); fd(); pu(); fdn(-1)
dg(); st()
```



We observe, for example, that the line drawn when the *Robot* executes an `fd()` command with a 48-pixel pen has an apparent length of 5 times the length of the side of the grid. This is explained by the fact that in its final position the pen leaves a square mark with sides of 48 pixels below and to the right of its stopping point, that is 4 times the length of the side of the 12-pixel grid. □

9.3 Speed of the robot

A last example of a procedure with a parameter is the command `sp` (speed) to choose the speed of the *Robot*. This speed is an integer between 0 and 10. Speed 10 corresponds to the default maximal speed. So the command `sp(10)` is equivalent to the command `ex()` for normal execution. The speed `sp(1)` corresponds to the minimal speed. It lets us observe all the details of the movements of the *Robot* which, at this speed, are very slow. Finally speed 0 corresponds to execution in step-by-step mode. The command `sp(0)` is therefore equivalent to the command `ss()`. Step-by-step mode is used to find errors in programs. In this mode of execution, the commands of the *Robot* are displayed at the bottom of the screen and are executed by pressing a key (or by clicking the mouse button). The paths of the *Robot* with the pen up are shown as dotted lines, which makes it possible to follow its trail perfectly.

Exercise 19 *Write a Python program to draw:*

– *the raised capital letter H of figure 1.12 (page 9);*

– *the cube in perspective, the eight-branched crystal and the pair of glasses of figure 9.2 (page 111);*

– *the helicopter and the passenger train of figure 1.13 (page 10);*

– *the 'spirolateral' curves of figure 9.3 (page 111), obtained by making the Robot move forward once, then twice, then three times, etc., turning each time a quarter turn to the right and repeating the whole thing four times in a row ([30]);*

– *the arabesque and the Jacob's ladder of figure 9.4 (page 112), a Jacob's ladder that you can also try to make by weaving a string between your fingers.* □

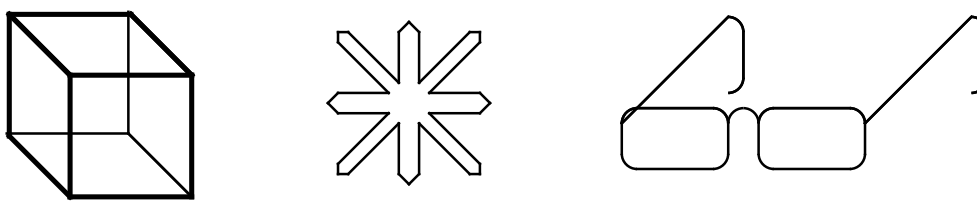


FIG. 9.2 – *Cube in perspective, eight-branched crystal and pair of glasses*

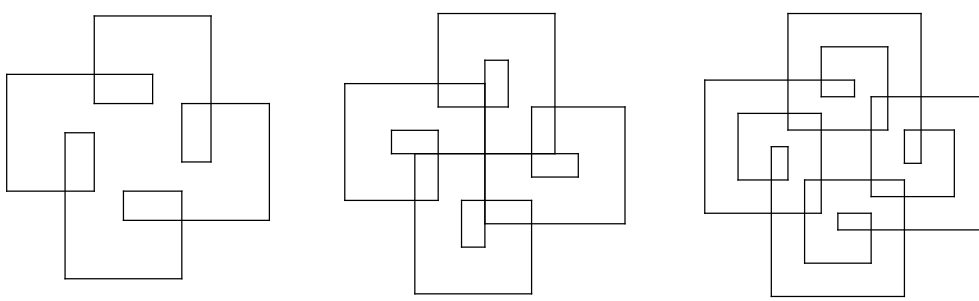


FIG. 9.3 – *90° spirolaterals of order 5, 7 and 9*

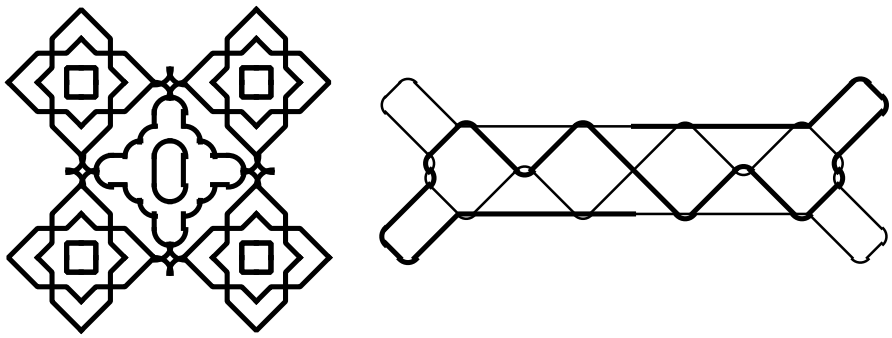


FIG. 9.4 – *Arabesque and Jacob's ladder*

Solution 19

```
from LibRobot import *
# Draw a 90-degree spirolateral of order 9 by
# Frank C. Odds.
pu(); fdn(-4); pd()
for i in range(4):
   for j in range(1, 10):
      fdn(j); qtr()
st()
```

□

Addenda

The use of parameters in procedures is one of the basic notions of programming: each call of the procedure depends on one or several values in terms of which the effect of the procedure is expressed. For the time being we only consider parameter passing *by value*, which allows information (one or several numbers in our case) to be passed to a procedure. This corresponds to the elementary notion of *function* (or *mapping*) in mathematics.

The use of subroutines with parameters in machine language is generally attributed to Alan Turing. The Swiss Heinz Rutishauser [35] and the Italian Corrado Böhm [5] introduced the notions of high-level programming language and of compiler to translate programs written in this high-level language into machine language (at the time people did not speak of a compiler but of *automatic coding*). They included the notion of subroutine with parameters in their languages without bringing it out very explicitly.

After these early theoretical contributions, the first compiler that actually worked, on the MARK I machine in Manchester, England, was built by Alick Glennie in 1952. The high-level language AUTOCODE used subroutines with parameters passed in the registers of the machine.

This work had no success because programming was relatively easy at the time compared with the effort needed to adapt programs to the malfunctions of the machine. Maurice Wilkes [44] introduced the use of libraries of procedures (like the ones we use for the commands of the $\mathcal{R}\text{o}\underset{\frown}{\text{b}}\text{o}t$) in 1951.

The first language in which these notions appeared in their modern form was FORTRAN (FORmula TRANslator), designed in 1954 by a team led by John Backus [1]. Thanks to very many modifications incorporating new programming concepts, to extensions taking into account recent machine architectures and to the wealth of available libraries, this language is still widely used for numerical scientific programming.

10

The robot's grid

So far we have drawn our pictures on paper ruled in small squares. The *Robot* has the additional ability to change the size of the squares. We can therefore enlarge the drawings by increasing the size of the squares before starting the drawing. It is also possible to change the size of the squares while the *Robot* is drawing. This makes it possible to draw segments of different lengths and arcs of circles whose radius can vary. By choosing rectangular cells we can draw ellipses and choose the angles of movement of the *Robot* freely.

10.1 Square grid

The standard grid shown in figure 2.2 (page 14) has a size of 12 pixels. We can change the length of the side of a square of the grid with the command `gs` (grid size), which takes as parameter the new length of the grid expressed in pixels. For example `gs(6)` sets a grid half the size of the usual grid whereas `gs(24)` doubles the size of the grid.

The length of the grid can also have a decimal part, as in `gs(7.5)` (seven and a half pixels). Such a number, which is called a *floating-point number*, is written with a point (and not with a comma as in French): `6.0` (or `6.`) is the floating-point number six whereas `6` is the integer six. The commands `gs`, `gsX` and `gsY` accept either an integer or a floating-point number: `gs(6)` and `gs(6.0)` have the same effect[1]. When the length of the grid is given by an

[1] This is not the case in all languages: in OCaml for example, one must write `gs 6.` with a floating-point number and convert integers into floating-point numbers.

integer expression such as (6 * i), we therefore simply write `gs(6 * i)`. We will study floating-point numbers in detail in the chapter devoted to them.

Example 15 (Superimposed squares) The program below draws superimposed squares, the *Robot* starting and ending at the lower left corner, facing north, the superimposition being obtained by choosing successive grid sizes of 6, 12, 18, ..., 84:

```
from LibRobot import *

# Draw overlapping squares by enlarging the
# grid
def square():
   fd(); qtr(); fd(); qtr(); fd(); qtr(); fd()
   qtr()

gb(); gl()
for i in range(1, 15):
   gs(6 * i); square()
st()
```

□

10.2 Rectangular grid

The *Robot* can walk on a rectangular grid whose horizontal side has a length different from that of the vertical side. This allows it to draw rectangles and ellipses like these:

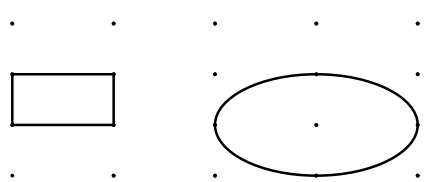


FIG. 10.1 – *Rectangle and ellipse on a* 40×20 *grid*

The initial grid has a size of 12×12 counted in pixels. The command `gsX(40)` gives a length of 40 pixels to the horizontal side of the grid without changing the length of the vertical side. After this command the grid therefore has a size of 40×12 pixels. In this notation 40×12 we give first the length of the horizontal side (called X) then the length of the vertical side (called Y) of the grid. The command `gsY(20)` gives a length of 20 pixels

to the vertical side of the grid without changing the length of the horizontal side. After this command the grid therefore has a size of 40×20 pixels. So we can change independently the horizontal size (with `gsX`) or the vertical size (with `gsY`) of the grid. We can return to a square grid by using the command `gs`. For example the command `gs(12)` returns to a standard 12×12 square grid. The rectangle and the ellipse above are built by the following program:

```
from LibRobot import *
# Draw a rectangle and an ellipse side by side.
# Size of the grid
gsX(40); gsY(20); gl()
# Grid
dg()
# Rectangle
fd(); qtr(); fd(); qtr(); fd(); qtr(); fd()
qtr()
# Shift to the right.
pu(); qtr(); fd(); fd(); qtl(); pd()
# Ellipse
cr(); cr(); cr(); cr()
st()
```

The figure below illustrates the effect of the commands `cr()`, `fd()` and `cl()` with a rectangular grid of dimensions 30×60.

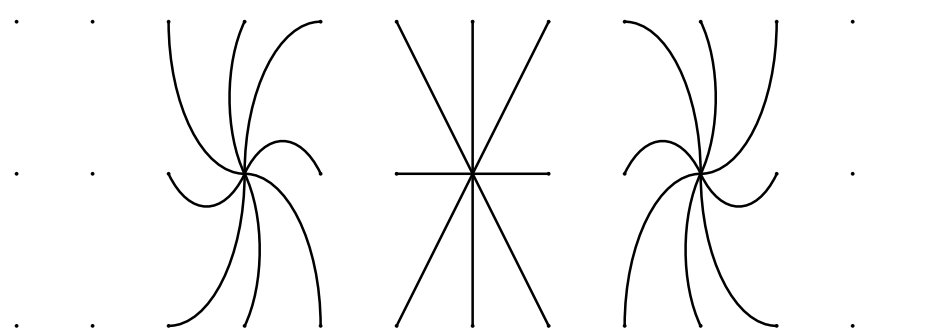


FIG. 10.2 – *Curve to the right, move forward and curve to the left with a* 30×60 *grid*

Example 16 The various segments and arcs of ellipses drawn by the commands `fd()`, `cl()` and `cr()` are illustrated by the two programs below, which

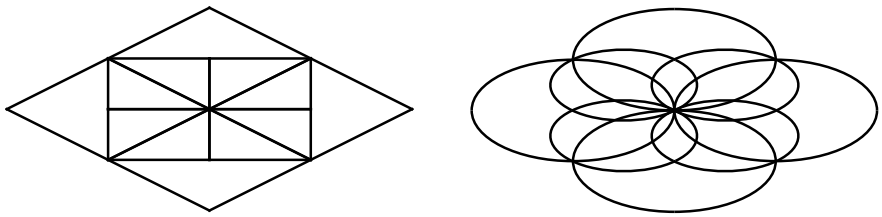


FIG. 10.3 – *Parallelograms and ellipses*

draw parallelograms and ellipses by turning them a quarter turn around the centre of the grid, as can be seen in figure 10.3 (page 118):

- **from** LibRobot **import** *

```
# Draw parallelograms on a rectangular
# grid
# Set the size of the grid to 40 x 20.
gsX(40); gsY(20)
# Draw eight parallelograms, turning their bottom
# left corner by an eighth turn around the
# centre of the grid.
for i in range(8):
   fd(); qtr(); fd(); qtr(); fd(); qtr(); fd()
   etr()
st()
```

- **from** LibRobot **import** *

```
# Draw ellipses on a rectangular grid
# Set the size of the grid to 40 x 20.
gsX(40); gsY(20)
# Draw eight ellipses, turning their major axis
# by an eighth turn around the centre of
# the grid.
for i in range(8):
   etr(); cr(); cr(); cr(); cr()
st()
```

□

Exercise 20 *Use the possibilities offered by changing the size of the square grid to draw:*

- *the spirals and concentric circles of figure 10.4 (page 119);*
- *the frieze of figure 10.5 (page 120);*
- *the propeller and the ball of figure 10.6 (page 120);*
- *the snail of figure 10.7 (page 120);*
- *the stegosaurus and its baby of figure 10.8 (page 121);*
- *the family of dogs of figure 10.9 (page 121);*
- *the tiling of figure 10.13 (page 123) inspired by a motif by van de Vecht ([26], page 26) shown in figure 10.10 (page 121).* □

Exercise 21 *Use the possibilities offered by rectangular grids to draw:*

- *the frieze of overlapping diamonds of figure 10.11 (page 122);*
- *the bicycle of figure 10.12 (page 122);*
- *the eight-point compass rose 2.1 (page 14);*
- *the tilings of figures 10.14 (page 124), 10.15 (page 125) and 10.16 (page 126) obtained from the tiling of figure 10.13 (page 123) by progressively decreasing the size of the grid on the right-hand side of the drawing, on the left-hand side and, in the last case, on all four sides of the rectangle, in the manner of the Dutch engraver M. C. Escher ([26], [11]).*

□

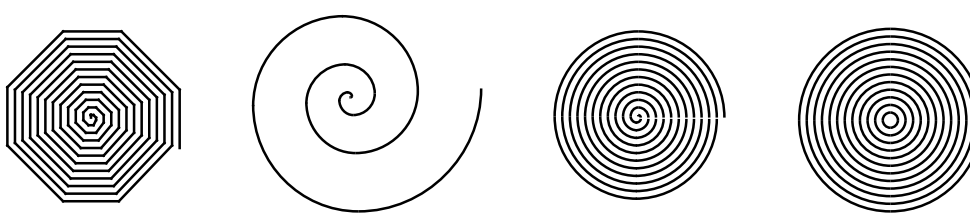


FIG. 10.4 – *Spirals and concentric circles*



FIG. 10.5 – *Double notched frieze*

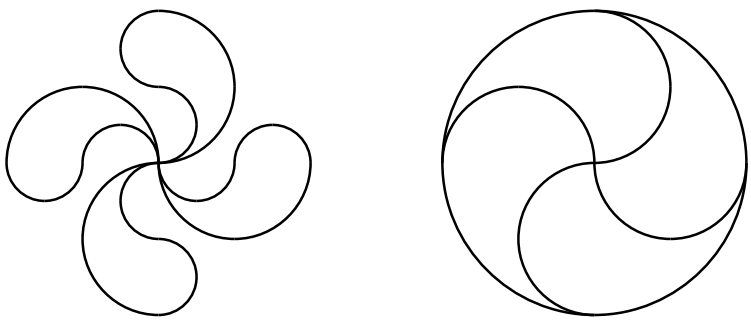


FIG. 10.6 – *Propeller and ball*

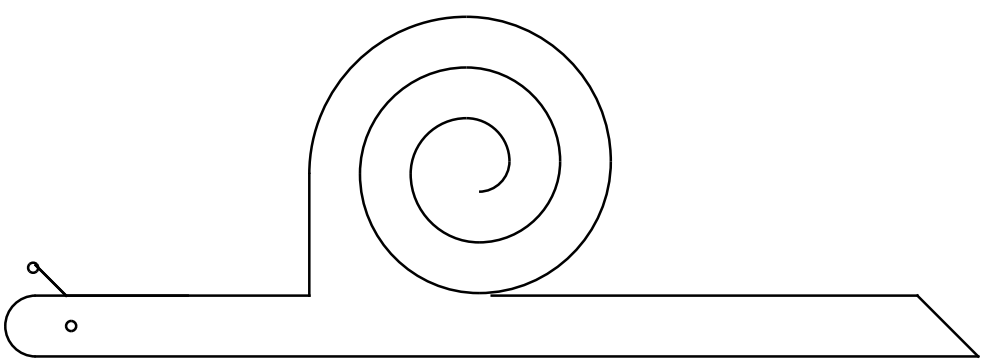


FIG. 10.7 – *Snail*

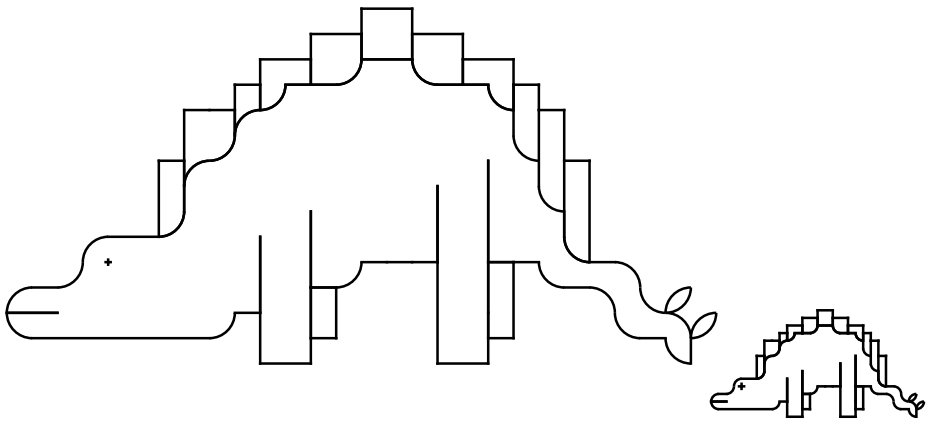


FIG. 10.8 – *Stegosaurus and its baby*

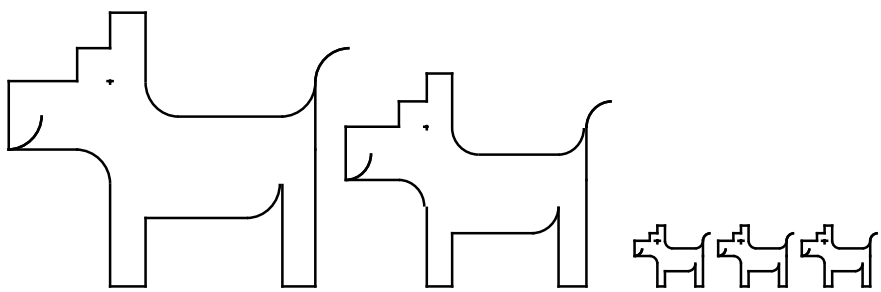


FIG. 10.9 – *Family of dogs out for a walk*

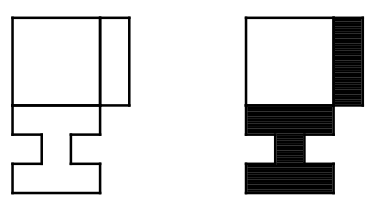


FIG. 10.10 – *Ornamental motif by N. J. van de Vecht*

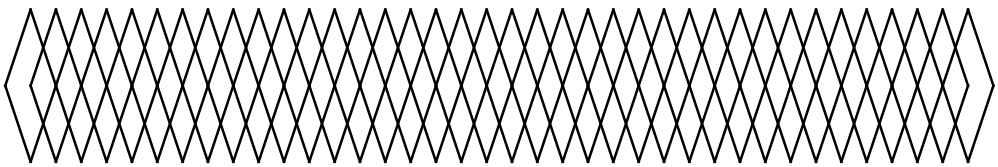


FIG. 10.11 – *Frieze of overlapping diamonds*

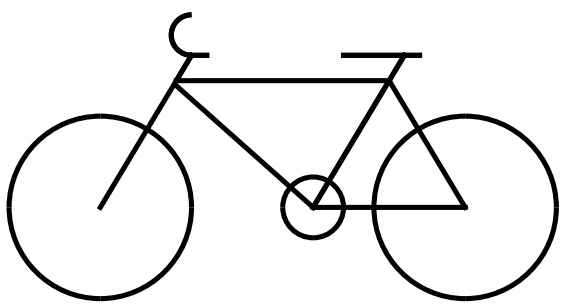


FIG. 10.12 – *Bicycle*

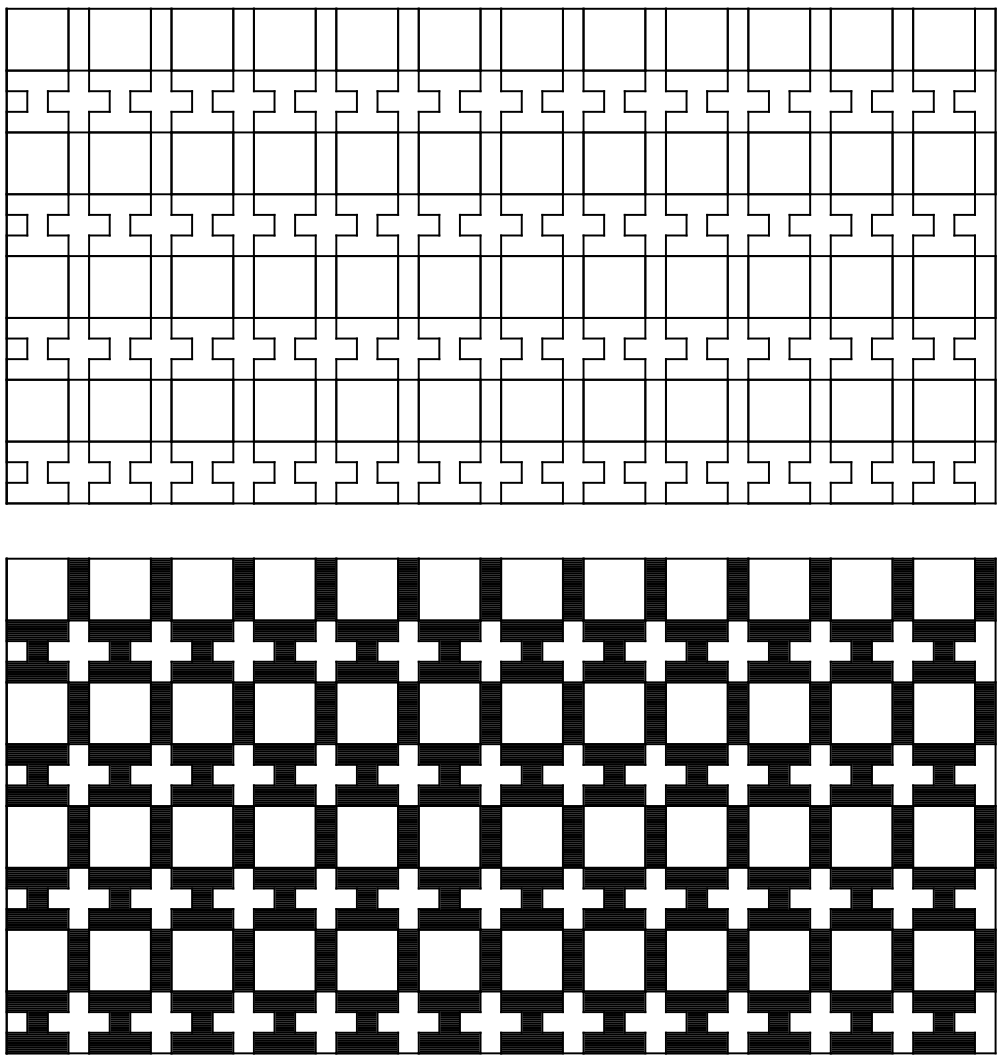


FIG. 10.13 – *Regular tiling inspired by an ornamental motif* by N. J. van de Vecht

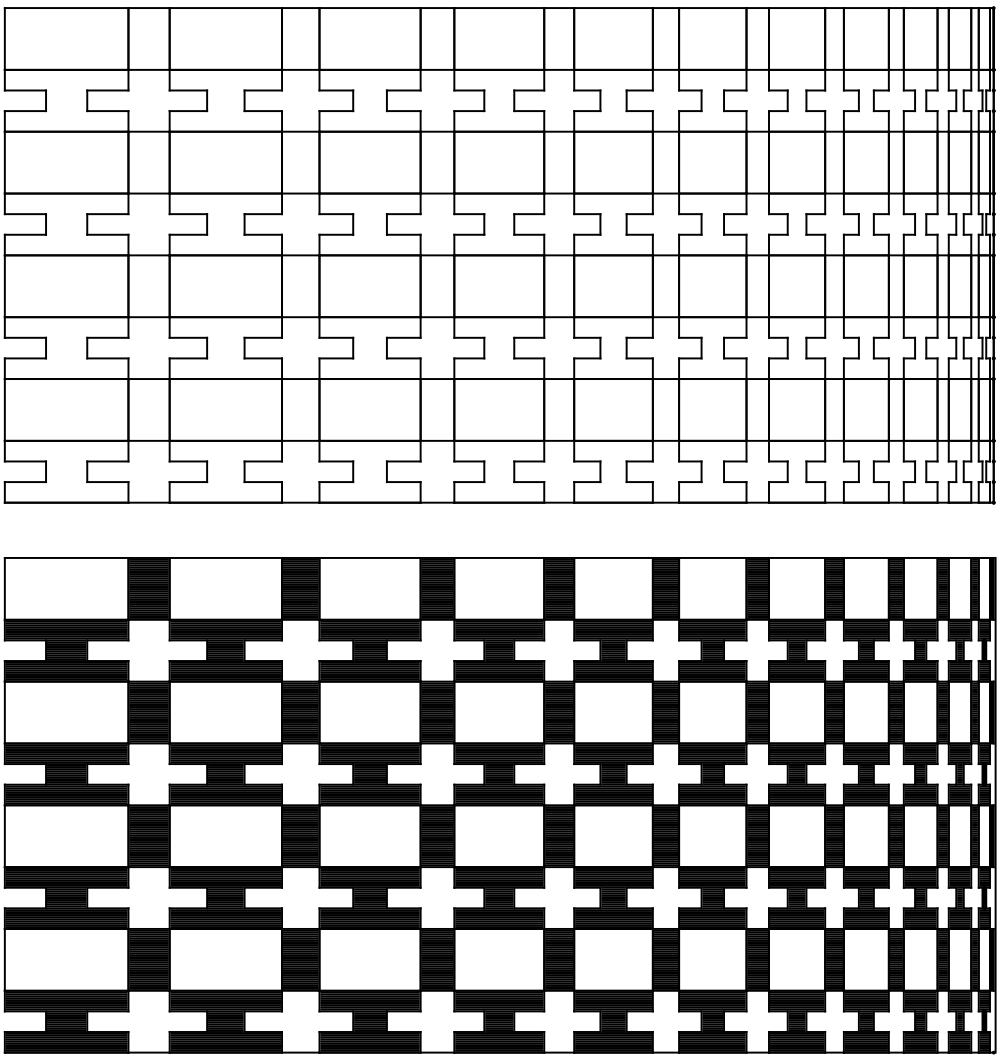


FIG. 10.14 – *Rectangular tiling with one limit*

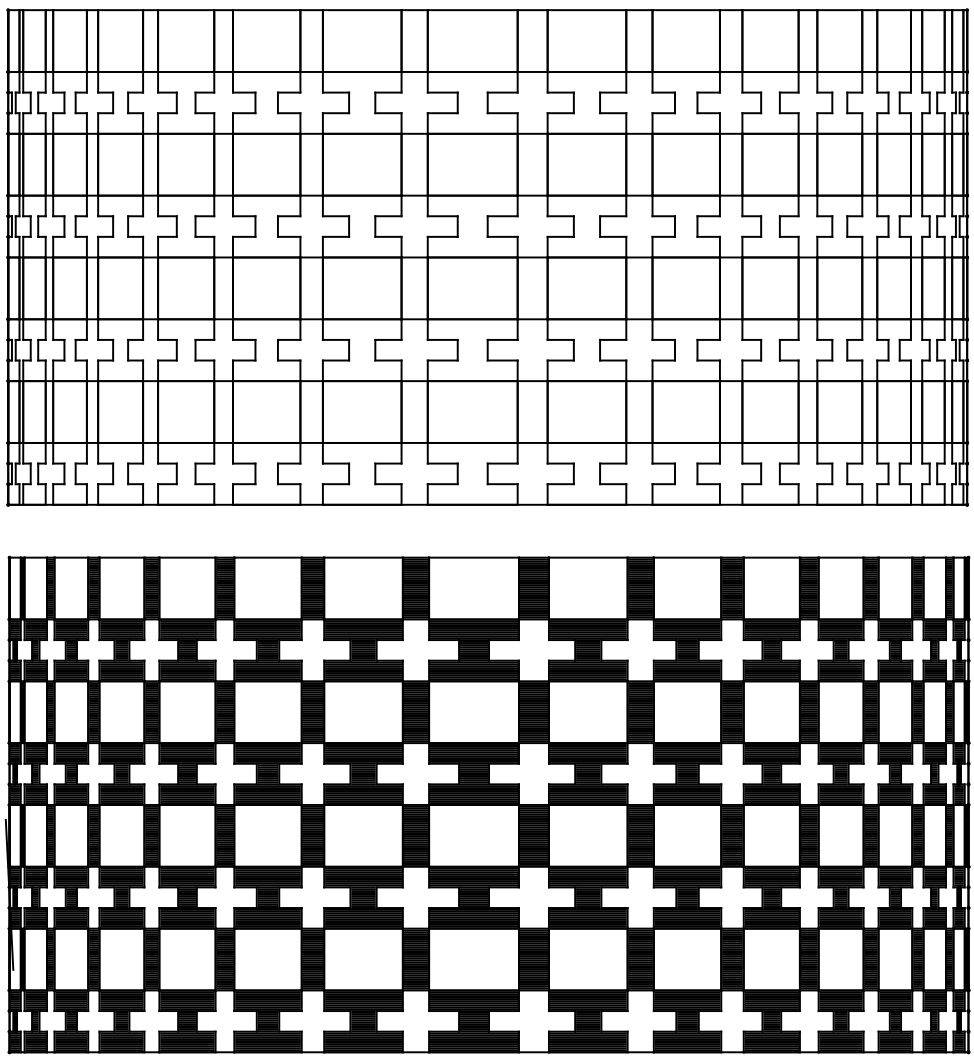


FIG. 10.15 – *Rectangular tiling with two limits*

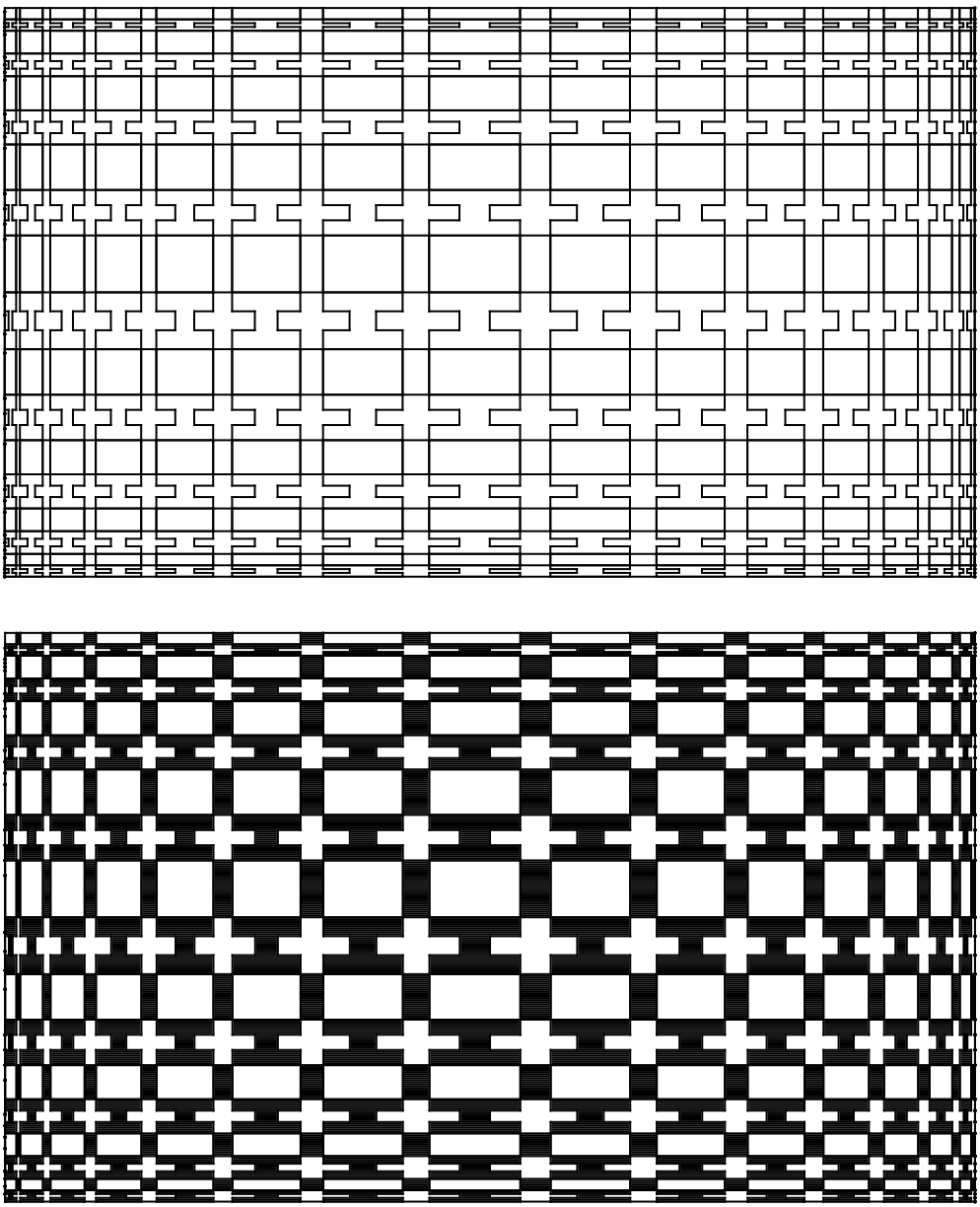


FIG. 10.16 – *Rectangular tiling with four limits*

Solution 20 *We give a few programs; the other programs are in the folder* `book/10_grid`*:*

- ```
  from LibRobot import *
  # Draw a spiral of circular arcs of radii
  # 1, 4, 9, 16, ...
  ht()
  for i in range(1, 13):
     gs(i * i); cl()
  st()
  ```

- ```
  from LibRobot import *
  # Draw a snail
  # Go up to the centre of the shell
  pu(); fdn(4); pd(); qtr()
  # Shell
  for i in range(1, 12):
     cl(); gs(12 + 5 * i)
  gs(12); fdn(4)
  # Body
  qtr(); fdn(9); cl(); cl(); fdn(31); ht(); etr()
  fdn(2); etl(); fdn(14)
  # Go to the antenna
  pu(); fdn(10); pd(); fdn(4)
  # Antenna
  etr(); fd(); gs(2); cl(); cl(); cl(); cl()
  gs(12); ht(); fd()
  # Eye
  etr(); pu(); fd(); pd()
  gs(2); cl(); cl(); cl(); cl()
  st()
  ```

□

Solution 21 *We give the most difficult program, namely the tiling of figure 10.16 (page 126) obtained from the tiling of figure 10.13 (page 123) by progressively decreasing the size of the grid on all four sides of the rectangle. The other programs are in the folder* `book/10_grid`*:*

```
from LibRobot import *
# Irregular tiling in the style of M. C. Escher,
# inspired by a pattern of N. J. van de Vecht.
# Pattern of N. J. van de Vecht, the robot starting
# at the top left and arriving at the bottom left
```

```
# facing north.
def pattern():
   qtr(); fdn(4); qtr(); fdn(3); qtr(); fd(); qtr()
   fdn(3); ht(); fdn(3)
   fd(); qtr(); fd(); qtl(); fd(); qtl(); fd()
   qtr(); fd(); qtr(); fdn(3)
   qtr(); fd(); qtr(); fd(); qtl(); fd(); qtl()
   fd(); qtr(); fd(); qtr(); fdn(3)
   ht(); fdn(3); qtr(); fdn(3); pu(); ht(); fdn(6)
   ht(); pd()

# Draw a column of patterns of heights 2, 4, 6,
# 8, 10, 8, 6, 4, 2, the robot starting at the top left
# and arriving at the bottom right (that is, at the
# bottom left of the next column) facing north.
def column():
   gsY(1); gt()
   for i in range(1, 6):
      gsY(2 * i); pattern()
   for i in range(1, 5):
      gsY(10 - 2 * i); pattern()
   qtr(); fdn(4); qtl()

# Draw an irregular tiling made of columns
# of widths 0, 2, 4, 6, ..., 14, 16, 14, ..., 2, 0.
def tiling():
   gsX(1); gl(); gsX(0)
   for i in range(1, 9):
      column(); gsX(2 * i)
   for i in range(1, 9):
      column(); gsX(16 - 2 * i)
   column()

tiling(); st()
```

□

Addenda

If all the grids of stopping points of the $\mathcal{R}^{o}\underline{b}^{o}t$ were aligned on the centre of the screen, it could happen that after a change of grid the $\mathcal{R}^{o}\underline{b}^{o}t$ would no longer be on a point of the grid. This is why the grid is always relative to the $\mathcal{R}^{o}\underline{b}^{o}t$ and not to the screen. When the size of the grid changes, all the points of the grid (which can be seen with the command `dg()`) move, except the one on which the $\mathcal{R}^{o}\underline{b}^{o}t$ stands. Consequently no command of the $\mathcal{R}^{o}\underline{b}^{o}t$ (except one, namely `gc()`) can make it leave one of the points of the current grid. For example the commands `gl()`, `gr()`, `gt()` and `gb()` move the $\mathcal{R}^{o}\underline{b}^{o}t$ horizontally or vertically to the point of the current grid (respectively to the left, to the right, at the top and at the bottom) closest to the edge. To be able to place the $\mathcal{R}^{o}\underline{b}^{o}t$ anywhere on the screen precisely, one needs an absolute reference point. This absolute reference point is the centre of the screen. The command `gc()` always brings the $\mathcal{R}^{o}\underline{b}^{o}t$ back to the same point, located exactly at the centre of the screen. This command is therefore the only command of the $\mathcal{R}^{o}\underline{b}^{o}t$ that is not performed on the same grid. If the centre of the screen is not on the current grid, the command `gc()` recentres the grid exactly at the centre of the screen so as to avoid making the $\mathcal{R}^{o}\underline{b}^{o}t$ leave its grid of movement.

To avoid possible misalignment problems, it is enough to program the drawing as if it were on graph paper, choosing a minimal grid size (1 millimetre on graph paper) so that all the changes of grid size are made when the $\mathcal{R}^{o}\underline{b}^{o}t$ is at the centre and correspond to multiples (5, 10, ... on graph paper) of the basic size. One can also proceed empirically by experimenting, which is sometimes more fun.

11

Constants

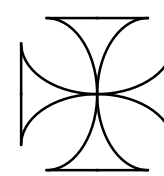
The number 1066 immediately reminds us of the battle of Hastings and the victory of William the Conqueror, Duke of Normandy; 1776 of the independence of the United States; 1789 of the French Revolution. When we read 3.14159 we know that it is an approximate value of π (written, as in English, with a decimal point, where French uses a comma). On the other hand, when we read the digit 2, we think of nothing in particular and we are almost certain to wonder "two what?". All this is to say that when we see a number in a program it is not always easy to understand immediately what it is about. To make it clear, we can, in Python, name the number with a constant declaration placed at the beginning of the program:

```
frenchRevolution = 1789  # Date of the revolution
```

and then use the name `frenchRevolution` instead of the number 1789 in the rest of the program.

Example 17 (Maltese cross) In the program below, which draws a Maltese cross, we use a constant `length` to define the length of an arm (that is to say the distance from its base to the centre of the cross) and another one, `width`, to define the width of an arm (that is to say the distance between the ends of its base):

```
from LibRobot import *
# Draw a Maltese cross
width = 20  # Width of an arm
length = 30  # Length of an arm
# Draw a vertical arm with the robot
```



```
# facing north or south
def verticalArm():
   gsX(width); gsY(length)
   cl(); ht(); fd(); fd(); ht(); cl(); qtl()

# Draw a horizontal arm with the robot
# facing east or west
def horizontalArm():
   gsX(length); gsY(width)
   cl(); ht(); fd(); fd(); ht(); cl(); qtl()

def malteseCross():
   for i in range(2):
      verticalArm(); horizontalArm()

malteseCross(); st()
```

To choose the best possible shape for this cross, there is only one place in the program where these constants need to be changed. For example, to obtain the smallest cross shown in figure 11.1 below, it is enough to take:

```
width = 10  # Width of an arm
length = 15  # Length of an arm
```

□

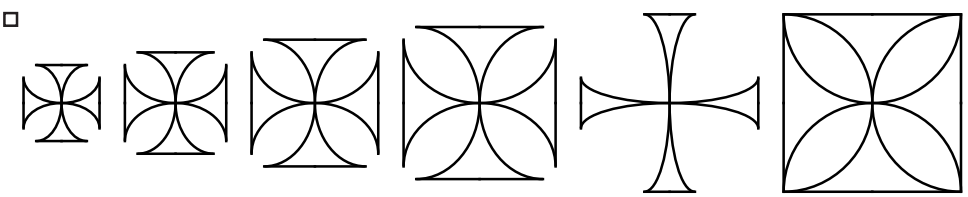


FIG. 11.1 – *Maltese crosses* 10×15, 15×20, 20×25, 25×30, 10×35, 35×35

□

Note that the numbers `20` and `30` (or `10` and `15`) are *integer* numbers. They are used as the lengths of the sides of the squares of the grid in the commands `gsX` and `gsY` which, like the command `gs`, also accept numbers with a decimal part, written with a point, such as `20.5`, which are called *floating-point* numbers. So we could just as well have written '`width = 20.5`'. Integers (such as `20`) and floating-point numbers (such as `20.5` or `20.0`) are two different kinds of numbers for Python, but it mixes them without difficulty in computations. We will see why they are different in chapter 19 on rational expressions.

We indicate that we use one or several *named constants* by *constant declarations*:

Constant declaration:

identifier1 = number1
identifier2 = number2
...

Each constant declaration is written on one line, at the beginning of the line (without indentation), like '$x = 3$' in mathematics. It is customary to put a space ␣ before and after the = sign, but it is not compulsory. As for procedure names, the identifier must follow the rules for identifiers of Python. The custom is to start it with a lower-case letter. Upper-case and lower-case letters are distinguished: if we declare '`Width = 20`' and then use `width`, Python answers '`NameError: name 'width' is not defined. Did you mean: 'Width'?`'. After these declarations, we can use the constant names 'identifier1' instead of 'number1' and 'identifier2' instead of 'number2', etc.

To tell the truth, Python has no real constants: nothing prevents us from giving, further on in the program, another value to the name `width` (we will see in chapter 22 that this is how *variables* are used). We speak of a constant because the program gives a value to this name only once, at the beginning, and never changes it afterwards.

In a Python program, a declaration defines constants (with the = sign) or procedures (with the word **def**). We will see later that it can also define functions and variables:

Declaration:

Constant declaration

or

Procedure declaration

Constant or procedure declarations are placed, in any order, before the instructions of the program:

Python program:

```
from ␣ LibRobot ␣ import *
# Comment explaining what the program does
... Declarations of the program ...
... Instructions of the program[a] (one per line or separated by ';') ...
```

[a] Using the declared constants and procedures.

The main rule to follow is that every identifier must be declared before it is used (otherwise, when the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ reaches the instruction that uses it, Python stops with a `NameError` error).

Exercise 22 *Write Python programs with named constants to draw:*

- *the singular propeller of figure 11.2 below;*
- *the duckling of figure 11.3 below.*

□

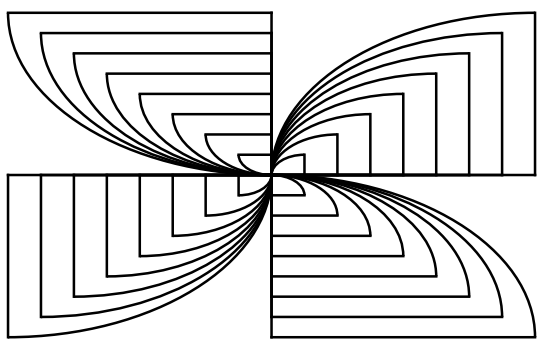


FIG. 11.2 – *Singular propeller*

Solution 22 *We give one program; the other one is in the folder* `book/11_constants`. *The horizontal side of the squares of the grid of the i-th leaf of a blade is* `xbase * i`. *It is an integer that can be given directly to the command* `gsX`*:*

```
from LibRobot import *
# Draw a singular propeller
```

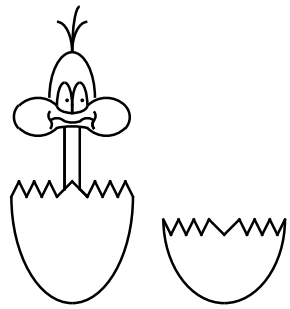


FIG. 11.3 – *Duckling hatching from its egg*

```
xbase = 13  # Width of the base grid
ybase = 8  # Height of the base grid
numberOfLeaves = 8  # Leaves of a blade
numberOfBlades = 4  # Blades of the propeller
def blade():
   for i in range(1, numberOfLeaves + 1):
      # Draw a quarter of an ellipse, of size
      # (xbase * i, ybase * i), without changing
      # the initial orientation of the robot
      gsX(xbase * i)
      gsY(ybase * i)
      fd(); qtl(); fd(); qtl(); cl(); qtl()
for j in range(numberOfBlades):
   blade(); qtr()
st()
```

□

Addenda

Computer integers are an approximation of the integers of mathematics (but not too large ones). Floating-point numbers (more precisely floating-point numbers) are a subset of the rational numbers (and therefore of the real numbers) written in floating-point notation.

A *constant declaration* `name = number` gives a name to a particular number. Afterwards the name, which is more meaningful, can be used instead of the number. The immediate advantage is that to change the number it is enough to do so in the constant declaration, whereas otherwise one has to look for all its occurrences in the program.

This can be compared with the notion of variable in mathematics, that is to say with the use of letters x, y, z ... to denote arbitrary numbers. The difference is that in Python the name of a constant denotes a particular value whereas in mathematics it is an arbitrary value.

Historically, the first mathematical problems dealt with by the Babylonians (1800 years BC) included an algorithmic solution given for one example and separate numerical tables to solve other similar problems [34]. In the Middle Ages people used rhetorical language such as 'a number added to its square makes 12' for '$x + x^2 = 12$'. Chuquet[a], author of the oldest treatise on algebra written by a Frenchman (1484), used 1^1 to denote the unknown, 1^2 to denote its square and the symbols $\bar{p}$ and $\bar{m}$ for addition and subtraction. He could write '$12\ \bar{p}\ 3^1$ egaulx a 4^2' (Middle French for "equal to") for '$12 + 3x = 4x^2$'. Viète[b] took an important step towards symbolism in algebra by introducing the use of letters to denote unknown quantities. Around 1590 he wrote: '1 QC $-$ 15 QQ + 85 C + 225 Q + 274 N equal 120', that is, if C means cube, hence x^3, Q means square, hence x^2, and N means number, hence x, the equation $x^5 - 15x^4 + 85x^3 + 225x^2 + 274x = 120$ ([4]). It took four French mathematicians, Desargues[c], Descartes[d], Fermat[e] and Pascal[f] (the latter having been chosen by Wirth to name his programming language PASCAL), to enter the modern era of mathematics.

---

[a] Nicolas Chuquet, French mathematician, c. 1445–1500.
[b] François Viète, French mathematician, 1530–1603.
[c] Girard Desargues, French mathematician, 1591–1661.
[d] René Descartes, French mathematician, 1596–1650.
[e] Pierre de Fermat, French mathematician, 1601–1665.
[f] Blaise Pascal, French mathematician, 1623–1662.

12

Painting

12.1 Black and white

The $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ can colour in its drawings. We will start with black and white colouring. The command `paint()` lets black paint escape from the back of the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$. The paint spreads over the largest possible part of the screen bounded by a black line. An example is given in figure 12.1 below. We start

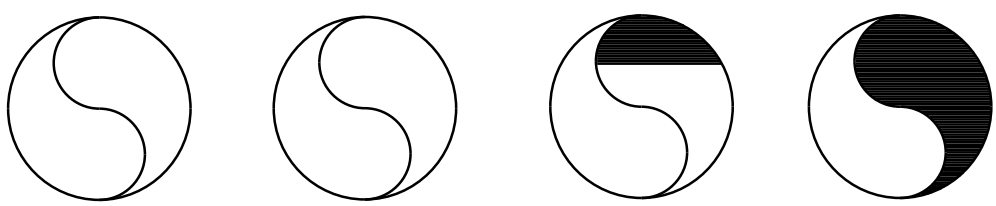


FIG. 12.1 – *Steps in painting a drawing*

by drawing the outline of the area to be painted, then we place the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ in the area to be painted or on its edge so that the back of the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$, through which the paint escapes, is in the area to be painted. The paint spreads line by line above, to the left of, to the right of and below the back point of the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$. The area covered is the largest one containing the back of the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ and bounded by a continuous black line. The $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ can paint without moving, whether its pen is up or down.

Example 18 (Yin Yang) The program below draws the Chinese Yin Yang symbol:



```
from LibRobot import *
# Draw the Chinese Yin Yang symbol
gridSize = 3
def circle():
   cr(); cr(); cr(); cr()
# Lower half circle
gs(6 * gridSize)
qtr(); cl(); cl()
# Upper half circle
gs(6 * gridSize)
cr(); cr()
# Outer circle
gs(12 * gridSize)
circle()
# Small white circle
gs(2 * gridSize)
pu(); qtr(); fdn(2); qtl(); pd(); circle()
# Paint the black area
pu(); qtr(); fdn(4); paint()
# Small black circle
fdn(2); qtl(); pd(); circle(); qtl(); paint()
st()
```

□

A frequent mistake is to leave a hole in the boundary of the area to be painted through which the paint will leak. A hole of a single pixel is enough to cause a leak. In case of a leak, we can stop the painting by pressing one of the keys esc or § or #. The *Robot* goes on drawing. Press one of these keys at least a second time to stop it for good.

12.2 Colour

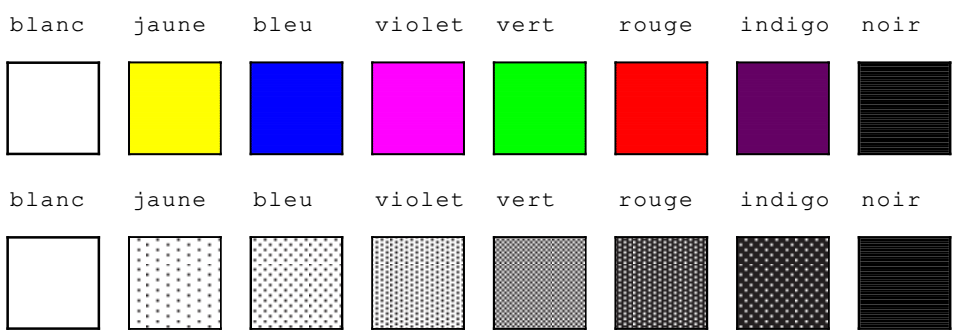
To draw and paint, the *Robot* has pens in the colours white, yellow, blue, violet, green, red, indigo and black. The command `pc` (pen colour) chooses the colour of the pen, which is given as a parameter. For example, after the command `pc(red)` all lines and painting are in red. The command `pc(black)` returns to black lines and painting. The colour names `white`, `yellow`, `blue`, `violet`, `green`, `red`, `indigo` and `black` are constants declared by `LibRobot`,

which stand for the colour numbers 0, 1, 2, ..., 7 (in this order): `pc(red)` is therefore the same thing as `pc(5)`, but much more readable. As we saw in chapter 9, the command `pw(n)` chooses the thickness of the pen; it is independent of its colour.

On a colour screen, the *Robot* draws and paints with the colours of the first line below.

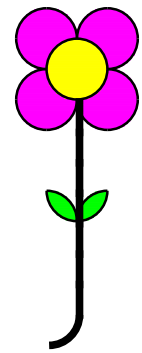
When we do not have a colour screen but a screen with several levels of grey, each pixel of which can be more or less dark, the colours are represented by greys, in the order white, yellow, blue, violet, green, red, indigo and black from lightest to darkest.

When we only have a black and white screen, the colours are represented by grey shades made of patterns of white dots and black dots side by side. The order is again white, yellow, blue, violet, green, red, indigo and black from lightest to darkest. The darker the shades, the denser the small dots used to represent them on the screen, as can be seen on the second line below:



When painting, the edges of the pattern must be drawn with no possible leak, in any colour except white.

Example 19 (Flower in colour) The program below paints the flower opposite in colour:



```
from LibRobot import *
# Draw and paint a stylized flower
gridSize = 12
stemThickness = 3
edgeThickness = 1
gb(); gs(gridSize)
# Bottom of the stem
pw(stemThickness); qtr(); cl()
fd(); fd(); fd()
# Right leaf:
# draw the outline
pw(edgeThickness); cr(); qtr(); cr()
# paint the inside
pc(green); etl(); paint(); tetr()
# Left leaf:
# draw the outline
pc(black); cl(); qtl(); cl()
# paint the inside
pc(green); etr(); paint(); tetl()
# Top of the stem
pc(black); pw(stemThickness)
fd(); fd(); fd(); fd()
# Centre:
# draw the outline of the centre
pw(edgeThickness); qtr()
cl(); cl(); cl(); cl()
# petals
pc(black); qtr(); cl(); cl(); cl()
pc(violet); etr(); paint(); etr()
pc(black); qtr(); cl(); cl(); cl()
pc(violet); etr(); paint(); etr()
pc(black); qtr(); cl(); cl(); cl()
pc(violet); etr(); paint(); etr()
pc(black); qtr(); cl(); cl(); cl()
pc(violet); etr(); paint(); etr()
# paint the inside of the centre
qtr(); pc(yellow); paint()
st()
```

□

Exercise 23 *Write Python programs to draw and colour in black and white:*

- *the tilings of the square of figure 12.2 below;*
- *the mosaics of the Hall of the Abencerrages (14th century) of the Alhambra, the residence of the Moorish kings in Granada, Andalusia, as they were copied by the artist Escher in 1936 [26] (figure 12.3 on page 141).*

□

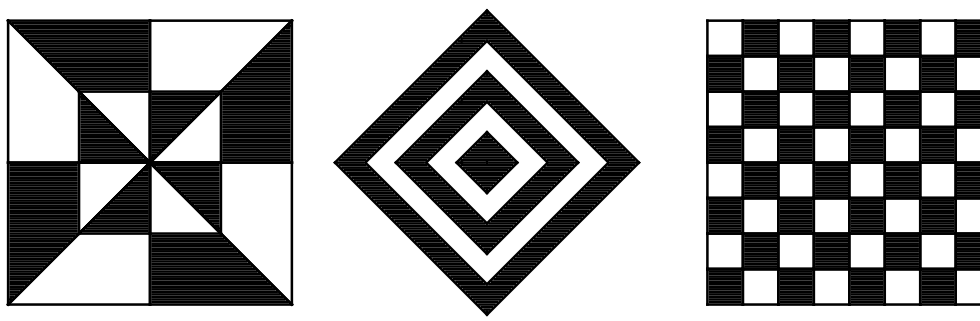


FIG. 12.2 – *Tilings of the square*

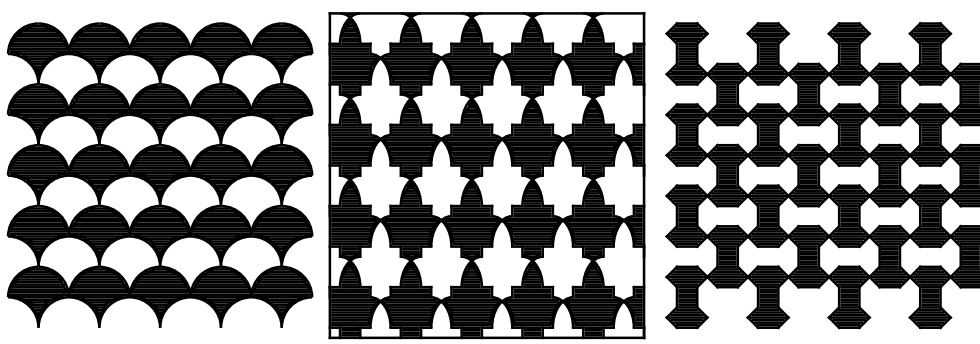


FIG. 12.3 – *Mosaics of the Alhambra*

Exercise 24 *Write Python programs to draw and colour in colour:*

- *the ball and the three-colour tiling of figure 12.4 (page 142);*
- *the mosaic of the Alhambra [26] of figure 12.5 (page 142);*

– *the tiling of disjoint squares of figure 12.6 (page 143);*

– *the tilings of figures 10.13 (page 123), 10.14 (page 124), 10.15 (page 125) and 10.16 (page 126) using the ornamental motif by N. J. van de Vecht as it is painted in figure 10.10 (page 121).*

□

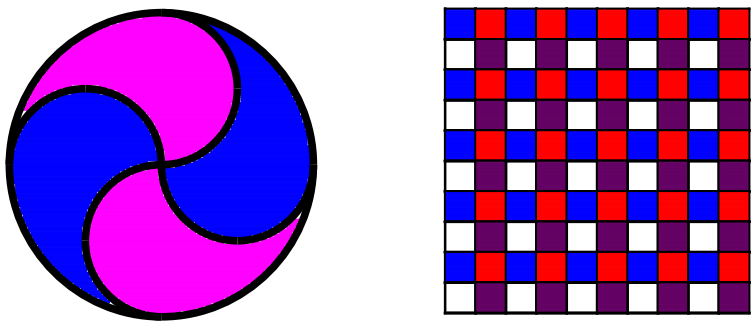


FIG. 12.4 – *Ball, three-colour tiling*

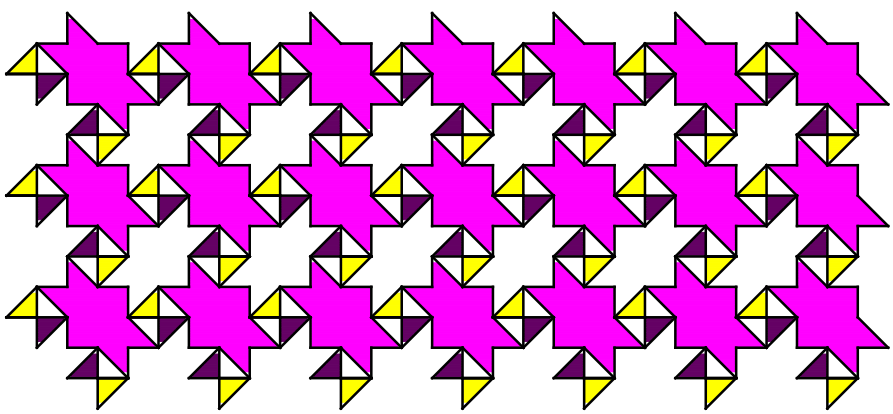


FIG. 12.5 – *Mosaic of the Alhambra in colour*

Exercise 25 *What is the motif used to make the black and white tiling of figure 12.7 (page 144)?* □

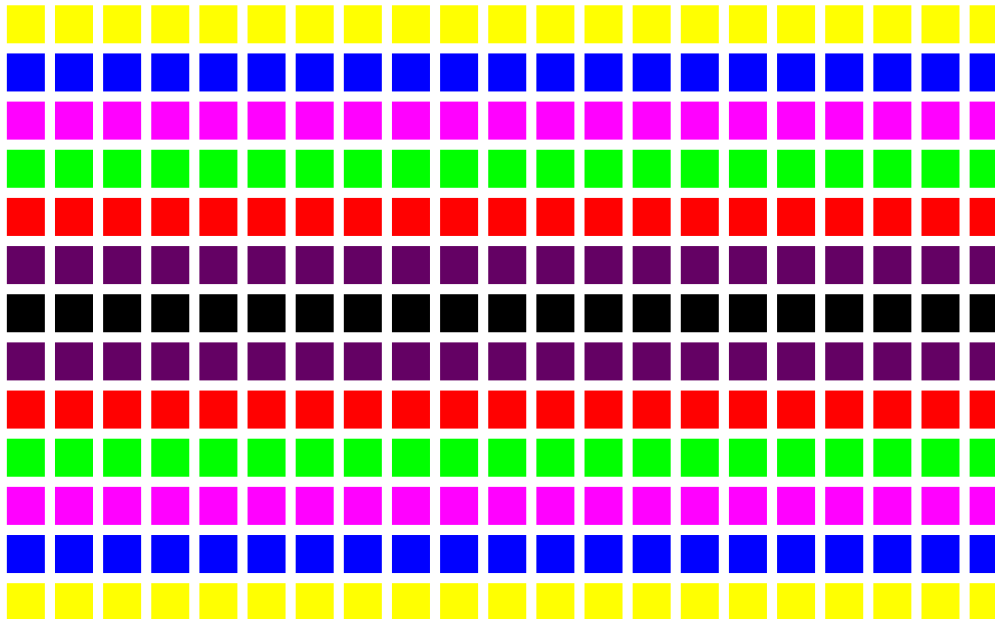


FIG. 12.6 – *Tiling of disjoint squares in colour*

Solution 23 *We give one program; the others are in the folder* `book/12_painting`*:*

```
from LibRobot import *

# Alhambra mosaic
gridSize = 3  # pixels
patternWidth = 4  # times the grid size
patternHeight = 6  # times the grid size
rowLength = 7  # patterns of an odd row
numberOfRows = 12  # even

# Draw an 8-shaped pattern, starting and
# ending at the end of the lower left
# point, with the robot facing north.
def pattern():
    for i in range(2):
        # Left/right side (above/below the
        # lower point).
        etr(); fd(); etl(); fdn(patternHeight - 4)
        etl(); fd(); qtr(); fd()
        # Top/bottom
        etr(); fdn(patternWidth - 2); etr()
```

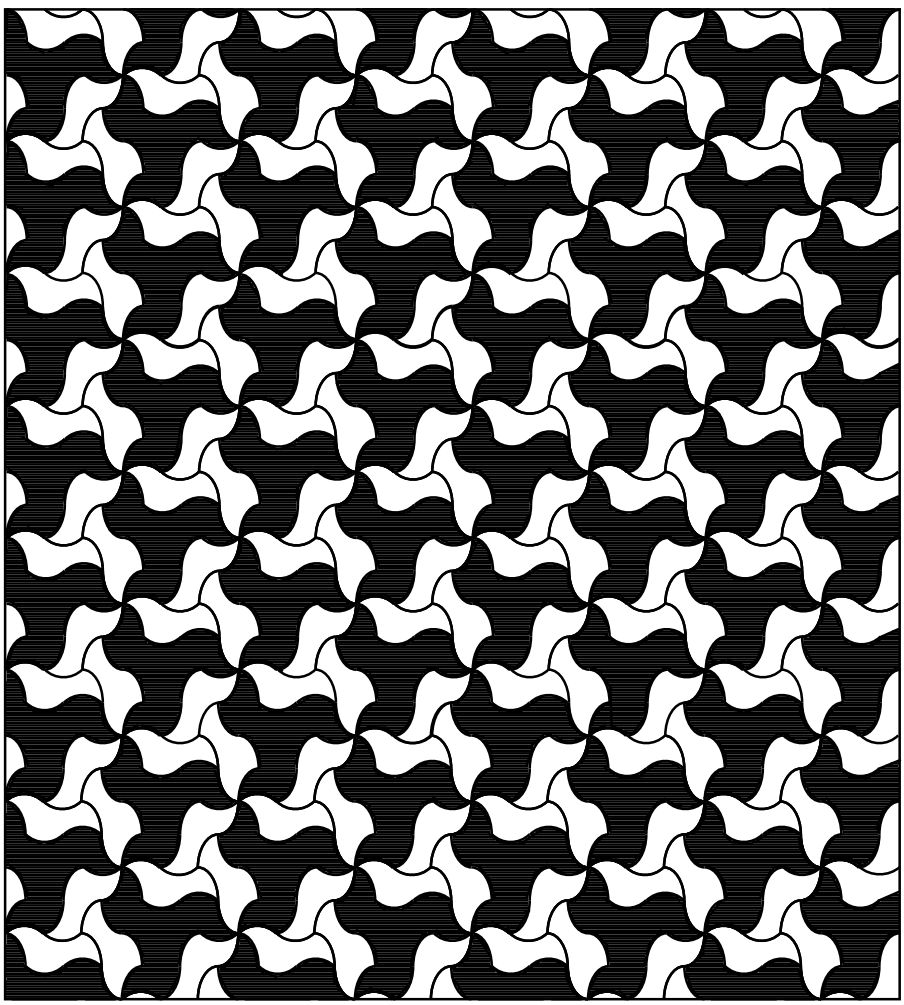


FIG. 12.7 – *Tiling of the plane in black and white*

```
      # Top/bottom of the right/left side (above/below
      # the upper/lower point).
      fd(); etr()
   # Colour and turn the robot to its initial
   # orientation.
   qtl(); paint(); qtr()

# Go from the lower left point of a pattern to that
# of the next pattern on the same horizontal
# row to the right.
def nextPattern():
   pu(); qtr(); fdn(2 * patternWidth); qtl(); pd()

# Draw a horizontal row containing
# rowLength patterns.
def oddRow():
   for i in range(rowLength):
      pattern(); nextPattern()

# Draw a row containing (rowLength - 1)
# patterns.
def evenRow():
   for i in range(rowLength - 1):
      pattern(); nextPattern()

# Go back to the lower left point of the first
# pattern of a row.
def backToRowStart():
   pu(); qtl()
   fdn(2 * patternWidth * rowLength - patternWidth)
   qtr(); pd()

# Go from the lower left point of a pattern to
# that of the pattern straight below, on the
# next row.
def nextRow():
   pu(); fdn(-(patternHeight - 2)); pd()

# Tile with a number of rows equal to
# numberOfRows.
def tiling():
   for i in range(numberOfRows // 2):
      oddRow(); backToRowStart()
      nextRow()
      evenRow(); backToRowStart()
      nextRow()

# Place the robot at the lower left point of the
# left pattern of the first row.
```

```
pu(); gs(gridSize); gl(); gt(); ht()
fdn(patternHeight); qtl(); fd(); qtl(); pd()
tiling(); st()
```

□

Solution 24 *The motif of the black and white tiling of the plane of figure 12.7 (page 144) is given in figure 12.8 (page 146).* □

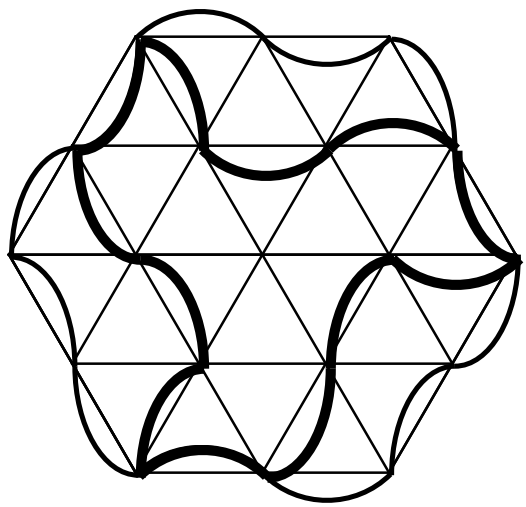


FIG. 12.8 – *Motif of the tiling of figure 12.7*

Solution 25 *We give one program; the others are in the folder* `book/12_painting`*:*

```
from LibRobot import *

# Draw and paint a ball in colour
edgeThickness = 3
halfRadius = 30

def quarterOfFigure():
    gs(halfRadius); cl(); cl()
    gs(2 * halfRadius); cl(); gc(); ht()

# Place the robot in the area to paint and paint
def paintQuarterOfFigure():
    gs(halfRadius // 2)
```

```
    pu(); gc(); qtr(); fd(); pd(); paint()
pw(edgeThickness)
for i in range(4):
    quarterOfFigure()
for i in range(2):
    pc(blue); paintQuarterOfFigure()
    pc(violet); paintQuarterOfFigure()
st()
```

□

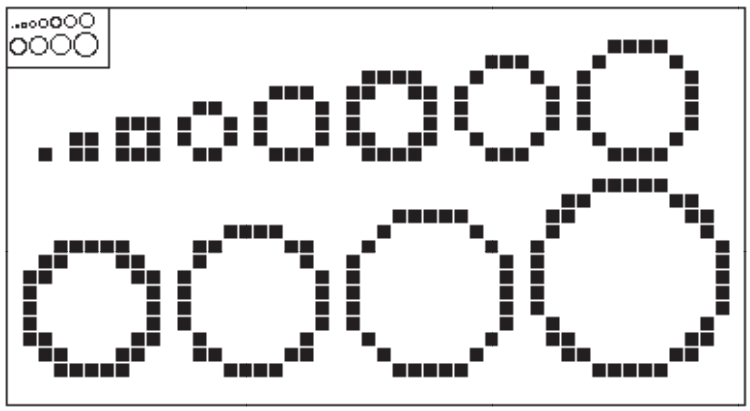
Addenda

We can imagine that, to paint, the $\mathcal{R}^{o}\underline{b}^{o}t$ places a seed on the screen, behind its stopping point. If a seed is placed on a pixel of the screen that is already coloured, then this seed disappears. In particular, if the initial seed placed by the $\mathcal{R}^{o}\underline{b}^{o}t$ is on a non-white pixel, the painting stops immediately. If the seed is placed on a white pixel of the screen, then the seed disappears, the white pixel on which it was is painted in the colour of the pen and four new seeds are placed to the east, to the west, to the north and to the south of the pixel that has just been painted. The painting is finished when all the seeds have disappeared.

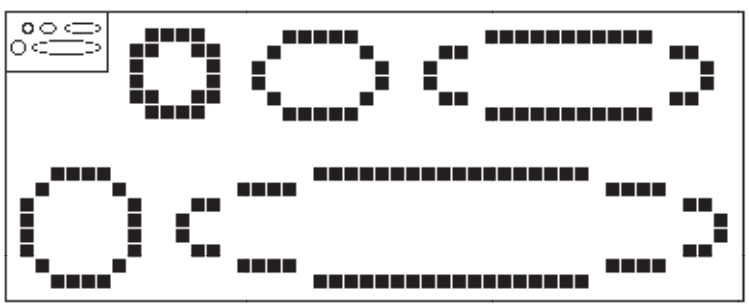
This painting algorithm is therefore based on the physical representation of the drawing on the screen and not on the logical representation which would be obtained by recording all the commands executed by the $\mathcal{R}^{o}\underline{b}^{o}t$. As a result, leaks sometimes occur that cannot be explained logically but that are due to the difference between the logical and physical representations.

Let us take the example of a circle. Mathematically a circle is a continuous line made up of an infinite set of dimensionless points. On a computer screen the circle will be represented by a discontinuous line made up of a finite set of "points" the size of a pixel. The mathematical and computer notions of circle are therefore completely different.

The approximation is acceptable when the size of the pixels is very small compared with the diameter of the circle, in the same way as a pair of compasses draws circles that give a good picture of the mathematical object. However, for very small circles, we get some aberrations, visible in the figure below, where the circles shown in the small frame at the top left of the figure have been enlarged eight times. Note in particular that the very small circles are square!



On screens with rectangular pixels, circles look like ellipses. The program must therefore correct the distortion by logically drawing ellipses in order to obtain circles on the screen. Circles are closed in the sense that every pixel painted on the circle has at least one painted neighbouring pixel in one of the eight directions of the compass rose. This is not the case for the very elongated ellipses that can be seen in the figure below:



The holes in the boundary of the elongated ellipses are points where the paint leaks when we want to paint the inside of the ellipse. To avoid these leaks one would have to use more complex colouring algorithms based on the logical rather than the physical structure of the drawing.

13

Declaring procedures with parameters

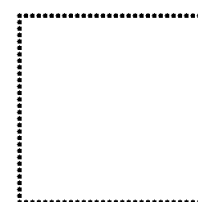
In chapter 5 we learned how to declare and call procedures without parameters, which let us repeat the same sequence of commands of the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ several times at different places in the program. In chapter 9 we used commands of the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ which are calls of procedures with parameters, the use of one or more parameters allowing the procedure to perform sequences of commands that are similar but differ according to the value of the parameters. We will now see how to declare procedures with parameters in Python. Let us start with a simple example:

```
from LibRobot import *

# Draw a dotted square

# Draw a dotted line of n dots,
# assuming n > 1.
def dottedLine(n):
   gs(8); pw(2); pu()
   for l in range(n - 1):
      dp(); fd()
   dp(); pd()

# Draw a dotted square with p dots per side
# (p > 1).
def square(p):
   for c in range(4):
```



```
        dottedLine(p); qtr()
    square(30); hideRobot(); st()
```

The procedure `dottedLine` draws a dotted line made up of a succession of `n` black pixels separated by (`n`−1) white pixels, the number `n` of dots being passed as a parameter of this procedure. In the declaration '**def** `dottedLine(n):`', the parameter `n` is written between the parentheses that follow the name of the procedure, parentheses which remain empty `()` for procedures without parameters. There is no need to state that `n` is an integer: Python does not ask for it, and it is only at execution time, when computing the expression `n - 1`, that it uses the value of `n`. The procedure `square` draws a dotted square, the number `p` of dots per side being passed as a parameter. The call `square(30)` says that the procedure `square` must be executed with a parameter `p` equal to 30. Consequently the procedure `dottedLine` is called four times with a parameter `n` equal to `p`, that is 30. We say that `p` is the *formal parameter* (whose value is not yet known at the time the procedure is declared, and which is designated by an identifier) and that 30 is the *actual parameter* (that is, the value of the formal parameter for this call of the procedure).

In general, declarations and calls of procedures with parameters have the following form:

Declaration of a procedure with parameters:

```
# Specification: what the procedure does
def␣procedureName(formalParam1[a], formalParam2, ...):
    ... Instructions of the body of the procedure[b] (indented) ...
```

[a] A formal parameter is an identifier.
[b] The expressions appearing in these instructions may use the formal parameters `formalParam1`, ... of the procedure.

Call of a procedure with parameters:

```
procedureName(actualParam1[a], actualParam2, ...[b])
```

[a] An actual parameter is an integer expression (or a colour, etc.).
[b] The actual parameters are given in the order in which the corresponding formal parameters appear.

Note that the parameters, formal ones in the procedure declaration as well as actual ones in the procedure call, are placed between parentheses and separated by commas. For example, to draw a rectangle of height 3 and width 4 with the procedure `rectangle` declared by '**def** `rectangle(height, width):`' in the program `MacintoshSE.py` of the folder `book/13_declaring_procedures`, we write `rectangle(3, 4)`. If we forget the parentheses and write `rectangle 3, 4`, Python rejects the program before running it with the message '`SyntaxError: invalid syntax`'. An actual parameter can be an expression, as in `rightHalfCircle(2 * gridSize)` (page 154) or `fdn(-3)`: it is computed before the procedure is called. Be careful to place the closing parenthesis correctly: `rightHalfCircle(2) * gridSize` would be understood as the multiplication by `gridSize` of the result of the call `rightHalfCircle(2)`, which makes no sense. After drawing the half circle, Python stops with the message: '`TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'`' (operand types not allowed for `*`: `NoneType` and integer). `NoneType` is the type of the value **None** ('nothing') returned by a procedure call.

The number of actual parameters must be equal to the number of formal parameters. If we write `rectangle(3)`, Python stops when the *Robot* reaches this call, with the message '`TypeError: rectangle() missing 1 required positional argument: 'width'`' and if we write `rectangle(3, 4, 5)`, with the message '`TypeError: rectangle() takes 2 positional arguments but 3 were given`'.

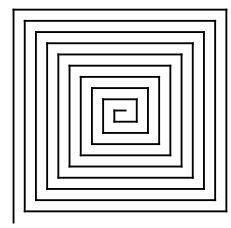
A procedure with formal parameters can be called several times in a program with actual parameters that can be different each time.

The formal parameters of a procedure are names that are *local* to the procedure: they can only be used in the body of the procedure (elsewhere, Python would answer for example '`NameError: name 'width' is not defined`'). Consequently different procedures can have formal parameters with the same name although they are different parameters.

These various points are illustrated by the example below.

Example 20 (Segmented spiral) The program below draws a spiral made up of pairs of straight line segments of successive lengths 1, 2, 3, . . . , 19:

```
from LibRobot import *
# Draw a segmented spiral
# Turn left then move the robot straight ahead
# l times the grid size to form a
# quarter turn.
def quarterTurn(l):
   qtl(); fdn(l)
# Draw two successive quarter turns to form
# a half turn.
def halfTurn(l):
   quarterTurn(l)
   quarterTurn(l)
# Draw a spiral of n half turns
def spiral(n):
   for l in range(1, n + 1):
      halfTurn(l)
gs(6); spiral(19); st()
```



□

Exercise 26 *Write Python programs containing procedures with parameters to produce the following drawings:*

– *the cross with three transverse arms, the coloured propeller, the microcomputer, the colour logo, the caterpillar (where the number of rings of the caterpillar's body is passed as a parameter of the drawing procedure), the canoe (where the number of rowers is a parameter) shown on page 153;*

– *the squares inscribed one inside another or the painted version of this drawing shown in figure 13.8 (page 154);*

– *the arabesque inspired by the dome of the mausoleum of Sultan Qayt-Bay shown in figure 13.1 (page 153);*

– *the barge of figure 1.13 (page 10) where the number of portholes is given as a parameter;*

– *the Golomb tiling of figure 8.6 (page 103).*

□

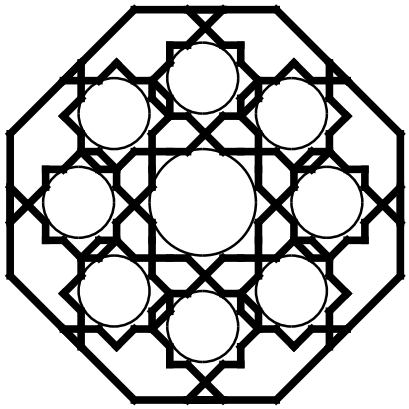


FIG. 13.1 – *Arabesque*

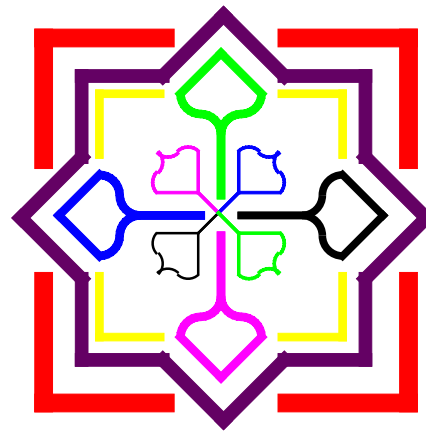


FIG. 13.4 – *Colour logo*

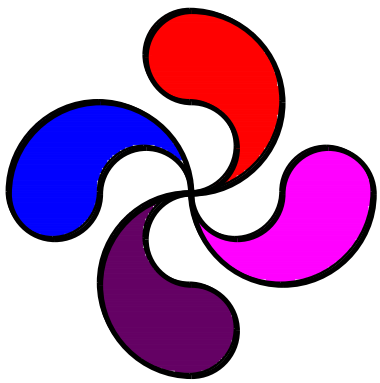


FIG. 13.2 – *Coloured propeller*

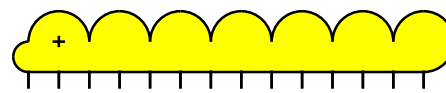


FIG. 13.5 – *Caterpillar*

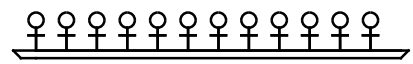


FIG. 13.6 – *Canoe*

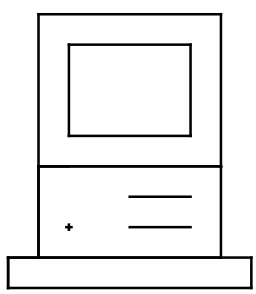


FIG. 13.3 – *Old microcomputer*

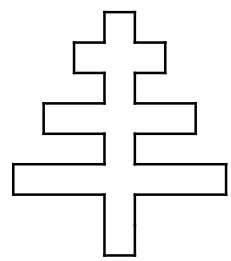


FIG. 13.7 – *Cross with three arms*

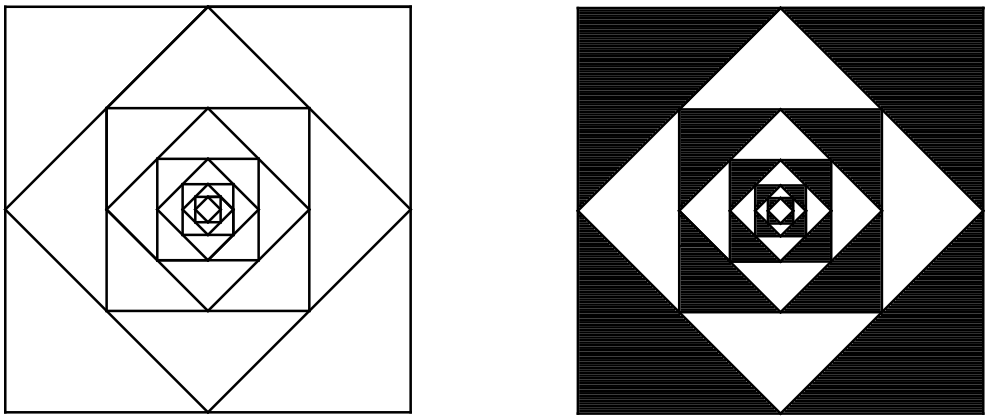


FIG. 13.8 – *Horemis's pattern of inscribed squares*

Solution 26 *We give one program, the others being in the folder* `book/13_declaring_procedures`*:*

```
from LibRobot import *

# Draw a four-bladed propeller in colour
gridSize = 18
edgeThickness = 2

def leftHalfCircle(r):
   gs(r); cl(); cl()

def rightHalfCircle(r):
   gs(r); cr(); cr()

def blade(c):
   # Draw the outline of the blade
   pw(edgeThickness); pc(black)
   leftHalfCircle(gridSize)
   rightHalfCircle(gridSize)
   rightHalfCircle(2 * gridSize)
   # Paint the inside of the blade
   qtr(); gs(gridSize)
   pu(); fdn(3); pc(c); paint(); fdn(-3); pd()

def propeller():
   blade(blue); blade(indigo); blade(violet); blade(red)

propeller(); st()
```

□

Addenda

In Python, the definitions of identifiers are distinguished from their uses. The *defining occurrences* appear in a constant or procedure declaration (**def**) as the name of a constant, of a procedure or of a formal parameter. The *using occurrences* of these identifiers are the names of constants or of formal parameters used in an integer expression, or the names of procedures used in calls.

The same identifier may have several defining occurrences in a program. Hence the problem arises of matching the using occurrences with the defining occurrences of the same identifier. For example the identifier can be used to name a constant of the program and formal parameters in procedures. In Python, these procedures can be nested one inside another (by declaring with **def** a procedure p2 inside the body of a procedure p1, with one more level of indentation), as in the example below:

```
from LibRobot import *
n = 17
def p1(n):
    def p2(n):
        ...
        ... n ...  #1
        ...
    ...
    p2(n)  #2
    ...
...
p1(n)  #3
...
```

We say that the program has a *block structure*. These blocks are formed by the procedures nested one inside another, the program being considered as the outermost block.

The *scope rule* of identifiers in Python says that a using occurrence of an identifier refers to the defining occurrence of the innermost block in which this using occurrence appears and which contains a defining occurrence of the identifier.

In the example above,

- the using occurrence of `n` on the line marked `#1` refers to the formal parameter of the procedure `p2`;
- the using occurrence of `n` on the line marked `#2` refers to the formal parameter of the procedure `p1`;
- the using occurrence of `n` on the line marked `#3` refers to the constant 17.

Block structure, which is found in Python, was introduced by ALGOL 60 [29]. This language was created by a group of European and American experts gathered in the working group WG2.1 of IFIP (International Federation for Information Processing), which brings together all the national computing societies. It is the ancestor of a family of languages which includes PASCAL, ADA and MODULA.

14

Cartesian coordinates

To locate the position of the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ on the screen, we use a *Cartesian coordinate system* as shown in figure 14.1 below:

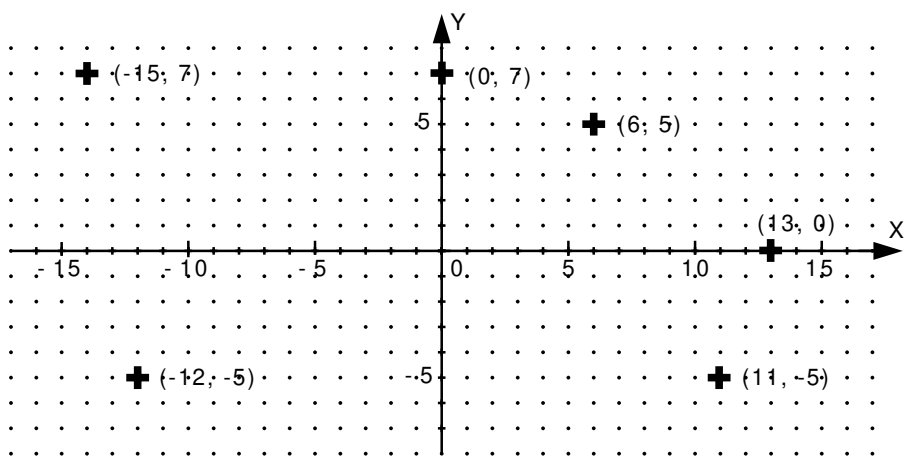


FIG. 14.1 – *Coordinates in a Cartesian coordinate system*

The *origin* 0 of the coordinate system is located at the centre of the grid. The horizontal axis OX is called the *x-axis* (axis of *abscissas*). The vertical axis OY is called the *y-axis* (axis of *ordinates*). Both axes are graduated. The length of the graduations on the x-axis is equal to the length of the horizontal side of a square of the grid counted in pixels. Likewise, the length of the graduations on the y-axis is equal to the length of the vertical side of a square of the grid counted in pixels.

A grid point is located by its *Cartesian coordinates*, which are two integers (X, Y), the first being its *abscissa* (or x-coordinate) and the second its *ordinate* (or y-coordinate). For example the grid point located at the centre of the grid has coordinates $(0, 0)$. The points located to the right of the origin O have a positive abscissa. The points located to the left of the origin O have a negative abscissa (preceded by the minus sign $-$). Likewise, the points located above the origin O have a positive ordinate while the points located below the origin O have a negative ordinate. As an example, some points of the grid are labelled with their coordinates in figure 14.1 (page 157).

The abscissa X of a point A is the distance from the origin of its projection on the axis OX (that is, of the point of intersection of the axis OX with the line parallel to the axis OY passing through A). Likewise, the ordinate Y of a point A is the distance from the origin of its projection on the axis OY (that is, of the point of intersection of the axis OY with the line parallel to the axis OX passing through A). For example in figure 14.2 below, we have $X = 6$ and $Y = 7$[1].

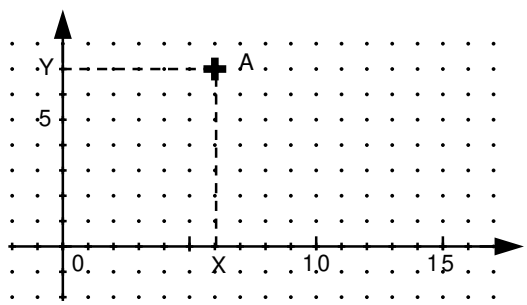


FIG. 14.2 – *Point A with abscissa X = 6 and ordinate Y = 7*

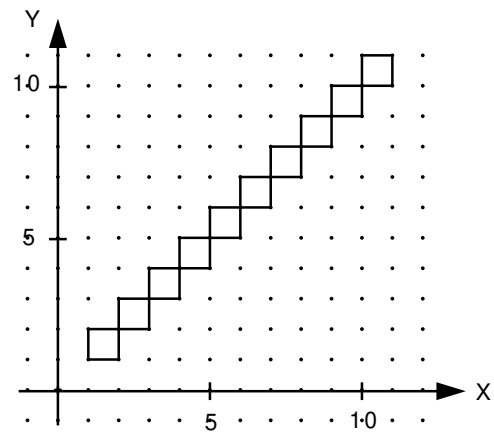
The command `moveTo(x, y)` places the $\mathcal{R}\text{o}\underline{\text{b}}\text{o}t$ on the grid point with Cartesian coordinates (x, y). The orientation of the $\mathcal{R}\text{o}\underline{\text{b}}\text{o}t$ and the position of its pen remain unchanged.

For example, the command `moveTo(0, 0)` places the $\mathcal{R}\text{o}\underline{\text{b}}\text{o}t$ at the origin of the Cartesian coordinate system, at the centre of the grid[2]. An example is given below:

[1] In the folder `book/14_cartesian_coordinates` you will find a Python program, `Learn_Coordinates.py`, for learning how to locate points of the plane with a Cartesian coordinate system.

[2] The command `gc()` has the additional effect of recentring, if necessary, the origin of the coordinate system exactly at the centre of the screen.

```
from LibRobot import *
# Row of squares (moveTo)
# Square of side 1
def square():
    fd(); qtr()
    fd(); qtr()
    fd(); qtr()
    fd(); qtr()
for i in range(1, 11):
    moveTo(i, i)
    square()
st()
```



The command `translate(x, y)` translates the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ by (x, y) without changing its orientation[3]. An example of the translation of a rectangle is given in figure 14.3 below.

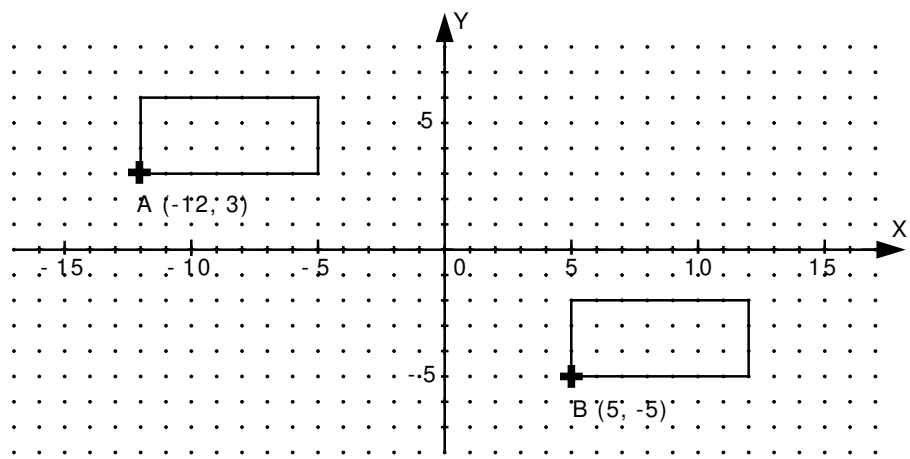


FIG. 14.3 – *Translation (17, -8) from A to B*

[3] In the folder `book/14_cartesian_coordinates` you will find a Python program, `Learn_Translation.py`, for learning how to translate points of the plane located by Cartesian coordinates.

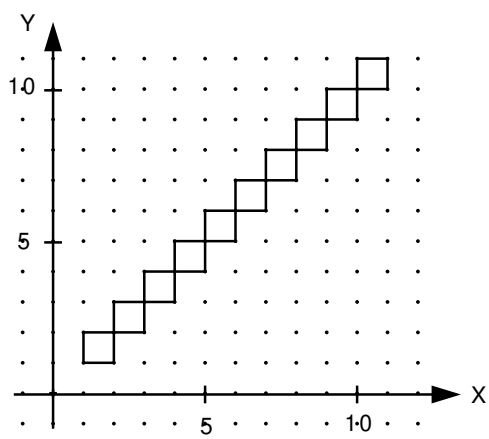
A translation of the *Robot* by (x, y) consists in increasing (or decreasing if x is negative) its abscissa by x and increasing (or decreasing if y is negative) its ordinate by y. If the centre of the grid, with coordinates $(0, 0)$, is exactly at the centre of the screen, we can define moving in terms of translation as follows:

```
def moveTo(x, y):
    gc(); translate(x, y)
```

As this example shows, the commands `moveTo` and `translate` are procedures with two parameters, the abscissa `x` then the ordinate `y`, placed between parentheses and separated by a comma, like the two parameters of the call `rectangle(3, 4)` in the previous chapter. The order matters: `moveTo(6, 7)` and `moveTo(7, 6)` do not place the *Robot* on the same point.

The program `RowOfSquares1` on page 159 can be rewritten using a translation command instead of a move command, as follows:

```
from LibRobot import *
# Row of squares (translate)
# Square of side 1
def square():
    fd(); qtr()
    fd(); qtr()
    fd(); qtr()
    fd(); qtr()
for i in range(10):
    translate(1, 1)
    square()
st()
```



With the command `translate(x, y)`, the movement of the *Robot* is relative to its starting position, whereas with the command `moveTo(x, y)`, the movement of the *Robot* is relative to the origin of the Cartesian coordinate system.

Exercise 27 *Write Python programs using the commands* `moveTo` *and* `translate` *to produce the following drawings:*

- *the tilings obtained by repeating various patterns inscribed in a square of size* (6×6) *in figure 14.6 (page 163);*
- *the tiling of quadrilaterals in figure 14.4 (page 161);*
- *the mosaic of the Alhambra recorded by Escher [26] in figure 14.5 (page 162).*

□

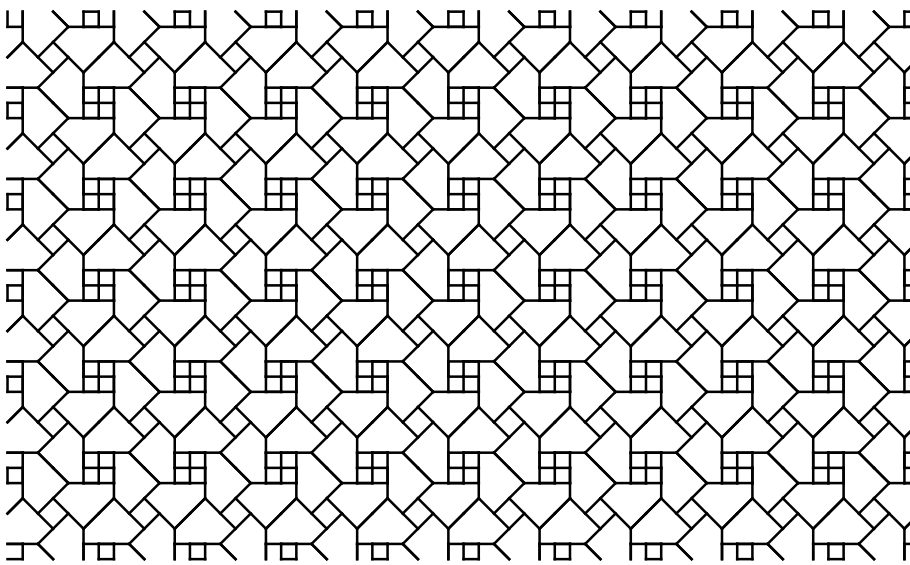


FIG. 14.4 – *Tiling of quadrilaterals*

Solution 27 *We give the program drawing the tiling of quadrilaterals of figure 14.4 (page 161), the other programs being in the folder* `book/14_cartesian_coordinates`. *It is a tiling of the screen with patterns inscribed in* (6×6) *squares, the basic pattern being drawn next to the procedure that draws it.*

```
from LibRobot import *
# Tiling of quadrilaterals produced by a pattern
```

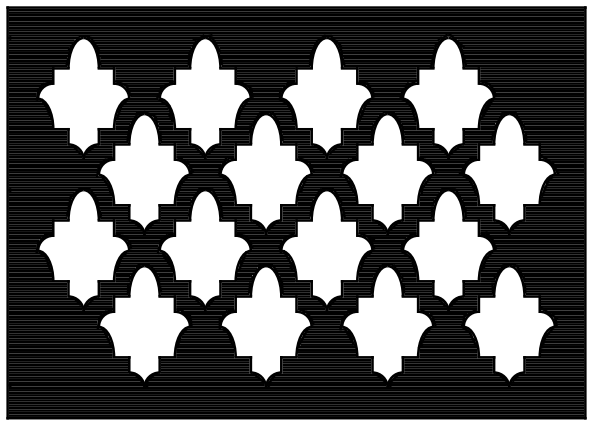


FIG. 14.5 – *Mosaic of the Alhambra*

```
# inscribed in a (6 x 6) square
gridSizeX = 6
gridSizeY = 6
patternX = 6  # Width of the pattern
patternY = 6  # Height of the pattern

# Draw a pattern in a (patternX x patternY) square,
# the robot starting and ending at the centre of
# the pattern, facing north.
def pattern():
   for i in range(4):
      pu(); qtl(); fd(); pd()
      tetr(); fd(); fd(); qtl(); fd(); ht(); fd()
      etl(); fd(); fd(); ht(); fd(); qtr(); fd()
      qtr(); fd()
      pu(); tetr(); fd(); fd(); fd(); pd()
      etr()

# Tile with y rows of x patterns
def tiling(x, y):
   # Shift the drawing to the left by half the
   # length of a row and upwards by
   # half the height of the tiling.
   translate(-(patternX * (x // 2)), patternY * (y // 2))
   # Draw (y - 1) rows of x patterns (with
   # translation to the bottom left of the next
   # row).
   for i in range(y - 1):
      # Draw a row of x patterns (without
```

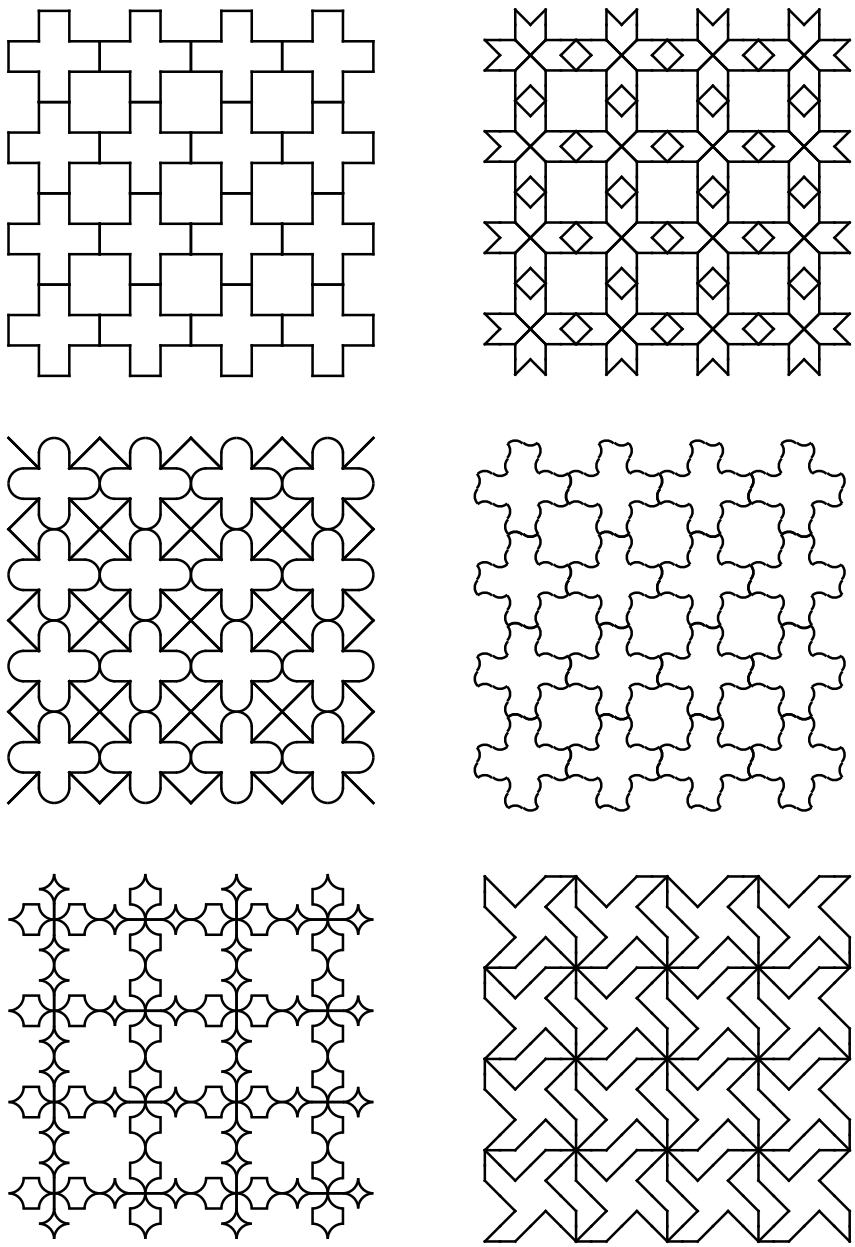


FIG. 14.6 – *Tilings of* (6×6) *patterns*

```
        # translation to the right after the last pattern
        # of the row).
        for j in range(x - 1):
           pattern()
           # Move to the next pattern on the right in the
           # same row.
           translate(patternX, 0)
        pattern()
        # Go to the start of the next row.
        translate(-(patternX * (x - 1)), -patternY)
     # Draw the last row of x patterns (without
     # translation to the bottom left of the next
     # row).
     for j in range(x - 1):
        pattern()
        translate(patternX, 0)
     pattern()
  gsX(gridSizeX); gsY(gridSizeY)
  tiling(10, 6)
  st()
```

□

Addenda

Following in the footsteps of Apollonius[a], who had informally given the equations of the conics (circles, ellipses, parabolas and hyperbolas), Fermat and Descartes were the first to "put into equations" geometrical problems. Descartes introduced the notion of a system of plane coordinates, each point of the plane being defined by its abscissa x and its ordinate y, obtained by projecting onto axes OX and OY, rectangular or oblique.

An algebraic curve is then defined by the relation $f(x, y) = 0$ that holds between the coordinates of each of its points. For example the Cartesian equation of the circle with centre (a, b) is $x^2 + y^2 - 2ax - 2by + c = 0$ with the radius R such that $R^2 = a^2 + b^2 - c$, assuming that $a^2 + b^2 \geq c$.

[a] Apollonius of Perga, Greek astronomer and mathematician, 295 – 230 BC.

Descartes set out his method in a work published in 1637 entitled "La Géométrie" (Geometry) which, together with "La Dioptrique" (Optics) and "Les Météores" (Meteorology), formed the appendices of one of the most famous texts of philosophy, the celebrated "Discours de la méthode pour bien conduire sa raison et chercher la vérité dans les sciences" (Discourse on the Method of Rightly Conducting One's Reason and of Seeking Truth in the Sciences), where he sets out his methodical and rational philosophy, since called *Cartesianism*. The idea of an equation appears more clearly in Fermat, who discovered independently of Descartes, around 1629, the basic principle of *analytic geometry*. He only published it in 1679 in his "Isagoge (Introduction aux lieux plans et solides)" (Introduction to plane and solid loci).

Descartes had only seriously studied the conics of the second degree. Newton[a] studied the cubics of the third degree and curves of higher degree. We also owe him the systematic use of negative coordinates, of which Descartes was wary. La Hire[b] introduced in 1679 the consideration of a system of spatial coordinates, which amounts to building a third coordinate axis OZ, perpendicular to the axes OX and OY defining the plane. A point in space is then determined by three real numbers (x, y, z) and the equation $f(x, y, z) = 0$ defines a skew surface. Möbius[c] and Plücker[d] extended the notion of coordinates by introducing the homogeneous coordinates (X, Y, Z, T) such that $x = \frac{X}{T}$, $y = \frac{Y}{T}$ and $z = \frac{Z}{T}$. This makes it possible to introduce the points at infinity $(X, Y, Z, 0)$ and the imaginary points $X = A_1 + iA_2$, $Y = B_1 + iB_2$, $Z = C_1 + iC_2$ and $T = D_1 + iD_2$ where $i^2 = -1$. One can then consider for example imaginary circles such that $a^2 + b^2 < c$.

[a] Isaac Newton, English mathematician, physicist and astronomer, 1642–1727.
[b] Philippe de la Hire, French astronomer and mathematician, 1640–1718
[c] August Ferdinand Möbius, German astronomer and mathematician, 1790–1868
[d] Julius Plücker, German mathematician and physicist, 1801–1868

15

The “if” test

In answer to our questions, our $\mathcal{R}^{o}\!\!\smile^{o}t$ can tell us which direction it is facing. Since it cannot speak, its answers are *boolean*, that is **True** (*true*) or **False** (*false*), written in Python with a capital letter. We can imagine that it gives its answers with an indicator light which it switches on to answer **True** and leaves off to answer **False**.

To ask questions about the orientation of the $\mathcal{R}^{o}\!\!\smile^{o}t$, we use the following test commands:

In Python, the “if” statement makes it possible to execute a sequence of instructions only if a certain *boolean expression* is true (**True**). To begin with, we will use simple boolean expressions, namely the commands testing the orientation of the $\mathcal{R}^{o}\!\!\smile^{o}t$ (`tn()`, `tne()`, `te()`, `tse()`, `ts()`, `tsw()`, `tw()`, `tnw()`). In its simplest form, the “if” statement is written as follows:

“if” statement with a single alternative:

if ␣ boolean expression[a] **:**
    ... Instructions (indented) executed only if
    the boolean expression is **True** ...

[a] With result **True** or **False**.

It reads “if ... then”. As for the body of a procedure or of a **for** loop, the line of the **if** ends with a colon ‘`:`’ and the instructions that depend on the test are indented (shifted four spaces to the right with respect to the **if**): they form a *block* which ends at the first line that is less indented. If we forget the colon, Python rejects the program with the message ‘`SyntaxError: expected`

Commands testing the orientation of the $\mathcal{R}$obot:	
`tn()`	**True** if the $\mathcal{R}$obot is turned towards the north (n) and **False** otherwise.
`tne()`	**True** if the $\mathcal{R}$obot is turned towards the north-east (ne) and **False** otherwise.
`te()`	**True** if the $\mathcal{R}$obot is turned towards the east (e) and **False** otherwise.
`tse()`	**True** if the $\mathcal{R}$obot is turned towards the south-east (se) and **False** otherwise.
`ts()`	**True** if the $\mathcal{R}$obot is turned towards the south (s) and **False** otherwise.
`tsw()`	**True** if the $\mathcal{R}$obot is turned towards the south-west (sw) and **False** otherwise.
`tw()`	**True** if the $\mathcal{R}$obot is turned towards the west (w) and **False** otherwise.
`tnw()`	**True** if the $\mathcal{R}$obot is turned towards the north-west (nw) and **False** otherwise.

TAB. 15.1 – *Tests of the orientation of the $\mathcal{R}$obot*

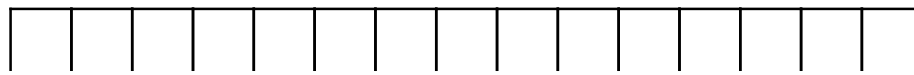
`':'` and if we forget to indent, with the message '`IndentationError: expected an indented block after 'if' statement on line 5`' (if the **if** is on line 5 of the program).

When there is only one instruction to execute, it can be written on the same line, after the colon, as in '**if** `tw(): qtr()`'. But beware: in '**if** `tw(): qtr(); fd()`' both instructions `qtr()` and `fd()` depend on the test. A serious error is to get the indentation wrong. For example in:

```
if tw():
   qtr()
fd()
```

only the instruction `qtr()` depends on the test: the instruction `fd()`, which is not indented, is always executed, whether the $\mathcal{R}$obot is facing west or not. Python cannot report this error since the program is correct (but does not do what we wanted). Note also that the "if" statement is *one* instruction: if other instructions follow it, they are written on the following lines, with the same indentation as the **if**.

In the example below, the $\mathcal{R}^{o}_{o}t$ moves forward twice and turns a quarter turn to the right, then one more quarter turn if it is facing west:



```
from LibRobot import *
# Draw a comb-shaped frieze
length = 15
gl()
for i in range(3 * length):
   fdn(2); qtr()
   if tw():
      qtr()
st()
```

In its first form above, the "if" statement does nothing when the boolean expression is **False**. If we want to choose the sequence of instructions to execute in both alternatives, we use the following form:

"if" statement with two alternatives:

if ␣ boolean expression[a]**:**
    ... Instructions (indented) executed if the boolean expression is **True** ...
else:
    ... Instructions (indented) executed if the boolean expression is **False** ...

[a] With result **True** or **False**.

It reads "if ... then ... otherwise ..." (**else** means 'otherwise').

The **else** is written exactly below the **if** (with the same indentation) and it is followed by a colon. As before, an alternative consisting of a single instruction can be written on the same line as the **if** or the **else**, after the colon. Forgetting the colon after the **else** or misaligning the **else** below the **if** is an error, but not a very serious one since it is reported by Python before execution ('`SyntaxError: expected ':'`' or '`IndentationError: unindent does not match any outer indentation level`'). Python also reports an error ('`SyntaxError: invalid syntax`') when we forget to indent one of the instructions placed between the **if** and the **else**.

In the drawing of the sawtooth frieze below, the $\mathcal{R}^{o}_{\smile}{}^{o}t$ turns three quarters of a turn to the right if it is facing north-east and to the left in the other cases (the only other possible case being, in fact, that it is facing south):



```
from LibRobot import *

# Draw a sawtooth frieze
numberOfTeeth = 31

gl(); etr()
for i in range(2 * numberOfTeeth):
   fd()
   if tne():
      tetr()
   else:
      tetl()
st()
```

When there are more than two cases to test, we can use several alternatives and do nothing when none of the boolean expressions is **True**, as in:

"if" statement with n alternatives:

if ␣ boolean expression 1**:**
    ... Instructions (indented) executed if boolean expression 1 is **True** ...
elif ␣ boolean expression 2**:**
    ... Instructions (indented) executed if boolean expression 2 is **True** ...
...
elif ␣ boolean expression n**:**
    ... Instructions (indented) executed if boolean expression n is **True** ...

We can also provide a last alternative which is executed when all the boolean expressions tested are **False**:

"if" statement with $(n+1)$ alternatives:

if ␣ boolean expression 1**:**
    ... Instructions (indented) executed if boolean expression 1 is **True** ...
elif ␣ boolean expression 2**:**
    ... Instructions (indented) executed if boolean expression 2 is **True** ...
...
elif ␣ boolean expression n**:**
    ... Instructions (indented) executed if boolean expression n is **True** ...
else:
    ... Instructions (indented) executed if boolean expressions 1, ..., n are all **False** ...

The word **elif** is a contraction of **else if**. The boolean expressions are evaluated in order and only the instructions of the first alternative whose expression is **True** are executed. This form of multiple test is illustrated below in the program that builds the following frieze:



```
from LibRobot import *

# Draw a frieze of ripples
length = 15

gl()
for i in range(2 * length):
   if tn():
      cr(); qtr()
   elif ts():
      cl(); qtl()
   elif te():
      qtr()
   else:
      qtl()
   fd()
st()
```

To finish, we are going to introduce new commands of the $\mathcal{R}^{o}_{b}{}^{o}t$ which make the $\mathcal{R}^{o}_{b}{}^{o}t$ turn on the spot to face a given direction. For example the command `fn()`, which makes the $\mathcal{R}^{o}_{b}{}^{o}t$ face north, can be defined with the following Python procedure:

```
# Turn the robot on the spot to face north
def fn():
    if tne():
        etl()
    elif te():
        qtl()
    elif tse():
        tetl()
    elif ts():
        ht()
    elif tsw():
        tetr()
    elif tw():
        qtr()
    elif tnw():
        etr()
```

The program `FacingNorth.py` of the folder `book/15_if` tries this procedure in the eight directions. We could define in the same way the commands that turn the *Robot* on the spot to face one of the eight main directions of the compass rose, which are given in table 15.2 below.

Commands turning the Robot on the spot:	
`fn()`	makes the *Robot* turn on the spot to face north (n).
`fne()`	makes the *Robot* turn on the spot to face north-east (ne).
`fe()`	makes the *Robot* turn on the spot to face east (e).
`fse()`	makes the *Robot* turn on the spot to face south-east (se).
`fs()`	makes the *Robot* turn on the spot to face south (s).
`fsw()`	makes the *Robot* turn on the spot to face south-west (sw).
`fw()`	makes the *Robot* turn on the spot to face west (w).
`fnw()`	makes the *Robot* turn on the spot to face north-west (nw).

TAB. 15.2 – *Turning the Robot on the spot to face a given direction*

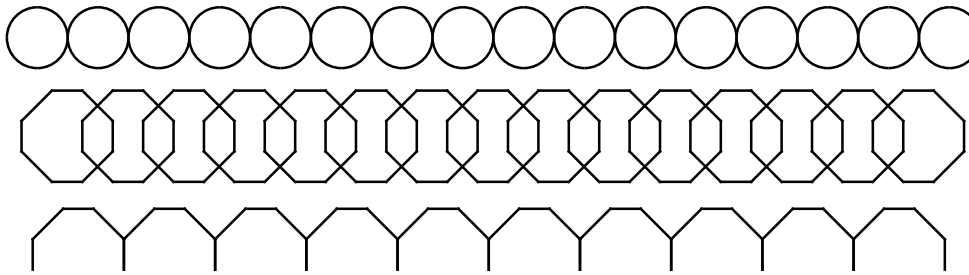


FIG. 15.1 – *Friezes of circles and (half-) octagons*

Exercise 28 *Write Python programs containing "if" statements to produce the friezes of figure 15.1 below.* □

Solution 28 *We give the program drawing the frieze of octagons of figure 15.1 (page 173), the others being in the folder* `book/15_if`*:*

```
from LibRobot import *

# Draw a frieze of octagons
numberOfOctagons = 15

gl(); qtr()
for i in range(8 * numberOfOctagons):
   if te():
      pu(); fdn(2); pd()
   fd(); etr()
st()
```

□

Addenda

The test statement `if ... then ... else ...` was introduced towards the end of the 1950s in the programming language ALGOL 60. This language, designed by a group of international experts, had a very great influence on the design of later programming languages, notably ALGOL W, PASCAL, ADA, MODULA and many others. Another form of test statement, by cases, was introduced in the programming language ALGOL W (also designed by N. Wirth) and taken up in PASCAL under the name "case". In OCaml, the test statement by cases is called *pattern matching* ("match"). Python also has, since version 3.10, a `match` statement, but we will not use it: in Python, a test by cases is usually written as a sequence of **if** ... **elif** ... **else**, comparing the value of an expression with each of the possible cases.

For example `valOrientation()` ("orientation value") is equal to the number of the orientation of the *Robot* (0 for north, 1 for north-east, 2 for east, etc.). The library `LibRobot` declares the constants `north`, `northEast`, `east`, `southEast`, `south`, `southWest`, `west` and `northWest`, equal to 0, 1, ..., 7. We can write:

```
if valOrientation() == north:          # north
    ...
elif valOrientation() == northEast:    # north-east
    ...
elif valOrientation() == east:         # east
    ...
elif valOrientation() == southEast:    # south-east
    ...
elif valOrientation() == south:        # south
    ...
elif valOrientation() == southWest:    # south-west
    ...
elif valOrientation() == west:         # west
    ...
elif valOrientation() == northWest:    # north-west
    ...
else:                                  # impossible
    pass
```

Note that the comparison is written with two equals signs '==', the single sign '=' being used to declare constants. With '**if** `valOrientation() = north:`', Python would answer '`SyntaxError: cannot assign to function call here. Maybe you meant '==' instead of '='?`' (a function call cannot be given a value). Two numbers can also be compared with `!=` (different from, $\neq$), <, <= ($\leq$), > and >= ($\geq$). The last case **else** groups together all the other values, like the word **otherwise** of some extensions of PASCAL. The **pass** statement does nothing; this last case can also be omitted. Several similar cases can be grouped together as in the example below, where we test whether the remainder of the division of the orientation number by 2 is zero:

```
if valOrientation() % 2 == 0:   # N, E, S, W
    ...
else:                           # other cases
    ...
```

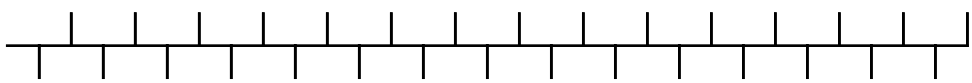
Note that with **if** tests, the cases can be names of constants such as `north` as well as numbers such as `0`, which makes the program more readable. This was not the case with pattern matching in OCaml, where '`| north ->`' does not mean 'if the orientation is north' but declares a new identifier `north` which matches all values (the same is true of the `match` of Python).

16

Boolean expressions

16.1 Parity

The tests we have made so far in "if" statements were about the orientation of the $\mathcal{R}\!\!\smile\!\!\circ t$. In Python it is also possible to make tests on the values of parameters and of "for" loop counters. It can be useful to test the counter in the body of a loop in order to perform different actions at each iteration. Take for example the drawing of the frieze shaped like a double comb below:



The teeth of the comb being drawn from left to right, the 1st, 3rd, 5th, ... point downwards whereas the 2nd, 4th, 6th, ... point upwards. In the drawing program we use a loop counter `i` which is 1 when we draw the 1st tooth, 2 when we draw the 2nd, ... Consequently when the value of the loop counter is *odd* (that is 1, 3, 5, ...) a tooth must be drawn downwards whereas if the value of the counter is *even* (that is 2, 4, 6, ...) a tooth must be drawn upwards. A number is odd when the remainder of its division by 2 is equal to 1. To test that the value of the counter `i` is odd we therefore use the boolean expression `i % 2 == 1` which is **True** when the remainder `i % 2` of the integer division of `i` by 2 is equal to 1 and **False** when this remainder is equal to 0, that is when `i` is even. Note that the equality test is written with two equal signs `==`, the single sign `=` being reserved for the declaration

of constants such as `numberOfTeeth = 30`.

```
from LibRobot import *

# Draw a frieze shaped like a double comb
numberOfTeeth = 30

gl(); fe()
for i in range(1, numberOfTeeth + 1):
   # Horizontal line
   fd()
   if i % 2 == 1:
      # Tooth pointing down
      qtr(); fd(); ht(); fd(); qtr()
   # Tooth pointing up
   else:
      qtl(); fd(); ht(); fd(); qtl()
st()
```

16.2 Comparison of integers

If `i` is a loop counter or an integer parameter of a procedure, the boolean expression `i > 10` is **True** if the value of `i` is strictly greater than 10 and **False** otherwise, that is if the value of `i` is less than or equal to 10. The mathematical symbols $\leq$ (less than or equal) and $\geq$ (greater than or equal) are written `<=` and `>=` in Python (because the mathematical symbols $\leq$ and $\geq$ are not found on computer keyboards whereas `<`, `>` and `=` are always there). For example the boolean expressions `10 <= 10` and `10 <= 20` are **True** whereas `10 <= 9` is **False**. In Python, as we have seen, equality is written with two equal signs `==` since the usual mathematical symbol `=` is used to declare constants. The mathematical symbol $\neq$ (different) is written `!=`. For example `10 == 10` and `10 != 20` are **True** whereas `10 != 10` and `10 == 20` are **False**[1]. Boolean expressions comparing integers or values of integer parameters can be used in the test of the "if" statement. For example the command `fdn(n)` which makes the *Robot* move forward `n` times if the value of `n` is positive, makes it

[1] If by mistake one writes **if** `i = 10:` instead of **if** `i == 10:`, Python rejects the program with the message `SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?`

move backward if the value of `n` is negative and which does not change its position when the value of `n` is zero can be defined as follows:

```
# Move forward n times if n is positive, back -n times if
# n is negative, do not move if n is zero.
def fdn(n):
   if n > 0:
      for i in range(n):
         fd()
   elif n < 0:
      ht()
      for i in range(-n):
         fd()
      ht()
```

Note that the negative actual parameter of the call `fdn(-5)` is simply written with its sign, between the parentheses of the call. To move backward, the *Robot* makes a half turn, moves forward `-n` times (which is positive) and makes a half turn again.

More generally, we can use the operators `<` (strictly less than), `<=` (less than or equal), `==` (equal), `!=` (different), `>=` (greater than or equal) and `>` (strictly greater than) to compare the values of two integer expressions, as for example `i <= l` in the following program which draws the paper streamer below. The command `gs` accepts an integer, as in `gs(20)` or `gs(i)`, as well as a number with a decimal part, written with a point as in `gs(20.5)` (we will come back to this in chapter 19, devoted to rational expressions):



```
from LibRobot import *

# Draw a streamer
def drawStreamer(l):
   for i in range(1, 2 * l + 1):
      if i <= l:
         gs(i)
      else:
         gs(2 * l + 1 - i)
      if i % 2 == 1:
         # Upper semicircle
         cr(); cr()
      # Lower semicircle
      else:
```

```
        cl(); cl()
gl(); drawStreamer(13); st()
```

16.3 Negation, conjunction and disjunction

The *negation* of a boolean expression E is written **not** E, which reads 'not E'. If the value of E is **True** then that of **not** E is **False** whereas if the value of E is **False** then that of **not** E is **True**. For example **not** `(5 % 2 == 1)` is **False** since `5 % 2 == 1` is **True**, `5` being odd. The parentheses are not indispensable but make the expression more readable (we will see why in the Addenda). This can be summarised by the following truth table:

E	**not** E
True	**False**
False	**True**

The *conjunction* of two boolean expressions E_1 and E_2 is written E_1 **and** E_2, which reads 'E_1 and E_2'. The value of E_1 **and** E_2 is **True** if and only if the value of E_1 and that of E_2 are **True**. If either of the boolean expressions E_1 or E_2 is **False** then the conjunction E_1 **and** E_2 is **False** too. For example `5 >= 0` **and** `5 % 2 == 1` is **True** since `5` is positive and odd. This can be summarised by the following truth table:

E_1	E_2	E_1 **and** E_2
True	**True**	**True**
True	**False**	**False**
False	**True**	**False**
False	**False**	**False**

The *disjunction* of two boolean expressions E_1 and E_2 is written E_1 **or** E_2, which reads 'E_1 or E_2'. The value of E_1 **or** E_2 is **True** if and only if the value of E_1 or that of E_2 is **True**. If the values of E_1 and of E_2 are **False** then the disjunction E_1 **or** E_2 is **False** too. For example `4 >= 0` **or** `4 % 2 == 1` is **True** since `4` is positive, even though `4` is even. This can be summarised by the following truth table:

E_1	E_2	E_1 **or** E_2
True	**True**	**True**
True	**False**	**True**
False	**True**	**True**
False	**False**	**False**

Conjunction and disjunction can be implemented simply with an electric circuit as shown in figure 16.1 below with the convention that **True** represents a closed switch and a lit bulb (the current flows) and **False** represents an open switch and an unlit bulb (the current does not flow).

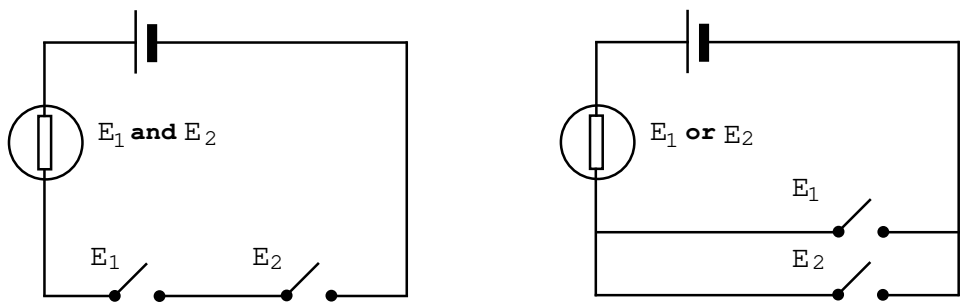


FIG. 16.1 – *Electric circuits implementing logical conjunction and* disjunction

A disjunction is used in the following program to test whether the *Robot* is facing north or east:

```
from LibRobot import *

# Draw a golden frieze
length = 31

gl()
for i in range(length):
   if tn() or te():
      cr()
   else:
      fdn(-1); cl(); qtl()
st()
```

16.4 Boolean expressions

The table below summarises the rules for writing boolean expressions and the rules for computing their value in Python:

Boolean expressions:	
True	Value 'true'.
False	Value 'false'.
Integer expr.$_1$ `<=` Integer expr.$_2$	**True** if and only if the value of Integer expr.$_1$ is less than or equal to that of Integer expr.$_2$.
Integer expr.$_1$ `<` Integer expr.$_2$	**True** if and only if the value of Integer expr.$_1$ is strictly less than that of Integer expr.$_2$.
Integer expr.$_1$ `==` Integer expr.$_2$	**True** if and only if the value of Integer expr.$_1$ is equal to that of Integer expr.$_2$.
Integer expr.$_1$ `!=` Integer expr.$_2$	**True** if and only if the value of Integer expr.$_1$ is different from that of Integer expr.$_2$.

Integer expr.$_1$ `>` Integer expr.$_2$	**True** if and only if the value of Integer expr.$_1$ is strictly greater than that of Integer expr.$_2$.
Integer expr.$_1$ `>=` Integer expr.$_2$	**True** if and only if the value of Integer expr.$_1$ is greater than or equal to that of Integer expr.$_2$.

not Bool. expr.	**True** if and only if the value of Bool. expr. is equal to **False**.
Bool. expr.$_1$ **and** Bool. expr.$_2$	**True** if and only if the values of Bool. expr.$_1$ and of Bool. expr.$_2$ are equal to **True**.
Bool. expr.$_1$ **or** Bool. expr.$_2$	**True** if and only if the value of Bool. expr.$_1$ or that of Bool. expr.$_2$ is equal to **True**.

A boolean expression can be given a name to make programs more readable. For example:

```
# is n odd?
def isOdd(n):
    return n % 2 == 1

# is n even?
def isEven(n):
    return n % 2 == 0
```

These declarations look like declarations of procedures with parameters but their body is not a sequence of instructions for the $\mathcal{R}^{o}\!\!\smile^{o}t$: the **return** instruction is followed by a boolean expression whose value is computed at each call and returned as the result of the call. We say that `isOdd` and `isEven` are *boolean functions* because their evaluation always returns a boolean (a value of type `bool`, that is **True** or **False**). For example `isOdd(5)` is **True** and `isEven(5)` is **False**. They can be used in a test such as **if** `isOdd(i):` ... or in a boolean expression such as **not** `isOdd(i)`. The orientation tests `tn()`, `te()`, ... of the $\mathcal{R}^{o}\!\!\smile^{o}t$ are also boolean functions, declared in the `LibRobot` library.

Exercise 29 *Write Python programs using parity tests to produce the following drawings:*

- *the pyramidal serpentine and the grid of figure 16.2 below;*
- *the octagonal tiling and the tiling of interlaced hexagons of figure 16.3 (page 184).*

□

Exercise 30 *Write Python programs using boolean expressions comparing integer values to produce the following drawings (the second one is difficult):*

- *the digital digits of figure 1.6 (page 8) where the digit to draw is passed as a parameter of the drawing procedure;*
- *the tiling of cogwheels of figure 16.4 below;*
- *the Egyptian carpet of figure 16.5 (page 185).*

□

Exercise 31 *Write Python programs using negations, conjunctions or disjunctions of boolean expressions to produce the following drawings (the second one is difficult):*

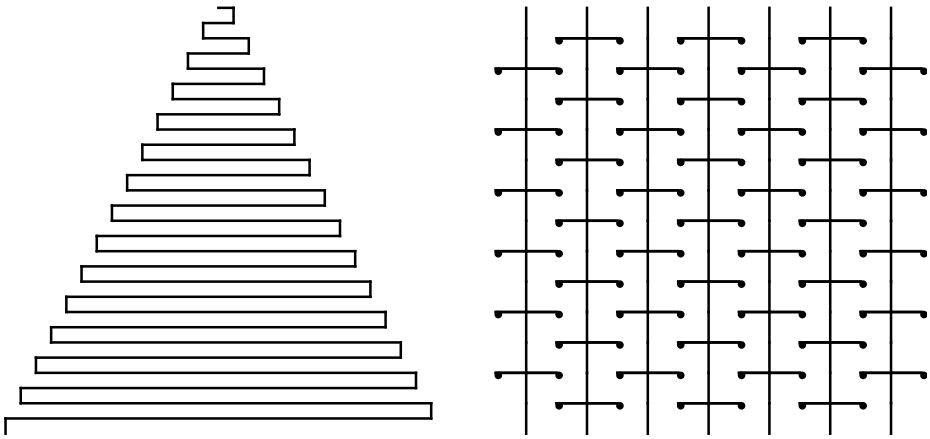


FIG. 16.2 – *Pyramidal serpentine and grid of crosses*

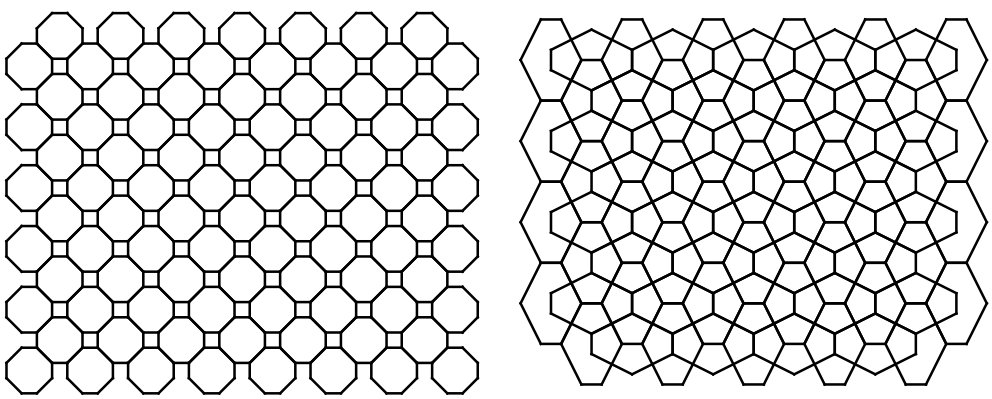


FIG. 16.3 – *Tilings of juxtaposed octagons and of interlaced hexagons*

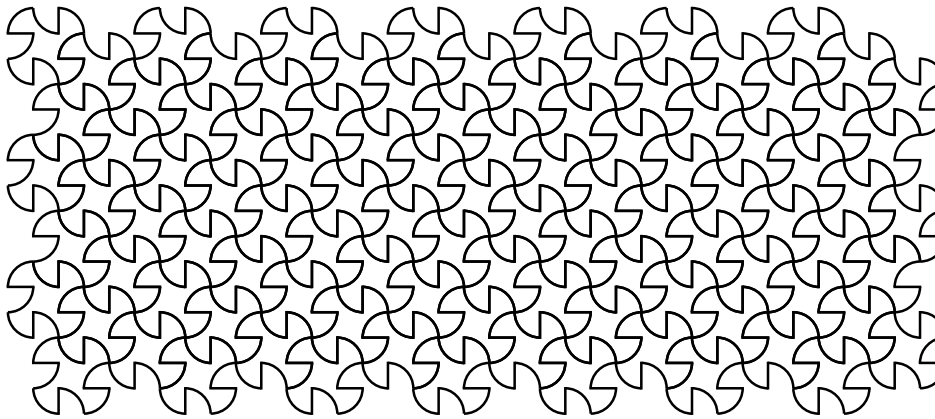


FIG. 16.4 – *Tiling of cogwheels*

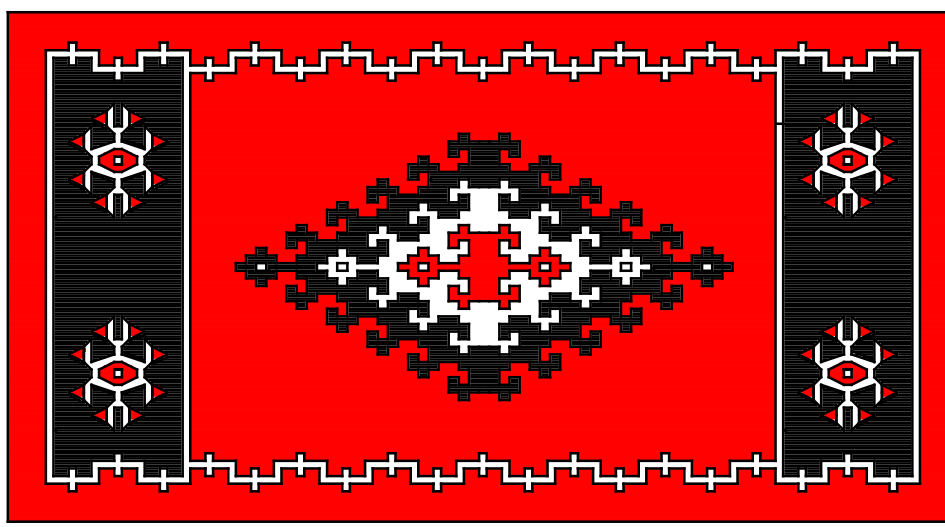


FIG. 16.5 – *Egyptian carpet*

– *the zebra-striped square of figure 16.6 (page 186);*
– *the Alhambra mosaic shown in figure 16.7 (page 186).*
□

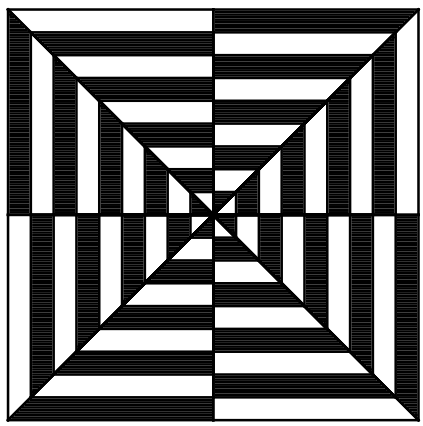


FIG. 16.6 – *Zebra-striped square*

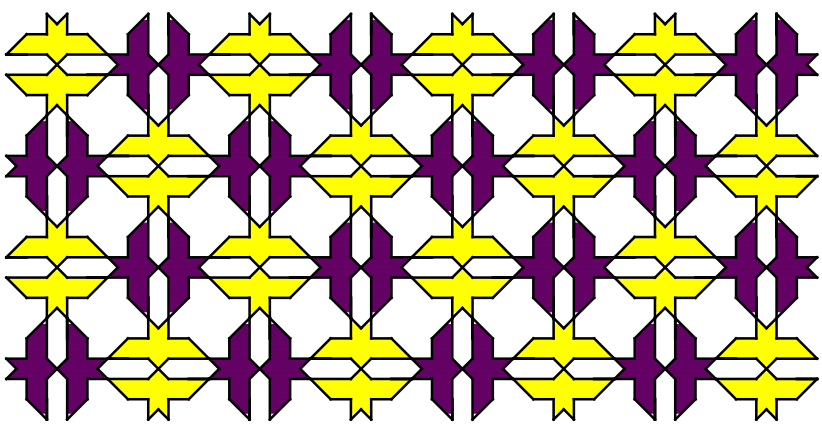


FIG. 16.7 – *Alhambra mosaic*

Solution 29 *We give one program, the others being in the folder* `book/16_boolean_expressions`*:*

```
from LibRobot import *

# Draw a pyramidal serpentine
gridSize = 6
height = 28

gs(gridSize); gt(); qtr()
for i in range(1, height + 1):
   fdn(i)
   if i % 2 == 1:
      qtr(); fd(); qtr()
   else:
      qtl(); fd(); qtl()
st()
```

□

Solution 30 *We give one program, the third one being in the folder* `book/16_boolean_expressions`*:*

```
from LibRobot import *

# Draw a tiling of cogwheels
gridSize = 10
numberOfPatternsPerRow = 7
numberOfRows = 14

# Move the robot straight on, up to two squares from
# the frame's north edge.
def nearTheTop():
   gt(); ht(); fdn(2)

# Draw the pattern of the tiling
def cogwheel():
   for i in range(4):
      fd(); qtr(); cr(); qtl(); cl(); qtl()

def tiling():
   for l in range(1, numberOfRows + 1):
      pu(); nearTheTop(); gl()
      fdn(l); qtl(); fd()
      # Offset at the start of the row
      if l % 5 == 2:
         fdn(2)
      elif l % 5 == 3:
         fdn(4)
      elif l % 5 == 4:
```

```
            fd()
         elif l % 5 == 0:
            fdn(3)
         # Draw a row of patterns
         qtl(); pd()
         for j in range(numberOfPatternsPerRow - 1):
            cogwheel()
            translate(5, 0)
         cogwheel()
   gs(gridSize); tiling(); st()
```

□

Solution 31 *We give one program, the first one being in the folder* `book/16_boolean_expressions`*:*

```
from LibRobot import *

# Draw an Alhambra mosaic
gridSize = 8  # must be even
numberOfRows = 4
numberOfColumns = 8

def halfPattern():
   etr(); fd(); etr(); fd(); qtl(); fd()
   gs(gridSize // 2)
   tetr(); fd(); qtl(); fd()
   gs(gridSize)
   tetr(); fd(); qtl(); fd(); etr(); fd()
   tetr(); fdn(2)
   gs(gridSize // 2)
   etl(); fd(); qtr(); fd()
   gs(gridSize); etl(); fdn(2)
   # Paint
   fdn(-1); etl()
   if tsw() or tne():
      pc(yellow)
   else:
      pc(indigo)
   paint()
   pc(black); etr(); fd()
   # Put the robot back in its initial orientation
   qtr()

def pattern():
   halfPattern()
   pu(); qtr(); fdn(5); qtr(); fd(); pd()
   halfPattern()
```

```
def doublePattern():
   pattern()
   # Next pattern
   pu(); etl(); fdn(2); tetr(); pd()
   pattern()
gs(gridSize); gt(); gl()
translate(1, -3)
for j in range(1, numberOfRows + 1):
   for i in range(numberOfColumns // 2):
      doublePattern()
      # Next double pattern
      pu(); etr(); fdn(2); tetl(); pd()
   # Next row
   if j % 2 == 1:
      pu(); ht(); fdn(6); pd()
   else:
      pu(); fdn(4); ht(); pd()
st()
```

□

Addenda

In Python the priorities of integer and boolean operators are defined as follows:

Priority of integer and boolean operators:		
`f(x)` (call)	function calls	highest priority
− (unary)	opposite	
∗, //, %	multiplicative operators	
+, − (binary)	additive operators	
<, <=, ==, !=, >=, >	relational operators	
not	negation	
and	conjunction	
or	disjunction	lowest priority

For example the boolean expression '**True and False or False**' is evaluated as '**(True and False) or False**' and is therefore equal to **False**.

These priorities are close to the usual mathematical conventions. For example '`0 <= i` **and** `i < 100`' is evaluated as '`(0 <= i)` **and** `(i < 100)`', which is indeed what we mean. This was not the case in older languages such as PASCAL, where this expression was incorrect and all the parentheses had to be written. Python even allows writing '`0 <= i < 100`' as in mathematics.

Since **not** has a lower priority than the relational operators, '**not** `i % 2 == 1`' is evaluated as '**not** `(i % 2 == 1)`', which is indeed what we mean. This is not the case in other languages such as PASCAL or OCaml, where **not** applies to what immediately follows it, like a function call, so that the parentheses must be written. When in doubt, one can always add parentheses.

Finally, Python evaluates conjunctions and disjunctions from left to right and stops as soon as the result is known: in E_1 **and** E_2, if E_1 is **False** then E_2 is not computed since the conjunction is necessarily **False**; likewise in E_1 **or** E_2, if E_1 is **True** then E_2 is not computed.

The term "boolean" (`bool` in Python) comes from Boole[a], creator of modern mathematical logic.

[a] George Boole, British mathematician, 1815–1864.

17

"while" loops

17.1 Is the robot at the edge of the frame?

We saw in the previous chapter that the boolean functions `tn()`, `tne()`, `te()`, `tse()`, `ts()`, `tsw()`, `tw()`, `tnw()` can be used in "if" statements to find out the orientation of the $\mathcal{R}\!\underline{o}\!\underline{b}\!^{o}t$. Another boolean function, `tb()` (test border), makes it possible to test whether the $\mathcal{R}\!\underline{o}\!\underline{b}\!^{o}t$ is at the edge (the border) of the frame. More precisely `tb()` is **True** if the supposed execution of the command `fd()` would take the $\mathcal{R}\!\underline{o}\!\underline{b}\!^{o}t$ out of the frame in which it moves. The boolean expression `tb()` is **False** if the supposed execution of the command `fd()` moves the $\mathcal{R}\!\underline{o}\!\underline{b}\!^{o}t$ only inside the frame in which it moves. For example, we can define a command `fdWithoutLeaving()` which orders the $\mathcal{R}\!\underline{o}\!\underline{b}\!^{o}t$ to execute an `fd()` if this does not take it out of the frame:

```
# Move forward if it keeps the robot inside the frame
def fdWithoutLeaving():
    if not tb():
        fd()
```

The empty parentheses of **not** `tb()` are indispensable. If we forget them and write **not** `tb`, Python reports no error, but the program does not do what we want: `tb` without parentheses denotes the function itself, and not the value computed by calling it. Now Python considers that a function, like any value that is not zero or empty, counts as **True**. The expression **not** `tb` is therefore always **False** and the $\mathcal{R}\!\underline{o}\!\underline{b}\!^{o}t$ never moves forward, whether it is at the edge of the frame or not. This is an error that is hard to find since nothing reports it!

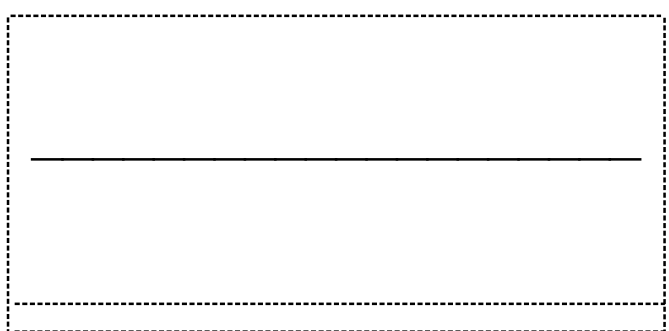
17.2 "while" loop

A "while" loop makes it possible to repeat the execution of a sequence of instructions zero or more times, testing at each iteration whether to stop or to continue. The English word "while" means here 'as long as'. The "while" loop makes it possible to repeat the execution of a sequence of instructions as long as the value of a boolean expression is **True**. For example, the following program draws a horizontal line across the middle of the screen by orienting the $\mathcal{R}^{o}\!\!\smile^{o}t$ towards the east and repeating `fd()` as long as the $\mathcal{R}^{o}\!\!\smile^{o}t$ does not leave the frame:

```
from LibRobot import *

# Draw a horizontal line in the middle of the screen
gl(); fe()
while not tb():
   fd()
st()
```

The result of the execution of this program is shown reduced in the figure below, the frames in which the $\mathcal{R}^{o}\!\!\smile^{o}t$ moves and in which error messages are displayed being dotted:



A "while" loop always has the following form:

"while" loop:

while ␣ Boolean expression ␣ :
    ... Repeated instructions (indented) ...

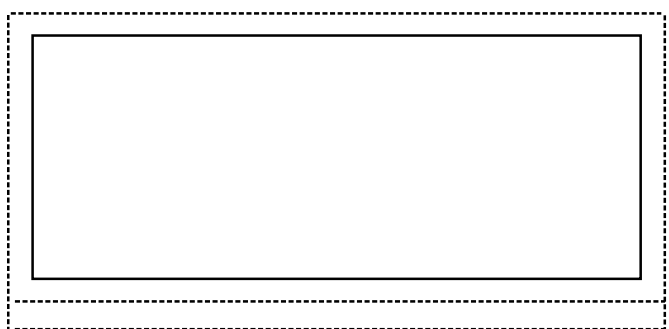
The sequence of instructions which is repeated as long as the value of the Boolean expression is **True** is called the *body of the loop*. As for the "for" loop,

the **while** line ends with a colon : and the instructions of the body of the loop are indented (shifted to the right), the loop ending at the first instruction which is no longer indented. If the Boolean expression is **False** when the execution of the "while" loop begins then this execution ends immediately without the body of the loop being executed. If the Boolean expression is always **True** at each iteration then the execution of the "while" loop never ends! One can still stop the computer by switching off the power. More recommendable methods are explained below.

A serious error is to write a loop body which does not change the value of the Boolean expression. For example if we write **while not** `tb(): qtr()`, the *Robot* turns on the spot without ever getting closer to the edge, and the computer loops forever.

We preferably use "for" loops (whose execution always terminates), reserving the use of "while" loops for the case where the number of iterations cannot be computed by an integer expression before starting to execute the body of the loop.

Example 21 (Frame) The program below draws a frame around a drawing:



```
from LibRobot import *

# Draw a frame around the drawing

# Reach the edge by moving straight ahead
def toEdge():
    while not tb():
        fd()

def frame():
    # Reach the top edge without drawing anything
    pu(); toEdge()
```

```
    # Draw the right half of the top edge
    qtr(); pd(); toEdge()
    # Draw the right edge
    qtr(); toEdge()
    # Draw the bottom edge
    qtr(); toEdge()
    # Draw the left edge
    qtr(); toEdge()
    # Draw the top edge
    qtr(); toEdge()
    # Place the robot at the centre, facing north
    gc(); fn()
frame(); st()
```

□

All the programs we had written until now terminated. This means that any execution of the program takes a finite time, which sometimes seems long, but we can be sure that by waiting long enough, we will always be able to observe a result. If for example we have lifted the pen of the $\mathcal{R}^{o}_{b}{}^{o}t$ and forgotten to lower it afterwards, the result is a blank sheet but the execution of the program terminates all the same.

With the "while" loop, we can write programs whose execution does not terminate. This is the case for the following program:

```
from LibRobot import *
# Make the robot turn forever. The body of the loop
# contains a robot command: its execution can be
# stopped by typing Esc, § or # in the robot's
# window.
pu()
while True:
   cr()
st()
```

The $\mathcal{R}^{o}_{b}{}^{o}t$ turns in the centre of the screen, as long as the computer is running and nobody intervenes. It can be stopped by pressing, in the window of the $\mathcal{R}^{o}_{b}{}^{o}t$, one of the keys [esc], [§] or [#]. The execution of the program ends because before executing a command, the $\mathcal{R}^{o}_{b}{}^{o}t$ checks whether the user has pressed one of these keys to ask it to stop.

On the other hand, if we execute the following Python program:

```
from LibRobot import *

# Loop forever. The body of the loop contains
# no robot command: so its execution cannot be
# stopped with Esc, § or #. You must type
# Ctrl+C in the terminal window.
pu()
while True:
    pass
```

the execution loops without stopping even if we press the stop keys esc, § or # since the Robot receives no command while the computer executes this loop, whose body is reduced to **pass**, an instruction which does nothing (Python requires at least one instruction in the body of a loop and **pass** serves that purpose).

To stop the execution of this program, one must go back to the terminal window in which it was launched, for example with the command:

```
./robot.sh book/17_while/Loop2.py
```

hold down the control key Ctrl and press the key C. The operating system of the computer then interrupts the program, whatever it is doing (and the window of the Robot closes).

If the program was launched by dragging its icon onto the application `LeRobotPythonEN.app`, a terminal window opens, which can be selected with the mouse in order to type Ctrl+C.

Exercise 32 *Write Python programs using "while" loops to produce the following drawings:*

- *the frieze of figure 17.1 below;*
- *the grid of the stopping points of the Robot of figure 17.2 (page 197), without of course using the command* `dg()`*;*
- *the square grid of figure 17.2 (page 197);*
- *the lattice of figure 17.2 (page 197) obtained by superimposing two square grids, one of which is turned by an eighth turn.*

In all cases the drawing must be as large as possible and fit inside the frame set for the movements of the Robot on the computer screen.
□

Exercise 33 *Define the commands* `gl()`, `gr()`, `gt()` *and* `gb()` *by procedures containing "while" loops (use the boolean function* `valPd()` *which is* **True** *if and only if the pen of the Robot is down).* □

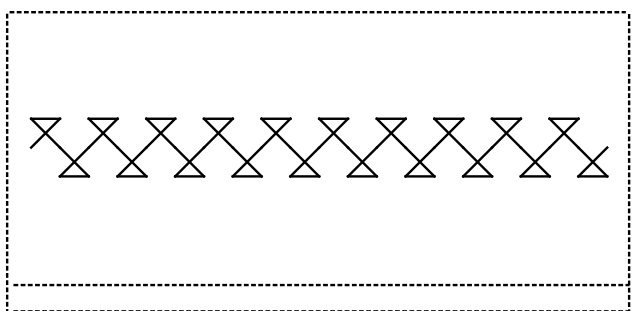


FIG. 17.1 – *Frieze of triangles (on a reduced screen)*

Solution 32 *We give one program, the others being in the folder* `book/17_while`*:*

```
from LibRobot import *
# Draw a frieze of triangles from one edge to the other
# Reach the edge by moving straight ahead
def toEdge():
    pu()
```

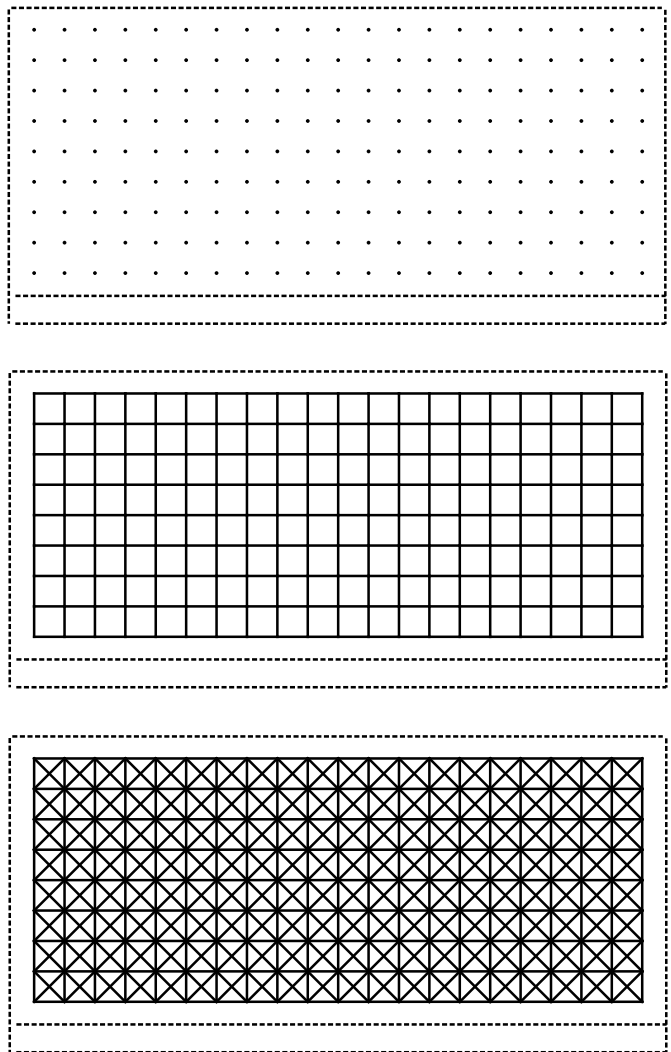


FIG. 17.2 – *Stopping points, square grid and lattice on a reduced screen*

```
    while not tb():
      fd()
    pd()
  def pattern():
    if not tb():
      fd(); tetl(); fd(); tetl(); fd()
      if not tb():
        fd(); tetr(); fd(); tetr(); fd()
  def frieze():
    fne()
    while not tb():
      pattern()
  fw(); toEdge(); frieze(); st()
```

□

Solution 33 *We give the solution for the command* `gl`*, the others being in the program* `GlGrGtGb` *of the folder* `book/17_while`*:*

```
  # Definition of the command gl
  def toWestEdge():
    pu(); fw()
    while not tb():
      fd()
  def farWest():
    if tn():
      toWestEdge(); fn()
    elif tne():
      toWestEdge(); fne()
    elif te():
      toWestEdge(); fe()
    elif tse():
      toWestEdge(); fse()
    elif ts():
      toWestEdge(); fs()
    elif tsw():
      toWestEdge(); fsw()
    elif tw():
      toWestEdge()
    else:
      toWestEdge(); fnw()
  def gl():
    if valPd():
      farWest(); pd()
```

```
else:
    farWest()
```

□

Addenda

In other programming languages, such as PASCAL, there is another form of unbounded repetition, the "repeat" loop. It has the following form:

repeat
  ... Repeated instructions (separated by ';') ...
until ␣ Boolean expression

The body of the loop is repeated (`repeat`) until (`until`) the value of the Boolean expression is equal to `true`. It is therefore always executed at least once.

The "repeat" loop does not exist in Python, but it can always be expressed with a "while" loop. A "repeat" loop:

repeat
  Body of the loop
until Boolean expression

can be written in Python with a "while" loop as follows:

Body of the loop
while not Boolean expression:
  Body of the loop

but the body of the loop must then be written twice. In Python one often prefers to write a "while" loop whose test is always **True** and which is exited with the instruction **break**, which immediately terminates the execution of the loop:

```
while True:
  Body of the loop
  if Boolean expression:
    break
```

Likewise, a "while" loop:

while Boolean expression**:**
  Body of the loop

could be rewritten with a "repeat" loop as follows:

if Boolean expression **then**
  repeat
    Body of the loop
  until not (Boolean expression)

The use of one or the other form of iteration depends only on whether the body of the loop may never be executed ("while" loop) or must always be executed at least once ("repeat" loop). The designers of Python decided that the "while" loop was enough.

18

Coding

The memory of a computer contains nothing but zeros and ones. So we must be able to represent all the information we want to store as sequences of zeros and ones. This translation of information into binary is called *binary coding*. The digits 0 and 1 used to represent information are called *bits*. We shall begin by studying other, older forms of coding.

18.1 Morse code

The electric telegraph, the ancestor of our telephone, was invented in 1837 by the American Samuel Morse [3]. The sending telegraph operator has a key, which is a simple spring lever used to send short signals (called dots) and long signals (called dashes) separated by silences or blanks (corresponding to the absence of a signal). The signal is carried along an electric wire, the return path being through the earth. Each movement of the key closes the circuit and causes a short or long movement of the receiver, which reproduces the signals on a paper tape. To send a message, one first shortens it into *telegraphic style*, then translates each sign (letter, digit, punctuation mark, ...) into a sequence of dots and dashes separated by blanks according to the *Morse code* of figure 18.1 (page 202). Seven blanks are left at the end of each word. Various other codes are used to mark the beginning and the end of the message, to signal errors, etc. Morse's telegraph could transmit about 25 words a minute.

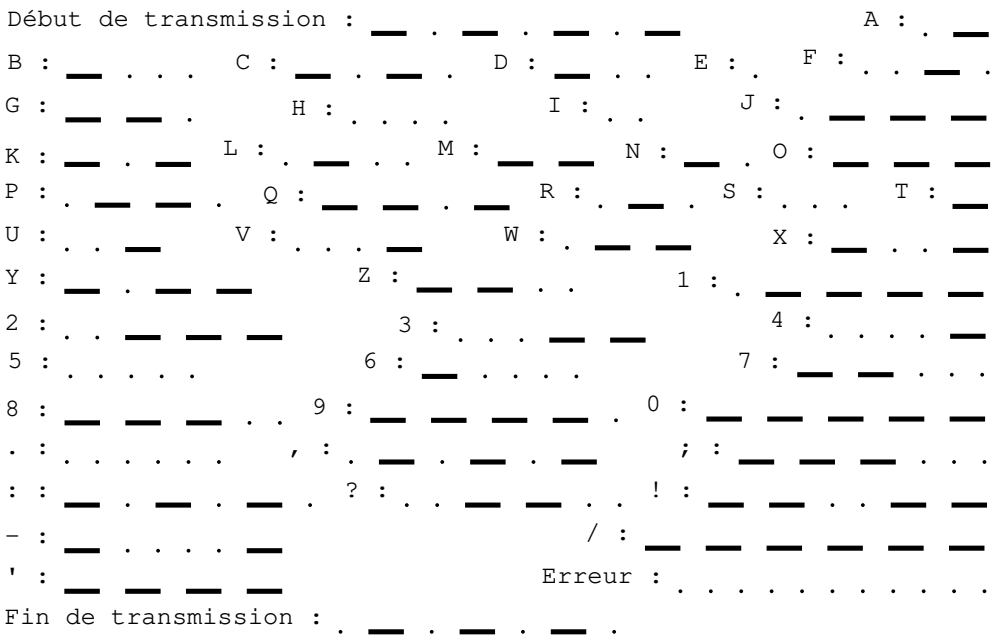


FIG. 18.1 – *Morse code*

18.2 Number systems

Our way of coding the natural numbers (positive or zero) is a positional, decimal number system. It is a decimal system because we use the ten digits 0, 1, ..., 9, which are of Indo-Arabic origin. It is a positional system because each digit in a number represents a value that depends on its position in the number. For example in 1789, the 1 represents 1000, the 7 represents 700, the 8 represents 80 and the 9 represents 9. So we have $1789 = 1000 + 700 + 80 + 9 = (1 \times 10^3) + (7 \times 10^2) + (8 \times 10^1) + (9 \times 10^0)$. For the digits after the decimal point we use the inverses. For example $0.147 = (1 \times 10^{-1}) + (4 \times 10^{-2}) + (7 \times 10^{-3}) = \frac{1}{10} + \frac{4}{100} + \frac{7}{1000} = \frac{100}{1000} + \frac{40}{1000} + \frac{7}{1000} = \frac{147}{1000}$. Mathematicians, ancient and modern, have invented many other ways of coding numbers. Figure 18.2 (page 204) shows different ways of coding the number 123,456,789, some of them going back to ancient civilisations [16]. The superiority of the positional, decimal coding of the natural numbers does not come from the choice of base ten but from the ease with which the usual arithmetic operations can be carried out, for example with an abacus (long

in use, and now gone from primary schools).

18.3 Binary coding

All the information stored in the computer's memory or on disk must be represented by sequences of zeros and ones. On microcomputers, as they are designed today, the length of these sequences of bits must be a multiple of 8. Depending on their length they are given special names, such as *byte* for a sequence of 8 bits, *half-word* for a sequence of 16 bits, *word* for 32 bits and *double word* for 64.

To show that any information can be represented by a sequence of bits (provided we know the code), we shall take the example of integers, then that of characters and of texts typed on the keyboard.

18.3.1 Natural numbers

To represent positive integers by a sequence of bits, it is enough to write them in base 2. For example:

$$\begin{aligned} 147 &= 128 + 16 + 2 + 1 \\ &= (1 \times 2^7) + (0 \times 2^6) + (0 \times 2^5) + (1 \times 2^4) + (0 \times 2^3) + (0 \times 2^2) \\ &\quad +(1 \times 2^1) + (1 \times 2^0) \end{aligned}$$

is written:

$$10010011$$

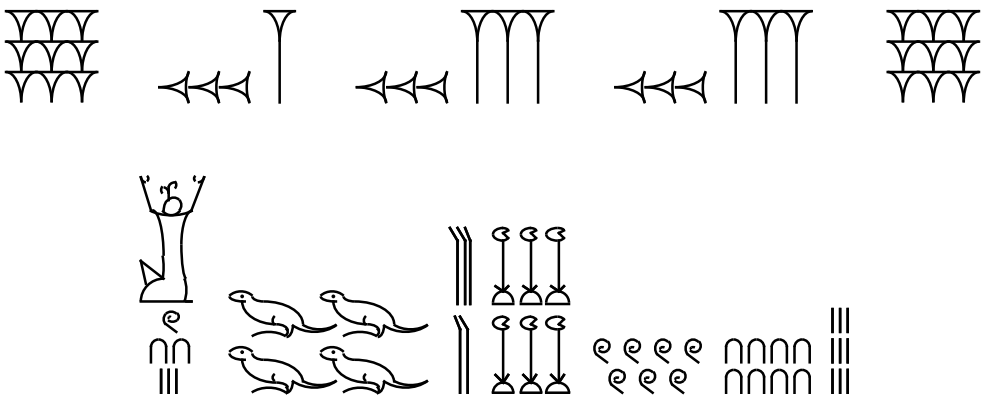
in base two. More generally, the number written:

$$a_n a_{n-1} \ldots a_1 a_0$$

in base two (where each of $a_n \ldots a_0$ is 0 or 1) is equal to:

$$(a_n \times 2^n) + (a_{n-1} \times 2^{n-1}) + \ldots + (a_1 \times 2^1) + (a_0 \times 2^0)$$

(where 2^n is equal to 2 multiplied by itself n times if $n > 0$, and $2^0 = 1$). With this convention we can therefore code on one byte the natural numbers between 0 (00000000) and 255 (11111111).



𒐜𒐜𒐜𒐐𒐐𒐐𒐐𒌋𒌋𒁹𒁹𒁹𒁹𒀭𒀭

MCCXXXIV LVI DCCLXXXIX

75BCD15

123456789

726746425

2212102202021220 0

111010110111100110100010101

FIG. 18.2 – *Coding of 123,456,789 in Babylonian, Egyptian, Greek, Roman,* hexadecimal, decimal, octal, ternary and binary number systems

18.3.2 Characters (ASCII code)

The Python programs we write are sequences of characters. Each time we press a key on the keyboard, the code of the corresponding character is entered into the computer's memory. The characters corresponding to the digits, to the lower-case or upper-case letters and to common characters such as +, ∗, (,), ... are called "printable" characters. Some other characters, which cannot be written, are called "special" characters. This is the case, for example, of the "control character" and the "escape character", which are obtained by pressing the keys Ctrl and Esc respectively. These characters (printable or special) are often represented on one byte (8 bits) according to an international code called the ASCII code. The printable characters whose ASCII code lies between 32 and 126 are given in table 18.1 below. For example, the ASCII code 65 of the character `'A'` is obtained from this table by adding the number 64 of the row containing the character `'A'` to the number 1 of the column of this character.

+	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
32	␣	!	"	#	$	%	&	’	(	)	*	+	,	-	.	/
48	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
64	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
80	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
96	‘	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
112	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	

TAB. 18.1 – *Characters with* ASCII *codes between 32 and 126*

When we say that the character `'A'` has ASCII code 65, we mean that this character is represented in the computer's memory by the byte 01000001, which also represents the number 65 written in base two. Notice that the character zero has code 48. The integer 0 (coded in the machine on one byte as 00000000) therefore has a code different from that of the character zero `'0'` (coded in the machine as 00110000). Table 18.2 (page 206) shows the ASCII codes of a few special characters.

Accented letters such as `'é'` or `'ç'` (used in French) do not appear in the ASCII code, which was designed for the English language. Today they are coded according to the international code UNICODE, which can represent the characters of all the world's writing systems. In the UTF-8 format used by

ASCII code	8	13	27	127
Keyboard key	←	↵	Esc	Del

TAB. 18.2 – ASCII *codes of a few special characters*

Python for the text of programs, ASCII characters are coded on one byte as above, and the other characters on two, three or four bytes. For example the letter `'é'` is coded by the two bytes 11000011 10101001.

18.3.3 Text

A text (such as a Python program) is a sequence of characters. The code of a text in the machine is the sequence of the codes of the characters that make it up. For example the text `"Python"` will be represented in the machine by the sequence of the ASCII codes of the characters `'P'`, `'y'`, `'t'`, `'h'`, `'o'`, `'n'`, that is, according to table 18.1 (page 205), the sequence 80 (01010000), 121 (01111001), 116 (01110100), 104 (01101000), 111 (01101111), 110 (01101110). The code of the text `"Python"` is therefore 01010000 01111001 01110100 01101000 01101111 01101110.

18.3.4 Character strings

A program can make the computer handle booleans and integers, but also texts, all coded in binary. So as not to confuse the text of the program with the texts handled by the program, we shall call the latter "character strings".

Use and notation in Python:

In Python we can handle texts as long as we like, which are called "character strings". They have type `str` (short for `string`). A character string is written between straight double quotes, as for example:

```
"This is a character string"
```

While it is easy to pick out an identifier or an integer in the text of a program, it is harder to recognise a text inside this text! The quotes therefore mark off the part of the text that forms the character string from the rest of the program. But this causes a problem if the character string contains a double

quote. That is why the convention is to put a backslash \ in front of the double quotes inside a character string, as in:

`"The \"Python\" language"`.

A character string consisting of a single double quote is therefore written `"\""` and a string consisting of a single backslash is written `"\\"`. In Python, a character string can also be written between single quotes (apostrophes), as in `'The "Python" language'`, which avoids having to put a backslash in front of the double quotes. Python has no special type for characters (characters, often called *char* in other languages): the character `'A'` is simply the character string `"A"`, of length 1.

Concatenation of character strings:

If c_1 and c_2 are character strings then c_1 + c_2 is the character string obtained by concatenating (that is, putting end to end) the character strings c_1 and c_2. Several strings can be concatenated: c_1 + c_2 + ... + c_n (so the + sign, which adds two numbers, puts two character strings end to end). For example:

`"The " + "\"" + "Python" + "\"" + " language"`

is equal to the character string that is also written:

`"The \"Python\" language"`

Writing on the robot's drawing and below it:

Whether its pen is up or down, the $\mathcal{R}^{o}_{b}{}^{o}t$ can write character strings, without moving and without changing its orientation. It can write in the drawing window or in the message box located below the drawing window.

The command `label(`c`)` tells the $\mathcal{R}^{o}_{b}{}^{o}t$ to write the character string c to the right of the point where it stands in its drawing window. The command `message(`c`)` tells the $\mathcal{R}^{o}_{b}{}^{o}t$ to write the character string c in the box provided below the window in which it moves. This erases the previous message displayed in this box. To give the user time to read the message, one can, if necessary, use the procedure `markPause()`, which waits until the user presses a key or clicks the mouse button before continuing. The command `clearMessage()` erases this message. The commands `textFont`, `textStyle` and `textSize` of the `LibRobot` library let you choose a font, a style or a character size other than the standard one (on some computers, they have no effect).

If n is an integer then `str(`n`)` is the character string denoting this integer in the usual decimal notation (the `LibRobot` library also offers `intToString(`n`)`, which does the same thing). If b is a boolean then `boolToString(`b`)` is `"True"` if b is equal to **True**, and `"False"` if b is equal to **False** (as does `str(`b`)`). For example if `x` is an integer parameter of a procedure equal to 17 then the command:

```
label("The value of x is " + str(x) + ".")
```

writes the concatenation of the character strings `"The value of x is "`, `"17"` and `"."`, that is:

```
The value of x is 17.
```

to the right of the $\mathcal{R}\text{o}\underset{\frown}{\text{b}}\text{o}t$. Notice that the conversion `str(x)` is essential: without it, Python would refuse to add an integer to a character string and would stop the program with the message `TypeError: can only concatenate str (not "int") to str`. If `b` is a boolean parameter of a procedure equal to **True** then the command:

```
message("b = " + boolToString(b))
```

writes the concatenation of the character strings `"b = "` and `"True"`, that is:

```
b = True
```

in the message box.

18.4 Music

This section is meant for musicians who know how to read music, that is, the way music is coded. The $\mathcal{R}\text{o}\underset{\frown}{\text{b}}\text{o}t$ can play music, in a rather limited way it must be said, since it can only play the piano with one finger. To code music for the $\mathcal{R}\text{o}\underset{\frown}{\text{b}}\text{o}t$ we have procedures with parameters. So all we need is to set up the correspondence with classical music theory [13] to define how to write music for the $\mathcal{R}\text{o}\underset{\frown}{\text{b}}\text{o}t$.

18.4.1 Note values

The procedure `note(p, d)` plays on the computer's loudspeaker a note of pitch `p` and value (duration) `d`. The pitch of the note is coded by an integer `p` as shown in figure 18.3 (page 209). The value of the note gives its duration, which is coded by a constant `d` that can be `whole` (whole note), `dotted_whole` (dotted whole note), `half` (half note), `dotted_half` (dotted
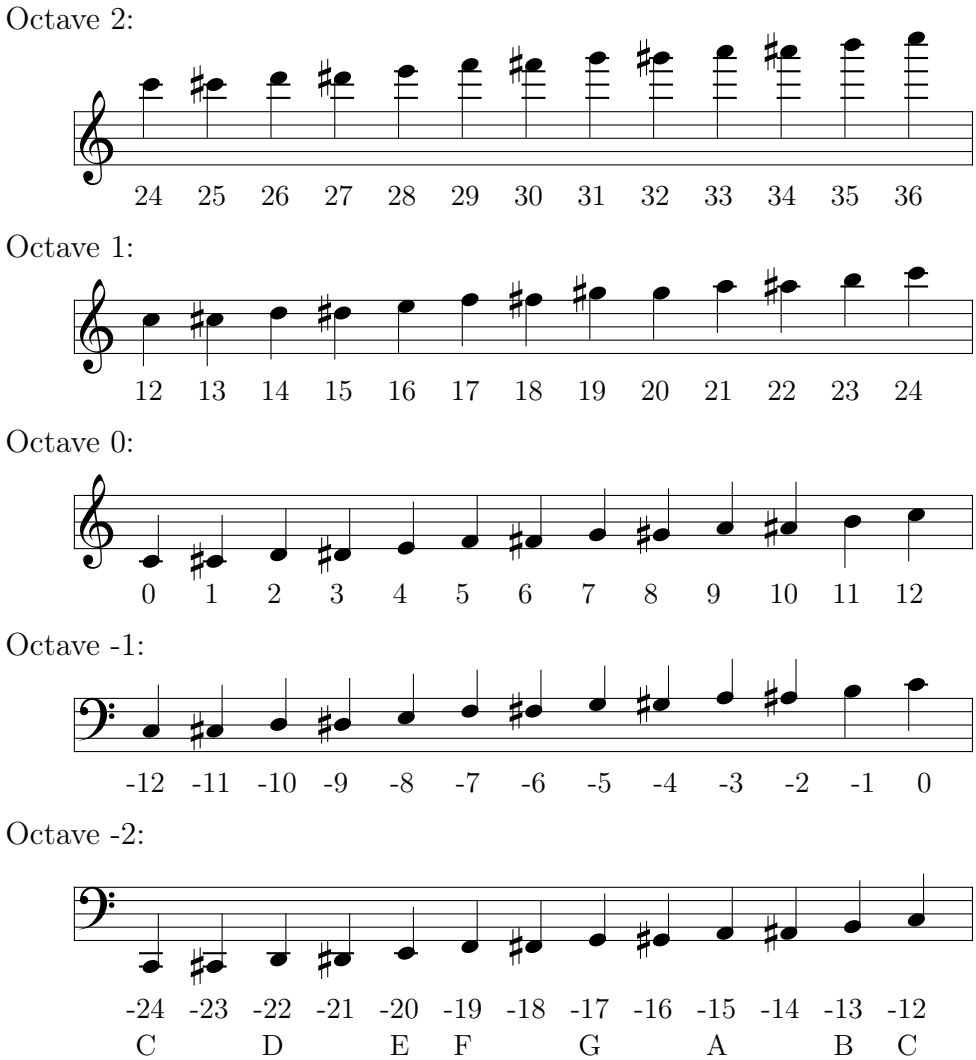


FIG. 18.3 – *Coding the pitch of musical notes by integers*

half note), `half_triplet` (half-note triplet), `quarter` (quarter note), `dotted_quarter` (dotted quarter note), `quarter_triplet` (quarter-note triplet), `eighth` (eighth note), `dotted_eighth` (dotted eighth note), `eighth_triplet` (eighth-note triplet), `sixteenth` (sixteenth note), `dotted_sixteenth` (dotted sixteenth note) or `thirty_second` (thirty-second note). These constants, declared in the `LibRobot` library, are numbers written with a decimal point (as we shall explain in chapter 19), which give the duration of the note in beats: a `quarter` is worth `1.0`, an `eighth` is worth `0.5`, a `whole` is worth `4.0`, etc. The relative duration of a `dotted_whole` is one and a half times that of a `whole`. The relative duration of a `whole` is twice that of a `half`, three times that of a `half_triplet`, ... Other note values can be defined, for example `sixty_fourth = 0.0625` (sixty-fourth note).

18.4.2 Tempo

The tempo, that is, the degree of speed or slowness with which the piece of music must be played, determines the absolute duration of the different note values. The tempo is set by the procedure `metronome(`n`)`, which gives the number n of beats of the metronome per minute, a quarter note being played on each beat. For example `metronome(60)` means that 60 quarter notes must be played per minute. Tempos are usually given by Italian terms (adagio, allegro, prestissimo, ...) which are placed at the beginning of a piece, above the staff. They are defined in the `LibRobot` library by the following constant declarations:

```
largo = 50
lento = 56
larghetto = 60
adagio = 70
andante = 85
andantino = 95
moderato = 110
allegretto = 115
allegro = 130
vivace = 150
presto = 175
prestissimo = 200
```

18.4.3 Notes

The numbering of the notes in figure 18.3 (page 209) is not easy to remember. That is why we shall code the notes by an integer expression over constants that are easy to remember.

A note is defined by its name n (namely C, D, E, F, G, A, B), its possible accidental a (that is, a sign that changes the sound of the note it is attached to, which can be the flat ♭ (which lowers the note by a semitone), the sharp ♯ (which raises the note by a semitone) or the natural ♮ (which cancels the effect of the sharp or the flat)) and its octave o (which tells to which series of notes with the same name this note belongs). We code the octaves from low to high by the integers -3, -2, ..., 3, 4, octave 0 starting with the C (middle C) at the centre of the piano keyboard. The following constants are declared in the `LibRobot` library:

```
# Notes
C = 0
D = 2
E = 4
F = 5
G = 7
A = 9
B = 11
# Accidentals
flat = -1
natural = 0
sharp = 1
# Octaves
octave = 12
```

The number of the note as defined in figure 18.3 (page 209) is given by the value of the integer expression o∗*octave*+n+a. If the note has no accidental, its code is simply $o * octave + n$. For example the C in the middle of the piano keyboard corresponds to $o = 0$, $n =$`C`, that is 0∗`octave`+0 = 0. The D sharp one octave higher has code 1∗`octave`+`D`+`sharp`= $1 * 12 + 2 + 1 = 15$. The neighbouring D flat has code 1∗`octave`+`D`+`flat`= $1 * 12 + 2 + (-1) = 13$. When the pitch of a note is given by an expression, we simply write this expression as the first parameter of `note`, as in `note(D + octave + sharp, quarter)`.
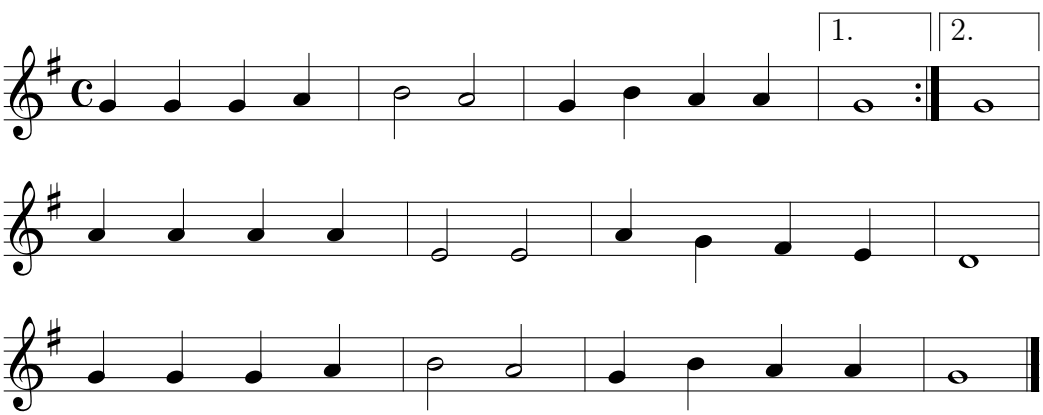
18.4.4 Rests

The procedure `rest(`*s*`)` interrupts the sound for a relative duration set by the rest value *s*, which can be one of the constants `whole_rest`, `dotted_half_rest`, `half_rest`, `quarter_rest`, `dotted_quarter_rest`[1], `eighth_rest`, `dotted_eighth_rest`, `sixteenth_rest` or `thirty_second_rest`.

18.4.5 Dynamics

Dynamics are the different degrees of loudness through which one or more sounds, a passage or a whole piece can go. The procedure `dynamics(`*n*`)` sets the dynamics with which the rest of the piece must be played. The possible dynamics terms *n* are, from the softest to the loudest: `pianississimo`, `pianissimo`, `piano`, `mezzo_piano` (or `mezza_voce`), `mezzo_forte`, `forte`, `fortissimo` or `fortissississimo`.

Example 22 (Au clair de la lune) "Au clair de la lune" ("By the light of the moon"), said to have been composed by the musician Jean-Baptiste Lully, is probably the most famous of all French children's songs:



The program below makes the computer play it. To save space, we have written two notes per line, with the corresponding syllables of the (French) lyrics in comments (those of the first and second lines of verse are separated by a slash `/`):

```
from LibRobot import *
# Au clair de la lune (song attributed to
```

[1] It is not customary to dot the quarter rest.

```
# Jean-Baptiste Lulli)
def stanza():
   metronome(moderato)  # time 4 / 4
   dynamics(piano)
   for i in range(2):
      note(G, quarter); note(G, quarter)  # Au clair/Prê-te
      note(G, quarter); note(A, quarter)  # de la/moi ta
      note(B, half); note(A, half)  # lu-ne/plu-me
      note(G, quarter); note(B, quarter)  # Mon a-/Pour é-
      note(A, quarter)
      note(A, quarter)  # -mi Pier-/-crire un
      note(G, whole)  # -rot./mot.
   note(A, quarter); note(A, quarter)  # Ma chan-
   note(A, quarter); note(A, quarter)  # -delle est
   note(E, half); note(E, half)  # mor-te.
   note(A, quarter); note(G, quarter)  # Je n'ai
   note(F + sharp, quarter); note(E, quarter)  # plus de
   note(D, whole)  # feu.
   note(G, quarter); note(G, quarter)  # Ou-vre
   note(G, quarter); note(A, quarter)  # moi ta
   note(B, half); note(A, half)  # por-te,
   note(G, quarter); note(B, quarter)  # Pour l'a-
   note(A, quarter); note(A, quarter)  # -mour de
   note(G, whole)  # Dieu.
   rest(whole_rest)
def song():
   for j in range(3):
      stanza()
song()
```

□

Exercise 34

_ *Write procedures* `startOfTransmission`, `a`, ..., `z`, `c0`, ..., `c9` *(digits),* `fullStop`, `comma`, `semicolon`, `colon`, `questionMark`, `exclamationMark`, `hyphen`, `apostrophe`, `slash`, `error`, `endOfTransmission`, `endOfWord` *and* `newLine` *(go to the next line), then use them to code messages in Morse, each sign of the plain message being coded by calling one of these procedures. For example "*coded message*" will be translated into the sequence of instructions "* `c(); o(); d(); e(); d(); endOfWord(); newLine(); m(); e(); s(); s(); a(); g(); e(); endOfWord()` *". You can add sound by using the following procedures of the Robot:*

- `beep()` *produces a short sound;*
- `longBeep()` *produces a long sound;*
- `delay(`*t*`)` *waits for* t *seconds (*t *can be a number written with a decimal point, as in* `delay(0.5)`*).*

_ *Before the invention of electricity, France used the optical (semaphore) telegraph invented in 1793 by Citizen Claude Chappe. The telegraph consists of three articulated wooden arms at the top of a mast set up on a tower on a hilltop or on a church steeple. As soon as he sees through his telescope a sign of activity at the nearest station, the* telegraph operator *hurries to work the arms of his own machine in turn to pass on the message (without understanding it) to the next station. Step by step the message reaches its destination. It is in a way an improved version of the smoke signals of the American Indians! After a first trial on 12 July 1793, the first line was opened between Paris and Lille on 17 August 1794 [3]. The signals of Chappe's optical telegraph could be decoded according to several codes. The alphabetic code is given in figure 18.4 (page 216). Write a program that displays the signals successively, one by one, and sends the message "Paris est tranquille et les bons citoyens sont contents" ("Paris is calm and the good citizens are content"), which Bonaparte had broadcast immediately after the coup d'état of 18 Brumaire (9–10 November 1799).*

To leave the signal visible for some time, use the procedure `delay(`*t*`)`*, which waits for t seconds. To clear the screen between signals, use the procedure* `bg(white)`*, which paints the background of the screen white (or another colour, depending on the parameter) and puts the Robot back in its initial*

state (visible at the centre of the screen, facing north, black pen of width 1 down, with a movement grid of (12 × 12)).

– *Write a Python program that translates the letters of the alphabet into the phonetic alphabet according to the international code of table 18.3 (page 215).*

– *Write a Python program that encrypts and decrypts messages using a simple secret code, obtained for example by a permutation of the letters of the alphabet.*

□

Exercise 35

– *What is the 16-bit binary representation of the number written 127 in base 10?*

– *What is the decimal notation of the positive integer represented in binary on 16 bits by 0011010101010101?*

– *What text is represented in* ASCII *code by the following bytes: 01000100 01100101 01110011 01100011 01100001 01110010 01110100 01100101 01110011?*

□

A	Alpha	B	Bravo	C	Charlie	D	Delta	E	Echo
F	Foxtrot	G	Golf	H	Hotel	I	India	J	Juliet
K	Kilo	L	Lima	M	Mike	N	November	0	Oscar
P	Papa	Q	Quebec	R	Romeo	S	Sierra	T	Tango
U	Uniform	V	Victor	W	Whiskey	X	Xray	Y	Yankee
Z	Zulu								

TAB. 18.3 – *International phonetic alphabet*

Exercise 36 *Songs are an inexhaustible source of exercises [6], [36], [37], [38]. For example, write Python programs to play* Greensleeves *(figure 18.5, page 217) and* O Tannenbaum *(figure 18.6, page 217). You can also scroll the score line by line on the screen while it is being played by the computer, drawing the staff and the notes as in figure 18.7 (page 218).* □

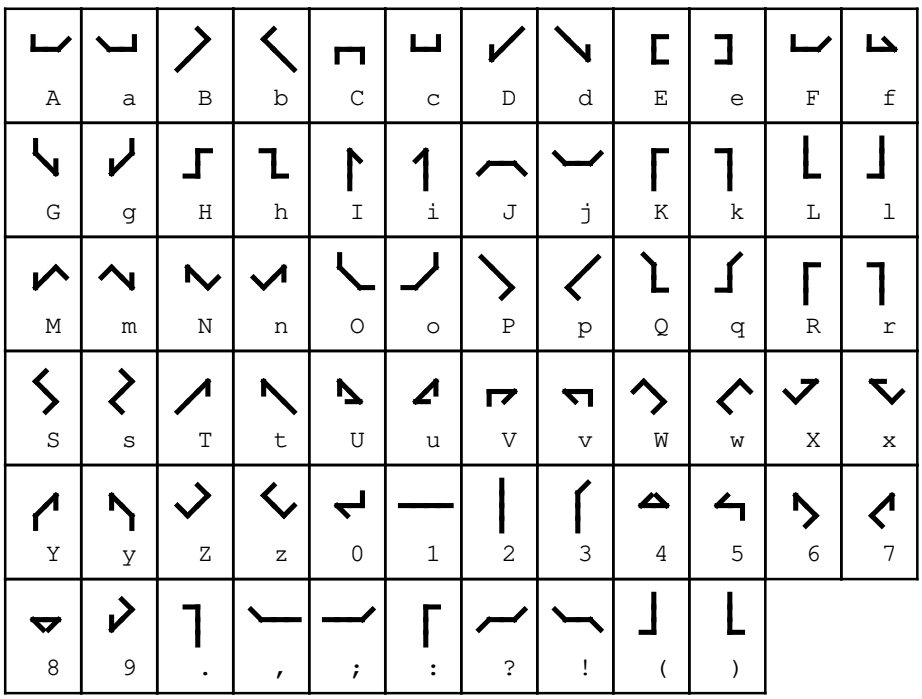


FIG. 18.4 – *Alphabet of Chappe's optical telegraph*
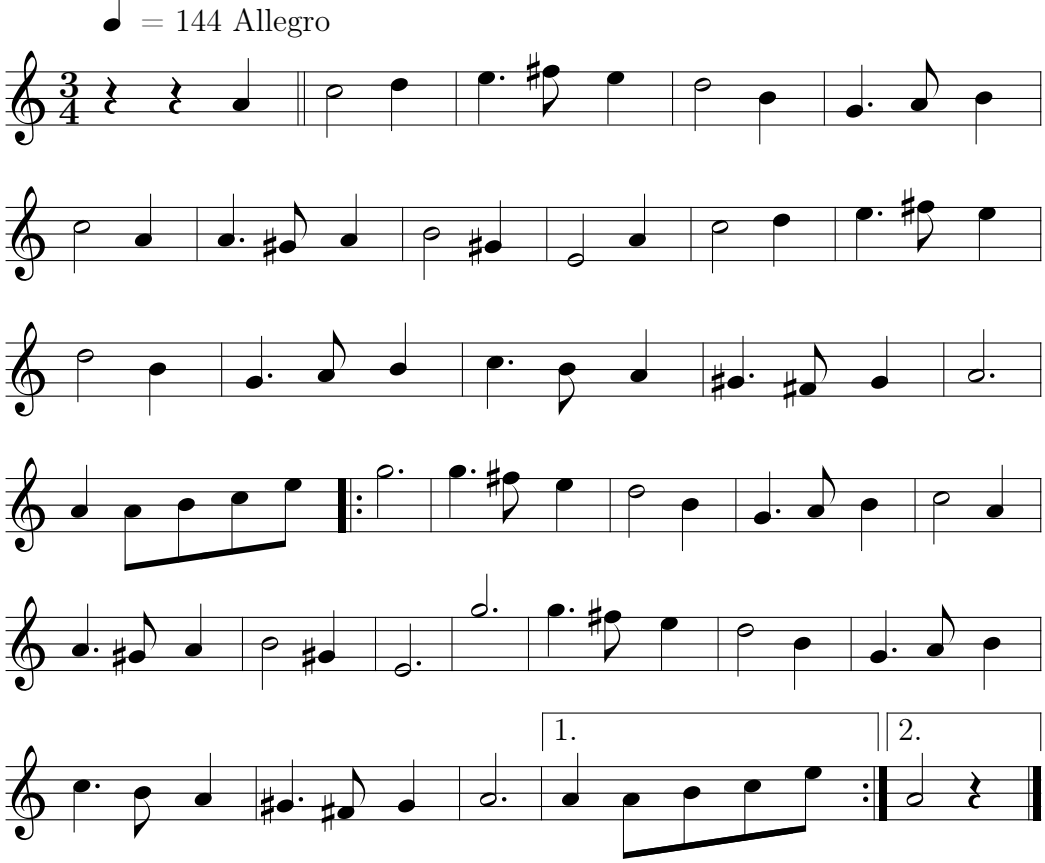


FIG. 18.5 – *Greensleeves (Irish song)*

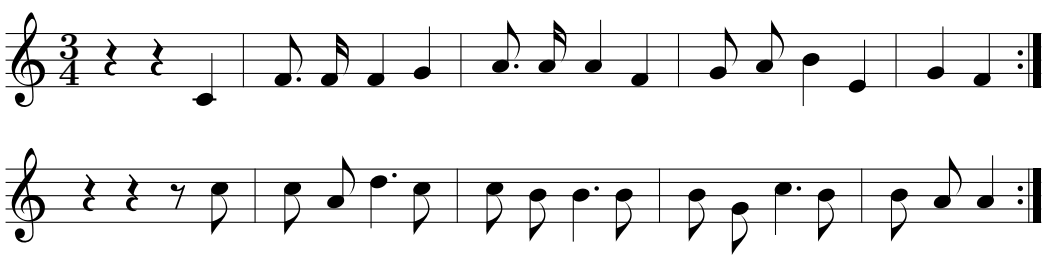


FIG. 18.6 – *O Tannenbaum ("O Christmas Tree", German song)*
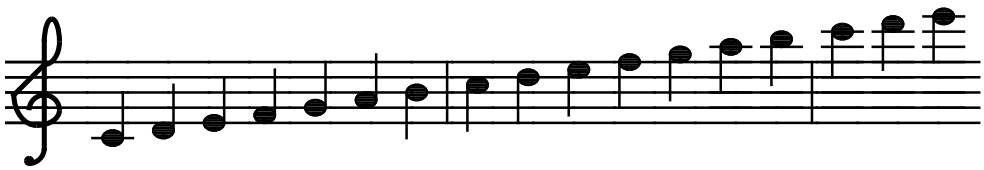


FIG. 18.7 – *Staff and notes drawn on the computer screen*

Solution 34 *We give one program, the others being in the folder* `book/18_coding`*:*

```
from LibRobot import *

# Write and sound out signs in Morse code

# Go to the start of the next line
def newLine():
   pu(); gl(); fs(); fd(); fe(); fd(); pd()

# Dot
def dot():
   dp(); beep(); pu(); fd(); pd(); delay(0.1)

# Blank
def blank():
   pu(); fd(); pd(); delay(0.1)

# Dash
def dash():
   fd(); longBeep(); pu(); fd(); pd(); delay(0.1)

# Alphabet
def a():
   dot(); dash(); blank(...)

def z():
   dash(); dash(); dot(); dot(); blank()

# Digits
def c1():
   dot(); dash(); dash(); dash(); dash(); blank(...)

def c0():
   dash(); dash(); dash(); dash(); dash(); blank()

# Punctuation
def fullStop():
   dot(); dot(); dot(); dot(); dot(); dot(); blank()

def comma():
   dot(); dash(); dot(); dash(); dot(); dash(); blank()
```

```
def semicolon():
   dash(); dash(); dash(); dot(); dot(); dot(); blank()
def colon():
   dash(); dot(); dash(); dot(); dash(); dot(); blank()
def questionMark():
   dot(); dot(); dash(); dash(); dot(); dot(); blank()
def exclamationMark():
   dash(); dash(); dot(); dot(); dash(); dash(); blank()
def hyphen():
   dash(); dot(); dot(); dot(); dot(); dash(); blank()
def apostrophe():
   dash(); dash(); dash(); dash(); blank()
def slash():
   dash(); dash(); dash(); dash(); dash(); dash(); blank()
# End of word
def endOfWord():
   pu(); fd(); pd(); delay(0.6)
def error():
   for i in range(11):
      dot()
   blank()
def startOfTransmission():
   pw(2); gt(); newLine()
   dash(); dot(); dash(); dot(); dash(); dot(); dash(); blank()
def endOfTransmission():
   dot(); dash(); dot(); dash(); dot(); dash(); dot(); blank()
startOfTransmission()
a(); b(); c(); d(); e(); newLine()
f(); g(); h(); i(); j(); k(); l(); newLine()
m(); n(); o(); p(); q(); r(); s(); newLine()
t(); u(); v(); w(); x(); y(); newLine()
z(); c1(); c2(); c3(); newLine()
c4(); c5(); c6(); c7(); c8(); newLine()
c9(); c0(); fullStop(); comma(); newLine()
semicolon(); colon(); questionMark()
exclamationMark(); newLine()
hyphen(); apostrophe(); slash()
newLine(); error(); endOfTransmission(); st()
```

The same folder contains various possible improvements of this program, in particular automatic line wrapping (`MorseCode2`*). The procedures* `a`*, . . . ,*

`z` *and* `c0`, ..., `c9` *have lower-case names because the custom in Python is to begin procedure names with a lower-case letter, names beginning with a capital letter usually being reserved for something else.* □

Solution 35

_ *The 16-bit binary representation of 127 is 0000000001111111.*

_ *The number represented by the half-word 0011010101010101 is written 13653 in base 10.*

_ *The sequence of bytes 01000100 01100101 01110011 01100011 01100001 01110010 01110100 01100101 01110011, that is, in decimal notation, 68 101 115 99 97 114 116 101 115, represents the character string* `'Descartes'`.
□

Solution 36 *We give one program, many other songs being programmed in the folder* `book/18_coding` *and in its sub-folder* `songs`*:*

```
from LibRobot import *

# O Tannenbaum (German song)
def chorus():
    note(C, quarter)  # O
    note(F, dotted_eighth); note(F, sixteenth)
    # Tan-nen-
    note(F, quarter)  # -baum,
    note(G, quarter)  # o
    note(A, dotted_eighth); note(A, sixteenth)
    # Tan-nen-
    note(A, dotted_quarter)  # -baum,
    note(F, eighth); note(G, eighth)  # wie treu
    note(A, eighth); note(B + flat, quarter)  # sind dei-
    note(E, quarter)  # -ne
    note(G, quarter); note(F, quarter)  # Blät-ter !
def verse():
    rest(eighth_rest)
    note(C + octave, eighth)  # Du
    note(C + octave, eighth); note(A, eighth)
    # grünst nicht
    note(D + octave, dotted_quarter)  # nur
    note(C + octave, eighth); note(C + octave, eighth)
    # zur Som-
    note(B + flat, eighth)  # -mer-
    note(B + flat, dotted_quarter)  # -zeit
    note(B + flat, eighth); note(B + flat, eighth)
```

```
    # Nein, auch
    note(G, eighth)  # im
    note(C + octave, dotted_quarter)  # Win-
    note(B + flat, eighth); note(B + flat, eighth)
    # -ter wenn
    note(A, eighth); note(A, quarter)  # es schneit.
# German song
def song():
    metronome(adagio); dynamics(mezza_voce)
    chorus(); verse(); chorus()
song()
```

□

Addenda

In addition, we explain two representations of signed integers, the second being the more widely used.

Sign-and-magnitude representation of signed integers: To code a signed integer (positive or negative) on n bits, one can reserve one bit for the sign and code the absolute value of the integer on the remaining $n-1$ bits. Let us adopt the convention of putting the sign first, choosing 0 for the sign $+$ and 1 for the sign $-$.

The code of -10 on one byte will be 10001010, that is, 1 for the sign followed by 0001010 for the absolute value 10 of -10. With this convention one can code on one byte the integers between -127 (11111111) and 127 (01111111). The integer zero is represented by 00000000 ($+0$) or by 10000000 (-0).

Two's complement representation of signed integers: In the two's complement representation on n bits, the positive numbers strictly less than 2^{n-1} are represented as before in base 2 on $n-1$ bits, the first (sign) bit being zero. A negative number $-x$ such that $x \leq 2^{n-1}$ is represented by a first sign bit equal to 1 followed by the representation of the positive number $p < 2^{n-1}$ such that $-x = -2^{n-1} + p$, coded on $n-1$ bits. For example, to code -1 on one byte, we write $-1 = -128 + 127$ and code 127 on 7 bits, which gives 1111111, which we precede by 1 to get 11111111. With this convention we can also code $-128 = -128 + 0$, which gives the code 10000000. This method makes it possible to code on one byte all the signed integers between -128 and $+127$. On 16 bits, one can code the integers between -32768 and 32767, which were the integers of the first microcomputers. On 32 bits one can code the integers between $-2\,147\,483\,648$ and $2\,147\,483\,647$. This representation leads to very simple algorithms for addition and subtraction. Moreover the algorithm for coding a negative number $-x$ is also simple. We code the positive number x on n bits, then invert the bits, 0 into 1 and 1 into 0, then add 1. For example to code -127, we code 127 as 01111111, then invert all the bits, which gives 10000000, and add 1 to get 10000001. The integers of type `int` in Python are not limited to 32 or 64 bits: Python uses as many bits as needed (within the limits of the computer's memory), so that `2 ** 100` (2 multiplied by itself 100 times) is exactly $1\,267\,650\,600\,228\,229\,401\,496\,703\,205\,376$.

19

Rational expressions

19.1 Rational numbers

The procedure parameters and the constants we have used so far have integer values. In Python, we say that they are of *type* `int` (short for integer). Unlike many other languages, Python imposes no limit on the size of integers: `2 ** 100` (2 to the power 100) is 1,267,650,600,228,229,401,496,703,205,376, which leaves plenty of room for counting the sides of a polygon or the squares of a grid. They are encoded in base two in the memory of the computer, on as many bits as needed.

In Python we can also use decimal numbers with a decimal point such as `3.14159`, called floating-point numbers (of type `float`). They are written according to the English-speaking convention with a point (whereas the French use a comma). Mathematicians call these numbers *rational numbers*. In Python, integers are distinguished from rational numbers by the presence of the point. For example `1000` is an integer whereas `1000.0` is a rational number, which can also be written `1000.` without any digit after the point. Likewise `.5` is allowed, but it is more readable to write `0.5`.

Integers and rational numbers are encoded in different ways in the memory of the computer. Rational numbers are of type `float`[1].

[1] From *floating-point number*. The computer keeps in memory a fixed number of significant digits and the position of the decimal point (which "floats" among these digits). These are not real numbers since numbers of type `float` cannot have infinitely many non-zero digits after the decimal point!

19.2 Parameters of type "float"

In Python, the type of the parameters of a procedure is not indicated: the same parameter can receive, from one call to another, an integer or a rational, and it is only during execution, when the value is used, that Python checks that it is suitable (we say that Python is a *dynamically typed* language). For example, in the procedure:

```
def arc(l): gs(l); cr()
```

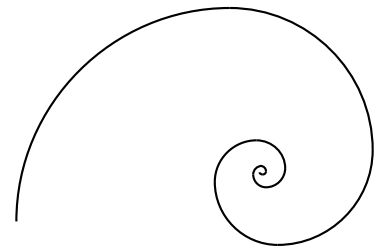
the formal parameter `l` (read the letter ℓ and not the digit 1) is passed to the command `gs` of the *Robot* which expects a number, integer or rational.

The corresponding actual parameter can therefore be either an integer, as in `arc(100)`, or a rational number, written with a point, as in `arc(66.666666667)`: Python automatically converts the integer `100` into the rational `100.0` when necessary. If we want to make this conversion ourselves, we use the function `float`: if `n` is an integer, `float(n)` is the corresponding rational number (for example `float(3)` is `3.0`). This is rarely useful.

The command procedures of the *Robot* `gs`, `gsX` and `gsY` (as well as `gsRT`, `rot` and `fdl` which we will see in the next chapter) accept rational parameters. This is why we can write `gs(20)` as well as `gs(20.5)`. The commands `fdn`, `bg`, `pw`, `moveTo` or `drawTo` on the contrary have integer parameters: `fdn(2.5)` stops the program with the message '`TypeError: 'float' object cannot be interpreted as an integer`'.

Example 23 (Arithmetic spiral) The spiral below is made of arcs of circles with successive radii 100; $100/1.5 = 66.666\ldots$; $66.666\ldots/1.5 = 44.444\ldots$; $44.444\ldots/1.5 = 29.629\ldots$; etc.:

```
from LibRobot import *
# Draw an arithmetic spiral
# Draw an arc of a circle of radius l
def arc(l):
   gs(l); cr()
arc(100); arc(66.666666667)
arc(44.444444444); arc(29.62962963)
arc(19.75308642); arc(13.16872428)
arc(8.7791495199); arc(5.8527663466)
arc(3.9018442311); arc(2.6012294874)
arc(1.7341529916); arc(1.1561019944)
st()
```



□

19.3 Rational expressions

Addition, subtraction, multiplication and division of rational numbers are written `+`, `-`, `*` and `/`, that is as for integers. For example `(((2 * (10.3 + 0.7)) - 1.0) / 3)` is `7.0`

When an operation involves an integer and a rational, Python converts the integer into a rational and the result is a rational: `2 * 3.5` is `7.0`. When both operands are integers, `+`, `-` and `*` give an integer. Division, however, requires attention: `/` *always* gives a rational, even between two integers. Thus `7 / 2` is `3.5` and `6 / 2` is `3.0`. Integer division, which gives an integer, is written `//`: `7 // 2` is `3`. Thanks to these two notations, the reader of the program always knows which division is performed. Since `6 / 2` is a rational, `fdn(6 / 2)` is rejected by the command `fdn` which expects an integer; one must write `fdn(6 // 2)`.

If r is a number, `sqrt(`r`)` is its square root $\sqrt{r}$ (short for square root), r `**  2` is its square r^2 (which can also be written r `*` r), `abs(`r`)` is its absolute value (for an integer as well as for a rational), `int(`r`)` is its integer part obtained by removing the digits after the decimal point (we say that it is truncated) and `round(`r`)` is the nearest integer (from the verb to round). The results of `int` and of `round` are integers. For example `int(0.3)` = `int(0.5)` =

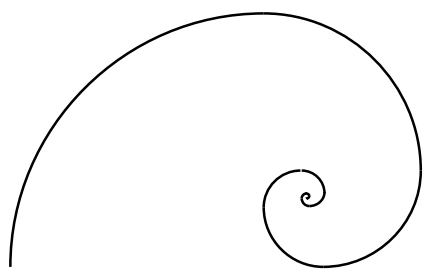
`int(0.9)` = `0` whereas `round(0.3)` = `0` and `round(0.5)` = `round(0.9)` = `1`[2]. The function `sqrt` is provided by the library of the $\mathcal{R}obot$ (line **from** `LibRobot` **import** `*`), the others are part of Python. As for procedures, the argument of a function is always placed between parentheses: `sqrt(x + 1)`.

In a rational expression, one can use the horizontal length `valGsX()` and the vertical length `valGsY()` of a square of the grid on which the $\mathcal{R}obot$ moves. For example `gsX(valGsX() * 2)` doubles the horizontal size of a square of the grid.

Example 24 (Golden spiral) The golden spiral below is made of successive arcs of circles whose radii have as ratio the *golden ratio* $\Phi = \frac{1}{2}(1+\sqrt{5}) \simeq 1.618033989$. For the advocates of traditional aesthetics, the golden spiral is said to be a geometric figure very harmonious to the eye.

```
from LibRobot import *

# Draw a golden spiral
goldenRatio = (1 + sqrt(5)) / 2
gs(100)
for i in range(12):
   cr()
   gs(valGsX() / goldenRatio)
st()
```



□

19.4 Rounding errors

For real numbers, which have infinitely many digits after the decimal point, such as $\frac{1}{3} = 0.3333333333333333\ldots$ or $\pi = 3.1415926535897932\ldots$ the computer can keep in its memory only a few digits after the decimal point, which leads, as with a pocket calculator, to rounding errors in computations. For

[2] The original `round` of Python rounds halves to the nearest even integer (`round(0.5)` is `0` and `round(2.5)` is `2`). The library of the $\mathcal{R}obot$ replaces it by a `round` which rounds halves away from zero, like Pascal and OCaml: `round(2.5)` is `3` and `round(-2.5)` is `-3`.

example in mathematics we have:

$$\left(\left(\left(\left(\left(\sqrt{\sqrt{\sqrt{\sqrt{2}}}}\right)^2\right)^2\right)^2\right)^2\right)^2 = \left(\left(\left(\left(\sqrt{\sqrt{\sqrt{\sqrt{2}}}}\right)^2\right)^2\right)^2\right)^2 =$$

$$\left(\left(\left(\sqrt{\sqrt{\sqrt{2}}}\right)^2\right)^2\right)^2 = \left(\left(\sqrt{\sqrt{2}}\right)^2\right)^2 = \left(\sqrt{2}\right)^2 = 2$$

whereas the computer finds that:

```
((((sqrt(sqrt(sqrt(sqrt(sqrt(2))))) ** 2)
    ** 2) ** 2) ** 2) ** 2 = 1.9999999999999964
```

Likewise, it finds that `0.1 + 0.2` is `0.30000000000000004` (the program `RoundingErrors` of the folder `book/19_rational_expressions` performs these two computations).

Similar rounding problems occur with the grid on which the $\mathcal{R}^{o}\underset{\sim}{b}^{o}t$ moves. Ideally, the Cartesian coordinates of the $\mathcal{R}^{o}\underset{\sim}{b}^{o}t$ should be real numbers. The computer keeps in memory a rational approximation of them. On the screen they must be rounded to the nearest integer to correspond to a pixel of the screen.

Exercise 37 *Write Python programs to produce the patterns of the figure below (only the central pattern, which uses the trigonometric functions introduced in the addenda of chapter 21, is difficult).* □

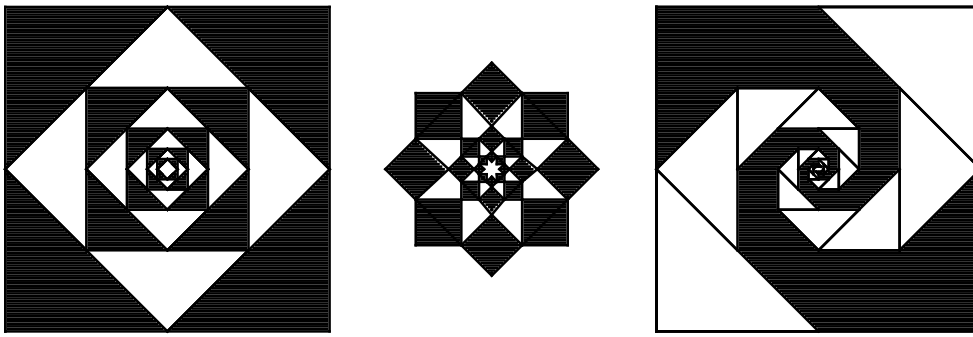


FIG. 19.1 – *Horemis and Beard patterns; Baravelle spiral*

Solution 37 *We give one program, the others being in the folder* `book/19_rational_expressions`*:*

```
from LibRobot import *

# Pattern by S. Horemis, 'Optical and geometrical
# patterns and designs', Dover, New York, 1970.
n = 6
twoToTheN = 64  # 2 to the power n

gb(); gl(); gs(twoToTheN)
for i in range(n):
   for j in range(4):
      fdn(4); qtr()
   fdn(2); etr()
   for j in range(4):
      fdn(2); fdn(-1); qtr(); paint(); qtl()
      fd(); qtr()
   fd(); etr()
   gs(valGsX() / 2)
st()
```

□

Addenda

Rational numbers are often called real numbers by abuse of language. They can be written using *scientific notation*, which consists in following the fractional part by a scale factor made of the letter e or `E` (which reads "ten to the power") followed by an exponent.
For example $-3.0\text{e}2 = -300.0$, $3.0\text{e}0 = 3.0$ and $3.0\text{e}{-2} = 0.03$. A rational number contains no blank. Consequently $-3.0 \text{␣} \text{e} \text{␣} 2$ is wrong. In Python, numbers of type `float` are represented with 15 to 16 significant digits in base 10, the smallest strictly positive rational (without loss of significant digits) being of the order of $2.2\text{e}{-308}$ and the largest of the order of $1.8\text{e}308$. Beyond that, the result of a computation is `inf` (infinity): `1e308 * 10` is `inf`.

The value of an arithmetic expression can be of type `int` or `float`, according to the following rules:

- integer constants (without a point) and the operators $+$, $-$, $*$, `//`, `%`, $**$ (with a positive exponent) and `abs`[a] applied to integers give an integer result of type `int`, as do `int` and `round` which apply to a rational;
- rational constants (with a point or an exponent `e` or `E`), division `/` and the function `sqrt` give a rational result of type `float`, as does `float` which applies to an integer;
- the operators $+$, $-$, $*$, $**$ and `abs` applied to rationals, or to an integer and a rational, give a rational result.

An expression which mixes both, such as `1 + 2.5`, is therefore allowed: the integer is converted into a rational and the result `3.5` is rational.

In Python, the type of parameters is not declared. Python automatically converts an integer into a rational when needed, but never a rational into an integer: it is up to the programmer to ask for it with `int` or `round`, for example `fdn(round(x))`. The expected type of a formal parameter can be indicated in the declaration of the procedure, as in **def** `arc(l: float): gs(l); cr()`, which is not necessary (Python does not check it) but can help the reader of the program.

[a] We use different typefaces for *keywords* (such as **def** or **for**) and *predefined identifiers* (such as `sqrt` or `abs`). The identifiers introduced in a Python program (such as the names of constants, of procedures, etc.) must be different from the keywords but may be identical to predefined identifiers. This practice is not recommended because programs which change the usual meaning of predefined identifiers are unreadable.

20

Rotations and translations

20.1 Size of the robot's movement grid in polar coordinates

The horizontal size X and vertical size Y of a square of the $\mathcal{R}^{o}_{\smile}o t$'s movement grid can be set in Cartesian coordinates by the commands `gsX(`X`)` and `gsY(`Y`)`, as shown in figure 20.1 below.

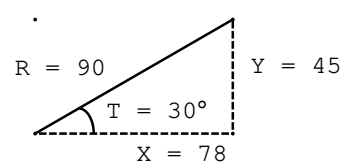


FIG. 20.1 – *The $\mathcal{R}^{o}_{\smile}o t$'s movement grid defined by* `gsX(`X`); gsY(`Y`)` or `gsRT(`R`, `T`)`

The size of a square of the grid can also be set in *polar coordinates* by the command `gsRT(`R`, `T`)`, giving the length R of the diagonal of this square measured in pixels and the angle T, in degrees, that this diagonal makes with a horizontal line, as illustrated in figure 20.1 above. The command `gsRT` has two parameters, which are integers or rational numbers, as in `gsRT(50, 70)` or `gsRT(50.5, 22.5)`.

Example 25 (Isosceles triangle) The program below draws an isosceles triangle with two angles equal to 70 degrees and two sides equal to 50 pixels:

```
from LibRobot import *
# Draw an isosceles triangle
sideLength = 50
angle = 70
gsRT(sideLength, angle)
etr(); fd()
qtr(); fd()
tetr(); fdn(2)
st()
```



□

20.2 Rotations

The command `rot(`t`)` makes the $\mathcal{R}\!\mathit{ob}\circ t$ perform a rotation of t degrees, t being an integer or rational expression (for example `rot(15)`, `rot(-90)` or `rot(22.5)`). If the value of the expression t is positive, the robot turns on the spot *counterclockwise*[1]. If the value of the expression t is negative, it turns on the spot clockwise. This command does not change the position of the $\mathcal{R}\!\mathit{ob}\circ t$ but it changes its orientation. The length of the segment drawn by the next `fd()` also stays the same. To achieve this, the size of the grid square must be changed accordingly. This is illustrated in figure 20.2 (page 233), which shows a grid square in dotted lines before and after successive rotations of 15 degrees (the $\mathcal{R}\!\mathit{ob}\circ t$ starting each time from the lower left corner of the square, and the rotation being followed by an `fd()` to show that the length of the segment drawn does not change). In this figure, the successive diagrams are arranged from top to bottom and from left to right.

When using the command `rot`, we advise you to avoid the commands `etr()`, `qtr()`, `tetr()`, ..., `fne()`, `fse()`, ..., replacing them by `rot` with a suitable parameter (only the commands `fn()`, `fs()`, ... are useful for absolute orientations). By using the command `rot` exclusively, there is no need to take into account the changes of coordinate system that it causes.

[1] The counterclockwise direction is what mathematicians call the *positive* (or *trigonometric*) *direction*.

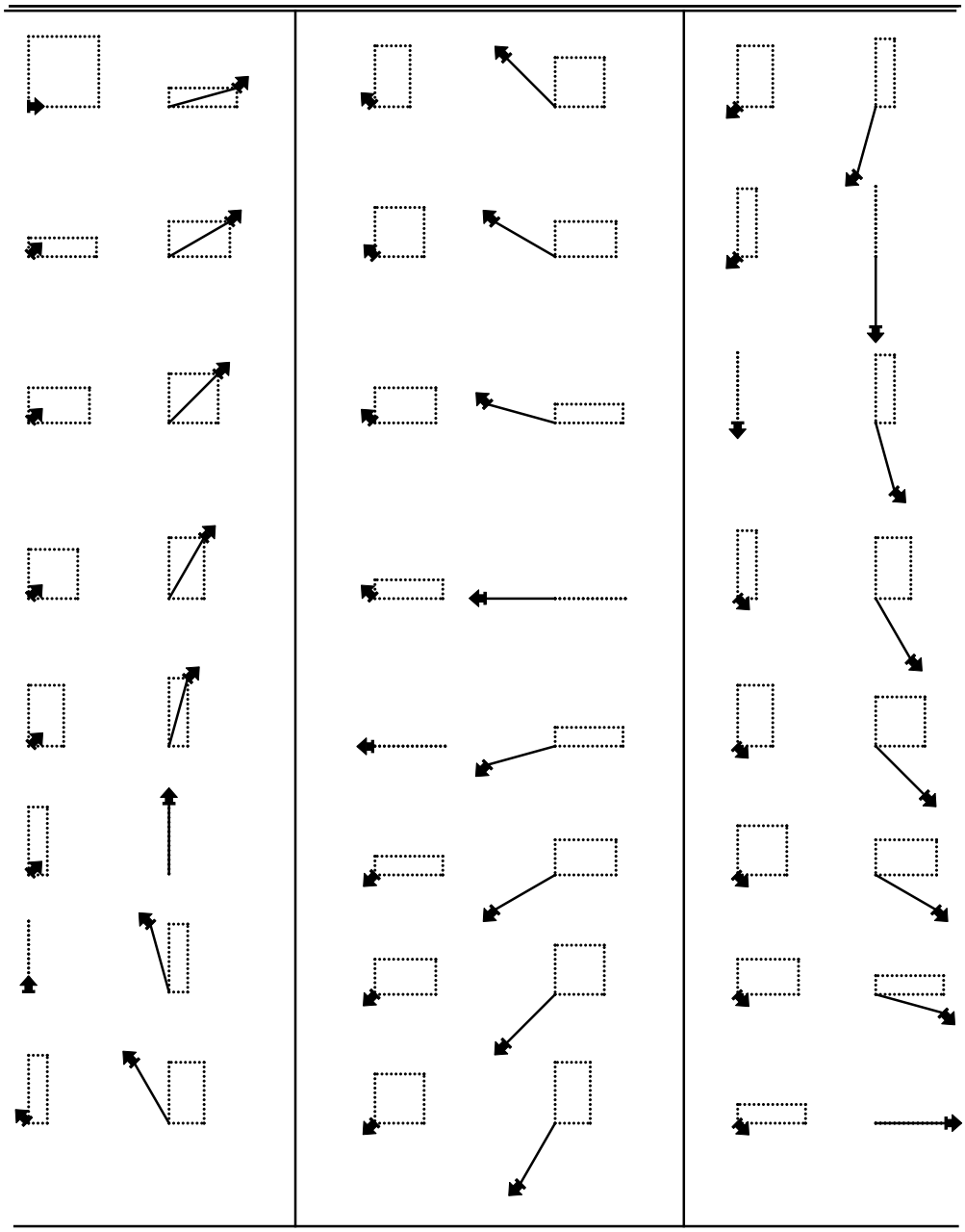
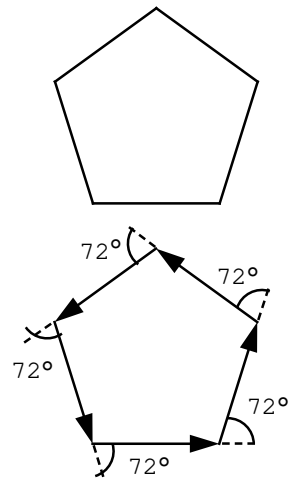


FIG. 20.2 – *Successive rotations and moves* `rot(15); fd()`

Example 26 (Pentagon) The program below draws a pentagon by repeating five times the drawing of a segment followed by a rotation of $(360/5) = 72$ degrees:

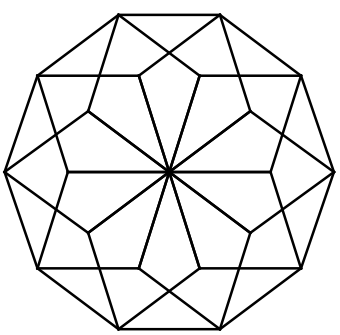
```
from LibRobot import *
# Draw a pentagon
sideLength = 50
gs(sideLength); fe()
for j in range(5):
   fd()
   rot(72)
st()
```



□

Example 27 (Rosette of pentagons) The rosette below is obtained by superimposing ten pentagons, each turned by 36 degrees from the previous one:

```
from LibRobot import *
# Draw a rosette of ten pentagons
sideLength = 40
qtr(); gs(sideLength)
for i in range(10):
   rot(36)
   for j in range(5):
      fd(); rot(72)
st()
```



□

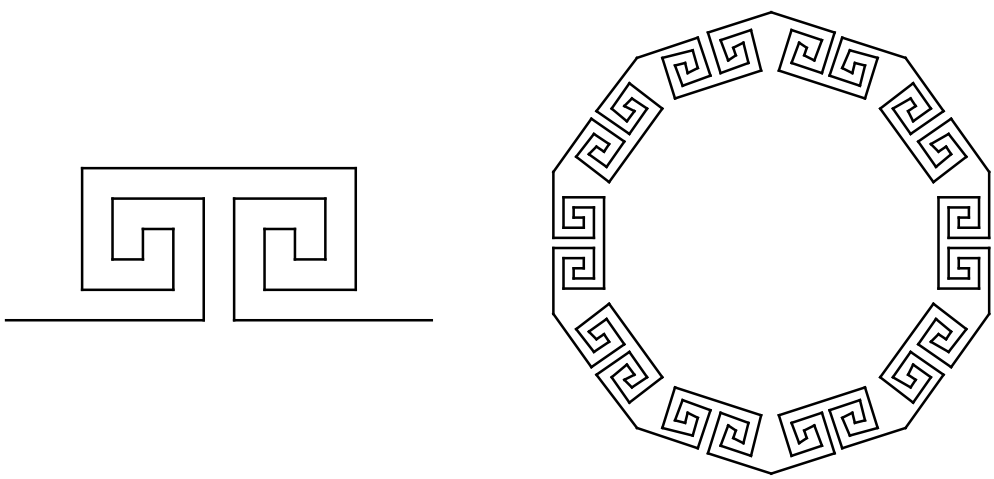
20.3 Translations

Whatever the orientation of the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$, the expression `valFdl()`, of type `float`, has as its value the length (measured in pixels) of a segment that would be drawn by an `fd()`.

To draw a segment of a different length, one can use the command `fdl(`E`)` (forward by a length), which takes as parameter an expression E (integer or rational) whose value is the length of this segment in pixels. If this value is positive, the segment is drawn moving forward in the current direction of the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$. If this value is negative, it is drawn moving backward. The $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ is moved to the end of the segment and does not change its orientation. The size of a grid square is changed to allow this move (if the pen is up) or this drawing (if the pen is down). Consequently, the segment can be extended by a segment of the same length using an `fd()` command.

For example, the sequence of commands `fdl(20); rot(90); fd(); rot(90); fd(); rot(90); fd()` draws a square with sides of 20 pixels.

Example 28 (Polygonal Greek key) The polygonal frieze below is obtained by replacing the drawing of each side of the polygon by that of the Greek key (meander):



```
from LibRobot import *
# Draw a polygonal Greek key
mf = 2  # Magnification factor
n = 10  # Number of sides
def pattern():
   # Left part
   fdl(13 * mf); rot(90); fdl(8 * mf); rot(90)
   fdl(6 * mf); rot(90); fdl(4 * mf); rot(90)
   fdl(2 * mf); rot(90); fdl(2 * mf); rot(-90)
   fdl(2 * mf); rot(-90); fdl(4 * mf); rot(-90)
   fdl(6 * mf); rot(-90); fdl(8 * mf); rot(-90)
```

```
    fdl(18 * mf); rot(-90)
    # Symmetric right part
    fdl(8 * mf); rot(-90); fdl(6 * mf); rot(-90)
    fdl(4 * mf); rot(-90); fdl(2 * mf); rot(-90)
    fdl(2 * mf); rot(90); fdl(2 * mf); rot(90)
    fdl(4 * mf); rot(90); fdl(6 * mf); rot(90)
    fdl(8 * mf); rot(90)
    fdl(13 * mf)
def greekKey():
    for i in range(n):
        pattern(); rot(360 / n)
greekKey(); st()
```

□

Exercise 38 *Write Python programs using the command* `gsRT` *to make the following drawings:*

- *an equilateral triangle;*
- *the pentagon, the crosses or the spider's web of figure 20.3 (page 236);*
- *the geometric patterns after Horemis [17] of figure 20.4 (page 237);*
- *the coloured tilings of figure 20.6 (page 238).*

The very many tilings given by Grünbaum and Shephard [15] are the source of countless exercises. □

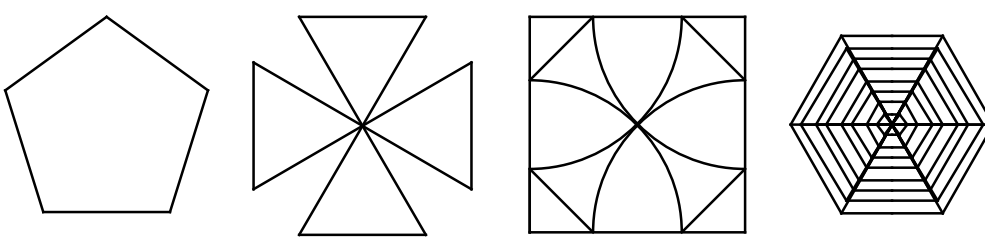


FIG. 20.3 – *Pentagon, crosses and spider's web*

Exercise 39 *Write Python programs using the command* `rot` *to make the following drawings:*

- *the stars, the decorated octagon or the spirolateral curve of order 3 at 135 degrees of figure 20.5 (page 237), the latter being obtained by moving forward*

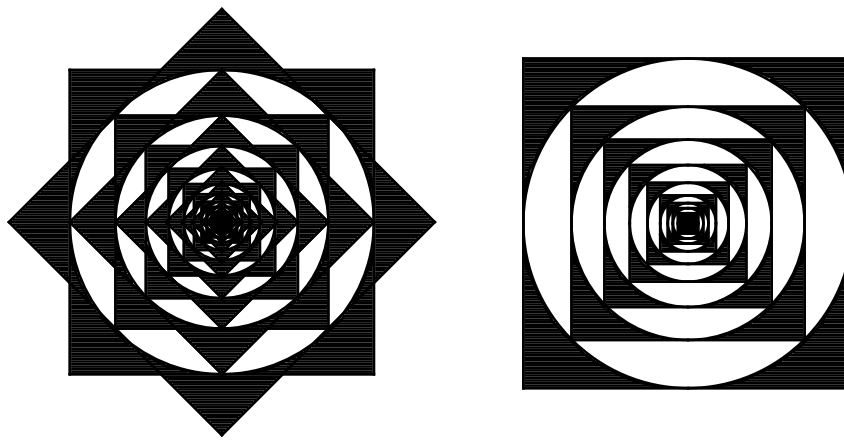


FIG. 20.4 – *Geometric patterns after Horemis*

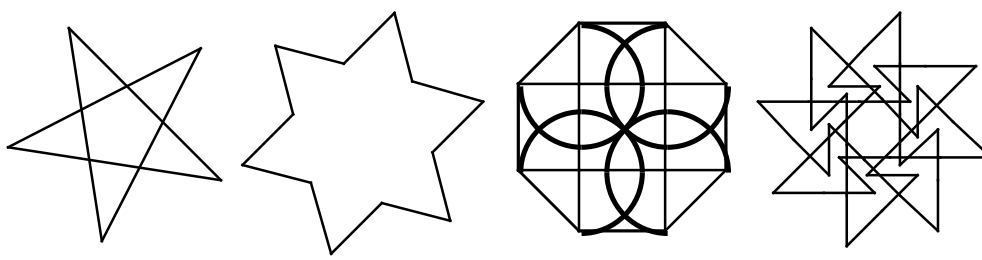


FIG. 20.5 – *Stars, decorated octagon and spirolateral*

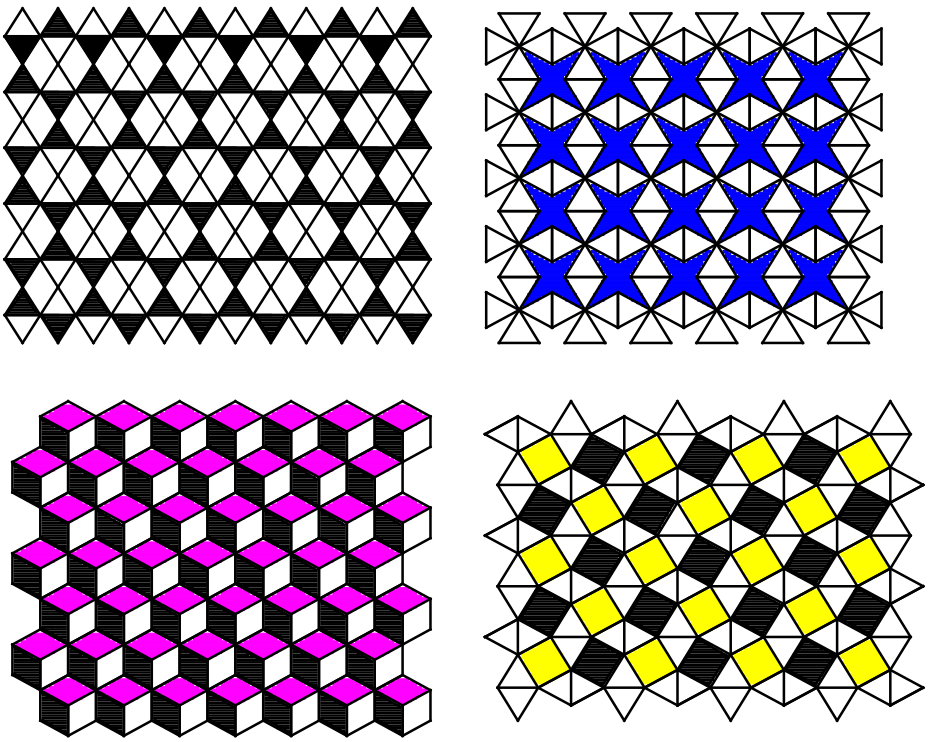


FIG. 20.6 – *Tilings with triangles, rhombuses and squares*

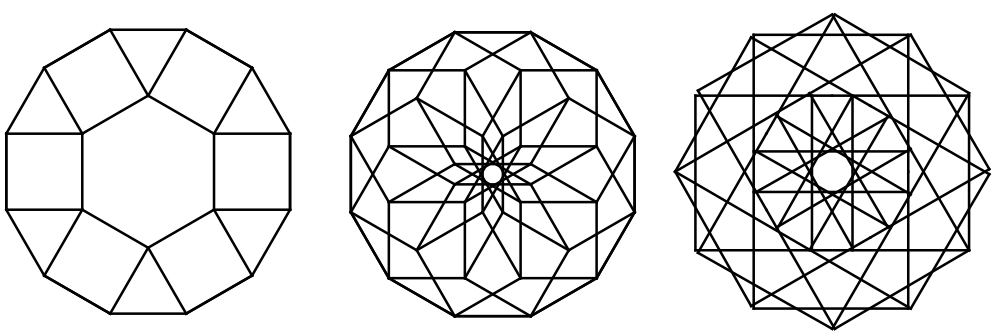


FIG. 20.7 – *Polygonal rosettes*

once, then twice, then three times, turning by 135 degrees each time, and repeating these moves until the curve closes up;

– *the regular polygons of figure 20.8 (page 239), the number of sides being a parameter of the drawing procedure;*

– *the stars of figure 20.9 (page 240), the number of points being a parameter of the drawing procedure;*

– *the protractor of figure 20.10 (page 240), using the command* `label(intToString(e))`*, which writes the value of the integer expression* e *to the right of the current position of the* Robot *(the irregularity of the graduations is due to rounding errors in integer screen coordinates, which make it impossible to draw straight lines of arbitrary slope);*

– *the tiling of the plane with hexagons or stars of figure 20.11 (page 240);*

– *the Escher-style [26] tiling of the disc and of the triangle of figure 20.12 (page 241);*

– *the tiling of the disc using Voderberg's tile [42] of figure 20.13 (page 241);*

– *the spirals using Voderberg's tile [42] of figure 20.14 (page 242).*

□

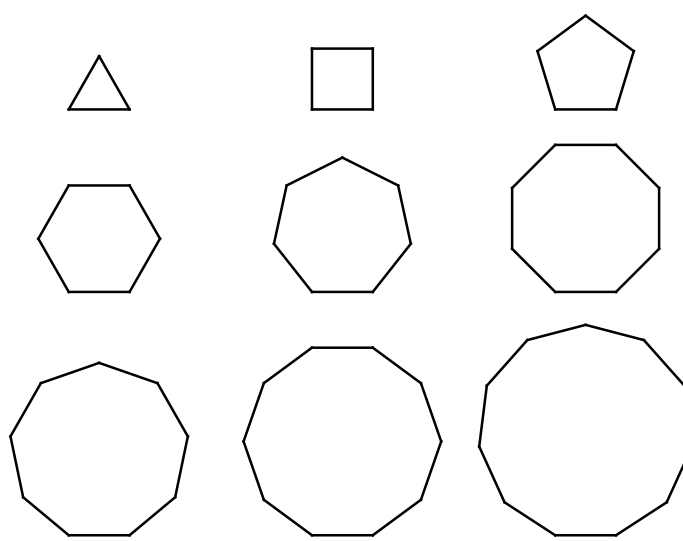


FIG. 20.8 – *Regular polygons*

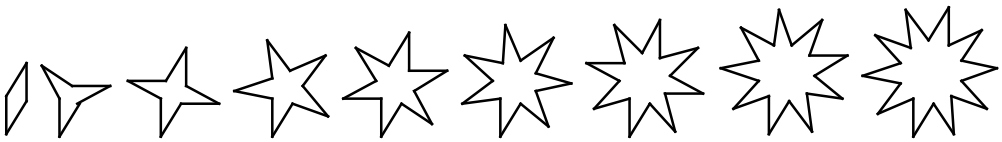


FIG. 20.9 – *Stars*

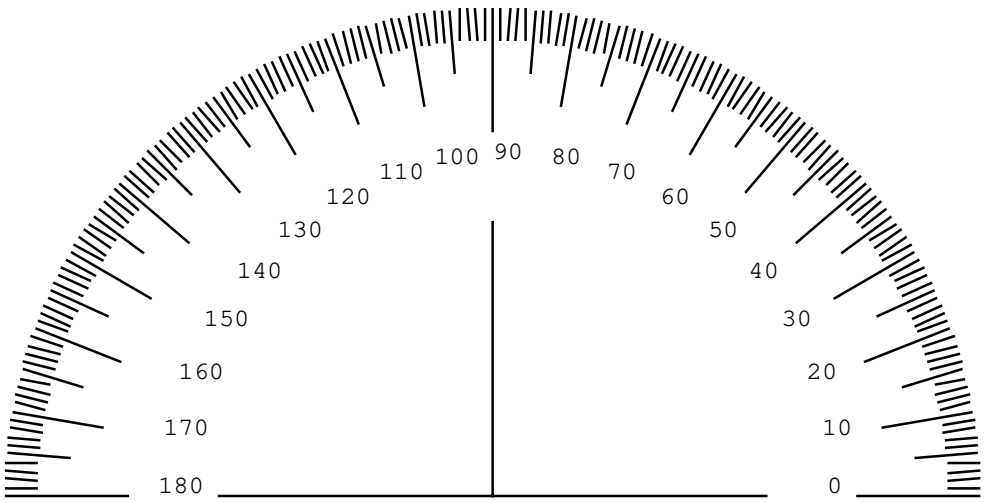


FIG. 20.10 – *Protractor graduated in degrees*

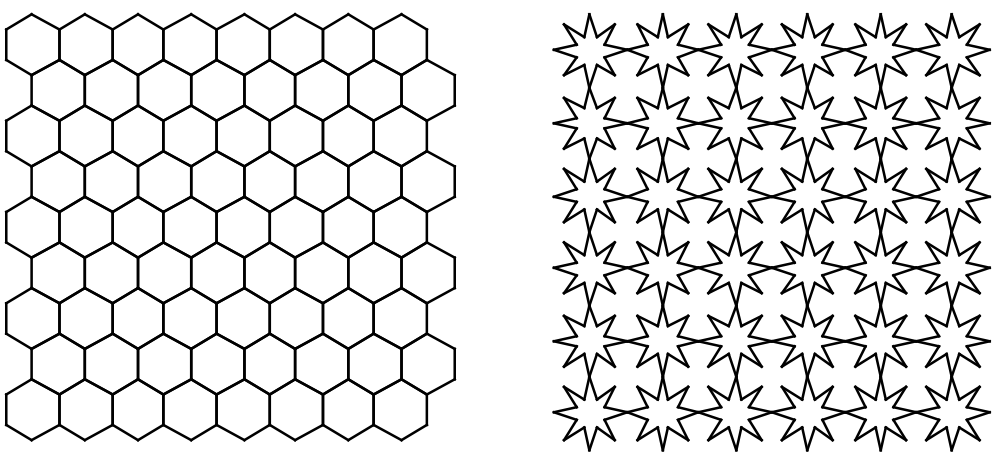


FIG. 20.11 – *Tiling of the plane with hexagons and stars*

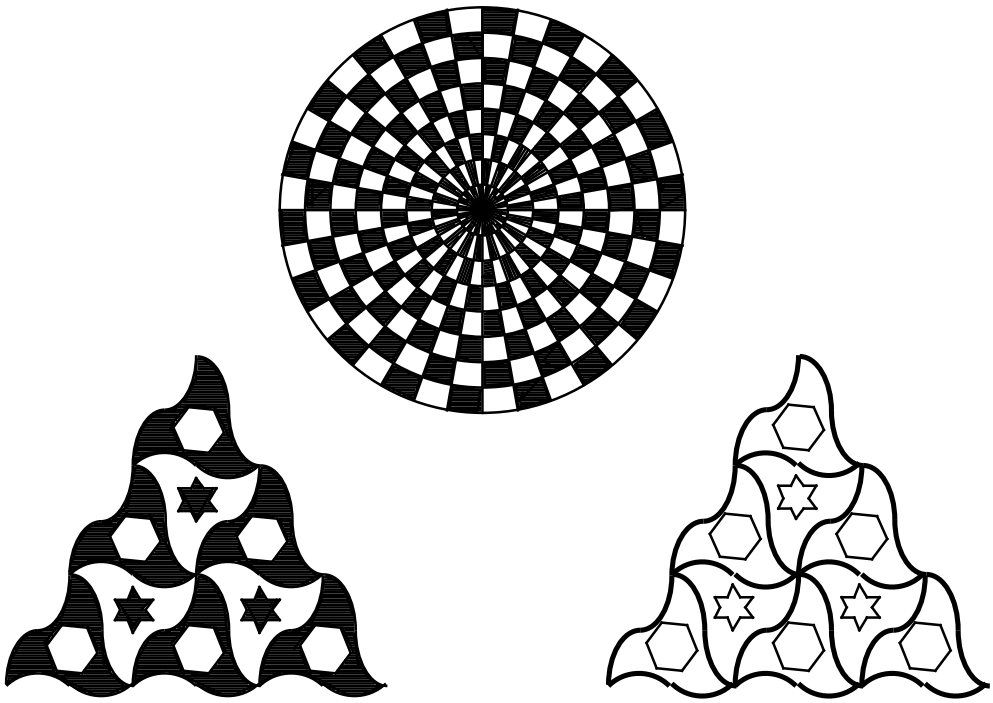


FIG. 20.12 – *Escher-style tiling of the disc and of the triangle*

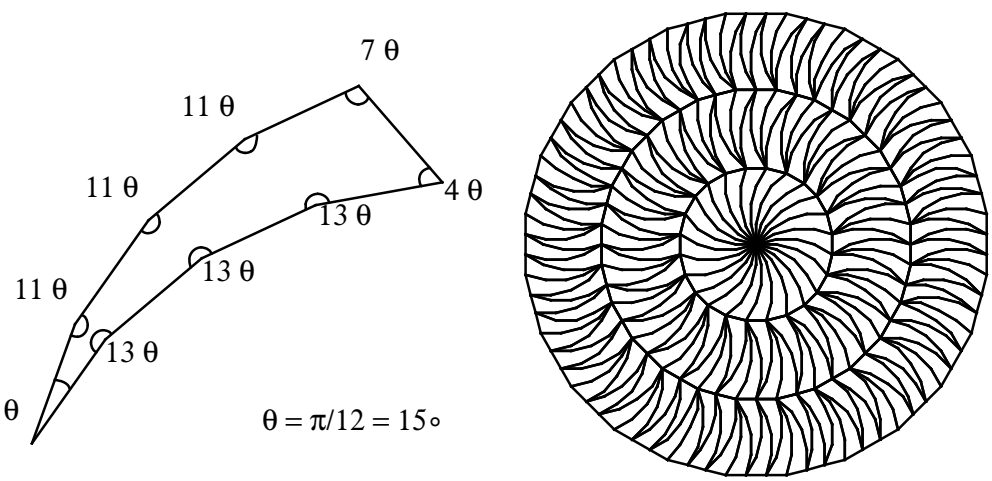


FIG. 20.13 – *Voderberg's tile and tiling of the disc*

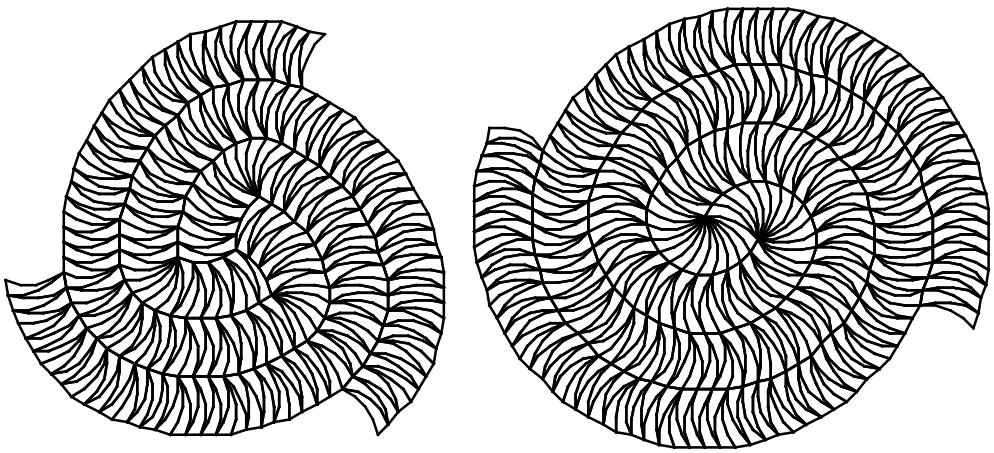


FIG. 20.14 – *Voderberg spirals*

Exercise 40 *Write Python programs using the commands* `gsRT` *and* `rot` *to draw the nets of the following solids:*

– *the Platonic solids (regular cube, tetrahedron, octahedron, dodecahedron and icosahedron) of figure 20.15 (page 242);*

– *the great and small rhombicuboctahedra and the truncated tetrahedron of figure 20.16 (page 243).*

The nets of solids given by Cundy and Rollett [12] are the source of many exercises. □

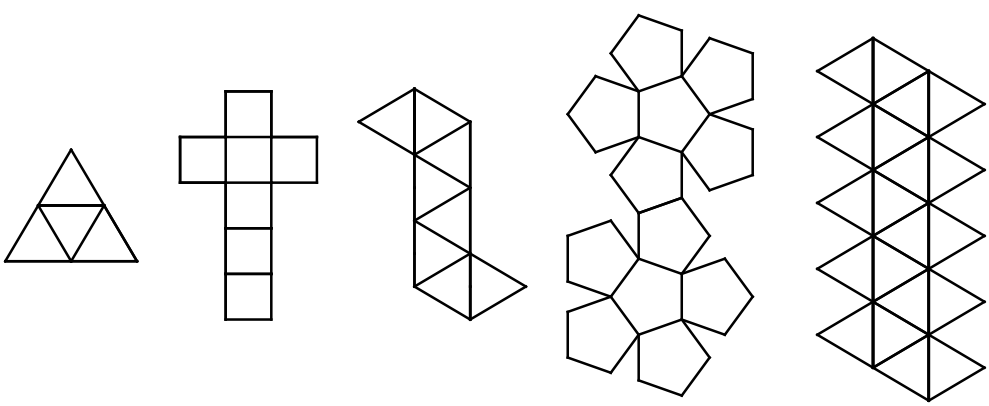


FIG. 20.15 – *Nets of the Platonic solids*

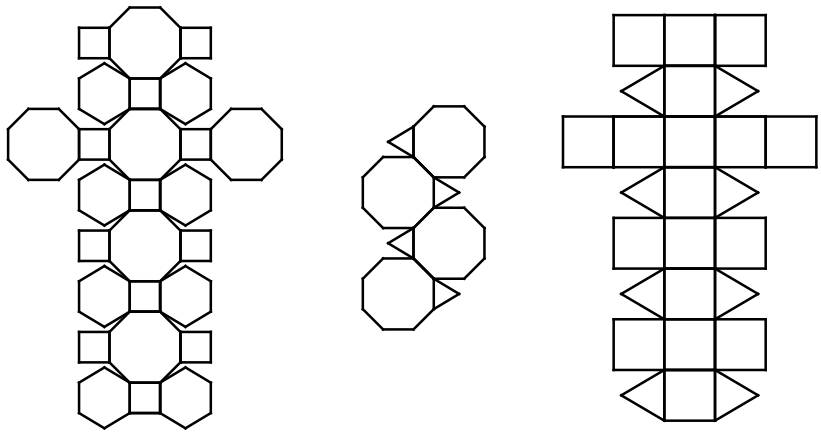


FIG. 20.16 – *Nets of the great and small rhombicuboctahedra* and of the truncated tetrahedron

Exercise 41 *Write Python programs using the commands* `rot` *and* `fdl` *to make the following drawings:*

– *the polygonal rosettes of figure 20.7 (page 238);*

– *the "jolygons" (from the French* joli, *"pretty", and* polygone*) of figure 20.18 (page 245), obtained by drawing successive segments whose length decreases by 5 % each time, two successive segments making an angle of θ degrees; θ being chosen equal to 15, 60, 72, 76, 89, 92, 118, 175 and 144 degrees respectively for the jolygons taken from left to right and from top to bottom;*

– *the plane curves of figure 20.19 (page 245), defined by a polar equation, as follows:*

- *the limaçon of Pascal, $\rho = a + b\cos\theta$ with $a \neq 0$, $b \neq 0$ and $\theta \in [0, 2\pi]$ (in the example, $a = 40$ and $b = 80$),*
- *the three-petalled rhodonea (rose curve), $\rho = a\cos n\theta$ with $a = 60$, $n = 3$ and $\theta \in [0, \pi]$,*
- *Moritz's curve, $\rho = a\cos\frac{9}{10}\theta$ with $a = 60$ and $\theta \in [-10\pi, 10\pi]$;*

– *the plane curves defined by the Cartesian equations $y = \sin x, x \in [-2\pi, 2\pi]$ (figure 20.17 below) and $y = \cos x, x \in [-2\pi, 2\pi]$, using the commands* `moveToReal` *and* `drawToReal` *(move to and draw to a point with real-number coordinates). The command* `drawToReal(x, y)`*, where* `x` *and* `y` *are rational expressions, draws a segment (if the pen is down) from the current position*

of the Robot to the point with Cartesian coordinates $(x,\ y)$ *and moves the Robot to this new position* $(x,\ y)$. *The command* `moveToReal(x, y)` *does the same without drawing. The functions* `sin` *and* `cos` *and the constant* `pi` *are explained in the addendum of chapter 21. Many other examples of curve drawing are given in [10].*
□

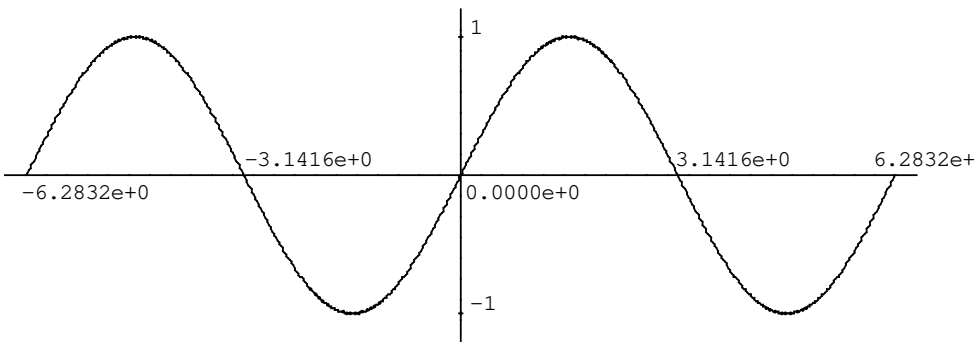


FIG. 20.17 – *Cartesian curve* $y = \sin x, x \in [-2\pi, 2\pi]$

Solution 38 *We give the program that draws the spider's web of figure 20.3 (page 236), the other programs being in the folder* `book/20_polar_grid`*:*

```
from LibRobot import *

# Draw a spider's web
def hexagon(r):
    gsRT(r, 60)
    gc(); fw()
    for i in range(2):
        pu(); fd(); fd(); pd(); tetr(); fd(); etr()
        fd(); fd(); etr(); fd(); gc()
        fe()

def diagonals(r):
    gsRT(r, 60)
    fsw()
    for i in range(4):
        fd(); gc(); qtr()
    gc(); fw(); fdn(2); gc(); fe(); fdn(2)

def web():
    for i in range(1, 9):
```

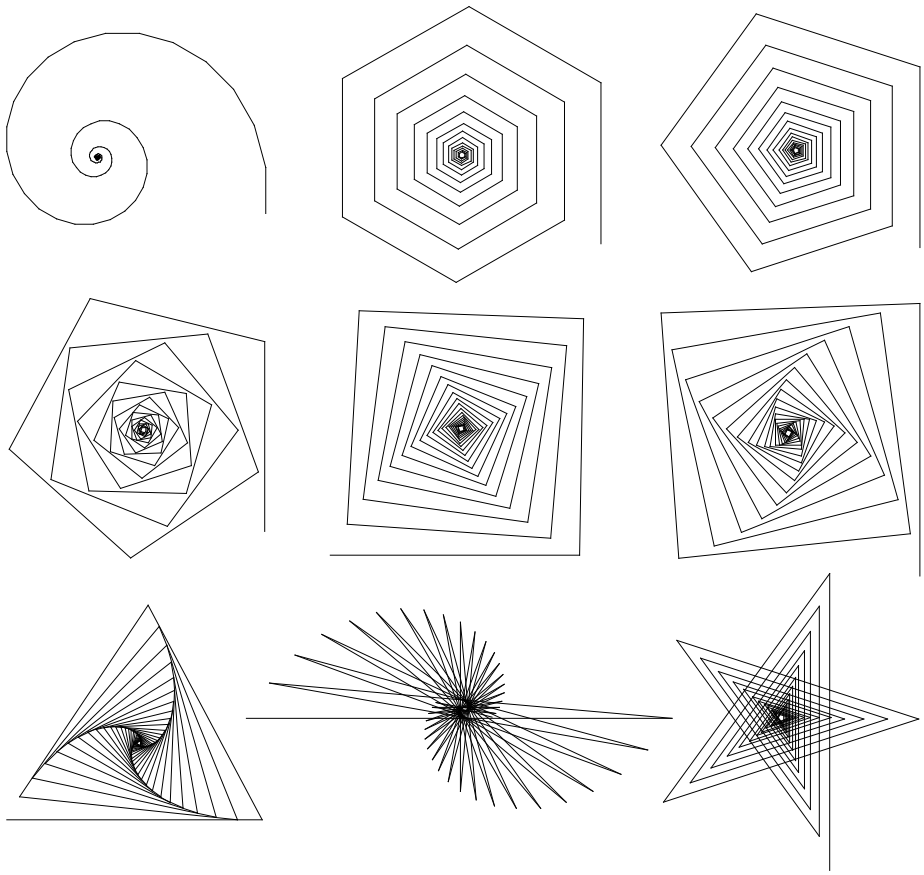


FIG. 20.18 – *Jolygons at 15, 60, 72, 76, 89, 92, 118, 175 and 144 degrees*

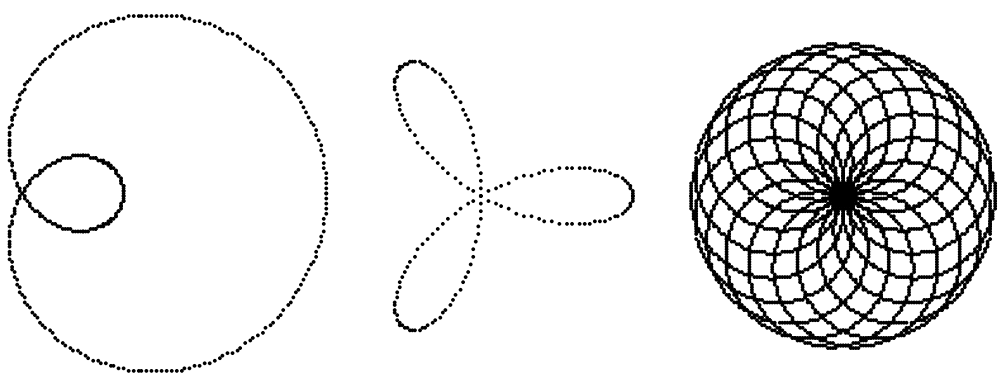


FIG. 20.19 – *Limaçon of Pascal, three-petalled rhodonea, Moritz's curve*

```
        hexagon(5 * i)
        diagonals(5 * i)
    web(); st()
```

□

Solution 39 *We give the procedure that draws a regular polygon with n sides (figure 20.8, page 239), the other programs being in the folder* `book/20_polar_grid`*:*

```
from LibRobot import *

# Draw a regular polygon
gridSize = 30
nbSides = 7

# Draw a regular polygon with n >= 1 sides
def polygon(n):
    for i in range(n):
        fd(); rot(360 / n)

gs(gridSize); polygon(nbSides); st()
```

□

Solution 40 *We give the program that draws the net of the truncated tetrahedron, the other programs being in the folder* `book/20_polar_grid`*:*

```
from LibRobot import *

# Draw the net of a truncated tetrahedron
sideLength = 24

def faceAndCorner():
    # Hexagon
    qtr(); gsRT(sideLength, 60); etl()
    for i in range(6):
        fd(); rot(60)
    # Equilateral triangle
    ht(); fd(); qtr(); fd()

def truncatedTetrahedronNet():
    for i in range(2):
        # Face, bottom corner
        faceAndCorner(); pu(); tetr(); fdn(2); etl(); fd()
        tetr(); pd()
        # Face, top corner
        faceAndCorner(); pu(); fd(); etl(); fdn(2); qtl()
```

```
        pd()
  truncatedTetrahedronNet(); st()
```

□

Solution 41 *We give the program that draws the jolygon at 175 degrees; the other jolygons are obtained by changing the value of the constant* `angle`. *The other programs are in the folder* `book/20_polar_grid`.

```
  from LibRobot import *

  # Draw a 175-degree jolygon
  initialLength = 350
  angle = 175
  nbSegments = 40

  gl()
  gs(initialLength); qtr(); fd()
  for i in range(nbSegments - 1):
     rot(angle); fdl(valFdl() * 0.95)
  st()
```

□

Addenda

The command `gsRT(r, t)` sets the length `r` of the diagonal of a grid square (in pixels) and the angle `t`, in degrees, that this diagonal makes with a horizontal line. It is equivalent to:

```
gsX(r * cos(t / 180 * pi))
gsY(r * sin(t / 180 * pi))
```

but avoids using trigonometric functions. The angle `t`, between 0 and 90, defines the north-east orientation, which coincides with east when `t` is 0 and with north when `t` is 90.

The command `rot(t)` makes the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ turn on the spot through an angle of `t` degrees to the left, in the positive (trigonometric) direction, that is counter-clockwise, if $\texttt{t} > 0$, and to the right, clockwise, if $\texttt{t} < 0$. The command `fdl(l)` makes the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ move straight forward (if $\texttt{l} > 0$) or backward (if $\texttt{l} < 0$) by `l` times the length of the side of a pixel of the screen.

These commands `rot` and `fdl` are directly inspired by the "turtle" graphics of LOGO. They change the coordinate system and the orientation of the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ so as to remain compatible with the use of Cartesian or polar coordinates.

The LOGO language was designed by Seymour Papert at the end of the 1970s [31]. It has many similarities with LISP [28], in particular the interpretation of data representing instructions, dynamic scoping of identifiers and the list structure. Interpreting data as programs entails some subtleties in writing programs (such as the use of the `quote` function, [45]) which are not within the reach of all beginners in computer science. Dynamic scoping of identifiers [45] has largely been abandoned, and the list structure is almost always supplemented by arrays in modern LISPs such as COMMON LISP, which in this follow the languages descended from ALGOL 60 [29], such as PASCAL or ADA. The Python language also offers both lists and arrays, does not interpret its data as instructions (unless explicitly asked to) and uses static scoping of identifiers.

21

Recursion

In Python, a procedure can call another procedure declared earlier. A procedure can also call itself. It is then said to be *recursive*. It is declared with **def**, like any other procedure: there is nothing special to write. Recursive procedures are useful for making drawings like those of figure 21.1 (page 250). These recursive drawings have the peculiarity of being found again and again, endlessly, in reduced form inside themselves (exactly like the infinite image obtained between two mirrors facing each other). Since in practice it is not possible to draw a line smaller than a pixel of the screen, the drawing stops for very small drawings.

21.1 Simple recursion

The structure of simple recursive drawings is understood by *induction* on the size of the drawing. We must first understand the *base*, that is, how to make a drawing of size 0 (or 1). Then we must understand the *induction step*, that is, how to build a drawing of size n using identical drawings of sizes $n-1$, $n-2$, ... strictly smaller than n.

Let us take the example of the recursive paving stones of figure 21.2 (page 251). A paving stone of size 0 is reduced to a point (more precisely a pixel of the screen). A paving stone of size n consists of a paving stone of size $n-1$ surrounded by a square whose side is n times the grid size, as explained in figure 21.3 (page 251). The recursive procedure `pavingStone` that draws a recursive paving stone receives as parameter the size `n` of the paving stone. If `n` is zero, it draws a point. If `n` is strictly positive then it draws a square of

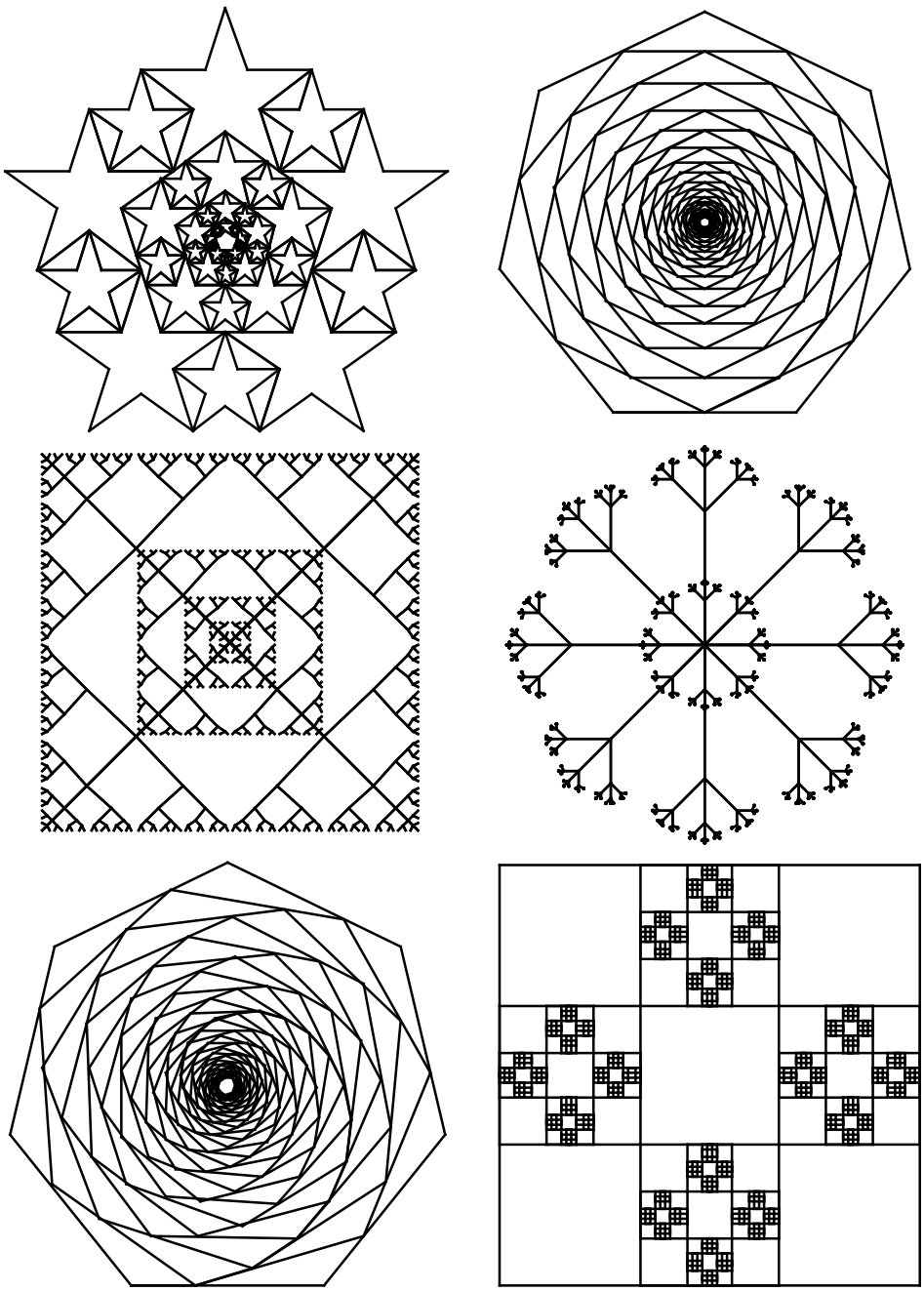


FIG. 21.1 – *Recursive drawings*

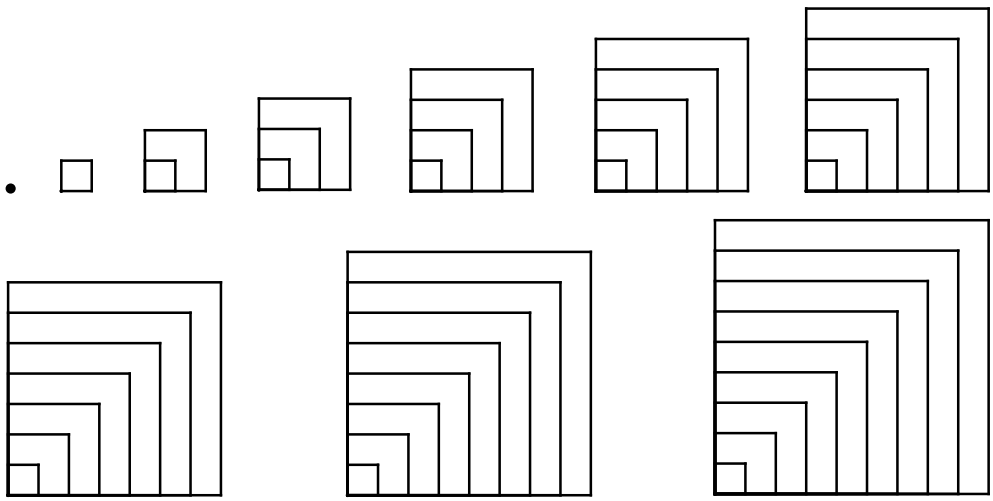


FIG. 21.2 – *Recursive paving stones*

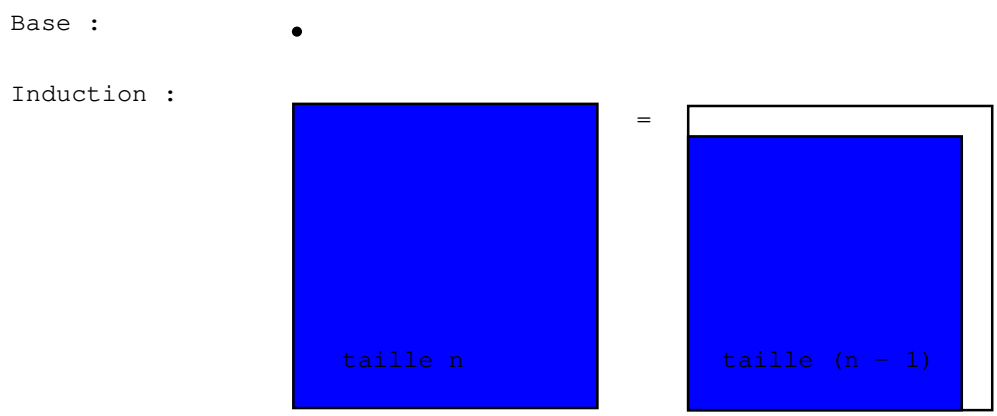


FIG. 21.3 – *Inductive construction rule of the recursive paving stones*

side `n` and then calls itself recursively to draw a paving stone of size `n - 1`.

```
from LibRobot import *
# Draw recursive tiles
def pavingStone(n):
    if n == 0:
        # Draw a square of side 0
        dp()
    # Draw a square of side n
    else:
        fdn(n); qtr(); fdn(n); qtr(); fdn(n); qtr()
        fdn(n); qtr()
        # Tile the inside
        pavingStone(n - 1)
gb(); gl(); pavingStone(10); st()
```

The name `pavingStone` can be used in the body of the procedure `pavingStone` itself: when Python executes the call `pavingStone(n - 1)`, the declaration of `pavingStone` is finished and the name `pavingStone` is therefore known. A recursive procedure is thus declared and called exactly like the procedures we have seen so far.

The recursive paving stones can also be drawn *iteratively*, that is, using a "for" or "while" loop. This would be more difficult with drawings like the draw sheet of a tennis tournament shown in figure 21.4 below.

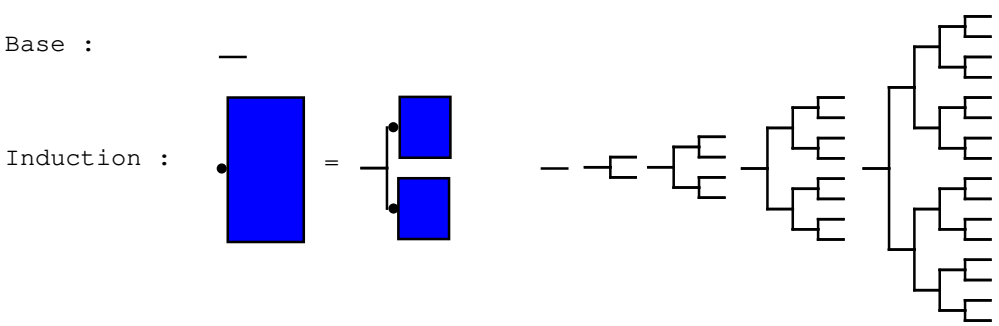


FIG. 21.4 – *Inductive construction rule of the draw sheet* of a tennis tournament

The draw sheet of a tennis tournament with only one entrant consists of one horizontal line on which to write the name of this entrant, who is also the winner. The draw sheet of a tennis tournament with `n` entrants is

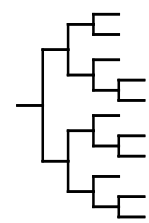
obtained by drawing a horizontal line on which to write the name of the winner, facing the point of a brace joining the sub-sheets used to determine the two finalists. Each of the sub-sheets contains roughly the same number of players, that is, `n // 2` and `n - n // 2` players respectively (recall that `//` is integer division). The recursive procedure that draws the draw sheet of a tournament of `n` players is given below and illustrated in the case where `n` is 11.

```
from LibRobot import *

# Draw the bracket of a tennis tournament
l = 5  # Length of a horizontal line

# Draw a bracket for a tournament of n players
def bracket(n):
    # Tip of the brace, for the winner
    fdn(l)
    if n > 1:
        # Upper branch of the brace
        qtl(); fdn(n); qtr()
        # Upper sub-bracket, containing
        # half of the players
        bracket(n // 2)
        # Go back to the centre of the brace
        pu(); qtl(); fdn(-n); qtr(); pd()
        # Lower branch of the brace
        qtr(); fdn(n); qtl()
        # Lower sub-bracket, containing the
        # rest of the players
        bracket(n - n // 2)
        # Go back to the centre of the brace
        pu(); qtr(); fdn(-n); qtl(); pd()
    # Go back to the end of the tip
    fdn(-l)

gs(3); fe(); bracket(16); st()
```



21.2 Invariants

When writing a recursive procedure it is important to think about the *invariant* of this procedure, that is, the hypotheses that the actual parameters of the procedure and the $\mathcal{R}^{o}_{b}{}^{o}t$ must satisfy for the drawing to be correct. Let us take the example of a binary tree (see figure 21.5 below), a basic

structure of programming, which we have already met in the form of the tennis tournament draw sheet.

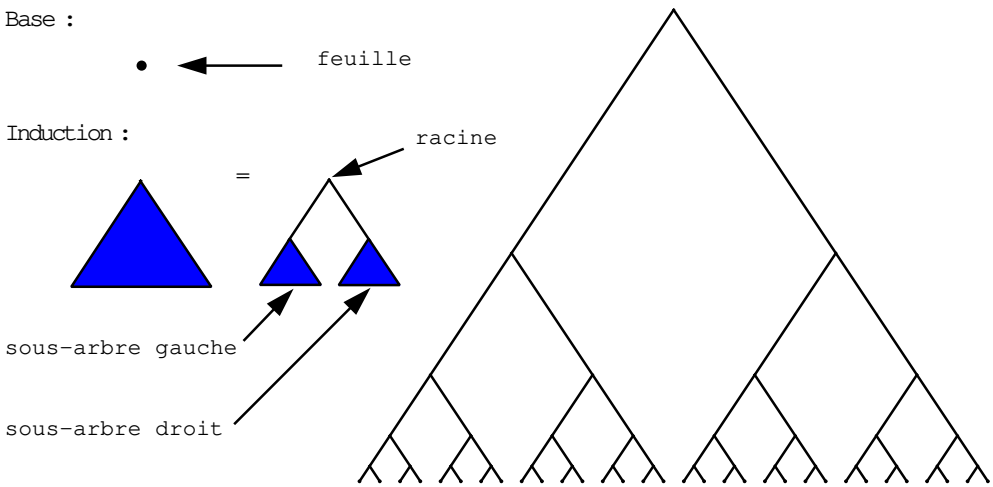


FIG. 21.5 – *Inductive construction rule of a binary tree*

The procedure that draws a binary tree is the following:

```
from LibRobot import *

# Draw a binary tree

# Draw a tree with l leaves, root at the top,
# the leaves at the bottom, assuming that l >= 1,
# the tree being reduced to a point when l = 1. Before
# and after the drawing, the robot must be placed at the
# root of the tree, facing north with its pen
# down.
def tree(l):
    if l == 1:
        # Draw a tree reduced to a single point (without
        # changing the position and orientation of the robot)
        dp()
    # With the robot placed at the root of the tree,
    # facing north with its pen down, draw
    # the south-west branch
    else:
        tetl(); fdn(l)
        # Turn the robot to face north
        tetr()
        # Draw the left subtree having l / 2 >= 1
```

```
        # leaves, the robot being placed at the root of the
        # subtree, facing north with its pen
        # down
        tree(l // 2)
        # Given that the robot is placed at the root of the
        # left subtree, facing north with its pen
        # down, go back to the root of the tree, without
        # drawing anything
        pu(); etr(); fdn(l)
        # With the robot placed at the root of the tree,
        # facing north-east with its pen up,
        # draw the south-east branch
        pd(); qtr(); fdn(l)
        # Turn the robot to face north
        tetl()
        # Draw the right subtree having l - l / 2 >= 1
        # leaves, the robot being placed at the root of the
        # subtree, facing north with its pen
        # down
        tree(l - l // 2)
        # Place the robot at the root of the tree, facing
        # north with its pen down
        pu(); etl(); fdn(l); etr(); pd()
gsX(2); gsY(3); gt(); tree(32); st()
```

The first invariant is that the parameter `l` of the procedure `tree` must be greater than or equal to 1. This is true for the main call since `l` $= 32$. Any call makes further calls only if `l` > 1. Since `l` ≥ 2, we have (`l // 2`) ≥ 1 and the invariant is true when the procedure is called recursively to draw the left subtree. Since `l` $= (2 * ($`l // 2`$))$ if `l` is even and `l` $= (2 * ($`l // 2`$)) + 1$ if `l` is odd, we have `l` $\geq (2 * ($`l // 2`$))$ hence (`l` $-($`l // 2`$)) \geq ($`l // 2`$)$ so that (`l` $-($`l // 2`$)) \geq 1$ since (`l // 2`) ≥ 1. The invariant is therefore also true when the procedure `tree` is called recursively to draw the right subtree. Since the invariant is true for the main call and remains true for all subsequent calls, it is always true.

In the same way one would check the invariant concerning the $\mathcal{R}^{o}_{\mathcal{O}}{}^{o}t$, namely that before and after drawing the binary tree, the $\mathcal{R}^{o}_{\mathcal{O}}{}^{o}t$ is placed at the root of the tree, facing north with its pen down.

21.3 Non-termination

When using recursive procedures, it can happen that they call one another endlessly, exactly as in the case of "while" loops whose execution does not terminate. The simplest example is that of the procedure `loopForever` below.

```
from LibRobot import *

# A program that does not terminate
def loopForever():
   loopForever(); st()

loopForever(); st()
```

A call of `loopForever()` immediately causes a new call of `loopForever()`, and so on endlessly. At each call, the computer uses a little of its memory (called the *stack*) to remember what remains to be done afterwards (here the `st()` that follows the call). To avoid filling up the whole memory, Python limits the number of calls in progress to about 1000. Beyond that, an error occurs (`RecursionError`), which the $\mathcal{R}$obot's library reports with the message '`Error line 5 of NonTermination.py: endless recursion (stack full)`' before the execution of the program stops (line 5 is the one with the call `loopForever()` in the body of the procedure). In general the error is less obvious and occurs only for certain values of the parameters. This limit also means that a correct recursive procedure cannot call itself more than about 1000 times in a row: this is more than enough for our drawings, since in practice the drawing stops well before that, at the level of the pixel.

21.4 Mutual recursion

Consider the serpentines of figure 21.6 (page 257). There are two kinds of them. The serpentines $\texttt{E}_n$ have a height n and are drawn starting from their lower west corner, by drawing a horizontal line from west to east and then a vertical line to move to the east end of the floor above. The serpentines $\texttt{W}_n$ have a height n and are drawn starting from their lower east corner, by drawing a horizontal line from east to west and then a vertical line to move to the west end of the floor above.
The procedures `serpentineE` and `serpentineW` in the program below draw a serpentine $\texttt{E}_n$ and $\texttt{W}_n$ respectively, their formal parameter `n` having as actual

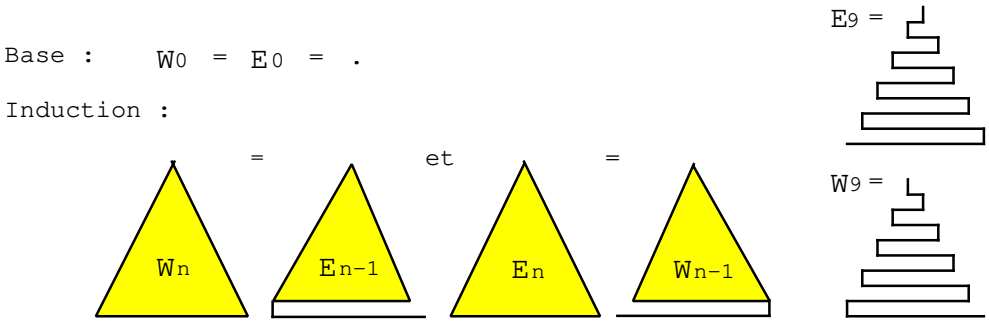


FIG. 21.6 – *Inductive construction rule of a serpentine*

value the height n of the serpentine to be drawn.

```
from LibRobot import *

# Draw serpentines

# Build a serpentine of height n starting from
# its lower west corner
def serpentineE(n):
   if n > 0:
      # Build the base of width n, from west to east
      fe(); fdn(n); fn(); fd()
      # Build the upper serpentine of height
      # n - 1 starting from its lower east corner
      serpentineW(n - 1)

# Build a serpentine of height n starting from
# its lower east corner
def serpentineW(n):
   if n > 0:
      # Build the base of width n, from east to west
      fw(); fdn(n); fn(); fd()
      # Build the upper serpentine of height
      # n - 1 starting from its lower west corner
      serpentineE(n - 1)

gs(6); gb(); gr(); serpentineW(18); gb(); gl()
serpentineE(18); st()
```

Since a serpentine `E0` or `W0` of height zero is empty, the procedures `serpentineE` and `serpentineW` do nothing when the value n of their parameter `n` is negative or zero. To draw a serpentine `En` or `Wn` whose height $n \geq 1$ is the actual value of their formal parameter `n`, the procedures `serpentineE` and `serpentineW` draw the base and then above it a serpentine `Wn-1` or `En-1` whose height is

the value of `n - 1`. Since the procedure `serpentineE` calls the procedure `serpentineW` and the procedure `serpentineW` calls the procedure `serpentineE`, these procedures are said to be *mutually recursive*.

When using mutually recursive procedures in Python, one must respect the rule that *every identifier must be declared before it is used*. Yet the body of `serpentineE` calls `serpentineW`, which is declared after it. This is allowed because Python looks up the meaning of the name `serpentineW` only when it executes this call, that is, during the execution of the main program `serpentineW(18)`, when both procedures have already been declared. It is therefore enough to declare the two procedures one after the other, *before* the first call of either of them. Each one can then call the other (and itself). If, on the contrary, `serpentineE` were called between the two declarations, execution would stop with the message '`NameError: name 'serpentineW' is not defined`'.

In general, declarations of mutually recursive procedures have the following form:

Declaration of mutually recursive procedures:

```
# Specification of the first procedure
def procedureName1(formalParam11, formalParam12, ...):
    ... Instructions of the body of the procedure (indented) ...

# Specification of the second procedure
def procedureName2(formalParam21, formalParam22, ...):
    ... Instructions of the body of the procedure (indented) ...

# Instructions of the program (calls of the procedures)
```

[a] A procedure name is an identifier.
[b] A formal parameter is an identifier.
[c] Including calls of the procedures $\texttt{procedureName}_1$ and $\texttt{procedureName}_2$.
[d] The expressions in these instructions can use the formal parameters $\texttt{formalParam}_1^1$, $\texttt{formalParam}_1^2$, ... of the procedure $\texttt{procedureName}_1$.
[e] The expressions in these instructions can use the formal parameters $\texttt{formalParam}_2^1$, $\texttt{formalParam}_2^2$, ... of the procedure $\texttt{procedureName}_2$.

In the same way, three or more mutually recursive procedures can be declared one after the other.

Exercise 42 *The exercise consists in drawing* fractals *[2] [27], that is, drawings that are found again endlessly, in reduced form, inside themselves.*

– *Write a recursive Python procedure to build a Von Koch curve*[1]*, as shown in figure 21.7 (page 260). The curve K_0 is a straight line segment. The curve K_1 is obtained by dividing the segment K_0 into three equal segments and replacing the middle one by an equilateral triangle without its base. The curve K_n is obtained in the same way by dividing into three each of the segments of the curve K_{n-1}. Write a program that draws a Von Koch snowflake, obtained by fragmenting in this way, outwards, the sides of an equilateral triangle. Do the same for a Von Koch anti-snowflake, for which the fragmentation is done towards the inside of the initial equilateral triangle. Use various fragmentations of a segment to obtain other kinds of snowflakes like those of figure 21.10 (page 262).*

– *Program the drawings of the recursive patterns of figure 21.1 (page 250).*

– *Write a Python program that draws the Sierpinski*[2] *squares (figure 21.11 page 263), triangles (figure 21.12 page 264) and pentagons (figure 21.13 page 265).*

– *Draw the plants of figure 21.14 (page 266).*

– *Draw the Sedgewick squares [39] of figure 21.15 (page 267).*

– *Draw the dragon curve of figure 21.16 (page 268) according to the construction rules of figure 21.8 (page 260). The dragon curve of order n is the one seen when looking edge-on at a sheet of paper that has been folded n times onto itself and then unfolded at right angles.*

– *Draw the Hilbert*[3] *curve of figure 21.17 (page 269) according to the construction rules of figure 21.9 (page 261).*

– *Draw the Peano*[4] *curve of figure 21.19 (page 271) according to the construction rules of figure 21.18 (page 270).*

---

[1] Von Koch, Swedish mathematician, 1870–1924.
[2] Waclaw Sierpinski, Polish mathematician, 1882–1969.
[3] David Hilbert, German mathematician, 1862–1943.
[4] Giuseppe Peano, Italian mathematician, 1858–1932.

_ *Write a Python procedure that writes in base 2 a number passed as parameter, noting that the binary representation $\beta(n)$ of a number n is defined by:*

- $\beta(-n) = -\beta(n)$ *if* $-n < 0$;
- $\beta(0) = 0$;
- $\beta(1) = 1$;
- $\beta(n) = \beta(n \texttt{ // } 2)\beta(n \texttt{ \% } 2)$ *if* $n > 1$.

Do the same in bases 3, 8 and 16 (using the digits `A`, `B`, `C`, `D`, `E`, `F` *to denote the numbers 10, 11, 12, 13, 14, 15 respectively, and "*`label("d"); translate(1, 0)`*" to write the digit* `d`*).*
□

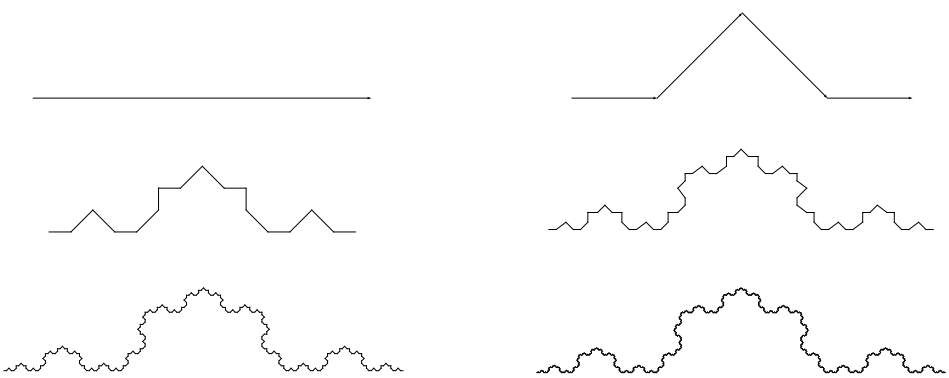


FIG. 21.7 – *Von Koch curve*

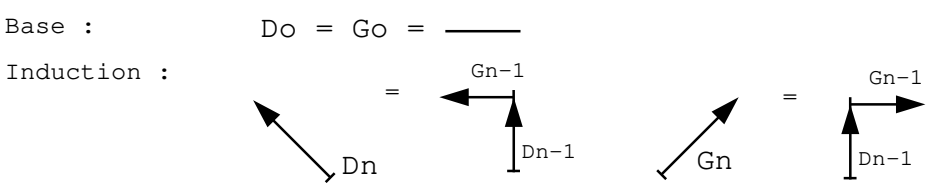


FIG. 21.8 – *Construction rules of the dragon curve*

Solution 42 *We give two programs, the others being in the folder* `book/21_recursion`*:*

■ **from** `LibRobot` **import** `*`

```
Base :          Do = Go = .
Induction :
```

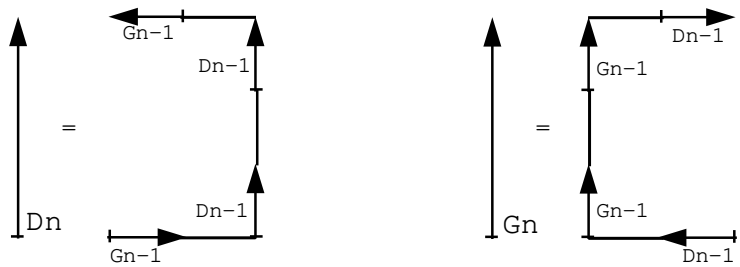


FIG. 21.9 – *Construction rules of the Hilbert curves*

```
# Draw a Von Koch curve
def segment(n):
   if n == 0:
      fd()
   else:
      segment(n - 1)
      etl()
      segment(n - 1)
      qtr()
      segment(n - 1)
      etl()
      segment(n - 1)
gs(6); gl(); fe(); segment(3); st()
```

-
```
from LibRobot import *
# Write a number in base 2
def decimalToBinary(n):
   if n < 0:
      label("-"); translate(1, 0)
      decimalToBinary(-n)
   elif n == 0:
      label("0"); translate(1, 0)
   elif n == 1:
      label("1"); translate(1, 0)  # n >= 2
   else:
      decimalToBinary(n // 2)
```

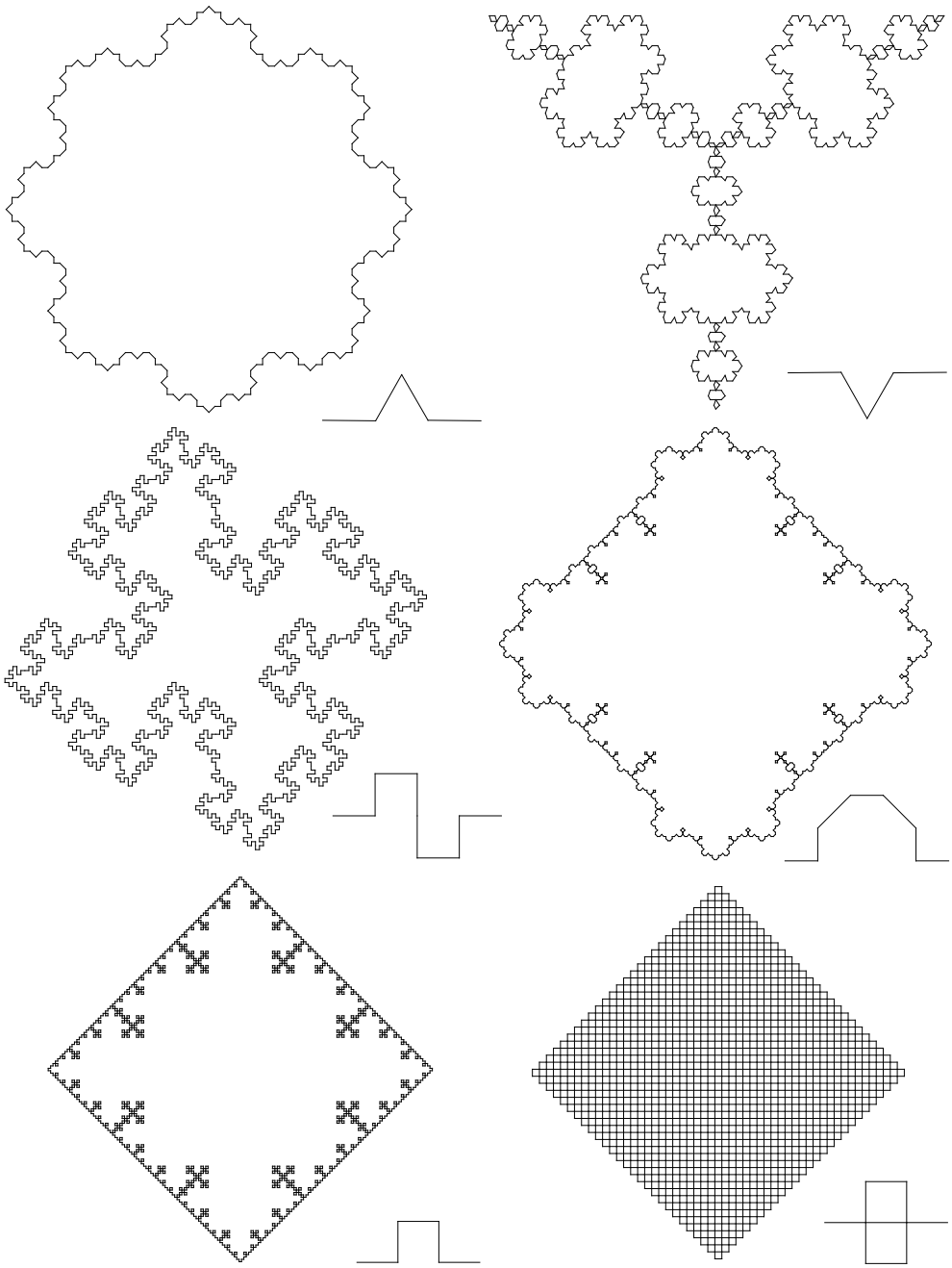


FIG. 21.10 – *Von Koch snowflakes*

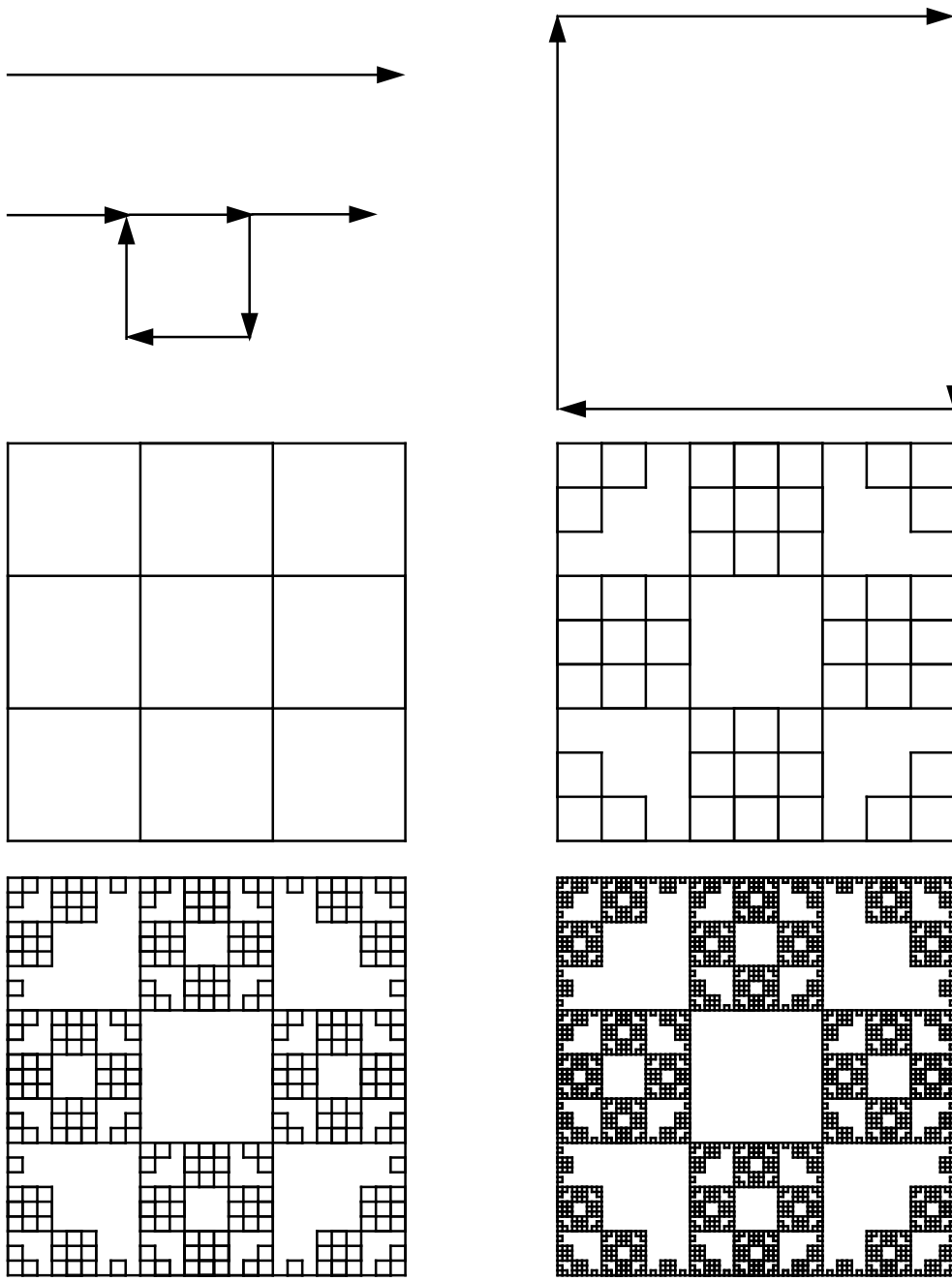


FIG. 21.11 – *Sierpinski squares*

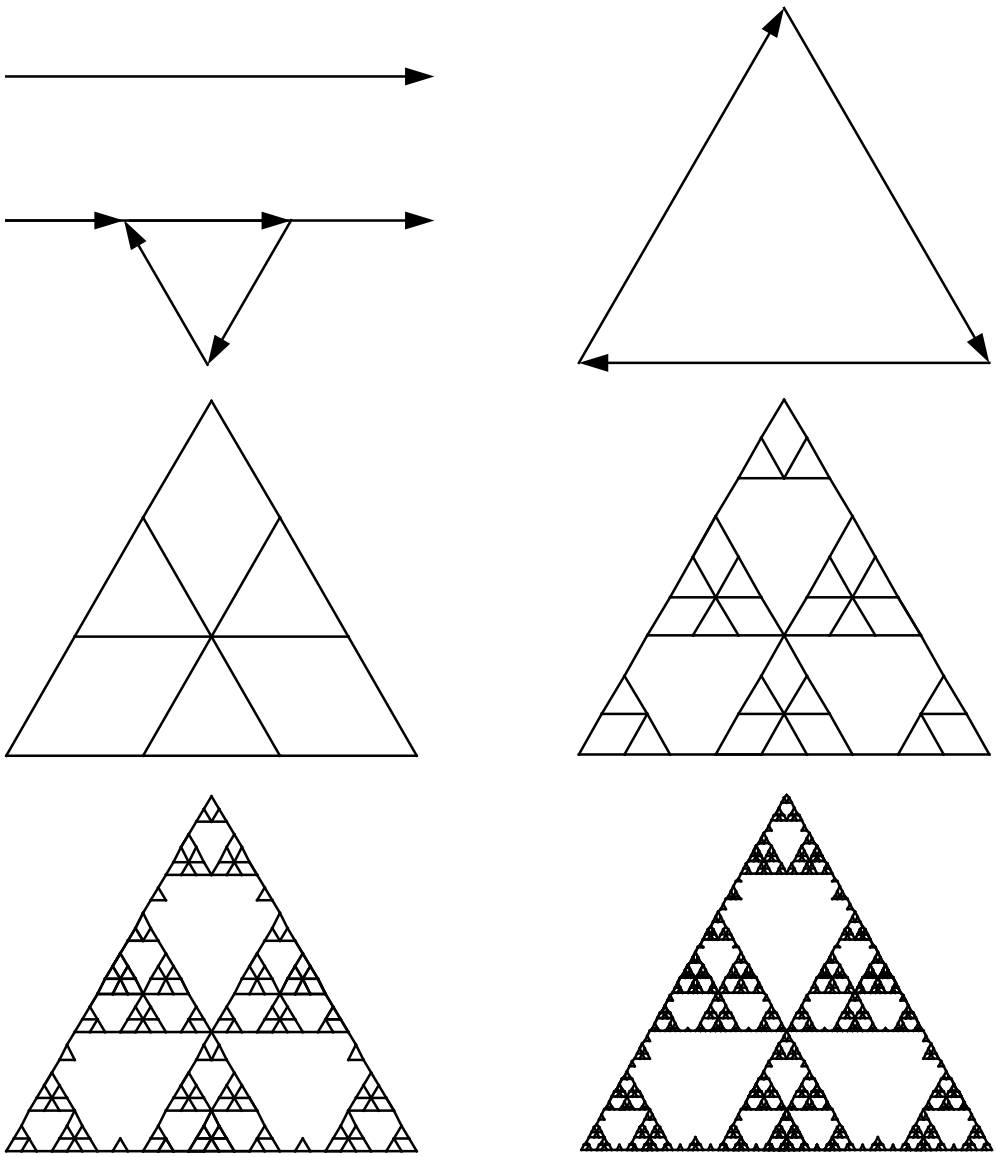


FIG. 21.12 – *Sierpinski triangles*

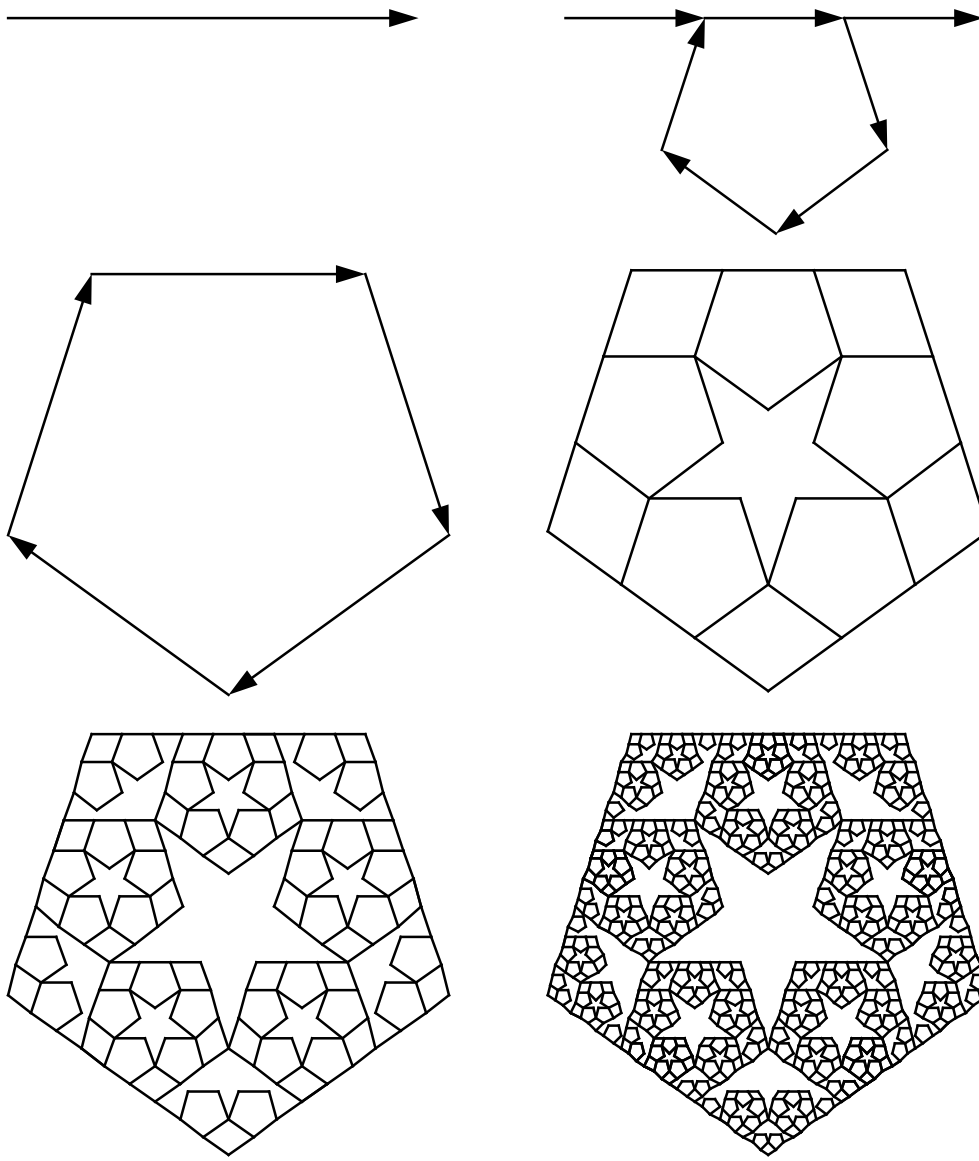


FIG. 21.13 – *Sierpinski pentagons*

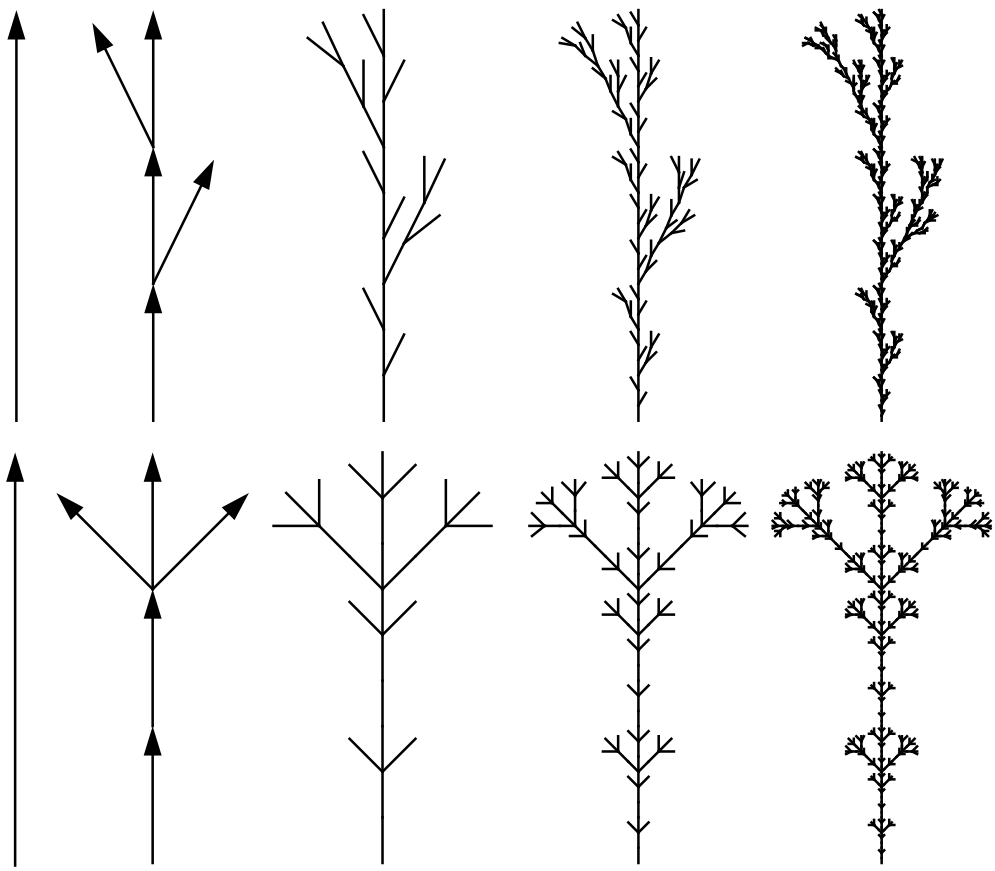


FIG. 21.14 – *Fractal plants*

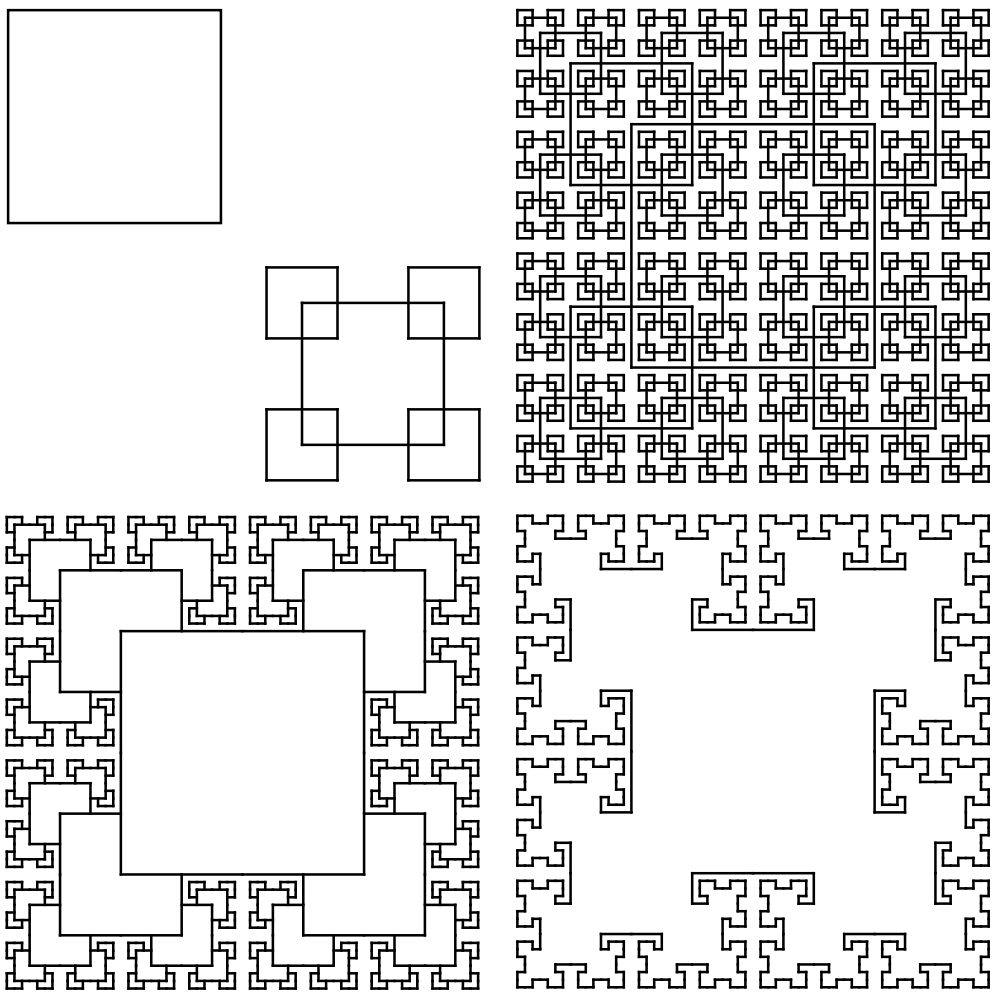


FIG. 21.15 – *Sedgewick's fractal star*

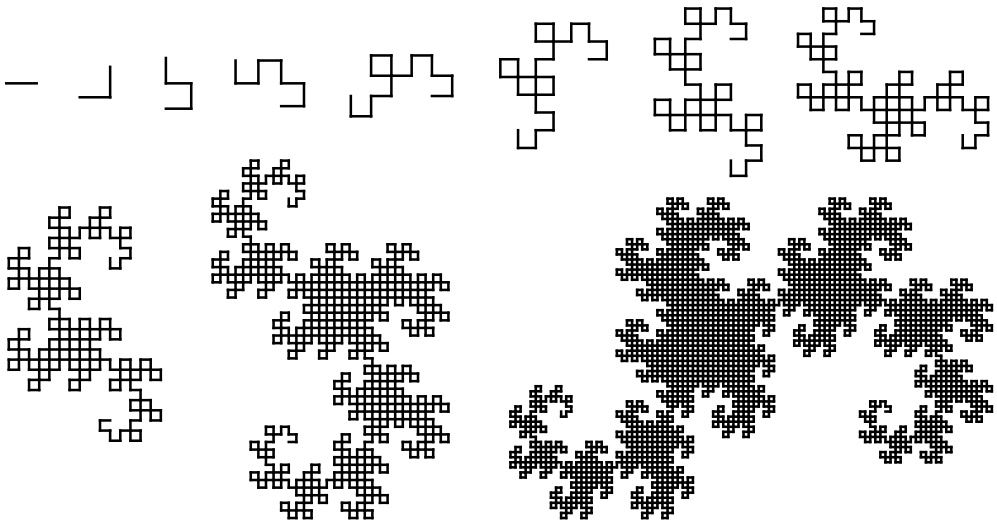


FIG. 21.16 – *Dragon curves of order 1, 2, . . . , 8, 10 and 12*

```
        decimalToBinary(n % 2)
  gs(7); decimalToBinary(123456789); st()
```

□

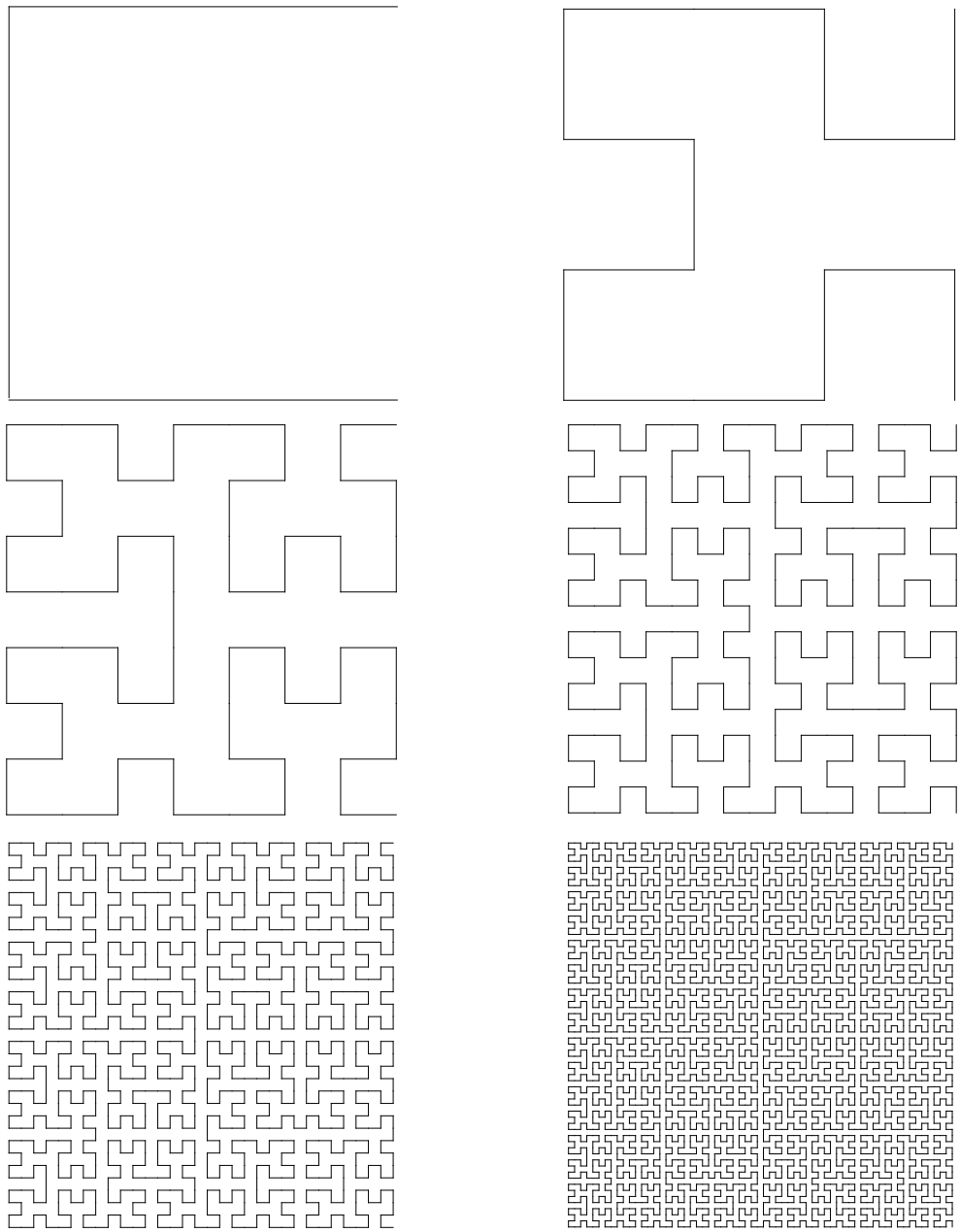


FIG. 21.17 – *Hilbert curves of orders 1 to 6*

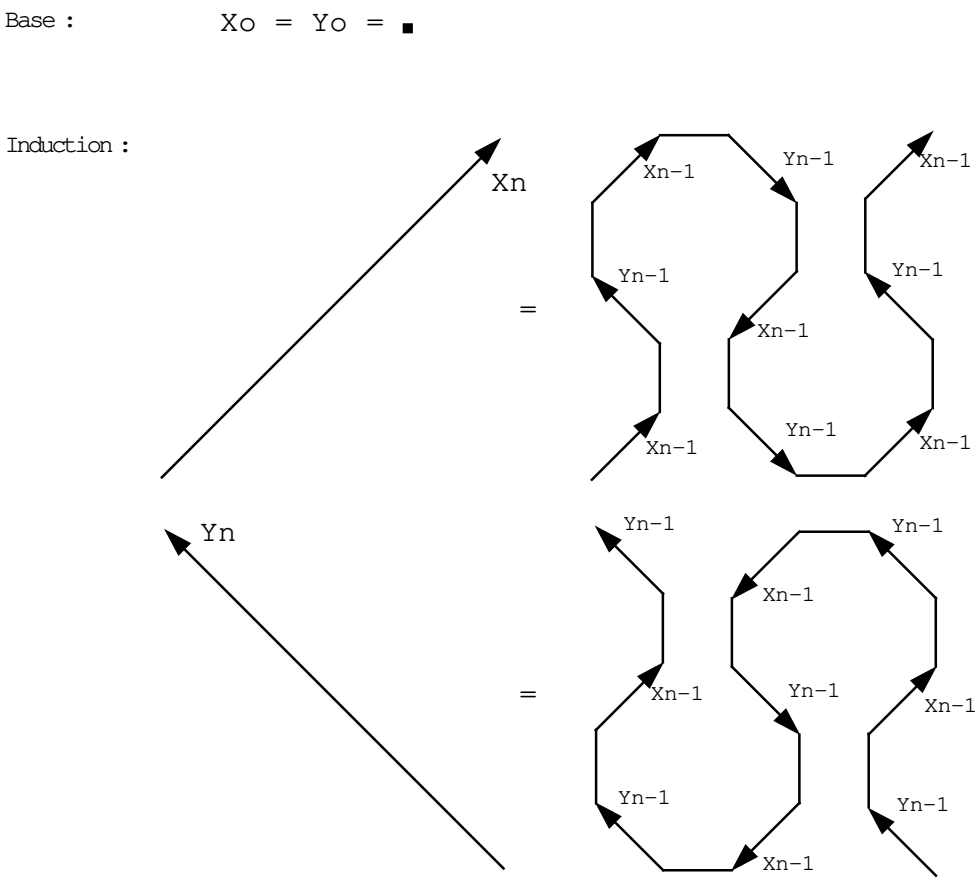


FIG. 21.18 – *Construction rules of the Peano curves*

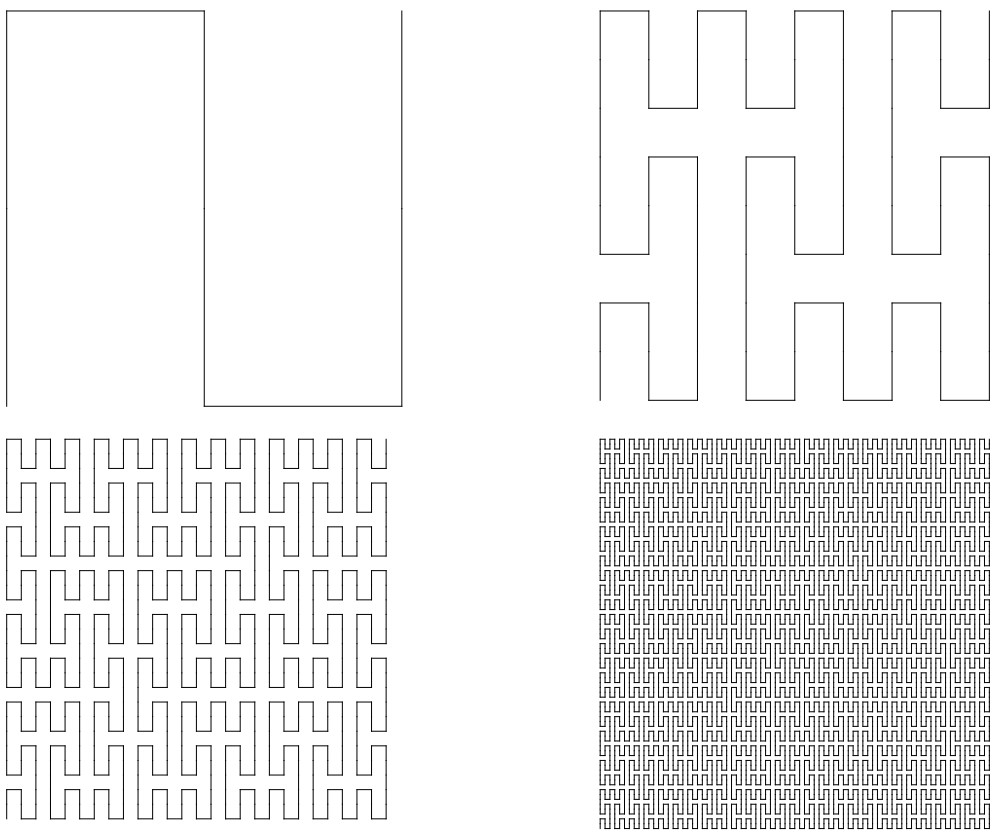


FIG. 21.19 – *Peano curves*

Addenda

The languages ALGOL60 [29] and then LISP [28] were the first to introduce the use of recursion. The early controversy over the efficiency of recursive procedures is now a thing of the past, compilation techniques being well established. The idea of recursion was at the root of many advances in computer science, whether recursive data structures (such as trees), recursive algorithms (such as fast sorting algorithms) or, more recently, functional programming, of which OCaml is an example (Python borrows some of its ideas).

The simplest way to understand a recursive procedure is to imagine that at each call the computer executes a new *copy* of the procedure.

To show that this succession of calls terminates, it is enough to show that there exists an integer function of the values of the parameters which is positive and strictly decreases over successive calls. For example the procedure `segment` of the program `VonKochSegment` (page 260) must be called with a parameter `n` that is positive or zero. If so, the recursive calls are of the form `segment(n - 1)` with $\texttt{n} > 0$, so that over successive calls the actual parameters strictly decrease while remaining positive. This cannot go on forever. Consequently the procedure `segment` does not loop when it is called with an initial parameter that is positive or zero.

Functions are to expressions what procedures are to instructions. In Python, a function declaration is written like a procedure declaration, but its body contains an instruction **return** *e* which terminates the execution of the function and returns as result the value of the expression *e*. Procedures are simply functions without **return** whose result we are not interested in, which is the special value **None** (nothing).

For example, a constant is a function without parameters, so that a declaration such as:

```
gridSize = 15
```

can be replaced by a function declaration:

```
def gridSize(): return 15
```

which is used by writing `gridSize()`, like `valGsX()`. As a second example, consider the computation of 2^n with $n \geq 0$, observing for the base that $2^0 = 1$ and for the induction step that $2^n = 2 * 2^{n-1}$:

```
def twoToThePower(n):
  if n == 0:
    return 1
  else:
    return 2 * twoToThePower(n - 1)
```

After this declaration, the expression `twoToThePower(2)` is 4 and the expression `twoToThePower(twoToThePower(2))` is 16 (the program `TwoToThePower` in the folder `book/21_recursion` draws the value of `twoToThePower(16)`). Here **if**. . . **else** chooses between two **return** instructions, hence between the two values that the function can return.

In Python one can use the trigonometric functions `sin` (sine), `cos` (cosine), `tan` (tangent) and `atan` (arc tangent). Angles are expressed in radians (an angle of d degrees is `d / 180 * pi` radians, the constant `pi` being $\pi \simeq 3.14159$). As for the functions `exp` (exponential) and `log` (natural logarithm), the parameter is a number (integer or rational) and the result is of type `float`. These functions and the constant `pi` are provided by the $\mathcal{R}\text{o}\underline{b}\text{o}t$'s library (they come from Python's `math` module). For powers, one uses `a ** x`, which is $a^x = e^{x \ln a}$. One could define the tangent function oneself by:

```
def tg(x): return sin(x) / cos(x)
```

When `cos(x)` is `0.0`, the division causes an error and stops the program with the message 'ZeroDivisionError: float division by zero'. Because of rounding errors, this does not happen for `tg(pi / 2)`: `cos(pi / 2)` is `6.123233995736766e-17` and not `0.0`, so that `tg(pi / 2)` is `1.633123935319537e+16`, a very large number instead of infinity.

22

Variables

We have used names to denote values: constants, parameters of procedures and counters of "for" loops. Until now, once fixed, the value denoted by such a name never changed. *Variables*, on the contrary, make it possible to store intermediate results of computations and to modify them during the execution of the program.

22.1 Variables and assignment

A variable is used to keep a value in the memory of the computer. In Python, a variable is created by a *variable declaration*, which gives the name of the variable (which is an identifier) and its initial value, placed after the sign `=`:

```
b = True
i = 0
r = 0.0
s = ""
```

This is written exactly like the constant declarations we have used so far: in Python, a constant is nothing but a variable whose value is never changed. It is Python that decides which part of the memory and which encoding will be used to represent the value of the variable, according to the type of this value: `b` is a boolean variable, `i` an integer variable, `r` a rational variable and `s` a variable containing a character string (here the empty string). In Python, a variable therefore always has a value, fixed as soon as it is created. This value can be used in an expression (such as `2 * i + 1`) simply by writing the name of the variable. A variable must not be used before it has been created: if the program begins with `i = i + 1` without first giving an initial

value to `i`, execution stops with the message '`NameError: name 'i' is not defined`'. The *assignment* instruction makes it possible to change the value of a variable. For example the instruction:

```
b = False
```

assigns the value "false" to the boolean variable `b`. In general, an assignment instruction has the following form:

Assignment instruction:

`variableName` = expression[a]

[a] The expression can use the old value of the variable.

The symbol `=` is read "becomes equal to". It must not be confused with the comparison `==` ("is equal to") used in the conditions of **if** and **while**. Executing this instruction consists in evaluating the "expression" to compute its value, which becomes that of the variable named `variableName`. The type of the variable is that of the value it contains:

Types of values:

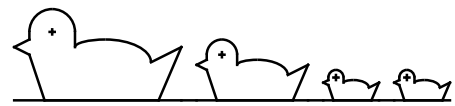
Example of value	Type	Name of the type
True, **False**	`bool`	boolean
`0`, `1`, `-5`	`int`	integer
`0.0`, `1.5`	`float`	rational
`""`, `"abc"`	`str`	character string

In Python, the type of a variable is not fixed once and for all: if `i` is an integer variable, the assignment `i = 1.5` is accepted and `i` becomes a rational variable. This is rarely a good idea, because the reader of the program gets lost and the error only shows up later, for example in `fdn(i)`, which requires an integer. It is better to keep the same type for each variable and to make *conversions* explicitly: for example `r = float(i)` or `i = int(r)`.

22.2 Saving the state of the robot

A first use of names is to save the state of the $\mathcal{R}^{o}\!\!\underset{\sim}{b}{}^{o}t$. Suppose for example that we want to design a procedure to make a drawing that requires various changes of the grid size. So that calling this procedure in a program does not modify the grid size used in the program, this size must be saved at the

beginning of the procedure and restored at the end. It is enough to give it a name by a *local declaration* `name = expression` placed in the body of the procedure, as in the following example:



```
from LibRobot import *
# Draw ducks
# Draw a duck on a
# grid of size l without changing
# the original grid
def duck(l):
   # Remember the dimensions of the original grid
   lX = valGsX(); lY = valGsY()
   # Draw a duck with a grid of size l
   gs(l)
   fdn(19)  # Waterline
   gsX(3 * l); gsY(7 * l); etl()  # Tail
   fd(); ht(); gsX(4 * l); gsY(2 * l); fd()
   gsX(6 * l); gsY(3 * l); tetr(); cl()  # Back
   gsX(4 * l); gsY(l); cl(); ht()
   gs(l); fdn(2); gs(3 * l); cl(); cl()  # Head
   pu(); qtl(); fd(); dc(); ht(); fd()  # Eye
   qtl(); pd()
   gsX(2 * l); gsY(l); fd(); etr(); fd()  # Beak
   qtl(); fd()
   gsX(2 * l); gsY(6 * l); fd()  # Breast
   etl()
   # Place the robot at the rear of the duck
   gs(l); fdn(18)
   # Restore the original grid
   gsX(lX); gsY(lY)
gs(6); fe(); duck(3); fd(); duck(2); fd()
duck(1); fd(); duck(1); st()
```

The declarations `lX = valGsX(); lY = valGsY()` compute the values of `valGsX()` and `valGsY()` at the beginning of the procedure and give them the names `lX` and `lY`, which can be used only in the rest of the body of the procedure `duck`. They are said to be variables *local* to the procedure: they do not exist outside its body.

22.3 Counting

Suppose we want to count the number of iterations of a "while" loop. We use an integer variable:

```
i = 0
```

Before executing the "while" loop, the variable must be initialized to 0 (it may have been modified since its declaration):

```
i = 0
```

After each pass through the loop, the value of the variable must be increased by 1:

```
i = i + 1
```

Suppose that the value of the variable `i` is 5 *before* the assignment `i = i + 1`. The value of the expression `i + 1` is $5+1 = 6$, so that the value of the variable `i` *after* the assignment will be 6. Since this instruction is very frequent, Python allows it to be abbreviated to `i += 1` ("increase `i` by 1"). Likewise `i -= 1` decreases `i` by 1 and `i *= 2` doubles it.

For example, this can help us determine the size of the window in which the $\mathcal{R}^{o}\!\!\smile^{o}t$ moves on the computer screen:

```
from LibRobot import *

# Compute the size of the window where the robot
# moves
width = 0
height = 0

def robotWindowWidth():
   global width
   gs(1); pu(); gc(); gl(); fe(); width = 0
   while not tb():
      width += 1; fd()

def robotWindowHeight():
   global height
   gs(1); pu(); gc(); gb(); fn(); height = 0
   while not tb():
      height += 1; fd()

robotWindowWidth(); robotWindowHeight()
message("(" + str(width) + " x " + str(height) + ")")
markPause()
```

The variables `width` and `height` are declared in the program, outside the procedures; they are said to be *global*. A procedure can use their value without saying anything special. On the other hand, to *modify* them, it

must declare so at the beginning of the body of the procedure with **global** `width`. Without this declaration, Python would consider that the assignment `width = 0` creates a new local variable `width`, unrelated to the global variable, which would keep the value 0 (and `width += 1` without `width = 0` before it would cause the error '`UnboundLocalError: local variable 'width' referenced before assignment`', the local variable `width` being used before it has received a value).

The operator `+` applied to two character strings puts them end to end, and the function `str` gives the character string that represents a number (for example `str(25)` is `"25"`). A string and a number cannot be put end to end: `"(" + width` causes the error '`TypeError: can only concatenate str (not "int") to str`'.

22.4 Entering a value from the keyboard

A value can be obtained by typing it on the computer's keyboard. To do so, the function `readInt()` (read an integer) waits for an integer to be typed and returns its value. The question asked by the program (written with `write`, see below) appears in the bottom bar of the *Robot*'s window, and the answer typed on the keyboard is displayed right after it (it can also be typed in the terminal from which the program was started). For example we write:

```
write("n = ")
n = readInt()
```

to give the variable `n` the value typed. Typing mistakes can be corrected with the ⟵ backspace key. The entry of the value must be ended by pressing the ↩ return key. If what has been typed is not an integer, the program asks for it again. On a French (AZERTY) keyboard, the digits can be typed with or without the shift key.

Likewise `readReal()` (read a real) returns a rational number (of type `float`), `readString()` (read a string) returns the character string typed (without apostrophes or quotation marks) and `readChar()` (read a character) returns the character of the first key pressed in the *Robot*'s window (without waiting for the return key). In Pascal, a single procedure `readln` made it possible to read a value of any type. In Python, each reading function returns a value of a well-defined type (Python's `input` function always returns a character string, which would have to be converted into an integer with `int`; `readInt` does it for us). On the other hand some functions, such as `write`, `str`

or `abs`, accept parameters of different types: they are said to be *polymorphic* (whereas other functions, such as `readInt` or `sqrt`, are *monomorphic* in the sense that their parameters and their result always have the same type).

The instruction `readln()` waits until the ↩ return key is pressed. It is therefore useful to leave time to read what the program has written before its execution ends.

22.5 Displaying the value of an expression

To display texts in the terminal (and in the bottom bar of the $\mathcal{R}$obot's window), one can use the instruction `write(`t_1`, `t_2`, ...)`, which writes t_1, t_2, ... one after the other, on the same line. The instruction `writeln(`t_1`, `t_2`, ...)` does the same and then goes to the next line. The t_i can be character strings or integer or rational expressions, whose value is written: if `x` is 3, `writeln("x = ", x)` writes `x = 3`. One can also convert the value of an expression E into a character string oneself with `str(`E`)`, as the program below does. For example this program reads an integer n and then writes its factorial $n! = (1 * 2 * \ldots * (n-1) * n)$. To ask again for the value of n as long as it is incorrect, it uses a recursive function `readN` (see chapter 21) which returns the value read:

```
from LibRobot import *

# Read an integer n and write its factorial n!
nmin = 0
nmax = 20

# Read the value of n from the keyboard, repeating
# until nmin <= n <= nmax
def readN():
    write("n = ")
    n = readInt()
    if nmin <= n and n <= nmax:
        return n
    else:
        writeln("n must be between ", str(nmin), " and ",
                str(nmax), "! Try again.")
        return readN()

n = readN()
# Compute f = n!
f = 1
for i in range(1, n + 1):
```

```
    f = f * i
# Write f
writeln("n! = ", str(f))
readln()
```

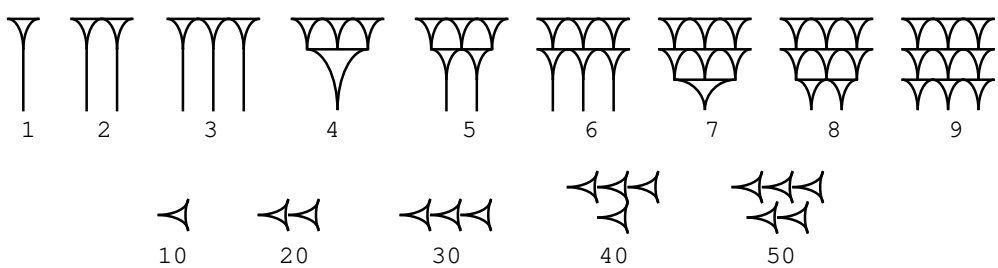
The variable `n` of the function `readN` is local to this function: it is a different variable from the variable `n` of the program, which receives the value returned by `readN()`. The maximal value of n is set to 20, as in the OCaml version of this program, where 21! exceeds the largest integer. In Python, integers have no limit: one could compute exactly 100!, which is written with 158 digits.

Exercise 43

– *Write a program that draws a frame as close as possible to the edge of the window in which the Robot moves.*

– *Write a program that sketches the drawings to be completed of figure 1.4 (page 7).*

– *Babylonian numeration [16], which dates from about 2000 BC, is in base 60. It is therefore a sexagesimal numeration, just like the one we still use for telling the time. The units, from 1 to 59, are written additively by accumulating "nails" (for 1) and "chevrons" (for 10), grouped and stacked as shown in the figure below.*



Beyond 60, the notation is positional, the value of a digit in the pth position (counted from right to left starting from zero) being multiplied by 60^p. The absence of sexagesimal units in a middle position (which we would write as 0) was first marked by a space and then, around the 3rd century BC, by "two slanting nails". The Babylonians thus took 15 centuries to invent a notation for zero, without however understanding its computational value!

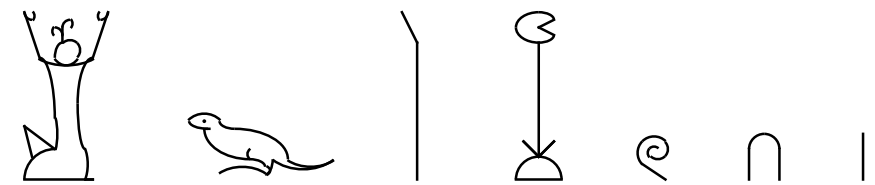
For example the number $3657 = 1 \times 60^2 + 0 \times 60^1 + 57 \times 60^0$ *is represented by juxtaposing the digits* 1, 0 *and* $57 = 50 + 7$*:*



"zéro" babylonien 3657

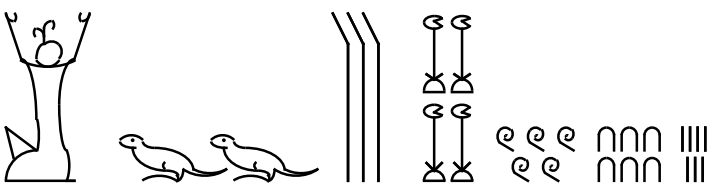
Write a Python program that converts any positive number into Babylonian numeration.

– *Egyptian numeration (*3rd *millennium BC) is decimal and additive [16]. It uses hieroglyphs to write the powers of 10 (the vertical stroke for 1, the handle for 10, the spiral for 100, the lotus for 1000, the finger for 10,000, the tadpole for 100,000 and the genie for 1,000,000):*



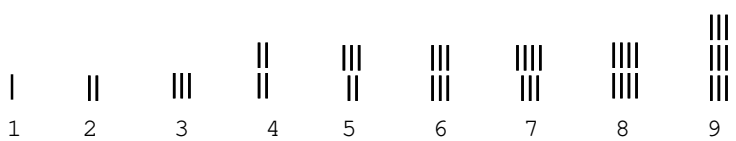
1000000 100000 10000 1000 100 10 1

The value of a sequence of signs is the sum of the values of these signs:



1 234 567

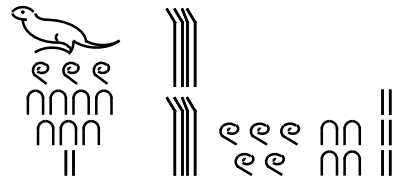
Writing goes from left to right or from right to left, the reading being done in the direction in which the hieroglyphs face. Identical signs are grouped on one, two or three lines as follows:



1 2 3 4 5 6 7 8 9

Between 2000 and 1600 BC, a hybrid representation (additive and partially positional) appeared. A number less than 1000 placed before, above or below the sign of the genie of the million, the tadpole, the finger or the lotus repre-

sents the product of this number and the value of this hieroglyph. For example, the number 37,280,549 can be decomposed into $((372 * 100{,}000) + 80{,}549)$ *and is written:*



37 280 549

Write a Python program that converts a number into Egyptian numeration.

_ *Greek acrophonic numeration [16] was used from the 5th century BC in Athens. It is an additive decimal numeration using the auxiliary base 5 to reduce repetitions. The digits greater than one are written with letters (or combinations of letters), each corresponding to the initial (or the abbreviation) of the name of the corresponding number:*



1 5 10 50 100 500 1000 5000 10000 50000

Πεντε ΠεντεΔεκα ΠεντεΗεκατον ΠεντεΧιλιοι ΠεντεΜυριοι

Δεκα Ηεκατον Χιλιοι Μυριοι

Write a Python program that converts a number less than 100,000 into Greek numeration. To represent large numbers, use the acrophonic numeration for currencies:
Obols:

1

Drachmas (1 drachma = 6 obols):



1 5 10 50 100 500 1000 5000

Talents (1 talent = 6,000 drachmas):



1 5 10 50 100 500 1000 5000

For example the number 239,686,966 is represented by:



6657 Talents 5827 Drachmes 4 Oboles

239 686 966

Roman numeration [16] is additive and subtractive (any sign placed to the left of a sign of greater value is subtracted from it). Its digits are well known:



1 5 10 50 100 500 1000

For large numbers, the horizontal bar multiplies by 1,000 the value of the number it overlines, and the rectangle without a base multiplies by 100,000 the value of the number it encloses, as in the example 135,792,468 below:



1 3 5 7 9 2 4 6 8

Write a Python program that converts a number into Roman numeration.

Write a program that draws the Sierpinski curve of any order of figure 22.1 (page 285).

A magic square of order n is a square array of n^2 numbers such that the sum of the numbers of each row, of each column and of each diagonal is the same. Examples are given in figure 22.2 (page 285). Let $C_{i,j}$ be the number in the ith row and the jth column of the square, counted from 1 and numbered from bottom to top for the rows and from left to right for the columns. Write a Python program that builds magic squares of odd order n by first placing 1 in $C_{i,j} = C_{\frac{n+1}{2},n}$ and then assigning the numbers 2, 3, ..., n^2 to the elements $C_{i,j}$ obtained by increasing i and j by 1 modulo n each time for $n-1$ steps, and by decreasing j by 1 while leaving i unchanged at every n^{th} step.

□

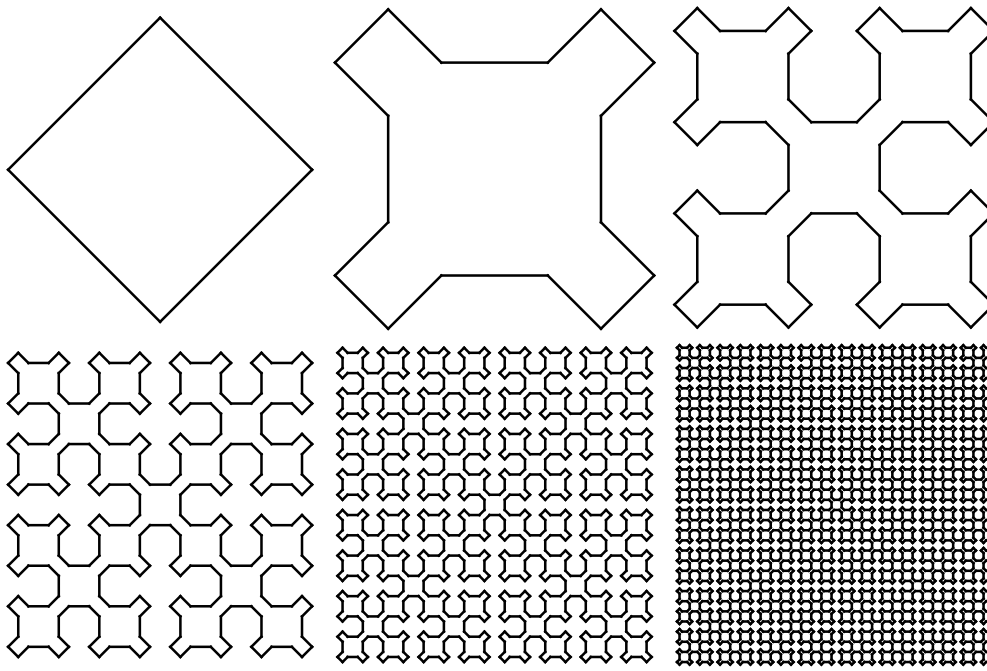


FIG. 22.1 – *Sierpinski curves of orders 0 to 5*

8	1	6
3	5	7
4	9	2

17	24	1	8	15
23	5	7	14	16
4	6	13	20	22
10	12	19	21	3
11	18	25	2	9

30	39	48	1	10	19	28
38	47	7	9	18	27	29
46	6	8	17	26	35	37
5	14	16	25	34	36	45
13	15	24	33	42	44	4
21	23	32	41	43	3	12
22	31	40	49	2	11	20

FIG. 22.2 – *Magic squares of orders 3, 5 and 7*

Solution 43 *We give one program (without the body of the technical procedure* `writeCentred`*). The others are in the folder* `book/22_variables`*:*

```
from LibRobot import *

# Build a magic square of odd order

# Read from the keyboard the (odd) order n of the square
def readOrder():
   write("Order of the magic square (odd), n = ")
   n = readInt()
   if n > 0 and n % 2 == 1:
      return n
   else:
      return readOrder()

# Put the robot at the bottom left corner of the square:
# one step above and to the right of the window's corner
def origin():
   gb(); gl(); translate(1, 1)

# Write the text s centred on the robot's position
# (technical procedure, see the complete program in
# the folder book/22_variables)
def writeCentred(s):
   ...

# Display the element c at the centre of cell (i, j)
def display(i, j, c):
   origin(); translate(2 * i - 1, 2 * j - 1)
   writeCentred(str(c))

n = readOrder()
# Draw the matrix in the robot's window
gs(20); hideRobot()
for k in range(n + 1):
   origin(); translate(2 * k, 0); fn(); fdn(2 * n)
   origin(); translate(0, 2 * k); fe(); fdn(2 * n)
# Fill in the magic square
# Row i and column j of the cell to fill
i = (n + 1) // 2
j = n
display(i, j, 1)
for c in range(2, n * n + 1):
   if c % n != 1:
      i = i % n + 1
      j = j % n + 1
   else:
```

```
        j -= 1
      display(i, j, c)
  st()
```

□

Addenda

A variable is used to give a *name* to a memory that keeps a *value* represented according to a code determined by the *type* of the variable. For a binary encoding, one can imagine that the memory is a row of lamps that are off (to represent the bit 0) or on (for the bit 1). In many languages, such as PASCAL, the initial value of a variable is *undetermined*, since it depends on the state of the memory at the moment the execution of the program begins, which is a source of errors. In Python, a variable receives an initial value as soon as it is created. The assignment instruction makes it possible to keep a new value by physically modifying the state of this memory. The new value is given by an expression (which can use the old value). The computing notion of variable is therefore very different from the mathematical notion of the same name. In mathematics a variable denotes an unknown value fixed once and for all, like the constants of a Python program whose value is never changed. For a mathematician, a computer variable would be the sequence of values it takes when it is created and then after each assignment.

Variables *local* to a procedure (or function) can be declared by writing `x = ...` in the body of the procedure, like the variable `n` of the function `readOrder` of the program `MagicSquare`. In this case the variable can be used only in the rest of the body of the procedure. At run time, it is created at each execution of the body of the procedure. For a recursive procedure, it is at each call a new variable without any link to the instances corresponding to earlier recursive calls.

In the body of the procedure one can also use *global* variables whose declaration appears in the program, outside the procedures, such as `width` and `height` in the program `ComputeRobotWindowSize` or `i` and `j` in the program `MagicSquare`. For a procedure to be able to modify them by assignment, they must be named after the keyword **global** at the beginning of its body. These global variables exist throughout the execution of the program. Consequently, in the case of recursive calls of a procedure, it is always the same global variable.

Finally, local and global variables can have the same name (like the variable `n` of the function `readOrder` and the variable `n` of the program `MagicSquare`). In this case it is the local variable that is visible in the procedure, the global variable continuing to exist but being inaccessible. In Python, a variable is local to a procedure as soon as it receives a value by assignment in the body of this procedure (unless it is declared **global**).

Bibliography

[1] J. W. Backus, R.J. Beeber, S. Best, R. Goldberg, L. M. Haibt, H. L. Herick, R. A. Nelson, D. Sayre, P. B. Sheridan, H. Stern, I. Ziller, R. A. Hugues & R. Nutt. *The Fortran automatic coding system*, Proceedings Western Joint Computer Conference, Volume 11, pages 188–198, 1957.

[2] M. Barnsley, R. Devaney, B. Mandelbrot, H.-O. Peitgen, D. Saupe & R. Voss. *The science of fractal images*, Springer-Verlag, Berlin, 312 pages, 1988.

[3] C. Bertho. *Télégraphes et téléphones, de Valmy au microprocesseur*, Le Livre de Poche, Paris, 541 pages, 1981.

[4] P. Benoit. "Calcul, algèbre et marchandise", in [40], pages 197–221, 1989.

[5] C. Böhm. "Macchina calcolatrice digitale a programma con programma preordinato fisso con tastiera algebrica ridotta atta a comporre formule mediante la combinazione dei singoli elementi simbolici", Patent application No. 13567, Milan, October 1st, 1952, 26 pages.

[6] M. Boutet de Monvel. *Chansons de France pour les petits enfants*, Gautier-Languereau, Paris, 77 pages, 1981.

[7] Commission ministérielle de terminologie de l'informatique. *Glossaire des termes officiels de l'informatique*, AFNOR, Tour Europe, 92080 Paris La Défense Cedex 7, 21 pages, 1989 (2nd edition).

[8] S. Conchon & J.-C. Filliâtre. *Apprendre à programmer avec OCaml. Algorithmes et structures de données*, Eyrolles, Paris, 2014.

[9] P. Cousot & R. Cousot. "Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints", Conference Record of the 4th ACM Symposium on Principles of Programming Languages, Los Angeles, California, U.S.A., p. 238–252, 1977.

[10] P. Cousot. *Introduction à l'algorithmique numérique et à la programmation en Pascal*, McGraw-Hill, Paris, 621 pages, 1988.

[11] H. S. M. Coxeter, M. Emmer, R. Penrose & M. L. Teuber. *M. C. Escher: Art and Science*, North-Holland, Amsterdam, 402 pages, 1986.

[12] H. M. Cundy & A. R. Rollett. *Mathematical models*, Tarquin publications (Stradbroke, Diss, Norfolk, G.B.), 286 pages, 1981 (3rd edition).

[13] A. Danhauser. *Théorie de la musique (édition revue et corrigée par H. Rabaud)*, Éditions Henry Lemoine, Paris, 128 pages, 1929.

[14] S. W. Golomb. "A geometric proof of a famous identity", *Mathematical gazette*, Vol. 69, p. 198–200, 1965.

[15] B. Grünbaum & G. C. Shephard. *Tilings and patterns*, W. H. Freeman & Co., New York, 700 pages, 1987.

[16] G. Guitel. *Histoire comparée des numérations écrites*, Flammarion, Paris, 857 pages, 1975.

[17] S. Horemis. *Optical and geometrical patterns and designs*, Dover, New York, 1970.

[18] K. E. Iverson. *A Programming Language*, J. Wiley & Sons, 1962.

[19] K. Jensen & N. Wirth. *Pascal, user manual and report*, Springer-Verlag (Heidelberg), 167 pages, 1976 (2nd edition).

[20] B. W. Kernighan & D. M. Ritchie. *The C programming language*, Prentice-Hall (Englewood Cliffs, New Jersey, U.S.A.), 228 pages, 1978.

[21] D. E. Knuth. *The art of computer programming, Volume 1 / Fundamental algorithms*, Addison-Wesley (Reading, Massachusetts, USA), 634 pages, 1973 (2nd edition).

[22] D. E. Knuth. *The TEXbook*, Addison-Wesley (Reading, Massachusetts, USA), 483 pages, 1987 (12th edition).

[23] L. Lamport. *LATEX a document preparation system*, Addison-Wesley (Reading, Massachusetts, USA), 242 pages, 1986.

[24] X. Leroy, D. Doligez, A. Frisch, J. Garrigue, D. Rémy, K. Sivaramakrishnan & J. Vouillon. *The OCaml system: documentation and user's manual*, Inria, https://ocaml.org/manual/.

[25] P. Lévy. "L'invention de l'ordinateur", in [40], pages 515–535, 1989.

[26] J. L. Locher. *The world of M. C. Escher*, Abrams (New York, USA), 1971; French translation: J. L. Locher, *Le monde de M. C. Escher*, Éditions du Chêne (Paris), 293 pages, 1986 (12th edition).

[27] B. Mandelbrot. *Les objets fractals* followed by *Survol du langage fractal*, Flammarion, Nouvelle collection scientifique (Paris), 1989 (3rd edition).

[28] J. McCarthy, P. W. Abrahams, D. J. Edwards, T. P. Hart & M. I. Levin. *LISP 1.5 Programmer's manual*, MIT Press, 1965.

[29] P. Naur *et al.* (Eds.). "Revised report on the algorithmic language Algol 60", *Communications of the Association for Computing Machinery*, volume 6, pages 1–17, 1963.

[30] F. C. Odds. "Spirolaterals", *Mathematics teacher*, volume 66, pages 121–124, 1973.

[31] S. Papert. *Mindstorms: children, computers and powerful ideas*, Harvester Press, 1980.

[32] Python Software Foundation. *The Python Language Reference*, official documentation of Python 3, https://docs.python.org/3/reference/.

[33] Python Software Foundation. *Le tutoriel Python*, official documentation of Python 3 (French translation), https://docs.python.org/fr/3/tutorial/.

[34] J. Ritter. "Babylone -1800", in [40], pages 17–37, 1989.

[35] H. Rutishauser. "Automatische Rechenplanfertigung bei programmgesteuerten Rechenmaschinen", *Mitteilungen aus dem Inst. für angew. Math. an der ETH Zürich*, No. 3, Birkhäuser, Basel, 1952, 45 pages.

[36] R. Sabatier. *Le livre des chansons de France*, Gallimard, Paris, 157 pages, 1984.

[37] R. Sabatier. *Deuxième livre des chansons de France et d'ailleurs*, Gallimard, Paris, 165 pages, 1986.

[38] C. Sabatier & R. Sabatier. *Troisième livre des chansons de France*, Gallimard, Paris, 165 pages, 1987.

[39] R. Sedgewick. *Algorithms*, Addison-Wesley, Reading, Massachusetts, U.S.A., 650 pages, 1988 (2nd edition), translated into French by J.-M. Moreau: R. Sedgewick, *Algorithmes*, InterÉditions, Paris, 1990.

[40] M. Serres. *Éléments d'histoire des sciences*, Bordas, Paris, 576 pages, 1989.

[41] S. Thévenet, A. Garioud & N. Pitot. *Mathématiques CM2*, Bordas, Paris, 160 pages, 1986.

[42] H. Voderberg. "Zur Zerlegung der Umgebung eines ebenen Bereiches in kongruente", *Jahresber. Deutsch. Math.-Verein.*, volume 46, pages 229–

231, 1936; "Zur Zerlegung der Ebene in kongruente Bereiche in Form einer Spirale", Ibid., volume 47, pages 159–160, 1937

[43] P. Weis & X. Leroy. *Le langage Caml*, InterÉditions, Paris, 1993; 2nd edition, Dunod, 1999.

[44] M. V. Wilkes, D. J. Wheeler & S. Gill. "The preparation of programs for an electronic digital computer: with special reference to the EDSAC and the use of a library of subroutines", Addison-Wesley, Cambridge, Massachusetts, 170 pages, 1951.

[45] L. B. Wilson & R. G. Clark. *Comparative programming languages*, Addison-Wesley publishing company, Reading, Massachusetts, 379 pages, 1988.

[46] N. Wirth. "The programming language Pascal", *Acta Informatica*, Vol. 1, Springer-Verlag (Heidelberg), pages 35–63, 1971.

Summary of the robot commands

The commands of the $\mathcal{R}o\!\smile\!o t$ are those of the library `LibRobot` (a program starts with **from** `LibRobot` **import** `*`). A command is written followed by parentheses, empty when it has no parameter, like `fd()`; the parameters are written between the parentheses, separated by commas, like `pw(2)`, `fdl(-3.5)` or `moveTo(2, 3)`. A real number (of type `float`) is written with a point, like `20.5`, an integer (of type `int`) without a point; an integer can be given where a real is expected, like `gs(20)`. A character string is of type `str`, a boolean of type `bool` (**True** or **False**). The functions `val`... return the current value of a setting of the $\mathcal{R}o\!\smile\!o t$.

Grid of the stopping points of the $\mathcal{R}^{o}\underset{\smile}{b}^{o}t$:
`dg()` (draw grid) draws the grid of the stopping points of the $\mathcal{R}^{o}\underset{\smile}{b}^{o}t$ (the drawing of the grid can be interrupted by Esc, § or #).
`gs(`n`)` (grid size) sets the new length n (a real $n \geq 0$) of the sides of a square of the grid (12×12 pixels by default).
`gsX(`n`)` sets the new length n (a real) of the horizontal side (X) of a square of the grid (grid size, 12 pixels by default).
`gsY(`n`)` sets the new length n (a real) of the vertical side (Y) of a square of the grid (grid size, 12 pixels by default).
`gsRT(`r`, `t`)` sets the new lengths of the sides of a square of the grid (grid size) by the polar coordinates r and t (two reals: the length $r \geq 0$ of the diagonal of the square is expressed in pixels and the angle t between this diagonal and the horizontal side is expressed in degrees, usually $0 \leq t \leq 90$).
`valGsX()` is equal to the value (of type `float`) of the length (grid size) of the horizontal side (X) of a square of the grid (counted in pixels).
`valGsY()` is equal to the value (of type `float`) of the length (grid size) of the vertical side (Y) of a square of the grid (counted in pixels).
`valGsR()` is equal to the value (of type `float`) of the length (grid size) of the diagonal (R) of a square of the grid (counted in pixels).
`valGsT()` is equal to the value (of type `float`) of the angle (T) of the diagonal of a square of the grid with its horizontal side (counted in degrees).
`valX()` is equal to the value (of type `float`) of the absolute abscissa (X) of the $\mathcal{R}^{o}\underset{\smile}{b}^{o}t$ (counted in pixels from the left edge of the window).
`valY()` is equal to the value (of type `float`) of the absolute ordinate (Y) of the $\mathcal{R}^{o}\underset{\smile}{b}^{o}t$ (counted in pixels from the bottom of the window), so that after `x = round(valX())` and `y = round(valY())`, later on `pu(); drawTo(x, y)` puts the $\mathcal{R}^{o}\underset{\smile}{b}^{o}t$ back at this place on the screen.

Relative turns on the spot:	
`etr()`	(eighth turn right) turns the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ on the spot an eighth of a turn to the right.
`qtr()`	(quarter turn right) turns the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ on the spot a quarter turn to the right.
`tetr()`	(three eighths of a turn right) turns the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ on the spot 3 eighths of a turn to the right.
`etl()`	(eighth turn left) turns the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ on the spot an eighth of a turn to the left.
`qtl()`	(quarter turn left) turns the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ on the spot a quarter turn to the left.
`tetl()`	(three eighths of a turn left) turns the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ on the spot 3 eighths of a turn to the left.
`ht()`	(half turn) turns the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ on the spot a half turn.
Absolute turns on the spot:	
`fn()`	(face north) turns the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ on the spot towards the north (N).
`fne()`	makes the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ face the north-east (NE), turning on the spot.
`fe()`	makes the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ face the east (E), turning on the spot.
`fse()`	makes the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ face the south-east (SE), turning on the spot.
`fs()`	makes the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ face the south (S), turning on the spot.
`fsw()`	makes the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ face the south-west (SW), turning on the spot.
`fw()`	makes the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ face the west (W), turning on the spot.
`fnw()`	makes the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ face the north-west (NW), turning on the spot.
`face(`o`)`	turns the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ on the spot to face the orientation given by the value of the integer o (between 0 and 7) equal to `north` = 0, `northEast` = 1, `east` = 2, `southEast` = 3, `south` = 4, `southWest` = 5, `west` = 6 or `northWest` = 7.
`rot(`t`)`	changes the coordinate system and the orientation of the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ so that it performs a rotation on the spot by an angle of t degrees (t real, counterclockwise if $t > 0$ and clockwise if $t < 0$, like `rot(15)` or `rot(-30)`).
`valOrientation()`	is equal to the value (of type `int`) of the orientation of the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ coded by `north` = 0, `northEast` = 1, `east` = 2, `southEast` = 3, `south` = 4, `southWest` = 5, `west` = 6 and `northWest` = 7. The directions `north`, `east`, `south` and `west` are parallel to the edges of the screen, `north` at the top. The directions `northEast`, `southEast`, `southWest` and `northWest` correspond to the orientation of the diagonals of a square of the grid.

Pen:	
`pd()`	(pen down) puts the pen down to draw while moving.
`pu()`	(pen up) lifts the pen to avoid drawing while moving.
`dc()`	(draw a cross) draws a small cross (as thick as the pen), without moving the $\mathcal{R}^{o}_{o}o^{o}t$.
`dp()`	(draw a point) draws a dot (as thick as the pen), without moving the $\mathcal{R}^{o}_{o}o^{o}t$.
`pw(`n`)`	(pen width) sets the new thickness n (an integer) of the pen (1 pixel by default).
`valPw()`	is equal to the value (of type `int`) of the thickness of the pen (pen width).
`valPd()`	is equal to the boolean value **True** if and only if the pen is down (pen down), otherwise to **False**.
Moves (drawing or not depending on whether the pen is up or down):	
`fd()`	(forward) moves the $\mathcal{R}^{o}_{o}o^{o}t$ straight ahead to the next stopping point on the grid.
`cr()`	(curve right) makes the $\mathcal{R}^{o}_{o}o^{o}t$ turn by a bend of a quarter circle to the right.
`cl()`	(curve left) makes the $\mathcal{R}^{o}_{o}o^{o}t$ turn by a bend of a quarter circle to the left.
`fdn(`n`)`	(forward n times) moves the $\mathcal{R}^{o}_{o}o^{o}t$ straight ahead n times (n an integer; doing nothing if $n = 0$ and moving backwards if $n < 0$, like `fdn(-2)`).
`fdl(`l`)`	(forward by a length) changes the coordinate system (but not the angle of the directions NE-E) and moves the $\mathcal{R}^{o}_{o}o^{o}t$ forward by a translation of l pixels (l real) in the direction it is facing (if $l > 0$) or in the opposite direction (if $l < 0$).
`drawToReal(`x`, `y`)`	(draw to the point with real coordinates) draws a straight line segment (if the pen is down) from the current position of the $\mathcal{R}^{o}_{o}o^{o}t$ to the point with real coordinates (x, y) of the Cartesian coordinate system of the grid (see `moveTo`) and moves the $\mathcal{R}^{o}_{o}o^{o}t$ to this point.
`drawTo(`x`, `y`)`	does the same up to the point with integer coordinates (x, y) counted in pixels from the lower left corner of the window (like `valX()` and `valY()`).
`valFdl()`	is equal to the value (of type `float`) of the length of the segment drawn by the last move forward by `fd()` or by `fdl()` (forward by a length), counted in pixels.

Positioning at the centre and on the edges of the frame:	
`gc()`	(go to the centre) places the *Robot* at the centre of *the screen*, which is also *the centre of the grid*, without changing the orientation of the *Robot* and without drawing anything.
`gt()`	(go to the top) places the *Robot* at the top of the frame, without changing its horizontal position or its orientation and without drawing anything.
`gb()`	(go to the bottom) places the *Robot* at the bottom of the frame, without changing its horizontal position or its orientation and without drawing anything.
`gl()`	(go to the left) places the *Robot* on the left of the frame, without changing its vertical position or its orientation and without drawing anything.
`gr()`	(go to the right) places the *Robot* on the right of the frame, without changing its vertical position or its orientation and without drawing anything.
Cartesian coordinate system:	
`translate(`x`, `y`)`	translates the *Robot* by x squares of the grid horizontally (to the right if $x > 0$ and to the left if $x < 0$) and y squares of the grid vertically (up if $y > 0$ and down if $y < 0$), without changing its orientation and without drawing anything (x and y integers).
`moveTo(`x`, `y`)`	(move to) moves the *Robot* to the point with integer coordinates (x, y) on the grid, without changing its orientation or drawing anything, the origin of the Cartesian coordinate system being at the centre of the grid.
`moveToReal(`x`, `y`)`	does the same with real coordinates x and y (of type `float`), like `moveToReal(1.5, -2.5)`.
Boolean test of the orientation of the *Robot*:	
`tn()`	(test north) tests whether the *Robot* is facing north (N).
`tne()`	tests whether the *Robot* is facing north-east (NE).
`te()`	tests whether the *Robot* is facing east (E).
`tse()`	tests whether the *Robot* is facing south-east (SE).
`ts()`	tests whether the *Robot* is facing south (S).
`tsw()`	tests whether the *Robot* is facing south-west (SW).
`tw()`	tests whether the *Robot* is facing west (W).
`tnw()`	tests whether the *Robot* is facing north-west (NW).
Boolean test of the position of the *Robot*:	
`tb()`	(test border) tests whether the *Robot* is at the edge (border) of the frame (a move of the *Robot* forward by `fd()` would take it out of the frame).

Time:
`clock()` returns three integers: the present hour (between 0 and 23), minute (between 0 and 59) and second (between 0 and 59), so that after `h, mi, s = clock()` the variables `h`, `mi` and `s` contain these three values. `date()` likewise returns three integers, the day (between 1 and 31), the month (between 1 and 12) and the current year, as in `d, mo, y = date()`.
`delay(`t`)` waits for a delay of t seconds (t of type `float`, like `delay(0.5)`). `tickCount()` is equal to the integer number of sixtieths of a second elapsed (to measure a duration as the difference of two values).
Interrupting the execution of the program:
Esc, § or # typed while the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ is moving (by `fd()`, `cr()`, `cl()`, ...) or running step by step, any of these keys stops the program and closes the window.

Debugging the program:
`st()` stops the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ and recalls in the bottom line of the window the keys that can be used: + increases and − decreases the speed of the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ (between 0 and 10, speed 0 switching to step-by-step mode), Esc or # closes the window and ends the program, any other key redraws the whole drawing from the beginning, at the chosen speed. A program that ends without `st()` waits for a key to be typed or a click before closing the window.
`ss()` (step by step) runs the rest of the program in step-by-step mode, at speed 0: the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ waits for a key to be typed before each command (Esc, § or # stopping the program).
`ex()` runs the rest of the program in normal execution mode, at speed 10.
`sp(`v`)` runs the rest of the program at speed v (an integer; step by step if $v \leq 0$, with a maximum of 10 if $v > 10$).
`showRobot()` makes the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ visible on the screen during the drawing (default option).
`hideRobot()` makes the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ invisible on the screen during the drawing.
`forbidExits()` forbids the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ to go out of the frame in the rest of the program (default option: a command that would take the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ out of the frame then stops the program with an error, writing in the terminal, for example, `Error line 4 of Program.py: the robot would leave the frame (fd)`; the window stays displayed).
`allowExits()` allows the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ to go out of the frame in the rest of the execution of the program.
`valSp()` is equal to the value (of type `int`) of the speed of the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ (between 0 (step-by-step mode) and 10).
`valVisible()` is equal to the boolean value **True** (respectively **False**) if the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ is visible (respectively invisible) on the screen.
`valAllowExits()` is equal to the boolean value **True** (respectively **False**) if exits out of the frame are allowed (allow exits) (respectively forbidden).

Colour:

Colours are integers: `white` = 0, `yellow` = 1, `blue` = 2, `violet` = 3, `green` = 4, `red` = 5, `indigo` = 6 and `black` = 7.

`pc(`c`)` (pen colour) sets the colour c of the pen, like `pc(red)`.

`paint()` paints with the colour of the pen the region of a single colour (usually white), bounded by the lines of the drawing, where the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ is.

`bg(`c`)` (background colour) completely erases the drawing then fills the screen with the background colour c. The $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ is placed at the centre of the screen in the initial conditions (visible, facing north, exits out of the frame forbidden, black pen of thickness 1 down and normal execution at speed 10); the dimensions of the grid do not change.

`valBg()` is equal to the value (of type `int`) of the background colour (background).

`valPc()` is equal to the value (of type `int`) of the pen colour (pen colour).

`blackAndWhiteScreen()` is equal to **True** if and only if the screen is black and white, otherwise to **False** (it is always **False** on today's screens).

Writing in the drawing window of the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$:

`label(`c`)` writes the character string c (between quotes `""` and `""`, where any quote must be preceded by a backslash: `\"`, like `label("Hello")`) to the right of the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ (or to its left if it is facing west), without changing its position or that of its pen.

`intToString(`i`)` (integer to string) is the character string representing the integer i in base 10 (like `str(`i`)`).

`realToString(`r`)` (real to string) is the character string representing the real r in the form $d.dddd$`e`$\pm ee$ where d is a digit of the mantissa, e is a digit of the exponent and `e` reads "times 10 to the power".

`boolToString(`b`)` (boolean to string) is the character string (`"True"` or `"False"`) representing the boolean b.

`concat([`c_1`, `c_2`, ..., `c_n`])` is the character string obtained by concatenating the strings c_1, c_2, ..., c_n one after the other (like c_1 `+` c_2 `+ ... +` c_n).

`textFont(`f`)` chooses the font f of the text (`courier`, `times`, `helvetica`, ...).

`textSize(`s`)` chooses the size s (an integer) of the text.

`textStyle(`s`)` chooses the style of the text (`normal`, `bold`, `italic`, `underline`, ...).

These three commands have no visible effect: the font of the window of the $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ cannot be changed.

`valTextFont()` value (of type `str`) of the current font of the text.

`valTextSize()` value (of type `int`) of the current size of the text.

Messages displayed in the bottom line of the window of the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$:
`message(`c`)` erases the message in the bottom line of the window of the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ then writes the character string c there. `markPause()` (mark a pause) pauses (making the symbol $\hookleftarrow$ blink in the message line) until the mouse button or a key of the keyboard is pressed. `clearMessage()` (clear the message) erases the message in the bottom line of the window of the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$.
`readChar()` (read a character) waits (making the symbol $\hookleftarrow$ blink in the message line) until a key of the keyboard or the mouse button is pressed, then reads and returns the character (a string of a single character, of type `str`) corresponding to the key pressed (or the character `'0'` if the mouse button was pressed).
`keyboardCommandsMemento` is a character string recalling the keyboard commands for interactive control of the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ (see `interaction`), to be displayed by `message(keyboardCommandsMemento)`. `valMessage()` is equal to the value (of type `str`) of the message written in the bottom line of the window of the $\mathcal{R}\text{o}\underset{\frown}{b}\text{o}t$ or to the empty string `""` if there is no message.

Reading and writing text:

The text written by `write` and `writeln` is displayed in the terminal. When the program waits for an answer (`readInt()`, ...), the question (the current line, preceded by the last two lines written) is displayed in the bottom line of the window of the *Robot*. The answer is typed in the window of the *Robot*, where it is displayed after the question (← to correct, ↩ to finish), or else in the terminal.

`write(`c_1`, ..., `c_n`)` writes the character strings (or the numbers) c_1, ..., c_n one after the other, like `write("N = ", n)`.

`writeln(`c_1`, ..., `c_n`)` does the same then goes to a new line (`writeln()` only goes to a new line).

`readInt()` (read an integer) reads the user's answer, ended by ↩, and returns the integer (of type `int`) that it represents. If the answer is not an integer, the question `"Enter an integer: "` is asked again. On a French keyboard, the digits can be typed without the Shift key.

`readReal()` does the same to read a real number (of type `float`).

`readString()` (read a string) reads the user's answer, ended by ↩, and returns it as a character string (of type `str`).

`readCharacter()` does the same and returns the first character of the answer (a string of a single character).

`readln()` waits until the user types ↩ (what was typed before is ignored).

`clearText()` (clear the text) erases the text of the terminal.

Useful functions:

`pi` is the real $\pi = 3.14159\ldots$ (of type `float`).

`round(`x`)` is the integer closest to the real x, like `round(2.5)` = 3 and `round(-2.5)` = −3 (the library replaces the `round` of Python, which rounds halves to the even number: 2 for 2.5).

`random()` is an integer drawn at random between −32767 and 32767.

Sounds:

`beep()` emits a short sound.

`longBeep()` emits a long sound.

Music:

`metronome(`n`)` sets the metronome to n quarter notes per minute (`largo` = 50, `lento` = 56, `larghetto` = 60, `adagio` = 70, `andante` = 85, `andantino` = 95, `moderato` = 110, `allegretto` = 115, `allegro` = 130, `vivace` = 150, `presto` = 175, `prestissimo` = 200; 100 by default).

`dynamics(`n`)` sets the dynamics n (between 1 and 9: `pianississimo` = 1, `pianissimo` = 2, `piano` = 3, `mezza_voce` = 4, `mezzo_piano` = 4, `mezzo_forte` = 6, `forte` = 7, `fortissimo` = 8, `fortississimo` = 9).

`note(`p`,` d`)` plays a note of pitch $p = o * \texttt{octave} + n + a$ (an integer) where $o = -3, -2, \ldots, 3$ is the octave of the note (0 for the octave of middle C of the piano, `octave` = 12), n = `C` = 0, `D` = 2, `E` = 4, `F` = 5, `G` = 7, `A` = 9, `B` = 11 is the name of the note, a = `flat` = -1, `natural` = 0, `sharp` = 1 is the accidental of the note and d (a real) is the duration of the note, counted in quarter notes: `whole` = 4., `dotted_whole` = 6., `half` = 2., `dotted_half` = 3., `half_triplet` = 4/3, `quarter` = 1., `dotted_quarter` = 1.5, `quarter_triplet` = 2/3, `eighth` = 0.5, `dotted_eighth` = 0.75, `eighth_triplet` = 1/3, `sixteenth` = 0.25, `dotted_sixteenth` = 0.375 or `thirty_second` = 0.125, like `note(octave + G + sharp, eighth)`.

`rest(`d`)` makes a rest of duration d = `whole_rest` = 4., `dotted_half_rest` = 3., `half_rest` = 2., `dotted_quarter_rest` = 1.5, `quarter_rest` = 1., `dotted_eighth_rest` = 0.75, `eighth_rest` = 0.5, `sixteenth_rest` = 0.25 or `thirty_second_rest` = 0.125.

Interaction with the user of the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$:

`interaction(coordinates, turns, drawing, mouseEnds, save)`

erases the screen then lets the user move the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ with the mouse, by clicking on a point of the grid (if `mouseEnds` is **True**: from square to square, button released) or with the arrow keys, and possibly drive it with the keyboard (the five parameters are booleans, **True** or **False**). The interaction ends with the key ↩ (or by pressing the mouse button if `mouseEnds` is **True**); the program then stops as with `st()`.

– If `coordinates` is **True**, the coordinates of the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ are displayed during the interaction.

– If `turns` is **True**, the user can turn the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ with the keys – (`qtl`), + (`qtr`), l (`etl`) and r (`etr`).

– If `turns` and `drawing` are **True**, the user can also draw with the keys f (`fd`), ((`cl`),) (`cr`), u (`pu`), d (`pd`), p (`dp`), c (`dc`), . (`dg`), * (change of the pen colour, `pc`) and w (`paint`), undoing the last command with ←.

– If `drawing` and `save` are **True**, the program asks at the end whether to save the drawing: by answering y, one obtains a program `my_drawing.py` whose execution reproduces the drawing.

```
x, y, c = interactionTP(coordinates, turns, drawing,
                        mouseEnds, save)
```

lets the user move the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ with the mouse (or the arrow keys) and drive it with the keyboard like `interaction` (except *, w and ←), but without erasing the screen, and then continues the program. The interaction ends with the key ↩, with Esc, with any key that is not a command of the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ or, if `mouseEnds` is **True**, by pressing the mouse button. At the end, `interactionTP` returns three values, put here in the variables `x`, `y` and `c`: the integers `x` and `y` are the horizontal and vertical displacement of the $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ counted in squares of the grid, and `c` is the last key typed (a space `' '` if the interaction ended with a click). The parameter `save` has no effect.

Index

Abscissa, 157
Abstraction, 64
Actual parameter, 150
Application program, 49
ASCII code, 205
Assignment, 276, 287

Binary number system, 203, 221
Block structure, 155
Boolean, 167
Boolean expression, 167, 182, 192
Boolean function, 182
break, 199
Bug, 27

Cartesian coordinate system, 157
Cartesian coordinates, 157, 158
Character, 205
Character string, 206
Command interpreter, 49
Command language, 25
Comment, 38
Compilation, 51
Compiler, 51
Computer, 27
Computer graphics, 11
Concatenation, 207
Conjunction, 180
Constant declaration, 132
Conversational mode, 27
Conversion, 276
Copy, 53

Debug, 27
Debugging, 27
Decimal number system, 202
Declaration, 133
Delete key, 20
Disjunction, 180
Documentation string, 38
Drawing rule, 5
Dynamic typing, 224

Error, 20
Escape key, 41
Execution, 29, 41
Execution (interruption), 42

False, 167
float, 224
Floating-point number, 115
"for" loop, 72, 73
Formal parameter, 150
Function, 59, 272

global, 279
Global variable, 278
Grid of the $\mathcal{R}^{o}_{o}\mathcal{b}^{o}t$, 14

Identifier, 57, 60, 150, 155, 258
Indentation, 59, 74
Induction, 249
Integer, 222

Integer expression, 93, 104, 230
Integer variable, 72
Interactive mode, 25, 27
Interpret, 28
Interpretation, 39
Interpreter, 25, 28, 33, 39
Interrupting execution, 42
Invariant, 65, 80, 253
Iteration, 72

Library, 34
Local declaration, 277
Logical error, 42
Loop body, 72, 73, 192
Loop counter, 72, 75

Machine language, 28, 39
Monomorphic, 280

Negation, 180

Operating system, 49
Operator priority, 189
Ordinate, 157

Parameter, 107
Parity, 177
Paste, 53
Pixel, 32
Polar coordinates, 231
Polymorphic, 280
Positive direction, 232
Predefined identifier, 230
Procedure, 56, 64, 258
Procedure body, 57, 60
Procedure call, 57, 60, 150
Procedure declaration, 57, 58, 150
Program, 27, 133
Programming, 27
Programming error, 40, 42
Programming language, 27

Rational, 223
Rational (real) expression, 225, 230
Real, 226
Recursive, 249
Reserved identifier, 230
return, 272
Rotation, 232

Scientific notation, 229
Scope, 155
Sequence, 23
Space bar, 35
Specification, 10, 65
Stack, 256
Step-by-step execution, 45
Syntax error, 33, 40
Syntax rule, 33

Test, 167, 169, 170, 174, 182
Translation, 234
True, 167
Type, 223
Type compatibility, 276

Variable, 72, 275, 287
Variable declaration, 275

Warning, 45
"while" loop, 192, 199